Seica
Test Solutions

VIP®
VIVA INTEGRATED PLATFORM

WWW.SEICA.COM

Functional Programming

FUNCTIONAL PROGRAMMING

PUBBLISHED BY:

Via Kennedy, 24 - 10019 STRAMBINO (TO)
Ph.: ++39 0125 63 68 11
Fax.: ++39 0125 63 68 99
e-mail: seica@seica.com
Website: http://www.seica.com

Copyright © 2013
All rights reserved

Microsoft Windows is a Trademark of Microsoft Corporation

Document code: MA-GEN0FUEN-03

Software release: VIVA

Date of issue: 2005 - 05 - 26

Seica S.p.A. reserves the right to change technical information without any notice.

INTRODUCTION....1

RUN-TIME AND COMPILED ENVIRONMENTS....2
INSTRUCTIONS....2
MODULE DESCRIPTION....2
FILE.SRG STRUCTURE....3
STRUCTURE OF THE FILE.PAT....3
FUNCTIONAL PROGRAM GENERATION....6
NOTES TO WRITE A TEST PROGRAM....6
FUNCTIONAL PROGRAM COMPILATION....7
THE STATIC RUN-TIME TERMINAL....8
THE DEBUG ENVIRONMENT FOR FUNCTIONAL PROGRAM....9
VARIABLES AND CONSTANTS....11
COMPILED ENVIRONMENT....11
RUN-TIME ENVIRONMENT....13
THE FUNCTIONAL TEST FILES....14
PARTIAL AND GLOBAL FLAGS....16
SYSTEM VARIABLES....17
Variables used both inside the File.srg and File.pat....17
Variables used only inside the File.pat....17
TIMING....18
THE TIMING DECLARATION FOR F40 MODULES....20
SUMMARY TABLE....25
TIMINGS....25
DECLARATIONS....25
CHANNEL PROGRAMMING....27
CONDITIONSUMPSRECYCLES....28
CALCULATIONS....29
INPUT/OUTPUT....29
UTILITIES....30
PATTERN OPTIONS....30
COMPILER DIRECTIVES....31
INSTRUCTION FOR THE LAM MODULE....31
INSTRUCTIONS FOR THE High Voltage/High Voltage Lite MODULES....31
INSTRUCTIONS FOR THE USB MODULE....32
INSTRUCTIONS FOR THE GEN MODULE....32
INSTRUCTIONS FOR THE F40 MODULE....33
INSTRUCTIONS TO PROGRAM THE SE2 MODULE....34
INSTRUCTIONS IN THE FILE.SRG....34

THE INSTRUCTIONS....36

SYNTAX CONVENTIONS....37
STRUCTURE FOR THE DESCRIPTION OF THE LANGUAGE....38
#INCLUDE INSTRUCTION #INCLUDE....39

#INCLUDE INSTRUCTION #INCLUDE....39

/ INSTRUCTION /....41

/ INSTRUCTION /....41

@CALCOLATE INSTRUCTION @CALCOLATE....43

@CALCOLATE INSTRUCTION @CALCOLATE....43

@COMPILER COMPILER DIRECTIVES @COMPILER....45

@COMPILER COMPILER DIRECTIVES @COMPILER....45

@FOR-@ENDFOR INSTRUCTION @FOR-@ENDFOR....47

@FOR-@ENDFOR INSTRUCTION @FOR-@ENDFOR....47

@GRAY INSTRUCTION @GRAY....49

@GRAY INSTRUCTION @GRAY....49

@IF-@ELSE-@ENDIF Instruction @IF-@ELSE-@ENDIF....50
@IF-@ELSE-@ENDIF INSTRUCTION @IF-@ELSE-@ENDIF....50
@L Instruction @L....51
@L INSTRUCTION @L....51
@LABEL Instruction @LABEL....53
@LABEL INSTRUCTION @LABEL....53
@MACRO-@ENDMACRO Instruction @MACRO-@ENDMACRO....54
@MACRO-@ENDMACRO INSTRUCTION @MACRO-@ENDMACRO....54
@macro name Instruction @macro name....58
@MACRO NAME INSTRUCTION @MACRO NAME....58
@WHILE-@ENDWHILE Instruction @WHILE-@ENDWHILE....59
@WHILE-@ENDWHILE INSTRUCTION @WHILE-@ENDWHILE....59
~ABS Instruction ~ABS....60
~ABS INSTRUCTION ~ABS....60
~ADD Instruction ~ADD....61
~ADD INSTRUCTION ~ADD....61
~AFORMAT Instruction ~AFORMAT....62
~AFORMAT INSTRUCTION ~AFORMAT....62
~AND Instruction ~AND....63
~AND INSTRUCTION ~AND....63
~API Instruction ~API....64
~API INSTRUCTION ~API....64
~ATAN Instruction ~ATAN....66
~ATAN INSTRUCTION ~ATAN....66
~ATEST Instruction ~ATEST....67
~ATEST INSTRUCTION ~ATEST....67
~BRANCH Instruction ~BRANCH....70
~BRANCH INSTRUCTION ~BRANCH....70
~CALC Instruction ~CALC....73
~CALC INSTRUCTION ~CALC....73
~CALL Instruction ~CALL....75
~CALL INSTRUCTION ~CALL....75
~CLEAR Instruction ~CLEAR....76
~CLEAR INSTRUCTION ~CLEAR....76
~CLEAR Esn Instruction ~CLEAR ESn....77
~CLEAR ESN INSTRUCTION ~CLEAR ESN....77
~CLEAR ISORES N= Instruction ~CLEAR ISORES N=....78
~CLEAR ISORES N= INSTRUCTION ~CLEAR ISORES N=....78
~CLEAR LGC Instruction ~CLEAR LGC....79
~CLEAR LGC INSTRUCTION ~CLEAR LGC....79

~CLEAR USER_BUSW Instruction ~CLEAR USER_BUSW....80
~CLEAR USER_BUSW INSTRUCTION ~CLEAR USER_BUSW....80
~CLEAR USER_DAC Instruction ~CLEAR USER_DAC....81
~CLEAR USER_DAC INSTRUCTION ~CLEAR USER_DAC....81
~CLEAR USER_GEN Instruction ~CLEAR USER_GEN....82
~CLEAR USER_GEN INSTRUCTION ~CLEAR USER_GEN....82
~CLEAR USER_LINE Instruction ~CLEAR USER_LINE....83
~CLEAR USER_LINE INSTRUCTION ~CLEAR USER_LINE....83
~CLEAR USER_LOAD Instruction ~CLEAR USER_LOAD....84
~CLEAR USER_LOAD INSTRUCTION ~CLEAR USER_LOAD....84
~CLEAR USER_WORD Instruction ~CLEAR USER_WORD....85
~CLEAR USER_WORD INSTRUCTION ~CLEAR USER_WORD....85
~COLOR Instruction ~COLOR....86
~COLOR INSTRUCTION ~COLOR....86
~COMP Instruction ~COMP....88
~COMP INSTRUCTION ~COMP....88
~COS Instruction ~COS....91
~COS INSTRUCTION ~COS....91
~CURSOR Instruction ~CURSOR....92
~CURSOR INSTRUCTION ~CURSOR....92
~DELAY Instruction ~DELAY....94
~DELAY INSTRUCTION ~DELAY....94
~DIV Instruction ~DIV....95
~DIV INSTRUCTION ~DIV....95
~DO ~DONE Instruction ~DO ~DONE....96
~DO ~DONE INSTRUCTION ~DO ~DONE....96
~EXP Instruction ~EXP....98
~EXP INSTRUCTION ~EXP....98
~FILEn CLOSE Instruction ~FILEn CLOSE....99
~FILEN CLOSE INSTRUCTION ~FILEN CLOSE....99
~FILEn OPEN_APPEND Instruction ~FILEn OPEN_APPEND....100
~FILEN OPEN_APPEND INSTRUCTION ~FILEN OPEN_APPEND....100
~FILEn OPEN_READ Instruction ~FILEn OPEN_READ....101
~FILEN OPEN_READ INSTRUCTION ~FILEN OPEN_READ....101
~FILEn OPEN_WRITE Instruction ~FILEn OPEN_WRITE....102
~FILEN OPEN_WRITE INSTRUCTION ~FILEN OPEN_WRITE....102
~FILEn READ Instruction ~FILEn READ....103
~FILEN READ INSTRUCTION ~FILEN READ....103
~FILEn REMOVE Instruction ~FILEn REMOVE....105
~FILEN REMOVE INSTRUCTION ~FILEN REMOVE....105
~FILEn REWIND Instruction ~FILEn REWIND....106

~FILEN REWIND INSTRUCTION ~FILEN REWIND....106
~FILEN WRITE INSTRUCTION ~FILEN WRITE....108
~FILEN WRITE INSTRUCTION ~FILEN WRITE....108
~FLIN INSTRUCTION ~FLIN....109
~FLIN INSTRUCTION ~FLIN....109
~FOR-~ENDFOR INSTRUCTION ~FOR-~ENDFOR....111
~FOR-~ENDFOR INSTRUCTION ~FOR-~ENDFOR....111
~FORMAT INSTRUCTION ~FORMAT....114
~FORMAT INSTRUCTION ~FORMAT....114
~GNO INSTRUCTION ~GNO....115
~GNO INSTRUCTION ~GNO....115
~GROUP INSTRUCTION ~GROUP....117
~GROUP INSTRUCTION ~GROUP....117
~IF-~ELSE-~ENDIF INSTRUCTION ~IF-~ELSE-~ENDIF....119
~IF-~ELSE-~ENDIF INSTRUCTION ~IF-~ELSE-~ENDIF....119
~IH INSTRUCTION ~IH....121
~IH INSTRUCTION ~IH....121
~IL INSTRUCTION ~IL....122
~IL INSTRUCTION ~IL....122
~LEVEL INSTRUCTION ~LEVEL....123
~LEVEL INSTRUCTION ~LEVEL....123
~LIN INSTRUCTION ~LIN....125
~LIN INSTRUCTION ~LIN....125
~LOAD INSTRUCTION ~LOAD....128
~LOAD INSTRUCTION ~LOAD....128
~LOG INSTRUCTION ~LOG....129
~LOG INSTRUCTION ~LOG....129
~MEAS COUNTER INSTRUMENT ~MEAS COUNTER....130
~MEAS COUNTER INSTRUMENT ~MEAS COUNTER....130
~MEAS PMM MAX= TIM= NRC= MRC=....132
~MEAS PMM MAX= TIM= NRC= MRC=....132
~MEAS USER_GEN INSTRUCTION ~MEAS USER_GEN....134
~MEAS USER_GEN INSTRUCTION ~MEAS USER_GEN....134
~MEAS PW1…8 INSTRUMENT ~MEAS PW1…8....135
~MEAS PW1…8 INSTRUMENT ~MEAS PW1…8....135
~MEAS VOLT INSTRUMENT ~MEAS VOLT....137
~MEAS VOLT INSTRUMENT ~MEAS VOLT....137
~MH INSTRUCTION ~MH....139
~MH INSTRUCTION ~MH....139
~ML INSTRUCTION ~ML....140

~ML INSTRUCTION ~ML....140
~MSG Instruction ~MSG....141
~MSG INSTRUCTION ~MSG....141
~MULT Instruction ~MULT....142
~MULT INSTRUCTION ~MULT....142
~MUXOUT Instruction ~MUXOUT....143
~MUXOUT INSTRUCTION ~MUXOUT....143
~OH Instruction ~OH....146
~OH INSTRUCTION ~OH....146
~OL Instruction ~OL....147
~OL INSTRUCTION ~OL....147
~OM Instruction ~OM....148
~OM INSTRUCTION ~OM....148
~ONBREAK Instruction ~ONBREAK....149
~ONBREAK INSTRUCTION ~ONBREAK....149
~ONERROR Instruction ~ONERROR....150
~ONERROR INSTRUCTION ~ONERROR....150
~OR Instruction ~OR....153
~OR INSTRUCTION ~OR....153
~PAUSE Instruction ~PAUSE....154
~PAUSE INSTRUCTION ~PAUSE....154
~PCT Instruction ~PCT....156
~PCT INSTRUCTION ~PCT....156
~PLn Instruction ~PLn....157
~PLN INSTRUCTION ~PLN....157
~PRINT Instruction ~PRINT....159
~PRINT INSTRUCTION ~PRINT....159
~PROTECTION Instruction ~PROTECTION....161
~PROTECTION INSTRUCTION ~PROTECTION....161
~PULL UP/DOWN Instruction ~PULL UP/DOWN....162
~PULL UP/DOWN INSTRUCTION ~PULL UP/DOWN....162
~READ BUSn Instruction ~READ BUSn....164
~READ BUSN INSTRUCTION ~READ BUSN....164
~READ_CLOCK Instruction ~READ_CLOCK....165
~READ_CLOCK INSTRUCTION ~READ_CLOCK....165
~READ_DATE Instruction ~READ_DATE....166
~READ_DATE INSTRUCTION ~READ_DATE....166
~READ_IEEE Instruction ~READ_IEEE....167
~READ_IEEE INSTRUCTION ~READ_IEEE....167
~READ_OPTOn Instruction ~READ_OPTOn....168

~READ_OPTON INSTRUCTION ~READ_OPTON....168
~READ_SCR Instruction ~READ_SCR....169
~READ_SCR INSTRUCTION ~READ_SCR....169
~READ STAUS PW1…8 Instrument ~READ STATUS PW1…8....170
~READ STAUS PW1…8 INSTRUMENT ~READ STATUS PW1…8....170
~READ_TIME Instruction ~READ_TIME....171
~READ_TIME INSTRUCTION ~READ_TIME....171
~REPORT Instruction ~REPORT....172
~REPORT INSTRUCTION ~REPORT....172
~RG Instruction ~RG....174
~RG INSTRUCTION ~RG....174
~ROUT RSn Instruction ~ROUT RSn....176
~ROUT RSN INSTRUCTION ~ROUT RSN....176
~SEND_IEEE Instruction ~SEND_IEEE....178
~SEND_IEEE INSTRUCTION ~SEND_IEEE....178
~SET ALM Instruction ~SET ALM....179
~SET ALM INSTRUCTION ~SET ALM....179
~SET BUS Instruction ~SET BUS....180
~SET BUS INSTRUCTION ~SET BUS....180
~SET BUS_LINE SERV value Instruction ~SET BUS_LINE SERV value....182
~SET BUS_LINE SERV VALUE INSTRUCTION ~SET BUS_LINE SERV VALUE....182
~SET CKGEN Instrument ~SET CKGEN....184
~SET CKGEN INSTRUMENT ~SET CKGEN....184
~SET DR1 Instrument ~SET DR1....185
~SET DR1 INSTRUMENT ~SET DR1....185
~SET DR2 Instrument ~SET DR2....187
~SET DR2 INSTRUMENT ~SET DR2....187
~SET EB0 Instruction ~SET EB0....188
~SET EB0 INSTRUCTION ~SET EB0....188
~SET EB1 Instruction ~SET EB1....189
~SET EB1 INSTRUCTION ~SET EB1....189
~SET ELM Instruction ~SET ELM....190
~SET ELM INSTRUCTION ~SET ELM....190
~SET EMX instrument=line Instruction ~SET EMX instrument=line....191
~SET EMX INSTRUMENT=LINE INSTRUCTION ~SET EMX INSTRUMENT=LINE....191
~SET Esn PLn IP= Instruction ~SET Esn PLn IP=....194
~SET ESN PLN IP= INSTRUCTION ~SET ESN PLN IP=....194
~SET GND Instrument ~SET GND....195
~SET GND INSTRUMENT ~SET GND....195
~SET GNDREF Instruction ~SET GNDREF....196

~SET GNDREF INSTRUCTION ~SET GNDREF....196
~SET GUA INSTRUMENT ~SET GUA....197
~SET GUA INSTRUMENT ~SET GUA....197
~SET ISOLEV N= MV= ON INSTRUCTION ~SET ISOLEV N= MV= ON....198
~SET ISOLEV N= MV= ON INSTRUCTION ~SET ISOLEV N= MV= ON....198
~SET ISORES N= OHM= INSTRUCTION ~SET ISORES N= OHM=....199
~SET ISORES N= OHM= INSTRUCTION ~SET ISORES N= OHM=....199
~SET PMM PP= PN= MODE= AC/DC INSTRUCTION ~SET PMM PP= PN= MODE= AC/DC....200
~SET PMM PP= PN= MODE= AC/DC INSTRUCTION ~SET PMM PP= PN= MODE= AC/DC....200
~SET OPC INSTRUCTION ~SET OPC....202
~SET OPC INSTRUCTION ~SET OPC....202
~SET PUL INSTRUMENT ~SET PUL....203
~SET PUL INSTRUMENT ~SET PUL....203
~SET PW1…8 INSTRUCTION ~SET PW1…8....204
~SET PW1…8 INSTRUCTION ~SET PW1…8....204
~SET SERVICE_BUS INSTRUMENT ~SET SERVICE_BUS....209
~SET SERVICE_BUS INSTRUMENT ~SET SERVICE_BUS....209
~SET USER_BUSWN INSTRUCTION ~SET USER_BUSWN....210
~SET USER_BUSWN INSTRUCTION ~SET USER_BUSWN....210
~SET USER_DACN INSTRUCTION ~SET USER_DACN....211
~SET USER_DACN INSTRUCTION ~SET USER_DACN....211
~SET USER_GEN INSTRUCTION ~SET USER_GEN....212
~SET USER_GEN INSTRUCTION ~SET USER_GEN....212
~SET USER_LINEN INSTRUCTION ~ SET USER_LINEN....213
~SET USER_LINEN INSTRUCTION ~ SET USER_LINEN....213
~SET USER_LOADN INSTRUCTION ~SET USER_LOADN....214
~SET USER_LOADN INSTRUCTION ~SET USER_LOADN....214
~SET USER_WORDN INSTRUCTION ~SET USER_WORDN....215
~SET USER_WORDN INSTRUCTION ~SET USER_WORDN....215
~SET ZTEST INSTRUCTION ~SET ZTEST....216
~SET ZTEST INSTRUCTION ~SET ZTEST....216
~SHL INSTRUCTION ~SHL....217
~SHL INSTRUCTION ~SHL....217
~SHR INSTRUCTION ~SHR....218
~SHR INSTRUCTION ~SHR....218
~SIN INSTRUCTION ~SIN....219
~SIN INSTRUCTION ~SIN....219
~SQR INSTRUCTION ~SQR....220
~SQR INSTRUCTION ~SQR....220
~SQUARE INSTRUCTION ~SQUARE....221

~SQUARE INSTRUCTION ~SQUARE....221
~STORE Instruction ~STORE....222
~STORE INSTRUCTION ~STORE....222
~SUB Instruction ~SUB....223
~SUB INSTRUCTION ~SUB....223
~SUBR Instruction ~SUBR....224
~SUBR INSTRUCTION ~SUBR....224
~SYS Instruction ~SYS....229
~SYS INSTRUCTION ~SYS....229
~TEST COUNTER Instrument ~TEST COUNTER....231
~TEST COUNTER INSTRUMENT ~TEST COUNTER....231
~TEST PW1…8 Instrument ~TEST PW1…8....233
~TEST PW1…8 INSTRUMENT ~TEST PW1…8....233
~TEST PMM MAX= TIM= LO= HI= ONERROR= ONPASS....234
~TEST PMM MAX= TIM= LO= HI= ONERROR= ONPASS....234
~TEST VOLT Instruction ~TEST VOLT....235
~TEST VOLT INSTRUCTION ~TEST VOLT....235
~URP Instruction ~URP....237
~URP INSTRUCTION ~URP....237
~URS Instruction ~URS....239
~URS INSTRUCTION ~URS....239
~UOP Instruction ~UOP....241
~UOP INSTRUCTION ~UOP....241
~WG Instruction ~WG....243
~WG INSTRUCTION ~WG....243
~WHILE-~ENDWHILE Instruction ~WHILE-~ENDWHILE....245
~WHILE-~ENDWHILE INSTRUCTION ~WHILE-~ENDWHILE....245
~WRITE Instruction ~WRITE....247
~WRITE INSTRUCTION ~WRITE....247
~XOR Instruction ~XOR....250
~XOR INSTRUCTION ~XOR....250
AG1…9 Instruction AG1…9....251
AG1…9 INSTRUCTION AG1…9....251
AH1…9 Instruction AH1…9....253
AH1…9 INSTRUCTION AH1…9....253
AL1…9 Instruction AL1…9....254
AL1…9 INSTRUCTION AL1…9....254
AS Instruction AS....255
AS INSTRUCTION AS....255
ASSERT1…/9 Instruction ASSERT1…/9....256

ASSERT1…/9 INSTRUCTION ASSERT1…/9....256
BEGINLOOP-ENDLOOP Instruction BEGINLOOP-ENDLOOP....259
BEGINLOOP-ENDLOOP INSTRUCTION BEGINLOOP-ENDLOOP....259
C1…9 Instruction C1…9....261
C1…9 INSTRUCTION C1…9....261
CALC Instruction CALC....262
CALC INSTRUCTION CALC....262
CLOCKF Instruction CLOCKF....265
CLOCKF INSTRUCTION CLOCKF....265
CLOCK Instruction CLOCK....266
CLOCK INSTRUCTION CLOCK....266
CLOCK2 Instruction CLOCK2....269
CLOCK2 INSTRUCTION CLOCK2....269
CONFIGURATION Compiler Directives CONFIGURATION....271
CONFIGURATION COMPILER DIRECTIVES CONFIGURATION....271
COPY Instruction COPY....272
COPY INSTRUCTION COPY....272
CR Instruction CR....273
CR INSTRUCTION CR....273
DEAD Instruction DEAD....275
DEAD INSTRUCTION DEAD....275
DECLARE ARGUMENT Instruction DECLARE ARGUMENT....276
DECLARE ARGUMENT INSTRUCTION DECLARE ARGUMENT....276
DECLARE CHANNEL Instruction DECLARE CHANNEL....277
DECLARE CHANNEL INSTRUCTION DECLARE CHANNEL....277
DECLARE GROUP Instruction DECLARE GROUP....278
DECLARE GROUP INSTRUCTION DECLARE GROUP....278
DECLARE PIN Instruction DECLARE PIN....280
DECLARE PIN INSTRUCTION DECLARE PIN....280
DECLARE PINLIST Instruction DECLARE PINLIST....282
DECLARE PINLIST INSTRUCTION DECLARE PINLIST....282
DECLARE RUNTIME Instruction DECLARE RUNTIME....284
DECLARE RUNTIME INSTRUCTION DECLARE RUNTIME....284
DECLARE RUNTIME ARGUMENT Instruction DECLARE RUNTIME ARGUMENT....286
DECLARE RUNTIME ARGUMENT INSTRUCTION DECLARE RUNTIME ARGUMENT....286
DECLARE RUNTIME ARRAY Instruction DECLARE RUNTIME ARRAY....288
DECLARE RUNTIME ARRAY INSTRUCTION DECLARE RUNTIME ARRAY....288
DECLARE RUNTIME INDEX Instruction DECLARE RUNTIME INDEX....290
DECLARE RUNTIME INDEX INSTRUCTION DECLARE RUNTIME INDEX....290
DECLARE SIGNAL Instruction DECLARE SIGNAL....291
DECLARE SIGNAL INSTRUCTION DECLARE SIGNAL....291

DECLARE SUBROUTINE Instruction DECLARE SUBROUTINE....292
DECLARE SUBROUTINE INSTRUCTION DECLARE SUBROUTINE....292
DECLARE VARIABLE Instruction DECLARE VARIABLE....293
DECLARE VARIABLE INSTRUCTION DECLARE VARIABLE....293
DECLARE VARIABLE ARRAY Instruction DECLARE VARIABLE ARRAY....294
DECLARE VARIABLE ARRAY INSTRUCTION DECLARE VARIABLE ARRAY....294
DELAY Instruction DELAY....296
DELAY INSTRUCTION DELAY....296
EOF Instruction EOF....297
EOF INSTRUCTION EOF....297
GR Instruction GR....298
GR INSTRUCTION GR....298
HI Instruction HI....300
HI INSTRUCTION HI....300
IG Instruction IG....301
IG INSTRUCTION IG....301
IH Instruction IH....303
IH INSTRUCTION IH....303
IL Instruction IL....304
IL INSTRUCTION IL....304
IN Instruction IN....305
IN INSTRUCTION IN....305
INCREMENTAL Instruction INCREMENTAL....306
INCREMENTAL INSTRUCTION INCREMENTAL....306
LABEL=label name Instruction LABEL= label name....308
LABEL=LABEL NAME INSTRUCTION LABEL= LABEL NAME....308
LO Instruction LO....309
LO INSTRUCTION LO....309
LOAD Instruction LOAD....310
LOAD INSTRUCTION LOAD....310
MG Instruction MG....311
MG INSTRUCTION MG....311
MH Instruction MH....313
MH INSTRUCTION MH....313
ML Instruction ML....314
ML INSTRUCTION ML....314
MODULE-ENDMODULE Instruction MODULE-ENDMODULE....315
MODULE-ENDMODULE INSTRUCTION MODULE-ENDMODULE....315
NR Instruction NR....318
NR INSTRUCTION NR....318
OG0…9 Instruction OG0…9....319

OG0…9 INSTRUCTION OG0…9....319
OH0…9 Instruction OH0…9....321
OH0…9 INSTRUCTION OH0…9....321
OL0…9 Instruction OL0…9....322
OL0…9 INSTRUCTION OL0…9....322
OM Instruction OM....323
OM INSTRUCTION OM....323
OU Instruction OU....324
OU INSTRUCTION OU....324
OVERLAY Instruction OVERLAY....325
OVERLAY INSTRUCTION OVERLAY....325
P1…9 Instruction P1…9....327
P1…9 INSTRUCTION P1…9....327
PERIOD Instruction PERIOD....329
PERIOD INSTRUCTION PERIOD....329
PHASE1…/9 Instruction PHASE1…/9....330
PHASE1…/9 INSTRUCTION PHASE1…/9....330
R0 Instruction R0....333
R0 INSTRUCTION R0....333
R1 Instruction R1....334
R1 INSTRUCTION R1....334
RG Instruction RG....335
RG INSTRUCTION RG....335
RZ1…9 Instruction RZ1…9....337
RZ1…9 INSTRUCTION RZ1…9....337
S1/2/3 Instruction S1/2/3....339
S1/2/3 INSTRUCTION S1/2/3....339
START-ENDEST Instruction START-ENDTEST....341
START-ENDEST INSTRUCTION START-ENDTEST....341
START DUTY_CYCLE=value Option START DUTY_CYCLE=value....343
START DUTY_CYCLE=VALUE OPTION START DUTY_CYCLE=VALUE....343
START PROBE_OFF Option START PROBE_OFF....344
START PROBE_OFF OPTION START PROBE_OFF....344
START PROBE_ON Option START PROBE_ON....346
START PROBE_ON OPTION START PROBE_ON....346
START STATIC Option START STATIC....348
START STATIC OPTION START STATIC....348
START STATUS_GENERATE Option START STATUS_GENERATE....349
START STATUS_GENERATE OPTION START STATUS_GENERATE....349
START STATUS_LEARN_FROM Option START STATUS_LEARN_FROM....350
START STATUS_LEARN_FROM OPTION START STATUS_LEARN_FROM....350

START STATUS_LEN=NUM OPTION START STATUS_LEN=NUM....351
START STATUS_LEN=NUM OPTION START STATUS_LEN=NUM....351
START STATUS_UPDATE OPTION START STATUS_UPDATE....352
START STATUS_UPDATE OPTION START STATUS_UPDATE....352
START SYNC_LOAD_OFF OPTION START SYNC_LOAD_OFF....353
START SYNC_LOAD_OFF OPTION START SYNC_LOAD_OFF....353
START SYNC_LOAD_ON OPTION START SYNC_LOAD_ON....355
START SYNC_LOAD_ON OPTION START SYNC_LOAD_ON....355
START THRESHOLD=VALUE OPTION START THRESHOLD=VALUE....356
START THRESHOLD=VALUE OPTION START THRESHOLD=VALUE....356
START TIMEOUT=VALUE OPTION START TIMEOUT=VALUE....357
START TIMEOUT=VALUE OPTION START TIMEOUT=VALUE....357
STROBE1…/9 INSTRUCTION STROBE1…/9....358
STROBE1…/9 INSTRUCTION STROBE1…/9....358
SYNC_ON/OFF INSTRUCTION SYNC_ON/OFF....360
SYNC_ON/OFF INSTRUCTION SYNC_ON/OFF....360
TIMING - ENDTIMING INSTRUCTION TIMING - ENDTIMING....361
TIMING - ENDTIMING INSTRUCTION TIMING - ENDTIMING....361
WG INSTRUCTION WG....363
WG INSTRUCTION WG....363
ZG INSTRUCTION ZG....364
ZG INSTRUCTION ZG....364
/ DEAD PATTERN OPTION / DEAD....366
/ DEAD PATTERN OPTION / DEAD....366
/ DMASTER PATTERN OPTION / DMASTER....368
/ DMASTER PATTERN OPTION / DMASTER....368
/ JUMP PATTERN OPTION / JUMP....369
/ JUMP PATTERN OPTION / JUMP....369
/ MASK PATTERN OPTION / MASK....371
/ MASK PATTERN OPTION / MASK....371
/ MASKC PATTERN OPTION / MASKC....372
/ MASKC PATTERN OPTION / MASKC....372
/ LABEL NAME PATTERN OPTION / LABEL NAME....373
/ LABEL NAME PATTERN OPTION / LABEL NAME....373
/ NOPROBE PATTERN OPTION / NOPROBE....374
/ NOPROBE PATTERN OPTION / NOPROBE....374
/ NOSTEP PATTERN OPTION / NOSTEP....375
/ NOSTEP PATTERN OPTION / NOSTEP....375
/ ONERROR PATTERN OPTION / ONERROR....377
/ ONERROR PATTERN OPTION / ONERROR....377
/ ONPASS PATTERN OPTION / ONPASS....378

/ ONPASS PATTERN OPTION / ONPASS....378
/ PROBE Pattern option / PROBE....379
/ PROBE PATTERN OPTION / PROBE....379
/ REPEAT Pattern option / REPEAT....380
/ REPEAT PATTERN OPTION / REPEAT....380
/ STATIC Pattern option / STATIC....381
/ STATIC PATTERN OPTION / STATIC....381
/ SYNC Pattern option / SYNC....382
/ SYNC PATTERN OPTION / SYNC....382
/ TIMING Pattern option / TIMING....384
/ TIMING PATTERN OPTION / TIMING....384
@CLUSTER Instruction @CLUSTER....385
@CLUSTER INSTRUCTION @CLUSTER....385
FUNCTIONAL INSTRUCTIONS of the File.SRG....387
CLK Instruction CLK....388
CLK INSTRUCTION CLK....388
CUR Instruction CUR....389
CUR INSTRUCTION CUR....389
DVH Instruction DVH....390
DVH INSTRUCTION DVH....390
DVL Instruction DVL....391
DVL INSTRUCTION DVL....391
GNO Instruction GNO....392
GNO INSTRUCTION GNO....392
LI5 Instruction LI5....393
LI5 INSTRUCTION LI5....393
NOS Instruction NOS....394
NOS INSTRUCTION NOS....394
PRB Instruction PRB....396
PRB INSTRUCTION PRB....396
THH Instruction THH....397
THH INSTRUCTION THH....397
THL Instruction THL....399
THL INSTRUCTION THL....399
TH2 Instruction TH2....401
TH2 INSTRUCTION TH2....401
TOU Instruction TOU....402
TOU INSTRUCTION TOU....402

INTRODUCTION

This chapter describes the programming language for developing functional tests of electronic boards. It is a high level, structured, proprietary language of the SEICA test systems and is composed of instructions, pattern options and instrument drivers. This manual describes each single element along with examples for its use.

RUN-TIME AND COMPILED ENVIRONMENTS

There are two functional programming environments, the COMPILED and the RUN-TIME.
Normally a test program starts in RUN-TIME ENVIRONMENT where it is possible to insert messages/instructions for the operator and switch the power supplies on. Here it is also possible to drive all instruments in the system (switch on/off decided by programmer) and to handle the graphic interface. The following step is to switch to the COMPILED ENVIRONMENT where it is possible to execute the test patterns at a certain frequency therefore applying test stimuli according to the timing of the UUT. It is possible to use the compiled environment to execute digital functional test also at high frequency.
A test system equipped with a DHF (real time controller) or F40 (high freq. channel card and real time controller) and one or more hybrid channel cards can use both the COMPILED and the RUN-TIME environment.
A test system equipped only with a LAM module and channel boards as a base configuration uses only the RUN-TIME environment. In this system is however possible to mount external instrumentation and drive it as the internal one.

This notation in the instruction description:

Compiled	Run-time
X	**X**

Indicates in which environment it can be used.

The notation:

Module	Compiled	Run-time
USB	**X**	**X**

Indicates the module that can use the instruction described.

INSTRUCTIONS

Instructions are divided in: compiler directives and run-time instructions. The first group is composed of those constructs that begins with the @ character or the DECLARE VARIABLE keyword. With the compiler directives it is possible to execute macros, to do FOR-ENDFOR cycles, to use IF-THEN-ELSE constructs, to perform arithmetic/logic expressions and “move” the functional channels at a certain frequency.
The run-time instructions begins with the ~ or & character and permits the programmer to change the test program execution (run-time), to handle the graphic interface (static Run-time terminal) to define subroutines, to execute FOR-ENDFOR cycles, to use IF-THEN-ELSE constructs, to perform arithmetic/logic expressions and “move” the functional channels. There is an overlay area between the two groups because there are constructs and instructions that work both in run-time and in compiled environment. The compiler directives/instruction applicability is indicated with the notation listed in the previous paragraph.

MODULE DESCRIPTION

LAM: logic/analogical measurement
SER: Power supply control
SE2: Power supply control with current programming
P32: 32-hybrid channel with programmable voltage levels
F40: 32-hybrid channel high frequency (25Mhz) and real time controller
USB: User board with programmable resources (loads, OP AMP, relay,) for customization
GEN: programmable waveform generator
HV: AC/DC high voltage/high current meter
EXL: Matrix to connect external instrumentation
DHF: real time controller for digital pattern up to 10Mhz

File.srg structure

A file.srg must be created to develop a test program (both in-circuit and functional). This file lists all in-circuit tests and list also the functional clusters as illustrated in the figure below:

The File.srg contains the “call” to the functional clusters.

@CLUSTER(“funct. Cluster name”, cluster type/diagnostic type);

Example:

@CLUSTER(“PRG1”,S);

@CLUSTER(“PRG2”,P)

STRUCTURE OF THE File.pat

The test program is divided into different sections as illustrated below:

Directives of the
COMPILER

TIMING
Declaration

CHANNELS, GROUPS,
PINS, PINLIST, SIGNALS,
VARIABLES, ARGUMENTS
Declarations

MACRO
Declaration

SUBROUTINE
Declaration

START program name STATIC

Static session

/ DMASTER
Dynamic session

/ STATIC
Static session

ENDTEST

EXAMPLE 1

COMPILER Directives	@COMPILER NUMBER_ON; @COMPILER BACKUP MACRO_ON;
TIMING Declaration	TIMING TIM1 PERIOD 10C; PHASE 4C 6C; STROBE 9C; ENDTIMING;
CHANNELS, GROUPS,... Declarations	DECLARE CHANNEL CL1=1; DECLARE CHANNEL CL2=2; DECLARE CHANNEL CH1=49; DECLARE CHANNEL CH2=50; DECLARE GROUP CAN=CH1,CH2; DECLARE VARIABLE RUNTIME INTEGER INT1;
MACRO Declaration	@MACRO CLOCK1; IL CL1; /; IH CL1; /; @ENDMACRO; @MACRO CLOCK2; IL CL2; /; IH CL2; /; @ENDMACRO;
Static session to set channel Levels, displaying messages	START PRG1 STATIC; ~LEVELA,IL=0.1,IH=4.7,OL=1.5,OH=2.4; ~GROUPA=1/32; ~CURSOR 40,1; ~WRITE “TEST PROGRAM A53D”; ~ CURSOR 20,3; ~WRITE “INSERT BOARD ON THE ADAPTER”; ~PAUSE;
Dynamic session to move the channels in frequency	/ DMASTER; IL CH1; /; @CLOCK1; /; IL CH2; /; @CLOCK2; /; @CALC INT1=1; @CALC INT1=INT1+1; /; IG CAN=B11;

```
/;
```

Static session to print results and messages

```
~CURSOR 20,3;
~WRITE “PROGRAM END”;
~DELAY 10s;
ENDTEST;
```

EXAMPLE 2

COMPILER Directives

```
@COMPILER COMPLEX=2000;
@COMPILER BACKUP LOOP_ON;
```

TIMING Declaration

```
TIMING TIM2;
PERIOD 10C;
PHASE 4C 6C;
OVERLAY 2C;
DEAD 4C;
STROBE 9C;
ENDTIMING;
```

CHANNELS, GROUPS,... Declarations

```
DECLARE CHANNEL CL1=1;
DECLARE CHANNEL CL2=2;
DECLARE CHANNEL CH1=49;
DECLARE CHANNEL CH2=50;
DECLARE VARIABLE RUNTIME INTEGER INT1;
```

SUBROUTINE Declaration

```
    ~SUBR CLOCK1;
    IL CL1;
    /;
    IH CL1;
    /;
    ~RETURN;
    ~ENDSUBR;
    ~SUBR CLOCK2;
    IL CL2;
    /;
    IH CL2;
    /;
    ~RETURN;
    ~ENDSUBR;
DECLARE SUBROUTINE CLOCK1, CLOCK2;

START PRG2 STATIC;
```

Test program Static session only

```
~IL CH1;
/;
~CALL CLOCK1;
/;
~IL CH2;
/;
~CALL CLOCK2;
/;
~CALC INT1=1;
~CALC INT1=INT1+1;
/;
ENDTEST;
```

FUNCTIONAL PROGRAM GENERATION

This is the procedure to generate a functional test

1) From the Desktop, enter the S20WIN environment double-clicking on its icon
2) Select the BOARD\Select menu and create a new board
3) Insert the name of the new board (up to 8 characters)
4) Select the BOARD\New and choose "Source Programs (*.srg)" to generate a file.srg
5) Write @CLUSTER(….) in the File.srg to call a functional cluster. Refer to the @CLUSTER description for the syntax.
6) Select the Cluster\Select menu to create a functional cluster
7) Choose the .pat extension
8) Write the name of the File.pat. NOTE: the file.pat name must be different from the board name.
9) An empty File.pat is opened where it is possible to write the functional program (refer to the description of the different instructions and to the "Notes to write a test program" paragraph here below.
10) Select the Cluster\Pattern Compiler menu to compile the functional cluster file.pat. Refer to the "functional program compilation" paragraph for details.
11) Select the Run\Compiler menu to compile the file.srg.
12) Repeat step from 6 to 10 if there are other functional clusters.
13) Select the RUN\Test menu to enter the test environment
14) Press the RUN button to execute the test program. To open the debug environment for functional test, press the View button on the left hand corner of the Static run-time terminal. If errors are present the debug dialog is automatically opened. **NOTE:** If the NOS=Y instruction in present in the source file (file.srg) the program halts with power supplies ON, it is possible to continue the test program pressing the Continue button. If the NOS=Y instruction in NOT present in the File.srg, the test program halts with power supplies OFF. By pressing the Continue or Abort button the program ends. Refer to the "debug environment for functional programs" for details.

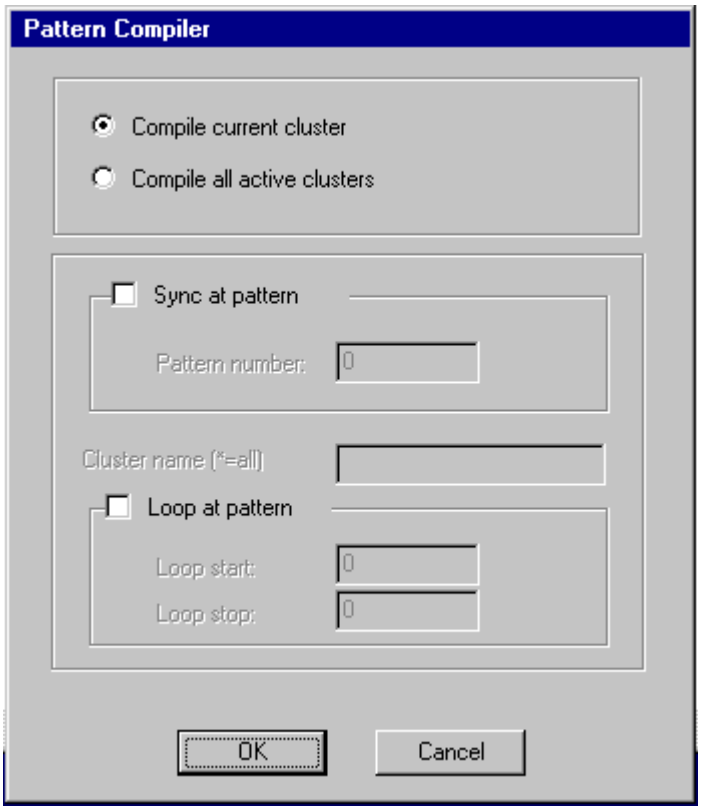
NOTES TO WRITE A TEST PROGRAM

The notes here below are usually followed when writing a functional test program:

- The command keywords are in CAPITAL LETTERS (optional but advised for a better clearness)
- Variable names are in capital letters (optional but advised for a better clearness)
- The ! character is used as a comment at the beginning or at the end of the line.
- The * character is used as a comment at the beginning of the line only
- The ; character ends the line
- The / character indicates a test pattern
- The & character is alias of ~

FUNCTIONAL PROGRAM COMPILATION

The Cluster\Pattern Compiler menu must be selected to compile a functional program (File.pat).
The following dialog appears:



Where:

Compile current cluster: compiles the current cluster (the last opened).

Compile all active cluster: compiles all clusters (all file.pat) set in the Cluster\Set Active menu

Sync at pattern: automatically inserts a sync instruction in the program code DURING COMPILATION. The sync instruction generates a synchronism signal that can be read with a scope as used to make the program debug easier. The sync signal is generated on the SYNC plug of the test system.

Pattern number: specifies at which pattern number the sync must be generated. The pattern number is usually read in the debug environment for functional program (Error Information dialog) that opens automatically when an error occurs.

Cluster name: specifies the name of the cluster where there is a loop.

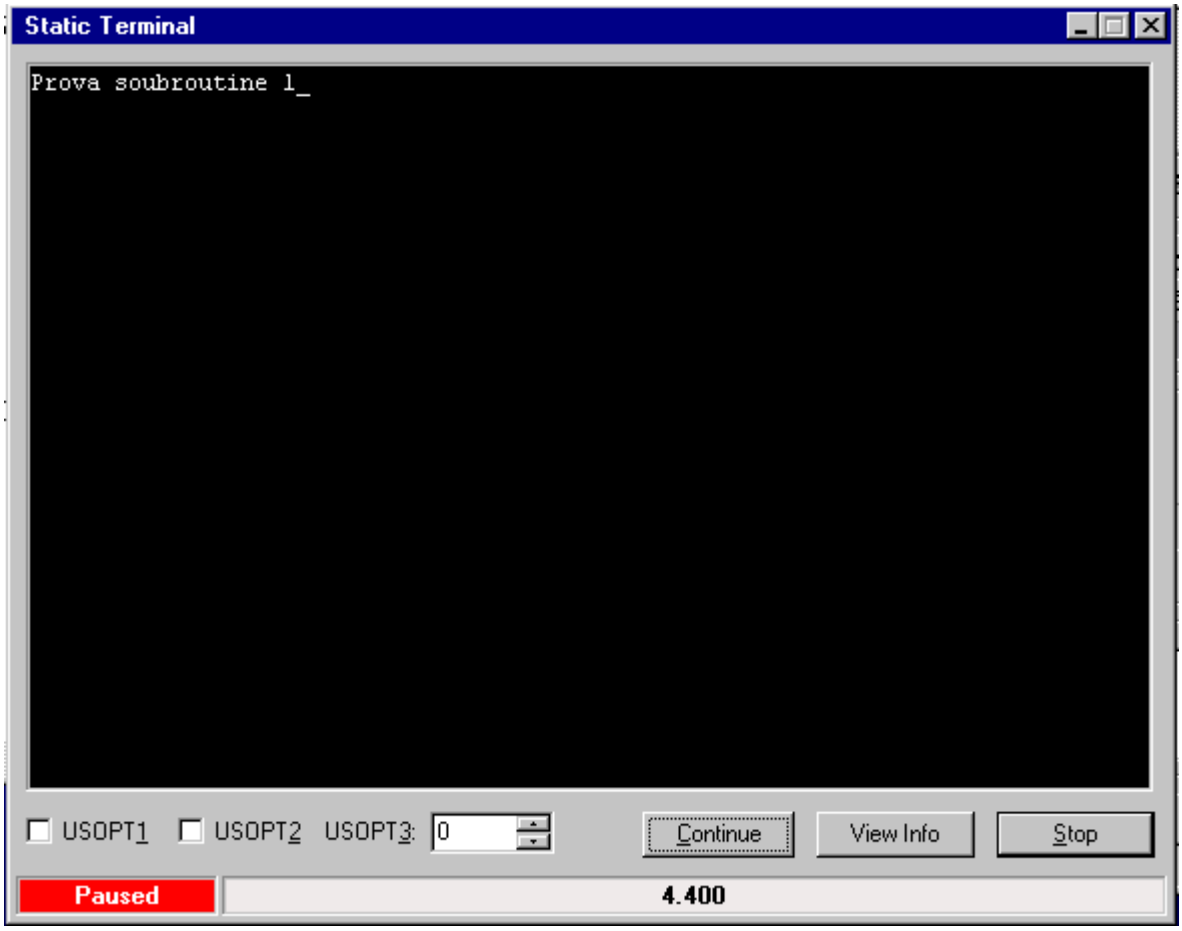
Loop at pattern: it sets a loop on the above-mentioned cluster

Loop start: start pattern for the loop

Loop stop: end pattern for the loop

THE STATIC RUN-TIME TERMINAL

The Static terminal is automatically opened if the functional program contains instructions that write messages on the screen. The Static terminal is illustrated in the figure below. In the upper part of the terminal appear the test results while the lower part displays the information about the program status (program in execution/ program in halt). USOPT1, USOPT2 and USOPT3 are system variables whose values can be checked or changed during program execution in the Static Terminal. (Refer to the “System variable” paragraph, later on described).



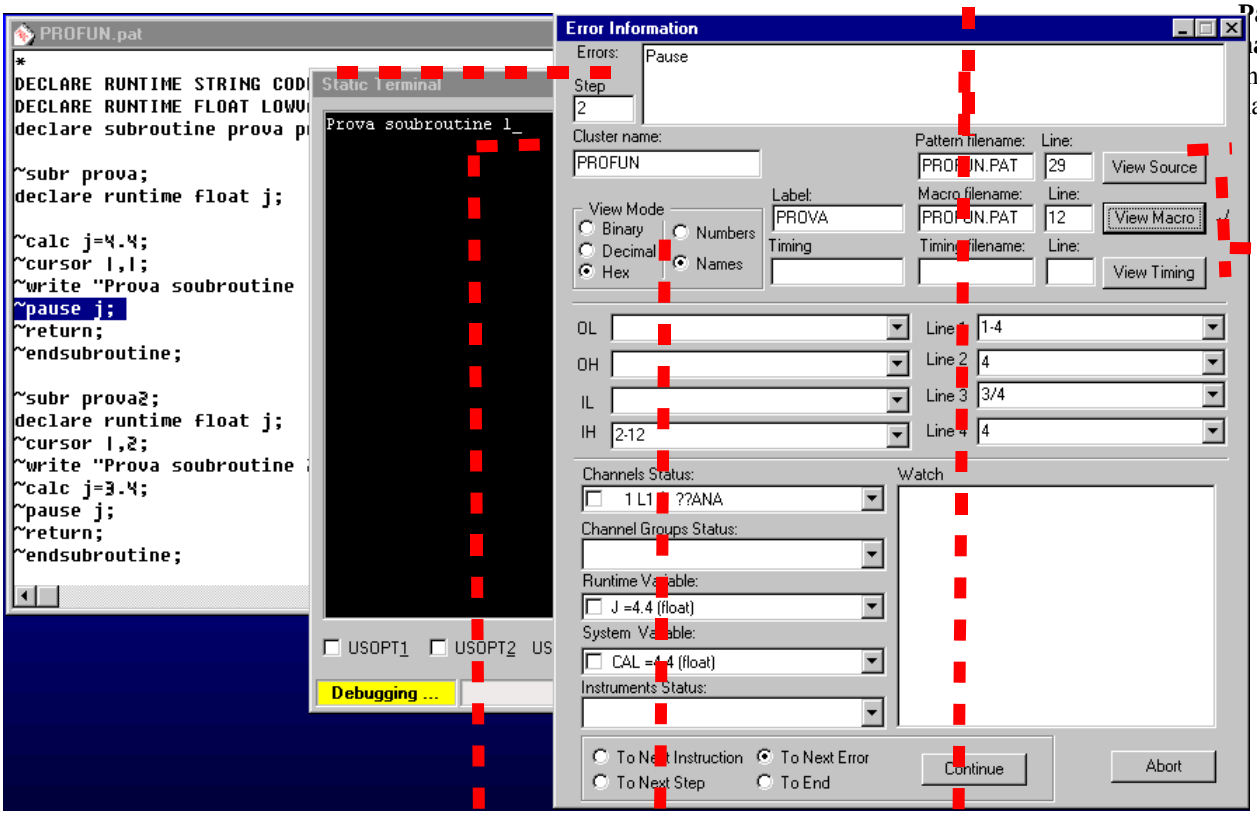
The **Continue** button permits the operator to continue the program execution after a pause (done with the ~PAUSE instruction).

The **View Info** button opens the Error Information dialog used during the program debug. The dialog appears in the following figure.

The **Stop** button, ends the program.

THE DEBUG ENVIRONMENT FOR FUNCTIONAL PROGRAM

The S20WIN, from release 3.12, gives the use a useful tool for the functional program DEBUG.
It is a dialog that is opened on the right part of the Static Terminal when an error occurs during the program execution. In the dialog are listed useful information for the user such as the number of the line in the program where the error occurred, the status of all channels, of signal busses etc. This info speeds up and makes easier the program debug especially for complex boards.



Pattern file name: indicates the pattern file name

If you press **view source**, the pattern file and the line in error is displayed on the left of the screen

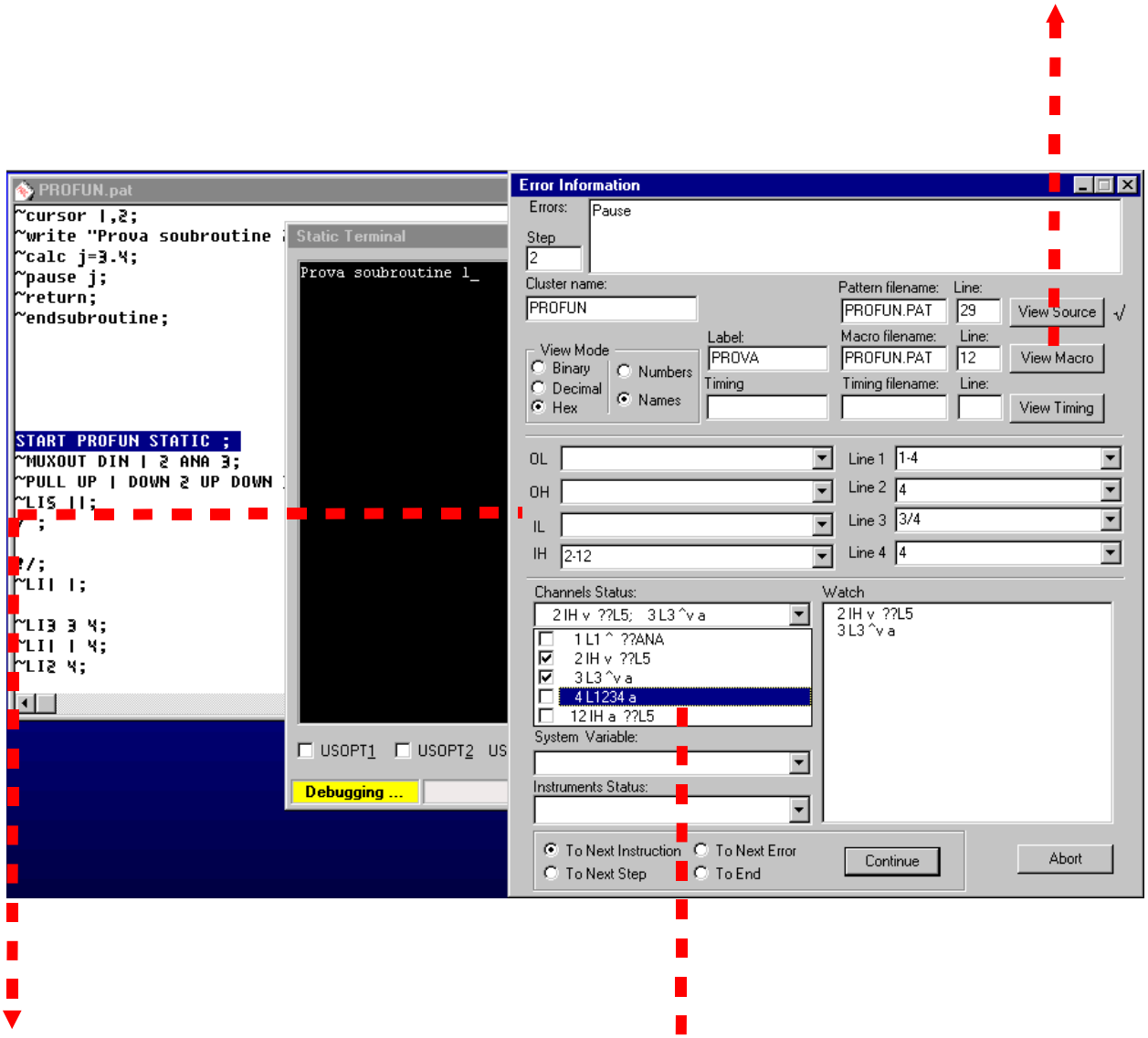
Errors: displays the signals in error. It lists the value read by the system.

Cluster name: indicates the current cluster in debug **(a START-ENDTEST section)**
Step number: is the number of the failing step in the current cluster

In **View mode** it is possible to display the value of the different signal busses in decimal, binary or hexadecimal format for better reading and understanding.

If **Numbers** is selected, instead of signal names (DO, D1, A0, etc…), the physical channel numbers are displayed (4, 33, 14,etc) on the right of the screen.

If you press the **View macro** button the cursor will move on the macro in error in the pattern file and the label in error (if present) and the line in error are displayed. If the macro is written in a library file, the library filename is displayed in the Macro filename field.



In the **OL and OH** fields is displayed the channel/s that must be checked low or high until the current step.
In the **IL and IH** fields are displayed the channels that are in the low or high state
The **LINE 1-4** fields display the channels connected to the analog lines of the LAM module.

In the **channel status** it is possible to view the status of all channel up to the pattern in error. It is possible to see also the channels moved in the previous patterns. The masked channels (OM) are not displayed.
If one or more channel is selected, they are displayed on the right so you can have a view only of those channels.

The * character beside the channel indicates that it is in error

The “a” character indicates that the channel involved is connected to the analog hardware on the channel module. Example: 4 L1234 a it means that channel 4 is connected to the analog lines 1,2,3,4 of the LAM module and to the analog hardware on the F40 module (SERANA relay).

The ^ character indicates that a pull-up resistor is connected to the channel. Example: 1 L1 ^ , It means that channel 1 is connected to line 1 and to a pull-up resistor.

The v character indicates that a pull down resistor is connected to the channel.

Example: 5 L3 v means: channel 5 is connected to line 3 and to a pull down.

The ^v characters indicate that a pull-up and pull down resistors are connected to the channel.
Example: 3 L3 ^v a means: channel 3 is connected to the analog part and to a pull-up and pull down.

The ?? characters indicate that the channel is disconnected from what indicated beside
Example: 12 IH a ??L5 means: channel 12 is at a logic high level, it is connected to the analog part of the channel, we are trying to use it as a digital channel but it is not connected to 5.

The ?? ANA characters indicate that the channel is connected to an analog line, we are trying to use it but it is not connected to the output (for F40 boards only. Refer to the ~MUXOUT instruction)

The **Channel Group Status** lists the signal bus value (example: data bus or address bus) up to the current pattern. The expected value and the read value (found) are displayed. It is possible to select the desired signals that will be highlighted on the right part of the screen.

The **Runtime variable** field lists the value of the variables used in the program. The **system variable** lists only the system ones.

The **Instrument Status** is not use. It will be utilized for future functions.

To Next Instruction, To Next Error, To Next Step, To End allows the operator to move forward in program execution.

In general: If the NOS=Y instruction is present in the file.srg, the test program halts with power supplies ON. It is then possible to go on pressing the **Continue** button.

If the NOS=Y instruction is NOT present in the file.srg, the test program halts and switched the power supplies OFF. The program ends both pressing Abort or Continue.

VARIABLES AND CONSTANTS

The constants and variables in a program, store values used in arithmetic/log expressions, loop cycles control instructions. In a Seica functional test program, constants and variables have a different meaning according to the environment (compiled/run-time) ad described below.

COMPILED ENVIRONMENT

- INTEGER constant
- INTEGER variables
- INTEGER array

Ranges

The allowed range for constant is:

-0x7FFFFFFF ≤ constant ≤ +0x7FFFFFFF

The allowed range for INTEGER variable is:

-0x7FFFFFFF ≤ variable ≤ +0x7FFFFFFF

Prefixes

Constants can have the following prefixes:

B	for binary constant
D or no character:	for decimal constant
H or 0X:	for hexadecimal constant
O	for octal constant

***Example*:**

B100101	binary con
D195	decimal constant
170	decimal constant
H4AF5	hexadecimal constant male
0X2C5	hexadecimal constant male

Syntax

The syntax to declare INTEGER variables is:

DECLARE VARIABLE variable name or DECLARE VARIABLE variable name=initialization value

NOTE: The compiled variables can be used also inside a run-time instruction. In this case the % character must be written first.

EXAMPLE 1

DECLARE VARIABLE IND;	Compiled variable declaration
DECLARE RUNTIME INTEGER NUM;	Run-time variable declaration
~CALC NUM=NUM+%IND;	Run-time instruction for an arithmetical/logic expression with a compiled variable (%IND)

EXAMPLE 2

DECLARE VARIABLE IND;	Compiled variable declaration
DECLARE RUNTIME INTEGER NUM;	Run-time variable declaration
DECLARE RUNTIME ARRAY ARR[3];	Run-time array declaration
DECLARE VARIABLE ARRAY AR[3]=1,3,4;	Compiled array declaration
~CALC NUM=ARR[%IND];	In this instruction, the run-time array index is a compiled variable. Since there is a compiled variable inside a run-time instruction, the index is preceded by the % character.
~CALC NUM= %AR[%NUM];	The index of the compiled array is a compiled variable. Since there are two compiled objects inside a run-time instruction, they both begin with the % character.

RUN-TIME ENVIRONMENT

- INTEGER constant
- FLOAT constant
- STRING constant

- INTEGER variable
- FLOAT variable
- STRING variable

- INTEGER array
- FLOAT array
- STRING array

Ranges

The allowed range for run-time INTEGER constants is:

-0x7FFFFFFF ≤ constant ≤ +0x7FFFFFFF

The allowed range for run-time FLOAT constants is:

-3.4E-38 ≤ constant ≤ + 3.4E+38

The allowed range for run-time STRING constants is:

255 characters enclosed in quotation marks (“”)

The allowed range for run-time INTEGER variables is:

-0x7FFFFFFF ≤ var ≤ +0x7FFFFFFF

The allowed range for run-time FLOAT variables is:

-1.7E-308 ≤ var ≤ + 1.7E+308

The allowed range for run-time STRING variables is:

80 characters enclosed in quotation marks (“”)

Prefixes

Constants can have the following prefixes:

B	binary run-time constant
D or no character:	decimal run-time constant
H or 0X	hexadecimal run-time constant
O	octal run-time constant

***Example*:**

B100101	binary run-time constant
D195	decimal run-time constant
170	decimal run-time constant
H4AF5	hexadecimal run-time constant
0X2C5	hexadecimal run-time constant

Syntax

The syntax to declare INTEGER run-time variables is:

DECLARE RUNTIME INTEGER

The syntax to declare FLOAT run-time variables is:

DECLARE RUNTIME FLOAT

The syntax to declare STRING run-time variables is:

DECLARE RUNTIME STRING

Conversions

In run-time environment it is possible to convert a variable type into an other one. (***Example***: a FLOAT variable converted into an INTEGER variable). The table below illustrated the conversions allowed and their result:

VARIABILE TYPE	CONVERTED INTO	RESULT
Integer	Float	The decimal part is considered equal to zero.
Integer	String	Refer to the ~AFORMAT instruction
Float	Integer	The decimal part is truncated without rounding off.
Float	String	Refer to the ~AFORMAT instruction
String	Integer	If the STRING contains a number, it is converted into a number. EXAMPLE: “41” → 41 (in numeric format) If the STRING contains a letter, the result will be 0 EXAMPLE: “A” → 0
String	Float	If the STRING contains a number, it is converted into a number.. EXAMPLE: “41.3” → 41.3 (in numeric format) If the STRING contains a letter, the result will be 0. EXAMPLE: “A” → 0

THE FUNCTIONAL TEST FILES

Different files are created when developing a test program. These files have the following description:

File.srg Source file where in-circuit tests are listed (if present) and functional clusters are “called”.

File.lib Library file

File.pat Functional cluster that contains functional tests

File.pac Listing file created during file.pat compilation. It has the same content of the file.pat but it is possible to have additional info (pattern number, macro listing…) inside. (Refer to the @COMPILER directive)

File.lis Listing file that contains compilation error of File.pat

File.sym Listing file crated during compilation

File.obj Object file of File.srg

File.obf Object file of File.pat

PARTIAL AND GLOBAL FLAGS

In the functional program, two error flags are set if there is an error (value out of limits, a different value from the one expected an error in the measure). These two flags are the partial and the global error flag. The partial error flag is referred to the error occurred in the current test. It is therefore set when the current test fails but it doesn't mean that the whole test program will give a FAIL as the final result. The partial error flag is used to take a decision, for example: if the result of a measure is inside a certain range the program will go to LABEL1, if the result is out of limit, the program will go toLABEL2. The partial error flag is reset by subsequently instructions in the program that give a positive result (see below).
The global error flag is referred to the global error of the program and, if it is active, the program will FAIL.
There are instructions that set the partial error flag and others that set the global one as indicated in the following table.

INSTRUCTION	Partial error flag	Global error flag
~ATEST	X	X
~CLEAR	X	
~COMP	X	
~MEAS	X	
~SET	X	
~TEST	X	X
/ or PATTERN	X	X
~FILE (*) instructions	X	

The partial error flag can be reset by the following instructions if they had a positive result:
~ATEST, ~CLEAR, ~COMP, ~MEAS, ~SET, ~TEST, PATTERN
If the global error flag is set it remains active however, advanced programmers can, if needed, reset the flag changing the value of the FAIL variable with a ~CALC instruction. To be more precise, if the FAIL variable value is different from 0, the global error flag is active so there will be a FAIL at the end of the test program. If the variable equals 0, the global error flag is not active so the test program will give a PASS.

(*) The ~FILE instructions set the partial error flag only if there are these conditions:

- An error is encountered when opening/closing the file
- The specified file doesn't exist
- The file is write-protected and we are trying to write it
- We are trying to read beyond the end of file

SYSTEM VARIABLES

In the in-circuit and functional test environments there is some system variables used to exchange values between programs. Some of them are used both inside the source file (file.srg) and the functional cluster (file.pat). Some of them are only used inside the file.pat

Variables used both inside the File.srg and File.pat

- 15 string variables STR[1-15] up to 80 characters long
- 15 float variables NUM[1-15] up to 80 characters long
- 7 special variables listed in the table below

SPECIAL VARIABLES	DESCRIPTION
CODPRO	Alphanumeric variable containing the PRODUCT CODE
OPERAT	Alphanumeric variable containing the OPERATOR CODE
BARCODE	Alphanumeric variable containing the BARCODE NUMBER
BATCODE	Alphanumeric variable containing the BATCH CODE
BATDATE	Alphanumeric variable containing the BATCH DATE
RICNUM	Alphanumeric variable containing the RECYCLE NUMBER of the different tests
PREVVAL	Alphanumeric variable containing the last in-circuit test value
NRTEST	Numeric variable containing the number of test program executions

Variables used only inside the File.pat

VARIABILE	DESCRIPTION
AR	Float type. It contains the result of arithmetic/logic expression and the result of measurements
USOPT1 and USOPT2	Numeric type. They appear in the Static Terminal and work as “buttons”. They can have value 0 or 1. The execution flow of the program changes if they are selected or not. They can be used in the same way as variables, so it is possible to assign them a value or check them (with the ~IF instruction for example)
USOPT3	Numeric type. It appears in the Static Terminal and works as “button”. It can have value from 0 to n (where n is an INTEGER run-time variable) The execution flow of the program changes if it is elected or not. It can be used in the same way as a variable, so it is possible to assign it a value or check it (with the ~IF instruction for example)
API	Integer type. It contains the returned value from an API function.
FAIL	Integer type. It contains the number of errors occurred during the run-time program execution.
LASTKEY	Integer type. It contains the numeric value of the last keyword stroke. **Example:** if the A character is pressed, the variable will assume value 41. If the pedal is pressed, the variable will be FF
ERRPAT	Integer type. It contains the number of the failing pattern

- 100 FLOAT registers (REG1-100). This registers are used in the ~CURSOR instruction. It is possible to write the column and row number inside the registers and change them inside a FOR cycle to draw simple graphs on the screen. Refer to ~CURSOR instruction for more details and examples. **NOTE:** When the REG1-100 are declared with the DECLARE RUNTIME FLOAT instruction it is possible to declare only the register with bigger index and all others will be automatically.

The register value can be changed in the File.srg with the @INPUT and @OUTPUT instruction, while the ~CALC instruction changes the value inside the File.pat. Usually the value of the 7 special registers is changed inside the File.srg only but not in the File.pat.

TIMING

The test of electronic boards often requires the application of stimuli according to a certain sequence and a determinate frequency. To achieve this, the test program is composed of different patterns, which contain the stimuli for the board under test. The stimuli are applied to the board at predefined instants via the test channels. The programming time, movement and read of these channels constitute the TIMING of the pattern.

There are two “moments” which can be identified within the pattern, one dedicated to programming the channels and another executive part of the pattern during which the stimuli are applied according to what is defined in the program. Thus, the test program can be considered to be an alternation between channel programming and application of the stimuli, as illustrated in the figure:

As you can see, the duration of the channel programming phase can vary. In fact, it depends on the number of channels that have to be programmed. The higher the number of channels, the longer the time that is dedicated to programming them. The duration of the stimuli application phase, on the other hand, is constant, and is regulated by the PERIOD parameter, which will be described later on.

The group of channels is considered as a set of a number of modules with 32 channels each. Each module is addressed separately. Therefore the total duration of the channel programming also depends on the physical location of the channels within the modules. ***Example:*** channels 1 through 33 have to be programmed. Channel 1 is situated on the first module. Channel 33 is situated on the second. Therefore it is necessary to first address channel 1 on the first module then channel 33 on the second module. It is obvious that the more channels there are on different modules, the longer it takes to program them}.

We will now examine the time relative to programming, movement of the driver channels and the instant of measurement of the sensor channels.

- PERIOD time
- DEAD time
- EXTRADEAD time
- OVERLAY time
- PHASE1, PHASE2, PHASE3
- ASSERT time
- STROBE time
- LOAD time

The following figures illustrate the various times:

PERIOD time: indicates the actual duration of the executive phase of the pattern (application of the stimuli)

DEAD time: Time dedicated to programming the channels between one executive phase and another (that is, between one pattern and another).

EXTRADEAD time: time dedicated to programming the channels between one executive phase and another, which is inserted automatically if the DEAD time is insufficient.

OVERLAY time: time dedicated to programming the channels inside the executive phase. In fact, to optimize the timing it is possible to use the time between the start of pattern execution and the first channel movement (which can be regulated by the ASSERT, LOAD, PHASE1/2/3 times).

PHASE1,PHASE2,PHASE3 Define, within the executive phase of the pattern, the various instants in which the channel or channels associated to the phase are commutated. In this way it is possible to associate a particular movement of the channels within the pattern. This is useful, for example, for generating clock signals that have to commutate at each step of the functional program.

ASSERT time Defines the instant of commutation of all the channels moved via the AL,AH,AS instructions. The definition of this time allows all of the channels in question to commutate at the same instant, excluding the programming time.

STROBE time Instant in which the responses from the board are read on the sensor channels within the pattern. The strobe time is also the instant of control of the driver channels.

LOAD time This allows the programming of the channel data to be moved within the executive phase of the pattern. This enables faster execution of the patterns. The LOAD time can be used only if the timing is specified with the OPTIMIZE option.

The various times which constitute the timing must be declared in the TIMING section located at the beginning of the test program. Refer to the chapter **structure of the File.pat**, which describes the different parts of the test file.

The following are two examples of timing declarations, shown graphically. For a more detailed description refer to the single instructions.

TIM1 TIMING

```
CLOCK 100NS;

TIMING TIM1;
```

```
PERIOD 10C;
PHASE1 2C 6C;
STROBE 9C;
ENDTIMING;

START PRG1;
.
.
.
ENDTEST;
```

TIM2 TIMING

```
TIMING TIM2;
PERIOD 10C;
PHASE1 2C 6C;
DEAD 4C;
STROBE 9C;
ENDTIMING;
```

THE TIMING DECLARATION FOR F40 MODULES

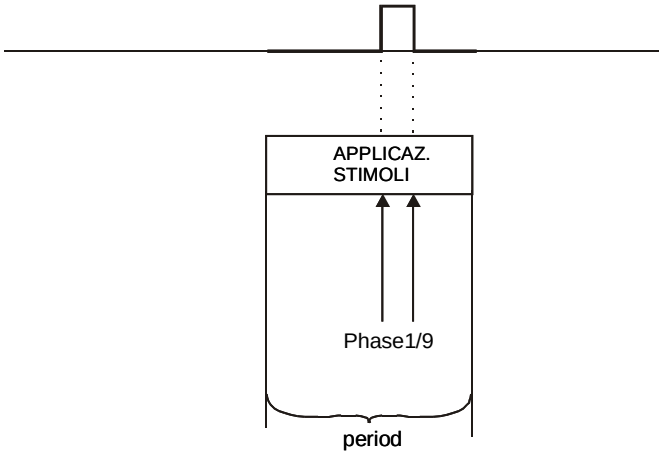
The test of electronic boards needs sometimes to apply stimuli with a certain sequence and with a fixed frequency. To achieve this result, the test program is composed of several test patterns. Inside the pattern, stimuli are applied to the UUT at fixed instants through the test channels. The channel programming, changing and reading instants form the pattern TIMING.
Let's describe better these instants (or times) that are:

- PERIOD time
- PHASE1…9
- ASSERT1…9 time
- STROBE1..9 time

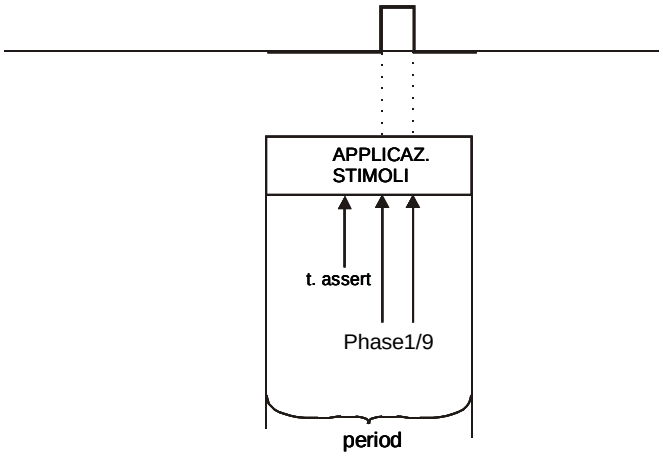
And are graphically represented in the following figures:

PERIOD time: indicates the total length of the executive part of the pattern (stimuli application)

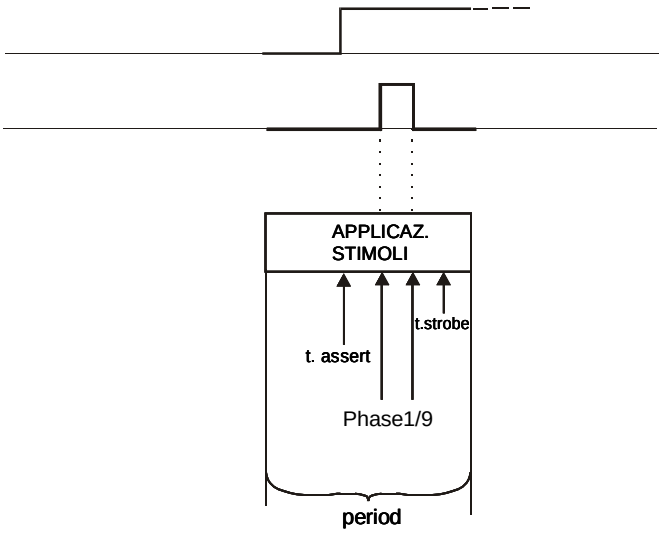
PHASE1… PHASE9 Defines the different commutation instants for channels, inside the executive phase of the test pattern. In this way it is possible to specify a special sequence of commutation for channels inside the pattern. This is very useful when you need for example, clock signals that commutates in each test pattern.



ASSERT (ASSERT1…9) time Defines the commutation instants for all signals that are moved with the AL,AH,AS instructions. All channels related to the ASSERT time, will move at the same instant avoiding the little programming delay between one channel and the other one.



STROBE (STROBE1…9) time Instant used to read and check the output signals form the UUT (on the sensor channels)



The different time declarations form the TIMING that is declared in the TIMING section at the beginning of the test program. Refer to the **Structure of the File.pat** paragraph for the description of the different file sections.

Here there are two examples of timing declaration and their graphic representations. For further details on timing declaration, refer to the description of the single instructions.

With the F40 module you can declare up to 9 different phases inside the test program but you can use only two phases for channels of the same F40 module (1-32 channels). You can have max two commutation instants inside each phase. You can declare up to 9 ASSERT time and 9 STROBE time. For the strobe, if you define two strobe times, you create a strobe window where the sensor channel is checked during all window length. You can define only one window strobe for each F40 module.
You can have max. two different assert time for each F40 module or up to four assert times if phases are not used.

If the test system is equipped with one F40 module, it is auto-synchronizing. If more than one F40 module is mounted in the system, the DFH module is used to synchronize them.

The clock has the following value: 20ns.
The period can be in the range: from 40ns to 5us
Phase can be in the range: from 20ns to 4980ns
The assert time can be in the range: from 20ns to 4980ns

EXAMPLE 1: Timing definition

```
TIMING TIM1;
STROBE1 900N;
PERIOD 1000N;
ENDTIMING;

START PRO1;
/ TIMING TIM1;
IL 1;
/;
IH 1;
OH1 2;
/;
ENDTEST;
```

EXAMPLE 2: Timing definition with phases

```
TIMING TIM1;
STROBE1 900N;
PERIOD 1000N;
PHASE1 20N 400N;
PHASE2 200N 800N;
ENDTIMING;

START PRO1;
/ TIMING TIM1;
IL  1;
IH 2;
/;
P1 1;
P2 2;
/;
/;
```

```
OH1 3;
/;
IH 1;
IL 2;
/;
ENDTEST;
```

EXAMPLE 3: Strobe window definition

```
TIMING TIM1;
STROBE1 900N;
STROBE2 500N 900N;
PERIOD 1000N;
PHASE1 100N 400N;
PHASE2 200N 800N;
ENDTIMING;

START PRO1;
/ TIMING TIM1;
IL  1;
IH 2;
/;
P1 1;
P2 2;
/;
/;
OH1 3;
/;
OL2 4;
/;
IH 1;
IL 2;
/;
ENDTEST;
```

EXAMPLE 4: ASSERT time and strobe0 definition

```
TIMING TIM1;
STROBE1 900N;
ASSERT1 160N;
ASSERT2 340N;
PERIOD 1000N;
ENDTIMING;

START PRO1;
/ TIMING TIM1;
IL  1;
IH 2;
/;
AH1 1;
AL2 2;
/;
/;
OH1 3;
OL0 5;
```

The sensor signal is checked at the end of the pattern

```
/;
IH 1;
IL 2;
/;
```

```
ENDTEST;
```

SUMMARY TABLE

The functional instructions are here divided into tables according to their function. In the tables are listed the instruction syntax and a short description. The instrument parameters are not listed in the table otherwise the description will be too heavy. Please refer to the single instrument description for further details.
The purpose of these tables is to give the user a quick idea of the instruction and to group them according to their function. **Example:** all Input/output instructions, all instructions to program the LAM module etc.

The instruction groups are:

- Timing
- Declarations
- Channel programming
- Condition/Jump/recycle
- Calculation
- Input/output
- Utility
- Pattern options
- Compiler directives
- Instructions to program the different modules in the system

TIMINGS

ASSERT value	Defines the assert instant in the timing
CLOCK value	Defines the period or the frequency for the clock used to execute the functional program
DEAD value	Defines the amount of time dedicated to the channel programming before the operating phase in the pattern
LOAD value	Defines the amount of time dedicated to the programming of the data inside the operating phase of the pattern.
OVERLAY value	Defines the amount of time dedicated to the channel programming inside the operating phase in the pattern
PERIOD value	Defines the length of the executive phase of the pattern
PHASE1/2/3 value1,value2,...	Defines the commutation instants of the driver channel for the specified phase
STROBE value	Defines the strobe instant for the driver and sensor channels in the pattern
TIMING timing name option **...** **...** **ENDTIMING**	It is the declaration of the timing.

DECLARATIONS

#INCLUDE file name	It includes the library file (if present) in the function program
/	Defines a functional step
@MACRO macro name	Defines a macro
...	
...	
@ENDMACRO	

DECLARE ARGUMENT = argument name1,argument name2,...	Declares an argument that will be used in the macro call
DECLARE CHANNEL channel name = channel number	Assigns a name to the physical channel of the system
DECLARE GROUP group name = channel group	Defines a group of channels
DECLARE PIN library = channel name	Assigns the channel name used in the library to the channel name defined in the DECLARE CHANNEL instruction (it creates an Alias)
DECLARE PINLIST = channels	Declares a group of channels that will be moved using the @L instruction
DECLARE ARRAY	Declares an integer array
DECLARE RUNTIME ARGUMENT argument type = argument name1, argument name2	Declares a global argument that will be used in the subroutine calls, in a Run-time cluster
DECLARE RUNTIME variable type = variable name1, variable name2	Declares a run-time variable
DECLARE SIGNAL = signal name	Declares a signal
DECLARE VARIABLE variable name1, variable name2,...	Declares a integer type variable used in a compiled cluster
LABEL = label name	Declares a label
MODULE module name	Declares a module
...	
...	
...	
ENDMODULE	
START program name [option 1,option 2,..]	Defines the start point of a stimuli section
...	
...	
ENDTEST	
EOF	
START program name **FULL_FUN**	Defines that the cluster is a run-time cluster with extended mode.
START program name **STATUS_LEARN_FROM**	During test program compilation it creates a listing file.pac starting from the status file
START program name **PROBE_OFF**	Disables the probe for the current cluster
START program name **PROBE_ON**	Enables the probe for the current cluster
START program name **SYNC LOAD_OFF**	Disables the synchro signal for the current cluster
START program name **SYNC LOAD ON**	Enables the synchro signal for the current cluster
START program name **STATIC**	Specifies that the current cluster is a run-time cluster
START program name **STATUS_GENERATE**	During test program compilation it creates a status file starting from the information written in the file. Pat
START program name **STATUS_LEN = num**	Defines the status file length
START program name **STATUS UPDATE**	During test program compilation, the status file is updated with the information of the File.pat
START program name **THRESHOLD = value**	Defines the probe threshold for the test program
START program name **TIMEOUT = value**	Indicates the max. length of time for the test program
~SUBR subroutine name[(argument type1, arg. Type2,...)]	Defines a subroutine
...	
...	
~RETURN [**ONERROR**] [**ONPASS**]	

CHANNEL PROGRAMMING

@L(logic states)	It moves the channel previously defined with the DECLARE PINLIST instruction
AL channel	Commutates the driver channel to the low level, at the assert time
AH channel	Commutates the driver channel to the high level, at the assert time
AS channel	Commutates the driver channel to the opposite state, at the assert time
C1/2/3 channel	It moves the driver channels related to the specified phase. The channels are not controlled
GR group name = value	Commutates the group of driver/sensor channels according to the value specified
HI channel	Converts the channel to the high level
IG group name = value	It moves and controls the group of driver channels according to the value specified.
IH channel	It moves and controls the driver channel to the high level
IL channel	It moves and controls the driver channel to the low level
IN channel	Converts the channel from driver to sensor
LO channel	Converts the channel to the low level
MG group name = value	It moves the group of driver channels according to the value specified. The channels are not controlled
MH channel	It moves and the driver channel to the high level. The channel is not controlled
ML channel	It moves and the driver channel to the low level. The channel is not controlled
OG group name = value	It controls a group of sensor channels
OL channel	It controls a low level on the sensor channel
OH channel	It controls a high level on the sensor channel
OM channel	It masks the sensor channel
OU channel	Converts the channel from sensor to driver
P1/2/3 channel	It moves the driver channel according to the related phase
S1/2/3 channel	It controls the sensor channel at the strobe time declared in the related phase.
ZG group name = value	It masks the group of sensor channel. The value indicates the channels to mask
~CLEAR	It clears the digital channels and the analog line
~GROUPn = channels	It defines the channel families. The voltage level of the family will be programmed with the instruction ~LEVEL
~IH channel	It moves and controls the driver channel to the high level. This is a run-time instruction
~IL channel	It moves and controls the driver channel to the low level. This is a run-time instruction
~LEVELn,IL=value**,IH**=value**,OL**=value**,OH**=value **,CUR**=value	Defines the voltage levels and current limits of the channel families defined with ~GROUP instruction
~LI line = [*,] [*] [channel]	Connects/disconnects to/from the requested line the channel listed in the instruction
~FLI line = [*,] [*] [channel]	Connects/disconnects to/from the requested line the channel listed in the instruction. The lines are considered as 8 separate lines. This instruction is used only when the test system is configured to work with 8 lines
~MH channel	It moves and the driver channel to the high level. The channel is not controlled
~ML channel	It moves and the driver channel to the low level. The channel is not controlled
~OH channel	It controls a high level on the sensor channel
~OL channel	It controls a low level on the sensor channel

~OM channel	It masks the sensor channel
~RG value= channel group	It reads the value of the group of channels and assigns it to a variable
~WG value= channel group	Assigns the specified value to a group of channels
~CLEAR LGC	It clears the digital and analog channels. After the execution of the instruction, the digital channels are switched in sensor low and they are masked. The analog channels are disconnected from the lines.

CONDITIONSUMPSRECYCLES

@macro name	Executes the macro
@FOR variable = initial value **TO** final value [**STEP** step value] **…** **…** **@ENDFOR**	Executes the instruction enclosed in the FOR cycle until the condition is false
... **...** **@ELSE** **...** **@ENDIF**	Executes the instructions enclosed in the IF branch if the specified condition is true. Otherwise it executes the ELSE branch
@WHILE variable 1 condition constant/variable 2 **...** **...** **@ENDWHILE**	Executes the @WHILE cycle until the initial condition is true
BEGINLOOP num **...** **...** **ENDLOOP**	Executes a LOOP, the times specified by "num"
~ATEST variable [LO=value][HI=value][ONERROR label][ONPASS label]	Executes the comparison between the specified value and the range in the instruction. If the value is out of range, the partial and global flags will be active
~BRANCH [**ONERROR**][**ONPASS**] label	It jumps to the specified label. The jump can be on condition
~CALL subroutine name[(parameter 1,parameter 2,...)][ONERROR][ONPASS]	It calls a subroutine. the call can be done on condition.
~COMP variable [LO=value][HI=value][ONERROR label][ONPASS label]	Executes the comparison between the specified value and the range in the instruction. If the value is out of range, the partial flag will be active
~DO num **…** **…** **~DONE**	Executes the instructions enclosed in the DO-DONE cycle, the times specified by "num"
~FOR [**NOT**] variable condition constant/variabile2/expression [**TO** final value][**STEP** step value][**DO** action] **...** **...** **~ENDFOR**	Executes the instructions enclosed in the FOR cycle until the condition is not true

~IF [**NOT**] variable condition constant/variabile2/expression **...** **...** **~ELSE** **...** **...** **~ENDIF**	Executes the instructions enclosed in the IF branch if the specified condition is true. Otherwise it executes the ELSE branch
~WHILE [**NOT**] variable condition costante/variabile2/espressione **...** **...** **~ENDWHILE**	Executes the @WHILE cycle until the initial condition is true

CALCULATIONS

@CALCOLATE variable=expression	Executes arithmetic/logic calculations
@GRAY variable= expression	Executes arithmetic/logic calculations and converts the result according to the GRAY code
~ABS	Calculates the absolute value of the AR register
~ADD value	Sums the specified value to the AR register
~AND value	Makes the AND function between the specified value and the content of AR register
~ATAN	Calculate the arc-tangent of the AR register
~CALC variable = expression	Executes arithmetic/logic calculations
~COS	Calculates the cosine of the AR register
~DIV value	Divides the AR register by the specified value
~EXP	Calculates the e^x of the AR register
~LOAD value	Loads the specified value into the AR register
~LOG	Calculates the natural logarithm of the AR register
~MULT value	Multiplies the AR register by the specified value
~OR value	Calculates the OR function between the AR register and the specified value
~SHL value	It shifts the AR register in the left direction
~SHR value	It shifts the AR register in the right direction
~SIN	Calculates the sine of the AR register
~SQR	Calculates the square root of the AR register
~SQUARE	Calculates the square of the AR register
~STORE	Unloads the AR register to the specified variable
~SUB value	Subtracts the specified value to the AR register
~XOR value	Calculates the XOR function between the AR register and the specified value
CALC value	In a COMPILED session of the program it executes logical arithmetic calculations and assignments between RUN-TIME constants and variables. This instruction is tipically used with WG and RG instructions.

INPUT/OUTPUT

~COLOR character, background	Specifies the character and background colors for the following output operations
~CURSOR column, row [attribute1, attribute2,...]	Defines the cursor position
~AFORMAT integer numbers, decimal numbers, format	Defines the format and notation for variables and constants

~FORMAT integer numbers, decimal numbers [**AUTOCR**] or **~FORMAT** format	Defines the format and notation for INTEGER and FLOAT variables and contents during the conversion into STRING format
~PRINT value1,value2,...	Prints the specified values
~READ_IEEE variable	Reads from the IEEE bus a float value and stores it in the variable
~READ_SCR value	Inputs a float number from the keyboard and stores it in the variable
~REPORT value1,value2,...	It displays the specified values in the TEST REPORTS dialog
~SEND_IEEE value	Writes the value on the IEEE bus
~WRITE value1,value2,...	It displays the specified values on the screen

UTILITIES

~API "module name","function name", parameters	Calls an API function in a DLL
~DELAY value	It waits for the length of time specified
~FILEn CLOSE	Close the file previously opened with the ~FILEn OPEN instruction
~FILEn OPEN_WRITE	Opens the specified file in WRITE mode
~FILEn WRITE	Writes a data into the file previously opened in WRITE mode
~FILEn OPEN_APPEND	Opens the specified file in WRITE mode and it is ready to append new data
~FILEn OPEN_READ	Opens the specified file in READ mode
~FILEn READ	Reads a data from the file previously opened in READ mode. The data is stored in the specified variable
~FILEn REWIND	Moves the pointer to the beginning of the file
~READ_CLOCK value	Reads the number of clock pulses of the Personal Computer clock
~READ_DATE value	Reads the Personal Computer date
~READ_TIME value	Reads the Personal Computer time
~PAUSE [value]	It stops the program execution and waits that the operator press a key
~MSG [value]	Displays the indicated value inside the lower status bar of the Static terminal
~ONBREAK option	Defines the action to be carried out when a pause occurs in the program execution
~ONERROR option	Defines the action to be carried out when an error occurs in the program execution
~SYS [view mode,][execution mode,] "program name","parameters"	Executes the specified application
CLOCK2 value	Defines the period of frequency of the auxiliary clock used for external instrumentations

PATTERN OPTIONS

DEAD value	Defines the time needed for channel programming before the executive phase in the pattern
DMASTER	It switches from the Run-time to the Compiled environment
JUMP label	It jumps to the specified label
Nome label	Defines a label
MASK	Masks all sensor channels in the pattern. Partial and global flags are masked
MASKC	Masks all sensor channels in the pattern. The partial flag is active but the global one is masked
NOSTEP	Disables the pattern calculation. In this way, the calculation will not be present in the status file and in the listing file (file.pac)

NOPROBE	Disables the probe for the current pattern
ONERROR label	If the partial flag is active it will jump to the specified label
ONPASS label	If the partial flag is NOT active it will jump to the specified label
PROBE	Enables the probe for the current pattern
STATIC	It switches from the Compiled to the Run-time environment
SYNC	Generates the synchronism signal in the current pattern
TIMING timing name	Apply the specified timing to the current pattern. The timing will be active from the current pattern, onwards

COMPILER DIRECTIVES

@COMPILER option	Specifies the options that the compiler will use during program compilation
@LABEL=label	Permits the programmers to declare different labels inside the @FOR, @WHILE cycles and macros @MACRO
CONFIGURATION=channel board 1, channel board 2,...	Permits the programmer to define a test system configuration different from the current one (**Example:** the customer test system configuration) and to develop the test program on it
COPY	With this instruction, the compiler programs the status of the channels in every pattern.
INCREMENTAL	With this instruction, the compiler DOES NOT program the status of the channels in every pattern.

INSTRUCTION FOR THE LAM MODULE

~CLEAR instrument [**ALL**]	Resets the specified instrument
~MEAS COUNTER	It executes a measurement using the COUNTER according to the specified parameters. If there is an error the partial flag will be active
~MEAS VOLT	It executes a measurement using the VOLTMETER according to the specified parameters. If there is an error the partial flag will be active
~SET ALM=value	Closes/opens the ALM relays
~SET ELM=value	Closes/opens the ELM relays
~SET DR1	Programs the DR1 instrument (voltage/current generator).
~SET DR2	Programs the DR2 instrument (voltage generator)
~SET CKGEN	Programs the CKGEN instrument (clock generator)
~SET GUA	Programs the GUA instrument (guarding generator)
~SET GND	Connects/disconnects the instrument ground reference to/from system line 2
~SET OPC=value	Drives the Open Collector signals
~SET PUL	Programs the resistor matrix
~SET PW	Programs the user power supplies
~SET SERVICE_BUS	Programs the connection of the SERVICE bus lines
~TEST VOLT [**HI**=value],[**LO**=value] [**ONERROR** label][**ONPASS** label]	It executes a measurement using the VOLTMETER and compares the result with the range listed in the instruction (HI/LO). If the value is out of range the partial and global flags will be active
~TEST COUNTER [**HI**=value],[**LO**=value] [**ONERROR** label][**ONPASS** label]	It executes a measurement using the COUNTER and compares the result with the range listed in the instruction (HI/LO). If the value is out of range the partial and global flags will be active

INSTRUCTIONS FOR THE High Voltage/High Voltage Lite MODULES

~MEAS PMM	It executes a voltage or current measurement with the

	High Voltage/High Voltage Lite meter, according to the specified parameters. If there is an error the partial flag will be active
~TEST PMM [**HI**=value],[**LO**=value][**ONERROR** label] [**ONPASS** label]	It executes a voltage or current measurement with the High Voltage/High Voltage Lite meter and compares the result with the range listed in the instruction (HI/LO). If there is an error the partial and global flags will be active
~SET ESn PL IP=	Connects a resource to the internal line listed in the instruction
~SET ISOLEV	For High Voltage module only. It programs a floating voltage level.
~SET ISORES	Programs the two resistive loads.
~SET PMM PP= PN= MODE= AC/DC	Connects the meter to the line and configures it according to do a voltage or current measurement
~CLEAR ESn	Disconnects a resource from the internal line
~CLEAR ISORES N=	Clears the programming on the two resistive loads
~PCT	Closes/open a PCT contact.
~PLn	Connects/disconnects a channel to/from the internal line indicated . The internal lines are PL1, PL2, PL3, and PL4.
~PROTECTION	It reads the current protection of the HV module. If the protection is active, there will be a 1 in the AR register otherwise, the register is equal to zero.

INSTRUCTIONS FOR THE USB MODULE

~SET USER_BUSW	Writes a data on the digital bus of the USB module. The address is present on USER A0 - USER-A7 signals. The data to write is present on USER-D0 - USER-D7 signals
~READ BUS	Reads the digital bus of the USB module. The address is present on the USER A0- USER-A7 signals. The data to read is present on the USER-D0- USER-D7 signals
~READ OPTO	Reads the status of the opto-coupled inputs on the specified USB module
~SET USER_DAC	Programs the 8 bit DACs
~SET USER_LINE	Closes the eight relays on the system lines
~SET USER_LOAD	Programs the two resistive loads
~SET USER_WORD	Writes a data on the CUSTOM1-15 lines of the USB module. This bus is available in the USB breadboard area for customization on JP18 connector.
~CLEAR USER_BUSW	Clears the DATA and ADDRESS busses of the USB modules (it writes 0X00 on them). The address bus is present on USER A0 -USER-A7 signals. The data bus is present on USER-D0 - USER-D7 signals
~CLEAR USER_DAC	Programs to 0V voltage all DAC of the USB modules.
~CLEAR USER_LINE	Opens all relay on the eight system lines
~CLEAR USER_LOAD	It disconnects the programmable resistive loads from the system lines, if they were connected with the ONLINE instruction in the ~SET USER_LOAD).
~CLEAR USER_WORD	Clears the CUSTOM1-15 lines of the USB module writing a 0x00 data.
~UOP	It drives the opto-coupled transistors
~URP	It closes/opens the double line power relays
~URS	It closes/opens the reed relays

INSTRUCTIONS FOR THE GEN MODULE

~SET USER_GEN	It programs the function generator

~MEAS USER_GEN	It execute an RMS to DC measurement using the GEN module
~CLEAR USER_GEN	It clears the function generator

INSTRUCTIONS FOR THE F40 MODULE

PHASES AND CLOCK

CLOCKF	Defines the reference clock period
PHASE1…9	Defines the commutation instants of the driver channels in the specified phase
STROBE1…9	Defines the strobe instant of the sensor channels in the specified phase
ASSERT1…9	Defines the commutation instant for those channels programmed with the AH1…9, AL1…9 instructions

OUTPUT CHANNEL PROGRAMMING

~ROUT RSn	Connects/disconnects the output resistor between the driver channel and the output on the connector. The resistor is used to isolate the channel from the UUT or to adapt the impedance of the channel transmission line.
~PULLUP/DOWN	Connects/disconnects the 1KΩ pull-up or pull-down resistor the output channel. The pull-up is referred to the high voltage threshold of the channel. The pull-down is referred to the low threshold.
~MUXOUT	Closes/opens the SERANA; LI5 and COAX_R relay of the output channel.

CHANNEL PROGRAMMING

AH1…9	It moves the driver channel to the high level at the ASSERT time
AL1…9	It moves the driver channel to the low level at the ASSERT time
AG1…9	It moves the group of driver channel at the ASSERT time
P1…9	It moves the driver channel according to the instants specified in the phase. The driver channel are checked at the strobe instant
C1…9	It moves the driver channel according to the instants specified in the phase. With C1…9 instructions, the driver channel is NOT controlled, that is, the channel is moved high or low but the system doesn't control if it has reached the voltage level required.
OL0…9	It checks that the sensor channel has moved to the low level.
OH0…9	It checks that the sensor channel has moved to the high level
OG0…9	It checks a group of sensor channels at the strobe instant
CR	It sets the clock in repetitive mode, inside the pattern
NR	It switches off the repetitive mode for the clock signals. After the instruction, the signal behaves according to the specified phase.
RZ1…9	The channels associated to this instruction are driven only between the first phase instant and the second one, after that, they are put in high impedance
R0	Return to 0. If the driver channel is at a low level (IL), it is driven high with a pulse, after that it returns to the low level. If the driver channel is at a high level (IH), it stays low for all pattern length. The pulse time is defined in the specified phase.
R1	Return to 1. If the driver channel is at a high level (IH), it is driven low with a pulse, after that it returns to the high level. If the driver channel is at a low level (IL), it stays high for all pattern length. The pulse time is defined in the specified phase.
RG	It reads the value of the channel group and it assigns it to the specified variable.
WG	It assigns the value (expressed with a variable) to a group of channel.

JUMPS and LOOPS

/ JUMP	It executes a jump to the specified label.
/ REPEAT number	It repeats the current pattern for the times specified in the instruction
BEGINLOOP-ENDLOOP	It repeats the instructions included inside the loop for the times specified.

PATTERN

DELAY	It sets a delay time between the current pattern and the following one
SYNC_ON/OFF	It generates a synchronism signal that can be used by the external instrumentation. The sync signal is generated from the pattern where the instruction is written, on until it is not switched off

INSTRUCTIONS TO PROGRAM THE SE2 MODULE

MEASUREMENTS

~MEAS PW1…8	It executes a voltage or current measurement on the specified power supply. The result of the measure is in the AR register.
~TEST PW1…8	It executes a voltage or current measurement on the specified power supply and it is possible to control the value directly in this instruction. The result of the measure is in the AR register.
~READ STSTUS PW1…8	It executes a status read of the specified power supply. The result of the measure is in the AR register.

INSTRUMENT CONNECTION AND SETTINGS

~SET BUS LINE SERV	It programs the lines connection of the LAM module(from LI1 to LI8) using the SE2 module.
~SET GNDREF	Closes/opens the GND_REF relay present on the SE2 module. This relay is used to refer to a common GND, the power supplies, the instruments and the UUT.
~SET PW1…8	It programs the specified power supply
~SET PW ALL	It switches ON/OFF all power supplies that were previously set with the ~SET PWn instruction and the PREPARE option
~SET EMX INSTRUMENT=LINE	This instruction programs the connection between the external instruments and the Board lines of the SE2 module.
~SET ZTEST	Some precision components (R, C,…) are mounted on the SE2 module. They can be connected on the system lines and measured by the LAM module to execute the system calibration. This instruction programs the connection between the precision components and the system lines

INSTRUCTIONS IN THE FILE.SRG

@CLUSTER	It executes the functional cluster whether it is a run-time or compiled cluster
CLK	Indicates the clock period that will be used to execute the functional cluster
CUR	Indicates the current limit value for the digital channels.
DVH	Defines the voltage value of the driver channel when it is high (IH).
DVL	Defines the voltage value of the driver channel when it is low (IL).
GNO	Connects or disconnect the channel ground to/from the UUT
LI5	Connects one or more digital channel on the UUT closing Line 5 (line for the functional test
NOS=Y/N	Indicates if the test program execution must go on or stop if an error occurred

	during program execution
PRB	Programs the probe for 1 or for 10
THH	Defines the high voltage threshold for the sensor channel (OH).
THL	Defines the low voltage threshold for the sensor channel (OL).
TH2=Y/N	Indicates if the test program uses one or two voltage thresholds for the sensor channels (the thresholds are declared using the DVH,DVL,THH,THL instructions)
TOU	Indicates the maximum timeout for a test program. After this time is expired, the program will end

THE INSTRUCTIONS

this chapter describes all the run-time and compiled functional instructions giving one or more examples of use.

SYNTAX CONVENTIONS

This manual uses the following syntax conventions in the descriptions of the instructions/pattern options and instruments of the functional language:

- The key names of the instructions/pattern options and instruments are indicated in boldface type.

  Example: **~BRANCH** label

- Everything included between square brackets [ ] is optional.

  Example: **~FORMAT** whole numbers, decimals [AUTOCR]

- Three periods (dots) which follow a value, variable, argument etc. indicate that several values, variables or arguments can be repeated with the same characteristics.

  Example: **PHASE1** value1, value2,...

- Three periods (dots) repeated vertically indicate the body of the instruction, which will actually be composed of a series of instructions with a certain meaning.

  Example: **@WHILE** variable1 condition variable2
  ...
  ...
  @ENDWHILE

A sequence of two or three periods repeated vertically within an example indicates that, for brevity, part of the example has been omitted intentionally.

Example:

```
START PRG1;
IL S1;
/;
IH S1;
/;
.
.
.
ENDTEST;
```

- For clarity, upper-case letters are always used in the examples, but both upper case and lower-case letters are accepted when writing programs.

- Files are indicated with the following syntax:

  File.nod
  File.srg ... etc.

where “File” is the generic identifier of the files and “.nod” and “.srg.” are the extensions.

STRUCTURE FOR THE DESCRIPTION OF THE LANGUAGE

The following structure is used for the description of the language:

The environment in which the instruction is active (compiled, runtime or both) is indicated under the title.

Compiled	Run-time
X	**X**

When the instructions can be applied ONLY on determined modules, (SE2, F40, HV) it is indicated as in the following structure:

Module	Compiled	Run-time
F40	**X**	**X**

This is followed by various sections, as explained below.

FUNCTION
Briefly describes what the instruction/pattern option does

SYNTAX
Indicates the syntax to follow when using the instruction/pattern option

DEFAULT VALUES
Indicates the default values of the instructions /options of the pattern/instruments, if present

SEE ALSO
Gives a list, where possible, of other instructions/options pattern/instruments which are related to the one in question

NOTE
Focuses attention on important information for using the instruction/option pattern/instrument

EXAMPLE
Gives a brief example of the instruction/option pattern/instrument. In some cases the example is extracted from a more complete program, and is included for the purpose of illustrating the use and syntax

#INCLUDE **Instruction** **#INCLUDE**

☛

Compiled	Run-time
X	**X**

FUNCTION:

Allows one or more library files File.lib to be included in the functional program File.pat. In this way, the instructions written in the library files will be completely visible in the File.pat. The library file has to be under the board directory. But if the name of the library starts with the three characters S20… (for example S20macro.lib), it can stay under the directory :\S20WIN\BOARD because it is a system library.

SYNTAX:

#INCLUDE file name or
$INCLUDE file name

file name: Name of the file to be included. The maximum length of file name is 8 characters. The file name must be written without the extension.

SEE ALSO:

@COMPILER INCLUDE_ON @COMPILER INCLUDE_OFF

NOTE:

It is possible to include up to 10 different files (also nested) in a File.pat. To include more than one file, write as many #INCLUDE lines as required. It is possible to insert the contents of the library file in the list file File.pac with the compiler directives @COMPILER INCLUDE_ON and @COMPILE INCLUDE_OFF. The compiler directives @COMPILER INCLUDE_ON/OFF must precede the #INCLUDE instruction in order to insert the contents in to the list file File.pac.

EXAMPLE:

```
EXAMPLE 1
#INCLUDE FUNZMAC;      ! Include of the functional
                       macro file
                       FUNZMAC.lib!
DECLARE CHANNEL S1=1;CLOCK=50N;
TIMING
TIM1;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG1;
.
.
ENDTEST;
EXAMPLE 2
```

```
@COMPILER INCLUDE_ON;        !this directive allows the
                             contents of the
                             MACRO.lib file to be
                             included in the list file
                             PRG2.pac!

$INCLUDE MACRO;  !Includes the MACRO.lib file!

DECLARE CHANNEL DAT1=34;
DECLARE CHANNEL DAT2=35;

TIMING TIM2;
PERIOD 5C;
STROBE 4C;
ENDTIMING;

START PGR2;
.
.
ENDTEST;
```

/ Instruction /

Compiled	Run-time
X	**X**

FUNCTION: Defines a step of a functional program. With the / instruction, the stimuli of the pattern are applied through the driver channels and the sensor channels are measured.

SYNTAX: /

NOTE: A program step is considered from the "/" instruction to the preceding one. See the example. The PATTERN instruction can be used in the same way as the "/" instruction, since they are identical.
It is advisable to use the / also as closing of a runtime pattern because with this instruction the error flags are activated.

EXAMPLE:

```
EXAMPLE 1
DECLARE CHANNEL WR=56
DECLARE CHANNEL RD=86
TIMING TIM1;
PERIOD 4C;
STROBE 3C;
ENDTIMING;
START PRG1;
/ TIMING TIM1;
IL WR;
IH RD;
/;          Pattern 1
IH WR;
IL; RD;
/;          Pattern 2
ENDTEST

EXAMPLE 2

DECLARE CHANNEL WR=56;
DECLARE CHANNEL RD=86;
TIMING TIM1;
PERIOD 4C;
STROBE 3C;
ENDTIMING;
START PRG1;
PATTERN TIMING TIM1;
IL WR;
```

```
IH RD;
PATTERN;        Pattern 1
IH WR;
IL; RD;
PATTERN;        Pattern 2
ENDTEST
```

@CALCOLATE Instruction @CALCOLATE

Compiled	Run-time
X	**X**

FUNCTION: Executes logical arithmetic calculations and assignments between constants and variables, storing the result in the destination variable.

SYNTAX: **@CALCOLATE** variable=expression

variable: Destination variable. This can be an INTEGER type variable.

expression: The expression can be a simple assignment:: (@CALCOLATE variable=value) or an arithmetic/logic expression.

The values used in the expression can be:

- INTEGER type variables
- constants

Constants can have the following prefixes:

B	indicates a constant in binary format
D or no character:	indicates a constant in decimal format
H or 0X:	indicates a constant in hexadecimal format
O	indicates a constant in octal format

The following operators can be used in the arithmetic/logic expression:

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Xor
&	And
|	Or
<n	Shift of n bits to the left
>n	Shift of n bits to the right

The priority of the operands is from left to right if no parentheses are used. When using parentheses, the entire expression must be enclosed by parentheses.

NOTE: If operations are carried out between variables and constants of different types and formats, each operand is converted into the type and format of the destination variable. See the **Variables and constants** chapter for the conversion between different types of variables and constants.

No control on overflow is done.

For string type variables and constants, the only valid operand is the sum (+), which is equal to the chain.

EXAMPLE:

```
DECLARE VARIABLE VAR,VAR1;

START PRG1;
/;
@CALCOLATE VAR=1;
@CALCOLATE VAR1=10;
/;
@CALCOLATE VAR=VAR1+15;
@CALCOLATE VAR=(56-5*(2/4)&0X0F|0X50);
/;
ENDTEST;
```

@COMPILER — Compiler Directives — @COMPILER

Compiled	Run-time
X	X

FUNCTION: Specifies the options to be used during compilation of the test program File.pat.

SYNTAX: **@COMPILER** option

option — The option of the @COMPILER command can be:

BACKUP INCLUDE_ON — Includes the contents of the file or files indicated in the #INCLUDE command in the list file File.pac. This directive must be specified before the #INCLUDE instruction.

BACKUP INCLUDE_OFF — Does not include the contents of the file or files indicated in the #INCLUDE command in the list file File.pac. This directive must be specified before the #INCLUDE instruction.

BACKUP LOOP_ON — Expands the loops expressed with the @FOR- @ENDFOR and @WHILE-@ENDWHILE instructions in the list file File.pac. It has no effect on the loops expressed with the instruction BEGINLOOP-ENDLOOP.

BACKUP LOOP_OFF — Expands the loops expressed with the @FOR- @ENDFOR and @WHILE-@ENDWHILE instructions in the list file File.pac. It has no effect on the loops expressed with the instruction BEGINLOOP-ENDLOOP.

BACKUP MACRO_ON — Expands the macros expressed with the instructions @MACRO-@ENDMACRO in the list file File.pac.

BACKUP MACRO_OFF — Does not expand the macros expressed with the instructions @MACRO-@ENDMACRO in the list file File.pac.

BACKUP NUMBER_ON — Numbers the patterns in the list file File.pac.

BACKUP NUMBER_OFF — Does not number the patterns in the list file File.pac.

COMPLEX=num — Defines the maximum number of iterations in the loops @FOR - @ENDFOR. @WHILE - @ENDWHILE. If this limit is exceeded because of an error, the loops end automatically. The maximum definable number is 32767.

MAX_ERROR=num — Changes the maximum number of errors given by the compiler. The maximum definable number is 32767.

DEFAULT VALUES:

@COMPILER BACKUP_INCLUDE_OFF
@COMPILER BACKUP_LOOP_OFF
@COMPILER BACKUP_NUMBER_ON
@COMPILER BACKUP_MACRO_OFF
@COMPILER COMPLEX=4000
@COMPILER MAX_ERROR=100

NOTE: The directives are found at the beginning of the program, before the START instruction, and are active from the point in which they are written, on.

EXAMPLE:

```
EXAMPLE 1
                                        DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
CLOCK=50N;

                                        TIMING CTS1;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

@COMPILER COMPLEX=5000;
@COMPILER LOOP_ON;
@COMPILER INCLUDE_ON;
@COMPILER MAX_ERROR=400;
#INCLUDE FUNZMAC;

                                        START PRG1;
/;
.
.
                                        .
                                        ENDTEST;


                                        EXAMPLE 2
                                        DECLARE CHANNEL S1=1;
                                        DECLARE CHANNEL S2=4;
                                        CLOCK=50N;

                                        TIMING CTS1;
                                        PERIOD 10C;
                                        STROBE 3C;
                                        ENDTIMING;

                                        @COMPILER NUMBER_ON;
                                        @COMPILER MACRO_ON;
                                        @COMPILER MAX_ERROR=400;

                                        START PRG2;
                                        /;
                                        .
                                        .
                                        .
                                        ENDTEST;
```

@FOR-@ENDFOR Instruction @FOR-@ENDFOR

☛

Compiled	Run-time
X	**X**

FUNCTION: Executes the body of the @FOR loop until the variable indicated assumes the specified "final value".

SYNTAX:

@FOR variable=initial value TO final value [STEP step value]
...
...
@ENDFOR

variable — Variable of the @FOR loop. It is an INTEGER type variable.

initial value — The initial value of the variable of the @FOR loop, and can be expressed as:

- a constant
- an INTEGER type variable
- an arithmetic/logic expression, whose operators can be:

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Xor
&	And
|	Or

final value — Expresses the final value of the variable of the @FOR loop. The body of the @FOR loop is repeated until the "var" variable has reached this value. The final value can be expressed as:

- a constant
- an INTEGER type variable

step value — Expresses the step value with which the variable must be incremented or decremented in order to reach the "final value". The step value can be expressed as:

- a constant
- an INTEGER type variable

If an increment of the step value must be expressed, it must be preceded by the character "+" or by nothing. If a decrement must be expressed, the step value must be preceded by the character "-".

DEFAULT VALUES: If no increment or decrement is specified, the step value is equal to 1.

EXAMPLE:

```
DECLARE VARIABLE I,J,K,VOLTE;
DECLARE CHANNEL CLK=2;
DECLARE CHANNEL A0=34;
DECLARE CHANNEL A1=35;
DECLARE CHANNEL A2=36;
DECLARE CHANNEL A3=37;
DECLARE GROUP ADD=A0,A1,A2,A3;

START PRG1;
/;
@FOR I=1 TO=10;
IL CLK;
/;
IH CLK;
/;
@ENDFOR;
.
.
@CALCOLATE K=1;
@CALCOLATE VOLTE =25;
/;
@FOR J=1 TO VOLTE STEP=2;
@CALC K=K*2;
IG ADD=K;
/;
@ENDFOR;
.
.
@CALCOLATE K=10;
@CALCOLATE VOLTE =2;
/;
@FOR I=K TO VOLTE STEP -2;
IL CLK;
/;
IH CLK;
/;
@ENDFOR;
/;
@CALCOLATE I=2;
@CALCOLATE VOLTE =15;
/;
@FOR J=I+1 TO VOLTE;
@CALC K=K*2;
IG ADD=K;
/;
@ENDFOR;
```

@GRAY — Instruction — @GRAY

Compiled	Run-time
X	**X**

FUNCTION: Executes logical arithmetic calculations and assignments between constants and variables, converts the result according to the GRAY code and stores the result in the destination variable.

SYNTAX: **@GRAY** variable=expression

variable — Destination variable. This can be an INTEGER type variable.

expression — The expression can be a simple assignment: (@GRAY variable=value) or an arithmetic/logic expression.

The values used in the expression can be:

- INTEGER type variables
- constants

The following operators can be used in the arithmetic/logic expression:

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Xor
&	And
|	Or
<n	Shift n bits to the left
>n	Shift n bits to the right

The priority of the operands is from left to right if no parentheses are used. If parentheses are used, the entire expression must be included between them.

NOTE: If operations are carried out between variables and constants of different types and formats, each operand is converted into the type and format of the destination variable. See the **Variables and constants** chapter for the conversion between different types of variables and constants.
For string type variables and constants, the only valid operand is the sum (+), which is equal to the chain.

EXAMPLE:

```
DECLARE VARIABLE VAR,VAR1;
START PRG1;
/;
@GRAY VAR=1;
@GRAY VAR1=10;
/;
@GRAY VAR=VAR1+15;
@GRAY VAR=VAR1=56-5*2/4&0X0F|0X50;
/;
ENDTEST;
```

@IF-@ELSE-@ENDIF — Instruction — @IF-@ELSE-@ENDIF

Compiled	Run-time
X	

FUNCTION:

Verifies the condition indicated. If this condition is true, the body of the @IF branch is executed, otherwise the body of the @ELSE branch is executed.

SYNTAX:

@IF variable1 condition constant/variable2
...
@ELSE
...
@ENDIF

variable1

Variable for which the specified condition must be verified. The variable can be INTEGER type.

condition

Specifies the condition to be applied to the variable. The following characters can be used to express the condition:

<	or “lt”	meaning “less than ...”
>	or “gt”	meaning “greater than ...”
=	or “eq”	meaning “equal to ...”
<=	or “le”	meaning “less than or equal to...”
>=	or “ge”	meaning “greater than or equal to ...”
<>	or “ne”	meaning “different from ...”

constant/variable2

Can be:
- a constant
- an INTEGER type variable

SEE ALSO:

@WHILE - @ENDWHILE

EXAMPLE:

```
DECLARE CHANNEL CH1=54;
DECLARE VARIABLE A,B;
START PRG1;
/;
@CALCOLATE A=12;
@CALCOLATE B=20;
/;
@IF A eq B;
IH CH1;
/;
@ELSE;
IL CH1;
/;
@ENDIF;
/;@CALCOLATE A=1;/;
@IF A < 15;
IH CH1;
/;IL CH1;/;
@ENDIF;
/;
ENDTEST;
```

@L Instruction @L

Compiled	Run-time
X	**X**

FUNCTION: Assigns the indicated logic states to the channels declared with the DECLARE PINLIST.

SYNTAX: **@L**(logic states)

logic states

Sequence of logic states to assign to the channels defined in the DECLARE PINLIST instruction.

The correspondence between @L instruction and the DECLARE PINLIST is the following:

DECLARE PINLIST CH1,CH2,...CHn;

@L(S1,S2,.....Sn);

The first state defined in the @L (S1) instruction is associated to the first channel defined in the DECLARE PINLIST (CH1) instruction, state 2 (S2) to channel 2 (CH2),.... state n (Sn) is associated to channel n (CHn).

The logic states can be:

H Executes an unmonitored forcing of the driver to high logic level (equal to an MH channel instruction).

L Executes an unmonitored forcing of the driver channel to low logic level (equal to an ML instruction).

1 Executes a test of the sensor channel at high logic level (equal to an OH instruction).

0 Executes a test of the sensor channel at low logic level (equal to an OL instruction).

h Executes an unmonitored forcing of the channel to high logic level. This condition is not indispensable to obtain valid outputs.

L Executes an unmonitored forcing of the channel to low logic level. This condition is not indispensable to obtain valid outputs.

T Executes the monitored forcing of the driver channel so that it follows phase 1. The channel is measured at the strobe time (equal to a P1 instruction).

. Masks the sensor channel (equal to an OM instruction).

SEE ALSO: DECLARE PINLIST

EXAMPLE:

```
DECLARE PINLIST = 1,2,3,4,5,6,7,8,9,10,11,12,13,14;

START IC1;


@L(HH0...........);
@L(Lh1...........);
@L(hL1...........);
@L(...HH0........);
@L(...Lh1........);
@L(...hL1........);
@L(.......0HH....);
@L(.......1Lh....);
@L(.......1hL....);
@L(..........0HH.);
@L(..........1Lh.);
@L(..........1hL.);


ENDTEST;
```

@LABEL — Instruction — @LABEL

Compiled	Run-time
X	**X**

FUNCTION: Allows the specification of the prefix for different labels within the @FOR, @WHILE loops and in the @MACRO (macros).

SYNTAX: **@LABEL** = variable

variable: Variable, which specifies the prefix of the label. It can be expressed as:
- an INTEGER type variable

SEE ALSO: ~BRANCH

NOTE: The labels must be preceded by the character"@".

The labels are created inserting "_" (under score) character between the prefix and the label name.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE RUNTIME INTEGER J;
DECLARE VARIABLE I;
START PRG1 STATIC;
/;
~CALC J=4;
@FOR I=1 TO 5;
@LABEL=I;          Definition of the label prefix!
~CALC J=J+1;
~IF J=7;           !If J=7 jumps to the label I_EQ7
                   where I assumes a different value at every
                   @FOR loop!
~BRANCH @EQ7;
~ENDIF;
~BRANCH @NOTEQ;        !If J is different from 7, high at
                       label I_NOTEQ where I assumes a
                       different value at each @FOR!
LABEL=@EQ7;
~WRITE "J=7";
~BRANCH @ENDCY;        !Jumps to the  I_ENDCY label
                       where  I assumes a different value
                       at each @FOR loop!
LABEL=@NOTEQ;

/;
~WRITE "J NOT 7";
LABEL=@ENDCY;
~PAUSE J;
@ENDFOR;
/;
ENDTEST;
```

@MACRO-@ENDMACRO Instruction @MACRO-@ENDMACRO

Compiled	Run-time
X	**X**

FUNCTION: Enables the definition of a sequence of instructions under the form of a macro. A macro thus defined can be called in the functional program.

SYNTAX:

@MACRO macro name (arg1, arg2,...)
...
...
...
@ENDMACRO

macro name: Alphanumeric string maximum 20 characters, which indicates the name of the macro.

arg1, arg2,...: Arguments passed in the call to the macro. The can be three types of arguments:

- global
- local
- “define”

Global arguments. These are the ones defined in the DECLARE ARGUMENT. These are global and visible to the entire program and to all the macros.

Local arguments. These are integer type variable not declared. The local arguments are visible only in the macro used and they are not visible to the rest of the program and to the other macros.
It is possible therefore to have calls to macros, which use the same name for the parameters, because they will be differentiated internally in each macro (this operation is completely transparent to the user).

Define type arguments. These are alphanumerical strings, which are simply replaced inside the macro just as they are. They are visible to all the program and to all the other macros. Define type arguments must be preceded by the character “@”. See EXAMPLE 5.

SEE ALSO:
@macro name
@COMPILER BACKUP MACRO_ON
@COMPILER BACKUP MACRO_OFF

NOTE: The macros must be declared before the START section of the test program.

It is possible to call macros from inside a macro up to 10 nesting levels.

Up to 500 macros can be declared in a test program.

To call a macro in the test program, use the @macro name instruction.

EXAMPLE:

EXAMPLE 1

Declaration of a macro and call to a macro inside the program.

```
TIMING TIM3;
PERIOD 10C;
PHASE 3C 7C;
STROBE 9C;
ENDTIMING

@MACRO CLKSU;
IH CLK;
IL CLKN,BRCLK,MN15P6;
/;
@ENDMACRO;

START PRG1;
/;
.
.
.
@CLKSU;
.
.
.
ENDTEST;
```

EXAMPLE 2

Declaration of a macro with passing of global parameters.

```
TIMING TIM1;
PERIOD 10C;
PHASE 3C 7C;
STROBE 9C;
ENDTIMING;

DECLARE ARGUMENT VAHH,VAHL,VALH,
VALL,VDHH,VDHL;

@MACRO RDROM (VAHH,VAHL,VALH,VALL,VDHH,VDHL);
IG ADDHH=VAHH;
IG ADDHL=VAHL;
IG ADDLH=VALH;
IG ADDLL=VALL;
IL FC0,FC2,SIZE0;
IH RW,FC1,SIZE1;
/;
IL ASN;
/;
IL DSN;
/;
```

```
OG DATHH=VDHH;
OG DATHL=VDHL;
/;
@ENDMACRO;

START PRG1;
/;
.
.
.
@RDROM (0X00, 0X01, 0X80, 0X 04, 0X55, 0XAA);
/;
.
.
.
ENDTEST;
```

EXAMPLE 3

Declaration of a macro with passing of local parameters.

```
TIMING TIM2;
PERIOD 10C;
PHASE 3C 7C;
STROBE 9C;
ENDTIMING;

DECLARE VARIABLE I, J,K;

@MACRO SOMMA (J,K);          !The J and K arguments
@CALC I=J+K;                 are local and are seen
ENDMACRO;                    only by the SOMMA
                                macro!

START PRG2;
/;
.
.
@SOMMA (2,3);
/;
.
.
ENDTEST;
```

EXAMPLE 4

Declaration of a macro with passing of local parameters.

```
TIMING TIM3;
PERIOD 10C;
PHASE 3C 7C;
STROBE 9C;
ENDTIMING;

DECLARE VARIABLE I, J,K;

@MACRO SOMMA (J,K);          !The J and K arguments
@CALC I=J+K;                 are local and are seen solo
@ENDMACRO;                   only by the SOMMA
                             macro!
```

```
@MACRO SOTTRAI (J,K);  ! Even though arguments J and K have
@CALC I=J-K;           the same name as those of the SOMMA
@ENDMACRO;             macro, they are different from the
                       preceding ones, are local and are seen
                       only by the SOTTRAI macro!

START PRG3;
/;
.
.
@SOMMA (2,3);
/;
.
.
@SOTTRAI (10,4);
.
.
ENDTEST;
```

EXAMPLE 5

Macro declaration with passage of “define” type parameters. When the described macro is compiled, it will develop the stimuli:

```
TIMING TIM4;
PERIOD 10C;
PHASE 3C 7C;
STROBE 9C;
ENDTIMING;

DECLARE CHANNEL CH1=16;
DECLARE CHANNEL CH2=45;

@MACRO CANALI (@STATO1,@STATO2);
@STATO1 CH1;
@STATO2 CH2;
@ENDMACRO;             !As you can see, the define type
                       arguments must be preceded by the
                       “@” character!

START PRG4;
/;
.
@CANALI (IL,IH);       !This macro develops the IL CH1 and
                       IH CH2 stimuli in this pattern!
/;
@CANALI (IH,IL);       !This macro develops the  IH CH1 and
                       IL CH2 stimuli in this pattern!
/;
ENDTEST;
```

@macro name **Instruction** **@macro name**

Compiled	Run-time
X	**X**

FUNCTION: Executes the specified macro.

SYNTAX: **@**macro name [(parameter1,parameter2,...)]

macro name — Alphanumeric string maximum length 9 characters which identifies the macro to be executed.

parameter1, parameter2,... — Parameters to be passed to the macro.

EXAMPLE:

```
DECLARE CHANNEL D0=29;
DECLARE CHANNEL CK=39;

@MACRO CLOCK;          macro definition
OM D0;
MH CK;
/;
ML CK;
/;
@ENDMACRO;

START PRG1;
.
.
@CLOCK;                Call to the macro
.
.
ENDTEST;
```

@WHILE-@ENDWHILE **Instruction** **@WHILE-@ENDWHILE**

☛

Compiled	Run-time
X	**X**

FUNCTION: Executes the body of the @WHILE loop until the condition is true. When the condition is false the loop ends.

SYNTAX:

@WHILE variable1 condition constant/variable2
...
...
...
@ENDWHILE

variable: Variable for which the specified condition must be verified. The variable can be INTEGER type.

condition: Specifies the condition to apply to the variable. The following characters can be used to express the condition:

<	or "lt"	meaning "less than ..."
>	or "gt"	meaning "greater than ..."
=	or "eq"	meaning "equal to ..."
<=	or "le"	meaning "less than or equal to..."
>=	or "ge"	meaning "greater than or equal to ..."
<>	or "ne"	meaning "different from ..."

constant/variable2: Can be:

- a constant
- an INTEGER type variable

SEE ALSO: @IF - @ELSE - @ENDIF

EXAMPLE:

```
DECLARE VARIABLE J,K;
DECLARE CHANNEL CH2=34;
START PRG1;
/;
@CALCOLATE J=15;
/;
@WHILE J ne 4;
IL CH2;/;IL CH2;
/;
@CALCOLATE J=J-1;
@ENDWHILE;
/;
@CALCOLATE K=10;
@CALCOLATE J=100;
@WHILE J ge K;
IL CH2;/;IH CH2;
/;
@CALCOLATE J=J-1;
@ENDWHILE;
/;
ENDTEST;
```

~ABS — Instruction — ~ABS

Compiled	Run-time
	X

FUNCTION: Calculates the absolute value of the AR register.

SYNTAX: **~ABS**

EXAMPLE:

```
DECLARE RUNTIME FLOAT TMB1K,TEMP44;

START PRG1 STATIC;
/;
~CALC TMB1K=124.59;
/;
~CALC TEMP44=TMB1K-100;
~CALC AR=TEMP44;
~ABS;
~COMP AR LO=-20 HI=20 ONPASS INTOLL;
~COLOR 1,12;
~CURSOR 24,21;
~WRITE "TEMPERATURE OUT OF RANGE”;
TOLLERANZA";
~BRANCH FINE;
/;
LABEL INTOLL;
~WRITE "TEMPERATURE OK";
/;
LABEL=FINE;
/;
ENDTEST;
```

~ADD — Instruction — ~ADD

Compiled	Run-time
	X

FUNCTION: Adds the specified value to the contents of the AR register.

SYNTAX: **~ADD** value

value — Value to be added to the AR register that can be expressed as a:
- constant
- runtime variable of the INTEGER, FLOAT type

SEE ALSO: ~SUB ~MULT ~DIV

EXAMPLE:

```
DECLARE CHANNEL WR=S1;
DECLARE CHANNEL RD=S2;
DECLARE RUNTIME INTEGER V1,V2;
DECLARE RUNTIME FLOAT V3;

START PRG1 STATIC;
/;
~CALC V1=6;
~CALC V2=5;
/;
~LOAD V1;
/;
~ ADD V2;
/;
~ADD 5;
/;
~WRITE “The contents of the AR register is: “,V3;
/;
~PAUSE 1;
/;
ENDTEST;
```

~AFORMAT — Instruction — ~AFORMAT

Compiled	Run-time
	X

FUNCTION:

Defines the format of the constants and the FLOAT and INTEGER type variables during their conversion in STRING type variables. This way it is possible to format strings.

SYNTAX:

~AFORMAT integers, decimals, format

integer — Whole numbers to consider in the INTEGER and FLOAT type variables.

decimals — Decimal number to consider (only in FLOAT type variables).

format — Defines the type of notation for the variables, which can be:

EXP: specifies the exponential notation

DEC: specifies the decimal notation

OCT: specifies the octal notation

HEX: specifies the hexadecimal notation

ASC: converts the number in the equivalent ASCII (41=A) format

DEFAULT VALUES:

If the ~AFORMAT is not specified, by default the number of decimals is 3 while the whole part is considered in its entirety but it is rounded to the fifteenth figure. The notation is decimal.

EXAMPLE:

```
DECLARE RUNTIME FLOAT SCREEN;
DECLARE RUNTIME STRING A;

START PRG1 STATIC;
/;
~FORMAT 10,10,EXP;
~READ_SCR SCREEN;
/;
~AFORMAT 10,1;
~CALC A=SCREEN;        Conversion of the SCREEN float
                       variable in one decimal number
/;
~CURSOR 10,10;
~WRITE A;
~PAUSE;
/;
ENDTEST;
```

~AND **Instruction** **~AND**

Compiled	Run-time
	X

FUNCTION: Calculates the AND between the contents of the AR register and the specified argument.

SYNTAX: **~AND** value

value Can be expressed as a:

- constant
- runtime variable of the INTEGER type

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR1,VAR2;
DECLARE RUNTIME FLOAT VAR;

START PRG1 STATIC;
/;
~CALC VAR1=3;
~LOAD VAR1;
~AND B01101001;
~STORE VAR;
~WRITE "AND di AR : ",VAR;
/;
~PAUSE 1;
/;
~CALC VAR1=0X55;
~CALC VAR2=0X0F;
~LOAD VAR1;
~AND VAR2;
~STORE VAR;
~WRITE "AND di AR : " VAR;
/;
~PAUSE 2;
/;
ENDTEST;
```

~API Instruction ~API

Compiled	Run-time
	X

FUNCTION: Allows the API function (Application Program Interface) present in a DLL (Dynamic Link Library) to be called. The return parameter of the function is stored in the API register.

SYNTAX: **~API** “module name”,”function name”, parameters

module name — Name of the DLL module which contains the API

function name — Name of the API function

parameters — Parameters of the API function

SEE ALSO: ~SYS

NOTE: The return value of the API function is stored in the INTEGER type system variable API.

A “cast” must be executed on the parameters using the following key words:

CHAR, INT, LONG, FLOAT, DOUBLE, UNSIGNED, POINTER

If the “cast” is not executed, the parameters are considered to be long variables.

EXAMPLE:

EXAMPLE 1

Call to the Windows API “Message Box” function

```
START PRG1 STATIC;
/;
~API "User32.dll","MessageBoxA",INT 0,"User Windows", "MY BOX",INT 1;
/;
ENDTEST;
```

EXAMPLE 2

Call to the API function “GetPrivateProfileIntA” for reading from the S20.INI file, section “Installation” and returning the integer value of the variable “Help”

```
START PRG2 STATIC;
```

```
/;
~API "KERNEL32.DLL","GetPrivateProfileIntA","Installation",
"Help", INT 333, "S20.INI";
~CALC API=API&0x0000FFFF;
~COLOR 9,15;
~CURSOR 12,12;
~WRITE API;
~PAUSE 1;/;
ENDTEST;
```

EXAMPLE 3

Call to the API function “GetCursorPos”. Note that the return parameter is loaded in REGL

```
DECLARE RUNTIME INTEGER REGL,MOUSEX,MOUSEY;
START PRG3 STATIC;
/;
~API "USER32.dll","GetCursorPos",POINTER REGL;
~CALC mouseX=REGL&0x0000FFFF;
~CALC mouseY=REGL&0xFFFF0000;
~CALC mouseY=mouseY>16;
~COLOR 5,1;
~CURSOR 10,8;
~WRITE "POSIZIONE CURSORE:";
~COLOR 5,1;
~CURSOR 10,9;
~WRITE mouseX;
~COLOR 5,9;
~CURSOR 10,10;
~WRITE mouseY;
~PAUSE 65;
/;
ENDTEST;
```

~ATAN Instruction ~ATAN

Compiled	Run-time
	X

FUNCTION: Calculates the arc tangent of the contents of the AR register. The AR value must be expressed in radians.

SYNTAX: **~ATAN**

SEE ALSO: ~COS ~SIN

EXAMPLE:

```
DECLARE CHANNEL WR=S1;
DECLARE CHANNEL RD=S2;
DECLARE RUNTIME INTEGER V1;
DECLARE RUNTIME FLOAT TR;

START PRG1 STATIC;
/;
~LOAD 0.785398163;
~ATAN;
/;
~STORE TR;
~WRITE “The value of the arc tangent is: “, TR;
~PAUSE 1;
/;
ENDTEST;
```

~ATEST **Instruction** **~ATEST**

Compiled	Run-time
	X

FUNCTION:

Executes the comparison between the value of the variable and the limits or range specified. The ~ATEST instruction activates the partial error flag and the global error flag if the comparison is outside of the limits or range, therefore it is possible to execute jumps on condition using the ONERROR and ONPASS options. See the section **Partial error flag and global error flag** for more information about the error flags.

Only the lower limit or upper limit can be specified in the ~ATEST instruction. If both limits are defined, it is considered to be a test range.

SYNTAX:

~ATEST variable [**LO=**value] [**HI=**value] [**ONERROR** label] [**ONPASS** label]

variable

Runtime variable to compare that can be of the type INTEGER, FLOAT, STRING

LO=value

Indicates the lower limit and can be expressed as:

- runtime constant of the type INTEGER or FLOAT in the formats: decimal (Example D15), binary (Example B01001), octal (O63), hexadecimal (Example 0X5A).

- runtime variable of the type INTEGER, FLOAT, STRING
- arithmetic expression

If the lower limit is expressed as a string type variable, its value must be written between quotation marks. See the EXAMPLE.

The arithmetic expression can be written with or without parentheses. In the first case, the execution priority of the operators is */ +-. If parentheses are used, the priority can be changed as required.

If only the lower limit is expressed, the comparison activates the partial and global error flags if the value of the variable is less than the value expressed by the lower limit (if it is equal, no error flags are activated).

HI=value

Indicates the upper limit and can be expressed as:

- runtime constant of the type INTEGER or FLOAT
- runtime variable of the type INTEGER, FLOAT, STRING
- arithmetic expression

If the upper limit is expressed as a string type variable, its value must be written between quotation marks. See the EXAMPLE.

The arithmetic expression can be written with or without parentheses. In the first case, the execution priority of the operators is */ +-. If parentheses are used, the priority can be changed as required.

If only the upper limit is expressed, the comparison activates the partial and global error flags if the value of the variable is less than the value expressed by the upper limit (if it is equal, no error flags are activated).

ONERROR label — A jump to the specified label is executed if the comparison is outside of the limits or range and partial and error flags are activated.

ONPASS label — A jump to the specified label is executed if the comparison is within the limits or range.

SEE ALSO: ~COMP

NOTE: The ~ATEST instruction activates the partial and global error flags only if the comparison is out of the specified range or limits. The global error flag remains active until the end of the program. the partial error flag remains active until another ~ATEST, ~CLEAR, ~COMP, ~MEAS, ~SET, ~TEST instruction or a pattern (/) with a positive result (that is, which does not generate an error) deactivates it.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE VARIABLE VAR1;
DECLARE RUNTIME FLOAT RREG;
DECLARE RUNTIME INTEGER RVAR2;
DECLARE RUNTIME STRING PAROLA;
CLOCK=50N;

START TCOMP STATIC;
/;
~CALC RREG=2;
~CALC RVAR2=3;
~CALC PAROLA="CHANNEL1";
IH S1 S2;
/;
IL S1 S2;
/;
~ATEST RREG LO=3 HI=5;        ! ~ATEST instruction which
                              specifies the test range!

/;
~ATEST RREG LO=3;             ! ~ATEST instruction which
                              specifies only the lower limit.
                              If the RREG value is less than
                              3 the partial error flag is
                              activated!

/;
```

```
~ATEST RREG HI=5;                    !~ATEST instruction which specifies only the upper limit. If the RREG value is more than 5 the partial error flag is activated!

/;
~ATEST RREG LO=3 HI=5 ONERROR LAB1;
                                     !If the RREG is out of the specified range, the system jumps to label LAB1!

/;
~ATEST RREG LO=3 HI=5 ONPASS LAB1;
                                     !If the RREG is within the specified range, the system jumps to label LAB1!
/;
~ATEST RREG LO=VAR1 HI=5 ONERROR LAB1;
                                     !The value of the lower limit is specified by a variable!

/;
~ATEST RREG LO=3 HI=RVAR2+5 ONERROR LAB1;
                                     !The value of the upper limit is specified by an expression!
/;
~ATEST PAROLA LO="CHANNEL3" HI="CHANNEL3";
                                     !The variable to compare is a string type!
/;
IH S2;
/;
/;
IH S1;
/;
LABEL LAB1;
~WRITE "Comparison done!";
/;
ENDTEST;
```

~BRANCH — Instruction — ~BRANCH

Compiled	Run-time
	X

FUNCTION: Executes a jump to the specified label. The jump can be conditioned. In this case the partial error flag is checked.

SYNTAX: **~BRANCH** [**ONERROR** label] [**ONPASS** label]

ONERROR label — Executes a jump to the specified label if the partial error flag is active.

ONPASS label — Executes a jump to the specified label if the partial error flag is not active.

DEFAULT VALUES: If no condition is specified, the jump is executed in any case.

NOTE: It is possible to execute a jump with the ~BRANCH instruction within a macro or a @FOR or @WHILE loop writing:

~BRANCH=@label

In this way, the compiler automatically creates a different label every time the same macro or subroutine is called.

~BRANCH ONPASS=@LABEL
~BRANCH ONERROR=@LABEL

See EXAMPLE 2

EXAMPLE: EXAMPLE 1

```
DECLARE RUNTIME INTEGER SUBERR;
DECLARE VARIABLE J;.

START TBRAN STATIC;
/;
.
.
~BRANCH TEST1;          !The jump to test 1 is executed in any case!
.
.
.
```

```
LABEL=TEST1;
.
.
.
~BRANCH ONPASS LAB2;        !The jump is executed if the partial error flag is not active!

~CALC SUBERR=SUBERR+1;
LABEL=LAB2;
.
.
.
~BRANCH ONERROR T2L2;       !The jump is executed if the partial error flag is active!

@FOR J= 0 TO 2;
~WRITE “MAIN”;
@ENDFOR;
LABEL=LAB2;
.
.
.
ENDTEST;
```

EXAMPLE 2

```
DECLARE VARIABLE J;

*Jump within the macro MESS*
@MACRO MESS VOLTE;
@FOR J=3 TO VOLTE;
@LABEL=j;
~BRANCH=@MAC;
~WRITE "MESSAGGIO IN PRINCIPALE";
label=@MAC;
@ENDFOR;
@ENDMACRO;
.
.
*Jump within the macro MESS1*
@MACRO MESS1 VOLTE;
@FOR J=3 TO VOLTE;
@LABEL=j;
~BRANCH ONPASS=@MAC1;
~WRITE "MESSAGGIO IN PRINCIPALE";
label=@MAC1;
@ENDFOR;
@ENDMACRO;
/;
*Jump within the macro MESS2*
@MACRO MESS2 VOLTE;
@FOR J=3 TO VOLTE;
@LABEL=j;
~BRANCH ONERROR=@MAC2;
~WRITE "MESSAGGIO IN PRINCIPALE";
label=@MAC2;
@ENDFOR;
```

```
@ENDMACRO;
.
.
*Call to a macro*
@MESS 7;
/;
* Call to a macro *
@MESS2 2;
/;
* Call to a macro *
@MESS1 3;
/;
* Call to a macro *
@MESS1 9;
/;
* Call to a macro *
@MESS2 6;
.
.
.
ENDTEST;
```

~CALC Instruction ~CALC

Compiled	Run-time
	X

FUNCTION: Executes logical arithmetic calculations and assignments between constants and variables storing the result in the destination variable.

SYNTAX: **~CALC** variable=expression

variable: Destination variable. It can be a INTEGER, FLOAT or STRING type variable.

expression: The expression can be a simple assignment (~CALC variable=value) or an arithmetic/logic expression:

The values used in the expression can be:

- runtime variables of the type INTEGER, FLOAT, STRING
- constants of the type INTEGER, FLOAT, STRING

The operators, which can be used in the arithmetic/logic expression, are:

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Xor
&	And
|	Or
<n	Shift n bits to the left
>n	Shift n bits to the right

The priority between the operands is from left to right if no parentheses are used. If parentheses are used, the entire expression must be enclosed between them.

NOTE: If operations are performed between variables and constants of different types and formats, each operand is converted into the type and format of the destination variable. See the chapter **Variables and constants** for the conversion of variables and constants of different types.

For string type variables and constants, the only valid operand is addition (+), which is the same as chaining.

EXAMPLE:

```
DECLARE RUNTIME INTEGER INT1;
DECLARE RUNTIME FLOAT FLT1;
DECLARE RUNTIME STRING STR1;

START PGR1 STATIC;
/;
~CALC INT1=478;
~CALC FLT1=23986.4713;
~CALC STR1="IL VALUE DI TENSIONE E':  ";
/;
~CALC INT1=B010110;
~CALC INT1=D76;
~CALC INT1= -7474;
~CALC INT1=O7156;
~CALC INT1= -0XFE4;
~CALC INT1=HA7F9;
/;
~CALC INT1=INT1+5/2*4^2-8;
~WRITE INT1;
~PAUSE 1;
/;
~CALC INT1=INT1&0XF0;
~CURSOR 1,2;
~WRITE INT1;
~PAUSE 2;
/;
~CALC INT1=INT1<4;
~CURSOR 1,3;
~WRITE INT1;
~PAUSE 3;
/;
~CALC INT1=INT1|B110001;
~CURSOR 1,4;
~WRITE INT1;
~PAUSE 3;
/;

!in the following ~CALC instruction, all of the operands are first converted into integer variables in decimal format, and then the operation is executed.!

~CALC INT1=B01011+D7586+O1721+0XFA2+15.32;

~CURSOR 1,5;
~WRITE INT1;
~PAUSE 4;
~CALC STR1="IL "+"VALUE E': ";
~CURSOR 1,6;
~WRITE STR1;
~PAUSE 5;
/;
ENDTEST;
```

~CALL Instruction ~CALL

Compiled	Run-time
	X

FUNCTION:

Executes a call to a subroutine. The call can be conditioned by the ONERROR and ONPASS options, which carry out a check on the partial error flag.

SYNTAX:

~CALL subroutine name [(parameter1,parameter2,...)] [**ONERROR**] [**ONPASS**]

subroutine name

Alphanumeric string maximum 9 characters which identifies the subroutine.

parameter1,parameter2,...

Parameter/parameters to pass to the subroutine; these can be expressed as:

- runtime constant of the type INTEGER, FLOAT or STRING

- runtime variable of the type INTEGER, FLOAT or STRING

EXAMPLE:

```
START PRG1 STATIC;
~START_SUB;

~SUB1 (INTEGER DAT,ADR);
~WG DATAB=DAT;
~WG ADB=ADR;
~RETURN;

~SUB2 (FLOAT PAR1,PAR2);
~CALC RIS=PAR1+PAR2);
~WRITE RIS;
~RETURN;

~SUB3 (STRING COMP, PAR);
~WRITE COMP “, ” PAR;
~RETURN;

~SUB4 (FLOAT OP1,OP2);
~CALC RIS=OP1+OP2;
~WRITE “RISULTATO = ”,RIS;
~RETURN;

~END_SUB;
.
~CALL SUB1 (0X55,0XAA);
~CALL SUB2 (12.5,14.7);
~CALL SUB3 (RESISTENZA, R12) ONERROR;
~CALC VAR1=12.67;
~CALC VAR2=VAR1+45.7;
/;
~CALL SUB4 (VAR1,VAR2);
/;
ENDTEST;
```

~CLEAR — Instruction — ~CLEAR

Compiled	Run-time
	X

FUNCTION: Executes the reset of the specified instrument.

SYNTAX: **~CLEAR** instrument [**ALL**]

instrument — This can be:

COUNTER	Pulse counter
CKGEN	Clock generator
DR1	Voltage/current generator
DR2	Voltage generator
GUA	Guarding generator
GND	Ground
POWER	Power supply
PUL	Resistor matrix
PWn	Power supply
VOLT	Voltage meter
PMM	Voltage/current meter of the High Voltage e High Voltage Lite module

ALL Option used to execute the reset of all the system instrumentation and switches the relay of the LAM module in default position, resets the USB module, disconnects possible channels connected to the analog and digital lines.

SEE ALSO: ~SET ~TEST ~MEAS

EXAMPLE:

```
~CLEAR PUL;
~CLEAR DR2;
~CLEAR GND;
~CLEAR ALL;
```

~CLEAR Esn **Instruction** **~CLEAR ESn**

☛

Module	Compiled	Run-time
HV		**X**

FUNCTION: Unlinks a resource from any internal line (PL1, PL2, PL3, PL4) of the High Voltage/High Voltage Lite module.

SYNTAX: **~CLEAR ES**n

n indicates the number of the resource to disconnect. It can be a number variable from 1 to 12 for the High Voltage module and a number from 1 to 4 for the High Voltage Lite module.

EXAMPLE: ~CLEAR ES1

~CLEAR ISORES N= **Instruction** **~CLEAR ISORES N=**

☛

Module	Compiled	Run-time
HV		**X**

FUNCTION: Resets the resistive load in the High Voltage module.
Since the resistive load is not physically connected, during reset, its value is simply programmed to zero.

SYNTAX: **~CLEAR ISORES** [**N**= resistive load]

[N= resistive load] Whole number that can be 1 or 2, which indicates the resistive load to program.

SEE ALSO: ~SET ISORES

EXAMPLE: ~SET ISORES N=1 OHM=10500 ON;

~CLEAR LGC — Instruction — ~CLEAR LGC

Compiled	Run-time
	X

FUNCTION: Executes a reset of the functional and analog channels. When this instruction is executed, the functional channels are put to low sensor and are masked. The analog channels are disconnected.

SYNTAX: **~CLEAR LGC**

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE CHANNEL S3=3;

START PRG1 STATIC;

~IL S1,S2;
/;
~IH S1,S2;
/;
~LI1=S3;
/;
~CLEAR LGC;          !The channels are put at low sensor and
                     masked!
/;
~OH S1;
/;
~OH S2;
/;
~OM S1,S2;
/;
ENDTEST;
```

~CLEAR USER_BUSW **Instruction** **~CLEAR USER_BUSW**

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Programs a writing at the address 0X00 of the 0X00 data on the digital busses of all the USB modules of the system. The address will be present on the lines USER A0- USER-A7 while the data on the lines USER-D0- USER-D7.

SYNTAX: **~CLEAR USER_BUSW**

SEE ALSO: ~SET USER_BUSWn

NOTE: The clear acts on all digital busses of the USB modules.

EXAMPLE: ~CLEAR USER_BUSW; !Executes the clear of the digital busses!

~CLEAR USER_DAC **Instruction** **~CLEAR USER_DAC**

Module	Compiled	Run-time
USB		**X**

FUNCTION: Programs at a voltage of 0V all of the DAC of the USB modules present on the test apparatus.

SYNTAX: **~CLEAR USER_DAC**

SEE ALSO: ~SET USER_DAC

NOTE: The clear acts on all the DAC of the USB modules present on the test apparatus.

EXAMPLE: ~CLEAR USER_DAC !Programs a 0V voltage on all the DAC present on all the USB modules!

~CLEAR USER_GEN **Instruction** **~CLEAR USER_GEN**

☛

Module	Compiled	Run-time
GEN		**X**

FUNCTION: Reset and disconnects the GEN module from the lines.

SYNTAX: **~CLEAR USER_GEN**

EXAMPLE: ~CLEAR USER_GEN

~CLEAR USER_LINE **Instruction** **~CLEAR USER_LINE**

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Opens all the relays of the USB module interfaced on the eight lines of the system.

SYNTAX: **~CLEAR USER_LINE**

SEE ALSO: ~SET USER_LINEn

NOTE: The clear acts on all the relays of the lines.

EXAMPLE:

```
~CLEAR USER_LINE      !Opens all the relays of the USB module
                      or modules interfaced on the lines of
                      the system!
```

~CLEAR USER_LOAD **Instruction** **~CLEAR USER_LOAD**

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Disconnects the programmable resistive loads from the lines of the system if they are connected (with ONLINE option of the ~SET USER_LOAD instruction).

SYNTAX: **~CLEAR USER_LOAD**

SEE ALSO: ~SET USER_LOAD

NOTE: The clear acts on all the programmable resistive loads present on the test apparatus.

EXAMPLE: ~CLEAR USER_LOAD !Disconnects the programmable resistive loads present on all the USB modules from the system's lines!

~CLEAR USER_WORD **Instruction** **~CLEAR USER_WORD**

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Writes a data equal to 0X00 on CUSTOM1-15 lines of all USB modules present in the test system.

SINTAX: **~CLEAR USER_WORD**

SEE ALSO: ~SET USER_WORDn

NOTE: This instruction has effect on all CUSTOM1-15 lines of all USB modules.

EXAMPLE: ~CLEAR USER_WORD; Clears all CUSTOM1-15 lines.

~COLOR Instruction ~COLOR

Compiled	Run-time
	X

FUNCTION: Specifies the foreground and background colors for the next output operations executed by the ~WRITE instruction.

SYNTAX: **~COLOR** background, character

background — Defines the background color and is a number 0 to 15. The number can also be expressed with a FLOAT type runtime variable. Refer to the following table:

COLORS TABLE

0	Black
1	Dark blue
2	Dark green
3	Blue/gray
4	Dark red
5	Violet
6	Ochre
7	Light gray
8	Dark gray
9	Light blue
10	Light green
11	Sky blue
12	Light red
13	Fuchsia
14	Yellow
15	White

character — Defines the character color and is a number 0 to 15. The number can also be expressed with a FLOAT type runtime variable. Refer to the table of colors above.

DEFAULT VALUES: If the ~COLOR instruction is not specified, the character color is white and the background is black.

SEE ALSO: ~CURSOR

NOTE: The ~COLOR command can be inserted in the ~WRITE instruction using this syntax:

~WRITE "\CO\character\background", value1, value2

where the character and the background are expressed with 3 characters each.

See the EXAMPLE.

EXAMPLE:

```
DECLARE RUNTIME FLOAT J,K;
DECLARE RUNTIME STRING R1;

START PRG1 STATIC;
/;
LABEL=OK;
~CURSOR 65,4;
~COLOR 1,2;
~WRITE “PASS”;
/;
~BRANCH FINE;
/;
LABEL=NOK;
~CURSOR 65,4;
~COLOR 1,5
~WRITE “FAIL”;
/;
LABEL=FINE;
/;
~CALC RES=“R2”;
/;
~WRITE “\CO\005\002\”, “Resistor ”, RES;
/;
ENDTEST;
```

~COMP Instruction ~COMP

Compiled	Run-time
	X

FUNCTION:

Executes the comparison between the value of a variable and the specified limit or range. The ~COMP instruction activates only the partial error flag if the comparison is out of the limits or out of the range. Therefore it is possible to execute a conditioned jump using the ONERROR and ONPASS options, and to execute different lines of the program without having an error condition. Refer to the chapter **Partial error flag and global error flag** for more information about the error flags.
In the ~COMP instruction it is possible to specify only the lower limit or only the upper limit. If both limits are defined, it is considered a test range.

SYNTAX:

~COMP variable [**LO=**value] [**HI=**value] [**ONERROR** label] [**ONPASS** label]

variable

Runtime variable to compare, which can be INTEGER, FLOAT or STRING type

LO=value

Indicates the lower limit, which can be expressed as:

- runtime constant INTEGER or FLOAT type in the following formats: decimal (Example D15), binary (Example B01001), octal (O63), hexadecimal (Example 0X5A).
- runtime variable INTEGER, FLOAT, STRING type
- arithmetic expression

If the lower limit is expressed as a string type variable, its value must be written between quotation marks. Refer to the EXAMPLE.

The arithmetic expression can be written with or without parentheses. In the first case the priority of the operators is */ +-. Using parentheses, the priority can be changed as required.

If only the lower limit is expressed in the syntax, the comparison activates the partial error flag if the value of the variable is less than the value expressed by the lower limit. (if it is equal, no flags are activated).

HI=value

Indicates the upper limit and can be expressed as :

- runtime constant INTEGER or FLOAT type
- runtime variable INTEGER, FLOAT, STRING type
- arithmetic expression

If the upper limit is expressed as a string type variable, its value must be written between quotation marks. Refer to the EXAMPLE.

The arithmetic expression can be written with or without parentheses. In the first case the priority of the operators is */ +-. Using parentheses, the priority can be changed as required.

If only the upper limit is expressed in the syntax, the comparison activates the partial error flag if the value of the variable is less than the value expressed by the upper limit. (if it is equal, no flags are activated).

ONERROR label — A jump to the specified label is executed if the comparison is out of the limits or range and the partial error flag is activated.

ONPASS label — A jump to the specified label is executed if the comparison is within the limits or range.

SEE ALSO: ~ATEST

NOTE: The ~COMP instruction activates the partial error flag only if the comparison is out of the range or limits specified. The partial error flag remains active until the execution of another ~ATEST, ~CLEAR, ~COMP, ~MEAS, ~SET, ~TEST or pattern (/) with a positive result (that is, which does not generate an error) deactivates it.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE VARIABLE VAR1;
DECLARE RUNTIME FLOAT RREG;
DECLARE RUNTIME INTEGER RVAR2;
DECLARE RUNTIME STRING PAROLA;
CLOCK=50N;

START TCOMP STATIC;
/;
~CALC RREG=2;
~CALC RVAR2=3;
~CALC PAROLA="CHANNEL1";
IH S1 S2;
/;
IL S1 S2;
/;

~COMP RREG LO=3 HI=5;  !Instruction ~COMP
                        specifying the test range!
/;
~COMP RREG LO=3;       ! Instruction ~COMP specifying only
                        the lower limit. If the RREG value is
                        smaller than 3 the partial error flag is
                        activated!

/;
  ~COMP RREG HI=5;     ! Instruction ~COMP specifying only
                        the upper limit. If the RREG value is
                        bigger than 5 the partial error flag is
                        activated!
```

```
/;
~COMP RREG LO=3 HI=5 ONERROR LAB1;
                    !If the RREG is out of the
                    specified range, the
                    system jumps to label LAB1!

/;
~COMP RREG LO=3 HI=5 ONPASS LAB1;
                         !If the RREG is within the
                         specified range, the system
                          jumps to label LAB1!

/;
~COMP RREG LO=VAR1 HI=5 ONERROR LAB1;
                    !The value of the lower
                    limit is specified by a
                    variable!
/;
~COMP RREG LO=3 HI=RVAR2+5 ONERROR LAB1;
                    !The value of the upper
                    limit is specified by an
                    expression!
/;
~COMP PAROLA LO="CHANNEL3" HI="CHANNEL3";
                         !The variable to compare
                         is a string type!
/;
IH S2;
/;
/;
IH S1;
/;
LABEL LAB1;
~WRITE "Comparison done !";
/;
ENDTEST;
```

~COS **Instruction** **~COS**

Compiled	Run-time
	X

FUNCTION: Calculates the cosine of the contents of the AR register. The AR register must be expressed in radians.

SYNTAX: **~COS**

SEE ALSO: ~SIN ~ATAN

EXAMPLE:

```
DECLARE CHANNEL WR=S1;
DECLARE CHANNEL RD=S2;
DECLARE RUNTIME INTEGER TR;
START PRG1 STATIC;
/;
~LOAD 1.570796327;
~ COS;
/;
~STORE TR;
~WRITE “The value of the cosine is: “,TR;
/;
ENDTEST;
```

~CURSOR Instruction ~CURSOR

Compiled	Run-time
	X

FUNCTION: Specifies the position in terms of line and column of the next input/output operation executed with the ~WRITE or ~READ_SCR instruction.

SYNTAX: **~CURSOR** column, line [attribute1, attribute2,...]

column: Specifies the column of the ~CURSOR instruction and is a number from 1 to 80 or a value within a register (REG1-100)

line: Specifies the column of the ~CURSOR instruction and is a number from 1 to 25 or a value within the register (REG1-100)

attribute n: Attribute or attributes for the next output operation. This can be:

CR	Carriage return	New line
EL	Erase line	erases from the cursor to the end of the line.
IC	Insert character	inserts the character to the right of the cursor.
IL	Insert line	inserts a line above the cursor.
CS	Clear screen	erases the contents of the Static Terminal window.
RI	Right	moves the cursor one position to the right
DO	Down	moves the cursor one line down.
HO	Home	moves the cursor to line 1 column 1.
EP	Erase page	erases from the cursor to the end of the page.
DC	Delete character	erases the character to the left of the cursor.
DL	Delete line	deletes the line of the cursor.
LE	Left	moves the cursor one character to the left.
UP	Up	moves the cursor one line up.

SEE ALSO: ~COLOR

NOTE: The cursor command can be inserted into the ~WRITE instruction using this syntax:

~WRITE “\CU\column\line”, value1,value2
where the column and the line are expressed with 3 characters each.
See the EXAMPLE 1.

It is possible to use the registers REG(1-100) to specify the values of the lines and of the columns. This way the value can change according to an index and it is possible to create simple graphs. See the EXAMPLE 2.

EXAMPLE:

EXAMPLE 1

```
DECLARE RUNTIME FLOAT TEMP,TEMP1,TEMP2;
START PRG1;
/;
~CALC TEMP1=100;
~CALC TEMP2= 3.3;
/;
~CALC TEMP=TEMP1/TEMP2*1000;
/;
~CURSOR 10,5;
~WRITE “channel impedance on positive voltage = ”;
~WRITE TEMP;
/;
~WRITE “\CU\001\025\” “CHANNEL NR.” CAN;   !moves the cursor on the column1, line 25 and writes “CHANNEL NR. ”!
~WRITE “\CU\002\030\” CAN1;
~WRITE “\CU\001\025\” CAN “CHANNEL NR.” CAN1;
~WRITE “\CR “FINE”;        !New line, and writes FINE!
/;
ENDTEST;
```

EXAMPLE2
Creation of a graph on video

```
DECLARE RUNTIME FLOAT REG2;
```

When the registers REG1-100 are defined it is also possible to declare only the biggest, in this case REG2, the others will be automatically declared.

```
DECLARE RUNTIME INTEGER I;
START PROFUN1 STATIC;
~CALC REG1=1;
~FOR I=REG1 TO 10;
~MEAS VOLT LI1 MAX=2 TIM=1;
~CALC REG2=1;
~CURSOR (REG1, REG2);
~WRITE ".";
~CALC REG1=REG1+1;
~ENDFOR;
~PAUSE;
ENDTEST;
```

~DELAY — Instruction — ~DELAY

Compiled	Run-time
	X

FUNCTION: Executes the specified delay.

SYNTAX: **~DELAY** value

value: Value, which indicates the delay time, which can be expressed as:

- time (1µs, 2µs,...)
- FLOAT type runtime variable

To express the value as time the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

If no letter is specified, the measuring unit is “seconds”. Example ~DELAY 100: delay of 100 seconds.

SEE ALSO: ~PAUSE

NOTE: The minimum delay time that can be defined with the ~DELAY instruction is 1ms.

EXAMPLE:

EXAMPLE 1

```
DECLARE RUNTIME INTEGER NCAN, SUBERR, MAXERR;
START PRG1 STATIC;
/;
~DO NCAN;
~CURSOR 60,3;
~WRITE "CHANNEL N. " CAN;
~CALC CAN=CAN+1;
~COMP SUBERR HI=MAXERR ONERROR FINE;
~DELAY 1M;
~DONE;
LABEL=FINE;
/;
ENDTEST;
```

EXAMPLE 2

```
DECLARE RUNTIME INTEGER T1;
START PRG2 STATIC;
/;
~CALC T1=10;
~DELAY T1;
/;
ENDTEST;
```

~DIV — Instruction — ~DIV

Compiled	Run-time
	X

FUNCTION: Divides the contents of the AR register (dividend) by the specified value (divisor).

SYNTAX: **~DIV** value

value — Divisor of the AR register. This can be specified as:

- a constant of the type INTEGER
  FLOAT

- a runtime variable of the type INTEGER, FLOAT

SEE ALSO: ~ADD ~SUB ~MULT

EXAMPLE:

```
DECLARE RUNTIME INTEGER V1;
DECLARE RUNTIME FLOAT RIS;

START PRG1 STATIC;
/;
~LOAD 4.194;
/;
~CALC V1=6;
/;
~DIV V1;
~STORE RIS;
/;
~WRITE "The value of the AR register is: ", RIS;
~DIV 5;
~STORE RIS;
~WRITE "The value of the AR register is: ", RIS;
/;
ENDTEST;
```

~DO ~DONE — Instruction — ~DO ~DONE

Compiled	Run-time
	X

FUNCTION: Executes the operations indicated in the ~do ~done cycles the number of times indicated by “num”.

SYNTAX:

~DO num
...
...
...
~DONE

num — Number which indicates how many times the operations defined in the ~do ~done cycle must be executed.

SEE ALSO: ~FOR - ~ENDFOR

EXAMPLE:

EXAMPLE 1

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;

START PRG1 STATIC;
/;
IH S1 S2;
/;
IL S1 S2;
/;
~DO 80;
~WRITE "--";
~DONE;
/;
.
.
ENDTEST;
```

EXAMPLE 2

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE CHANNEL S3=3;
DECLARE CHANNEL S4=4;
DECLARE VARIABLE VAR1;
DECLARE RUNTIME FLOAT NCAN;
DECLARE RUNTIME INTEGER
PRICAN,ULTCAN,SUBERR,MAXERR;
```

```
START PGR1 STATIC;
/;
IH S1 S2;
/;
IL S1 S2;
/;
~CALC NCAN=4;
~CALC CAN=1;
~CALC SUBERR=23;
/;
~CALC MAXERR=4;
/;
~DO NCAN;
~CURSOR 60,3;
~WRITE "CHANNEL N. " CAN;
~CALC CAN=CAN+1;
~MEAS VOLT=LI1,MAX=12,TIM=1;
~BRANCH ONERROR FINE;
~DONE;
/;
IH S2;
/;
IH S1;
/;
LABEL=FINE;
/;
ENDTEST;
```

~EXP Instruction ~EXP

Compiled	Run-time
	X

FUNCTION: Calculates the e^x value where X is the value contained in the AR register.

SYNTAX: **~EXP**

SEE ALSO: ~LOG

EXAMPLE:

```
DECLARE RUNTIME FLOAT VAR;

START PRG1 STATIC;
/;
~LOAD 2;
~EXP;
~STORE VAR;
~WRITE "e^x= " VAR;
/;
~PAUSE 1;
/;
ENDTEST;
```

~FILEn CLOSE **Instruction** **~FILEn CLOSE**

☛

Compiled	Run-time
	X

FUNCTION: Closes the file opened previously with the ~FILEn OPEN instruction. At the end of read or write operations, the file has to be closet with this instruction.

SYNTAX: **~FILEn CLOSE;**

n Whole number from 1 to 7 indicating the file to be closed.

SEE ALSO: ~FILEn OPEN_READ, ~FILEn OPEN_WRITE

EXAMPLE: Assuming that we are working on the ATD23 board:

START ATD23R STATIC;	Start of the program.
~FILE1 OPEN_WRITE “Risultati.txt”	Opening in writing of the Risultati.txt file under the directory .\S20WIN\BOARD\ATD23.
~FILE1 WRITE “PASS”;	Writing of the string PASS on the first line of the file.
~FILE1 CLOSE;	Closing of the file.
ENDTEST;	End of the program.

~FILEn OPEN_APPEND **Instruction** **~FILEn OPEN_APPEND**

☛

Compiled	Run-time
	X

FUNCTION:

Opens the file in writing mode and it adds new data at the end of the ones already present (APPEND operation). If the instruction can't open the file (because it doesn't exist or is protected from writing), the PARTIAL error flag is activated.

SYNTAX:

~FILEn OPEN_APPEND "file name"

n

Whole number from 1 to 7 that indicates the open file.

File name

Name and extension of the file. Example: Dati.txt. If there isn't a path in the file name, the file is created in the directory of the board, example: (:\S20WIN\BOARD\Piastra1). Otherwise, if the path is specified in the file name, the file is created in the directory specified. Example: "D:\Dati\Lettura.txt". The file Lettura.txt is created under the directory D\Dati.

SEE ALSO:

~FILEn OPEN_READ, ~FILEn OPEN_WRITE,

EXAMPLE:

Suppose to work on the ATD23 board:

START ATD23R STATIC;	Start of the program
~FILE1 OPEN_WRITE "Risultati.txt"	Opening in writing of the Risultati.txt file under the directory .\S20WIN\BOARD\ATD23.
~FILE1 WRITE "PASS";	Writing of the string PASS on the first line of the file.
~FILE1 CLOSE;	Closing of the file.
~FILE1 OPEN_APPEND "Risultati.txt"	Opening of the file in APPEND mode.
~FILE1 WRITE "FAIL";	Writing of the string FAIL, at the end of the file
~FILE1 CLOSE;	Closing of the file.
ENDTEST;	End of the program

The Risultati.txt file will contain the following values:
PASS
FAIL

~FILEn OPEN_READ — Instruction — ~FILEn OPEN_READ

☛

Compiled	Run-time
	X

FUNCTION:

Opens the specified file, in read mode. If the instruction can't open the file (because it doesn't exist), the PARTIAL error flag is activated.

SYNTAX:

~FILEn OPEN_READ "file name"

n

Whole number from 1 to 7 that indicates the open file.

file name

Name and extension of the file. Example: Dati.txt. If there isn't a path in the file name, the file is created in the directory of the board, example: (:\S20WIN\BOARD\Piastra1). Otherwise, if the path is specified in the file name, the file is created in the directory specified. Example: "D:\Dati\Lettura.txt". The Lettura.txt file is opened under D\Dati directory. Obviously the file has to be already previously created with the ~FILEn OPEN_WRITE instruction, to open it in read mode. If previously a file was created with associated the number 1 (~FILE1…..), to open the same file the syntax ~FILE1 OPEN_READ has to be used, that means that the file numbers must be the same.

SEE ALSO:

~FILEn OPEN_WRITE

EXAMPLE:

Suppose to work on the ATD23 board:

~FILE1 OPEN_READ "Risultati.txt" Opens in read mode the Risultati.txt file under the directory .\S20WIN\BOARD\ATD23.

~FILE2 OPEN_WRITE "C:\ARCHIVIO\Risultati.txt" Opens in read mode the Risultati.txt file under the directory C:\ARCHIVIO.

~FILEn OPEN_WRITE **Instruction** **~FILEn OPEN_WRITE**

☛

Compiled	Run-time
	X

FUNCTION:

Opens the specified file in reading mode. The file is an ASCII format and can be under the board directory or in a specific directory. Subsequently to write in the file use ~FILEn WRITE and ~FILEn OPEN_APPEND instructions. If the instruction can't write on the file (because it does not exist or protected from writing), the PARTIAL error flag is activated.

SYNTAX:

~FILEn OPEN_WRITE "file name"

n

Whole number from 1 to 7 that indicates the open file.

File name

Name and extension of the file. Example: Dati.txt If there isn't a path in the file name, the file is created in the directory of the board, example: (:\S20WIN\BOARD\Piastra1). Otherwise, if the path is specified in the file name, the file is created in the directory specified. Example: "D:\Dati\Lettura.txt". The Lettura.txt file is created under the D\Dati directory.

SEE ALSO:

~FILEn OPEN_READ, ~FILEn OPEN_APPEND,

EXAMPLE:

Assuming that we are working on the ATD23 board:

~FILE1 OPEN_WRITE "Risultati.txt" creates the file Risultati.txt under the directory .\S20WIN\BOARD\ATD23. The file is empty at the moment.

~FILE2 OPEN_WRITE "C:\ARCHIVIO\Risultati.txt" Creates the file Risultati.txt under the directory C:\ARCHIVIO. The file is empty at the moment.

~FILEn READ **Instruction** **~FILEn READ**

Compiled	Run-time
	X

FUNCTION:

Reads a data within the file opened previously in reading mode with the instruction ~FILEn OPEN_READ. The data read from the file is stored in the specified variable. If you try to read a data besides the file, the PARTIAL error flag is activated.

SYNTAX:

~FILEn READ variable

n

Whole number from 1 to 7 that indicates the open file.

variable

It can be a variable of the type INTEGER or FLOAT or STRING, previously defined with the instruction DECLARE RUNTIME INTEGER/FLOAT/STRING, which contains the value, read from the file.

SEE ALSO:

~FILEn OPEN_READ, ~FILEn OPEN_APPEND,

NOTE:

The first read operation, reads the contents of the first line of the file. If other readings are done, the data will be read from the following lines. If the file is closed and opened again, the first reading will be of the data on the first line again.

EXAMPLE:

EXAMPLE

Assuming that we are working on the ATD23 board and to have a file Risultati.txt under this directory with the following values:
23
26

DECLARE RUNTIME INTEGER VAL;	Declaration of the integer type variable VAL
START ATD23R STATIC;	Beginning of the program
~FILE1 OPEN_READ “Risultati.txt”	Opening in reading mode of the file Risultati.txt under the directory .\S20WIN\BOARD\ATD23.
~FILE1 READ VAL;	Reading of the first line of the file, the

	value is stored in the variable VAL
~CURSOR 10,10;	Positioning of the cursor
~WRITE “VALUE DEL FILE: ” VAL;	Display of the first value of the file (23)
~FILE1 READ VAL;	Reading of the second value of the file
~CURSOR 20,10;	Positioning of the cursor
~WRITE “VALUE DEL FILE: ” VAL;	Display of the second value of the file (26)
~FILE1 CLOSE;	Closing of the file.
ENDTEST;	End of the program.

~FILEn REMOVE **Instruction** **~FILEn REMOVE**

☛

Compiled	Run-time
	X

FUNCTION: Cancels the file specified (identified by the number indicated in the parameter n).

SYNTAX: **~FILEn REMOVE "Path\Filename";**

n Whole number from 1 to 7 that indicates the file to cancel.

SEE ALSO: ~FILEn OPEN_READ, ~FILEn OPEN_WRITE,~FILEn OPEN_APPEND,

EXAMPLE: EXAMPLE

Assuming that we are working on a ATD23 board and to have a file Risultati.txt under this directory that we want to cancel.

DECLARE RUNTIME INTEGER VAL;	Declaration of the integer type variable VAL
START ATD23R STATIC;	Beginning of the program
~FILE1 OPEN_READ "Risultati.txt"	Opening in reading mode of the file Risultati.txt under the directory .\S20WIN\BOARD\ATD23.
~FILE1 READ VAL;	Reading of the first line of the program, the value is stored in the variable VAL.
~CURSOR 10,10;	Positioning of the cursor
~WRITE "VALUE DEL FILE: " VAL;	Display of the first value of the file
~FILE1 CLOSE;	Closing of the file
~FILE1 REMOVE; "Path\Filename";	The file is deleted
ENDTEST;	End of the program.

~FILEn REWIND **Instruction** **~FILEn REWIND**

☛

Compiled	Run-time
	X

FUNCTION: Positions the file pointer at the beginning of the file, after having done previous reading/writing operations.

SYNTAX: **~FILEn REWIND**

n Whole number from 1 to 7 that indicates the open file.

SEE ALSO: ~FILEn OPEN_READ, ~FILEn OPEN_WRITE,~FILEn OPEN_APPEND,

EXAMPLE: EXAMPLE

Assuming that we are working on a ATD23 board and to have a file Risultati.txt under this directory with the following values:
23
26
09

DECLARE RUNTIME INTEGER VAL;	Declaration of the integer type variable VAL
START ATD23R STATIC;	Beginning of the program
~FILE1 OPEN_READ “Risultati.txt”	Opening in reading mode of the file Risultati.txt under the directory .\S20WIN\BOARD\ATD23.
~FILE1 READ VAL;	Reading of the first line of the program, the value is stored in the variable VAL.
~CURSOR 10,10;	Positioning of the cursor
~WRITE “VALUE DEL FILE: ” VAL;	Display of the first value of the file (23)
~FILE1 READ VAL;	Reading of the second value of the file
~CURSOR 20,10;	Positioning of the cursor
~WRITE “VALUE DEL FILE: ” VAL;	Display of the second value of the file (26)

~FILE1 REWIND;	Positioning of the cursor at the beginning of the file
~FILE1 READ VAL;	Second reading of the first line of the, the value is stored in the variable VAL.
~CURSOR 10,10;	Positioning of the cursor
~WRITE “VALUE DEL FILE: ” VAL;	Display of the first value of the file (23)
~FILE1 CLOSE;	Closing of the file.
ENDTEST;	End of the program.

~FILEn WRITE — Instruction — ~FILEn WRITE

Compiled	Run-time
	X

FUNCTION: Writes a data within a file opened previously with the instruction ~FILEn OPEN_WRITE.

SYNTAX: **~FILEn WRITE variable/constant**

n Whole number from 1 to 7 that indicates the open file.

variable Can be a variable of the type INTEGER or FLOAT or STRING, previously defined with the instruction DECLARE RUNTIME INTEGER/FLOAT/STRING that contains the value to write in the file.

constant Constant that indicates the value to write in the file.

SEE ALSO: ~FILEn OPEN_READ, ~FILEn OPEN_APPEND,

NOTE: The data is written at the beginning of a line within the file. If more writing operations are done while the file is open, the data are written one for every line. If the file is opened, written, closed, opened again in WRITE mode and written, it will start writing from the first line of the file, therefore the previous data will be overwritten. To avoid the overwriting, the file has to be opened in APPEND mode with the ~FILEn OPEN_APPEND instruction.

EXAMPLE: EXAMPLE 1 (writing with variable)

Assuming that we are working on a ATD23 board:

DECLARE RUNTIME INTEGER TENS;	Declaration of the integer type variable TENS
START ATD23R STATIC;	Beginning of the program
~MEAS VOLT LI1,MAX=2,TIM=1;	Voltage measure executed on the line 1 with end scale equal to 2V and time equal to 1ms. The result is stored in the accumulator.
~STORE TENS;	Assignment of the value in the accumulator to the variable TENS
~FILE1 OPEN_WRITE “Risultati.txt”	Opening in writing of the file Risultati.txt under the directory .\S20WIN\BOARD\ATD23.
~FILE1 WRITE TENS;	Writing of the voltage value, container in the variable TENS in the file.
~FILE1 CLOSE;	Closing of the file.
ENDTEST;	End of the program.

EXAMPLE 2 (writing with constant)

Assuming that we are working on a ATD23 board:

START ATD23R STATIC;	Beginning of the program
~FILE1 OPEN_WRITE “Risultati.txt”	Opening in writing mode of the file Risultati.txt under the directory .\S20WIN\BOARD\ATD23.
~FILE1 WRITE “PASS”;	Writing of the string PASS on the first line of the file
~FILE1 CLOSE;	Closing of the file.
ENDTEST;	End of the program.

~FLIn **Instruction** **~FLIn**

Compiled	Run-time
	X

FUNCTION: Connects and disconnects the specified channels from the line indicated. Unlike the ~LI instruction, the eight lines are driven separately.

SYNTAX: **~FLI line**=[*,][*][channel]

line — Indicates the line to connect/disconnect the channel to/from. Expressed as a number from 1 to 8.

channel — Indicates the channel or channels to connect/disconnect to/from the specified line. The channel can be:

- a single channel
- a group of channels
- a FLOAT type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

If the channel is not specified in the instruction (Example: ~FLI1;), all channels connected to the indicated line, will be disconnected.

[*,] — Indicates the incremental connect/disconnect of the channels. That is, the preceding connections are maintained and the actions specified in the current ~LI instruction are executed upon this situation.

[,] — If no character is specified in place of the "*" character, this indicates the normal channel connect/disconnect mode. That is, the channels specified in the current ~LI instruction are connected while those not specified are disconnected. If the current ~LI instruction includes the connection of channels which are already connected, the status of these channels remains the same.

[*] — Indicates the channel, which follows, must be disconnected from the indicated line.

[ ] — If no character is specified in place of the "*" character, this indicates that the channel which follows must be connected to the indicated line.

NOTE: If the value of the channel is passed on via a variable, any errors will come out during execution and not during compilation.

EXAMPLE:

```
DECLARE RUNTIME INTEGER CH;
DECLARE CHANNEL A3=65;
.
START PRG1 STATIC;
/;
.
.
~FLI2=98;
```

BE CAREFUL:This instruction works only on a system configured to work with 8 lines. It has no effect on a system configured to work with 4 lines because it is not possible to connect an even channel to LI1-LI2-LI3-LI4. The instruction connects channel 98 to line 2 and disconnects all of the other channels (normal connection mode)

~FLI4=1-3-5-7;	!Connects channels 1-3-5-7 to line 4 and disconnects all of the other channels (normal mode)
~FLI3;	!Disconnects all of the channels present on line 3
~CALC CH=15; ~FLI1=CH;	!Connects channel 15 to line 1 and disconnects all of the other (normal mode)
~FLI5=*,2	!Connects channel 2 while all of the other channels remain in the preceding situation (incremental connection mode)
~FLI4=*,*88	!Disconnects channel 88 while all of the other channels remain in the preceding situation (incremental connection mode)
~FLI4=*,TRICH;	
~FLI3=A3;	!Connects signal 65 defined with the DECLARE CHANNEL instruction to line 3 and disconnects all of the other channels (normal mode)

~FOR-~ENDFOR **Instruction** **~FOR-~ENDFOR**

Compiled	Run-time
	X

FUNCTION: Executes the instructions contained in the ~FOR cycle until the condition is true.

SYNTAX: **~FOR** [NOT] variable condition /constant/variable2/expression [**TO** final value] [**STEP** step value] [**DO** action]

[NOT] Enables the execution of a NOT of the condition specified.

variable Variable of the cycle. Runtime variable for which the specified condition is to be verified. This can be INTEGER, FLOAT, and STRING type.

condition Specifies the condition to apply to the variable. The following characters can be used to express the condition:

< or “lt” meaning “less than ...”
> or “gt” meaning “greater than ...”
= or “eq” meaning “equal to ...”
<= or “le” meaning “less than or equal to ...”
>= or “ge” meaning “greater than or equal to ...”
<> or “ne” meaning “different from ...”

costante/variabile2/espressione This can be:

- a constant
- a runtime variable type INTEGER, FLOAT, STRING

- an arithmetic/logic expression whose operands can be:

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Xor
&	And
|	Or
<n	Shift n bits to the left
>n	Shift n bits to the right

The arithmetic/logic expression has to be enclosed by parentheses (). There can be several levels of parentheses in the expression within global parentheses.

TO final value The value, which the variable must reach.

STEP step value

Step with which the variable reaches the limit value. NOTE: The field STEP is alternative to the field DO. It means that if the STEP value is present there can't be the action of DO at the same time. If the STEP field is present, the variable of the FOR cycle increase its value by 1 at every cycle.

DO action

If the key word DO is present, the ~FOR cycle is executed followed by the DO action; at this point, if the condition is still valid the cycle is executed again, followed by the DO action and so on, otherwise it ends. NOTE: the DO action is an alternative to the STEP field. That is if the DO action is present the STEP field cannot exist at the same time. When there is the DO action, the increasing of the FOR cycle variable is not automatically managed, therefore it has to be increased/decreased within the cycle with the instruction ~CALC.

DEFAULT VALUES:

If the key words "TO final value", "STEP step value" and "DO action", are omitted, by default the condition is > 0 and the step value is -1.
Refer to the EXAMPLE 1.

If the key word "STEP step value" is omitted, by default the step value is +1.
Refer to EXAMPLE 2.

If the key word "DO action" is omitted, no action is carried out by default.

SEE ALSO:

@FOR - @ENDFOR

NOTE: If the condition is false, the ~FOR loop ends without executing the body of the instructions.

The ~FOR- ~ENDFOR instruction accepts a nesting level of 10.

EXAMPLE:

EXAMPLE 1

!executes the ~FOR loop until A is greater than zero and each time decrements the variable by 1!

```
~FOR A;
~WRITE "A= ", A;
~ENDFOR;
```

EXAMPLE2

!executes the ~FOR loop until A reaches the value of 10 incrementing the variable each time by 1!

```
~FOR A=1 TO 10;
~WRITE "A= ", A;
~ENDFOR;
```

EXAMPLE 3

!executes the ~FOR loop until A reaches the value of 10 incrementing the variable each time by 2!

```
~FOR A=1 TO 10 STEP 2;
~WRITE “A= ”, A;
~ENDFOR;
```

EXAMPLE 4
Executes the ~FOR loop the first time with A=1, executes A=B+1 (A=5)
executes the FOR loop the second time with A=5, executes A=B+1 (A=6)
executes the FOR loop the third time with A=6, executes A=B+1 (A=7) and so on until A reaches or exceeds the value 30

```
~CALC A=1;
~CALC B=1;
~FOR A=1 TO 30 DO A=B+3;
~WRITE “CICLO”, A;
~CALC B=B+1;
~ENDFOR;
```

~FORMAT — Instruction — ~FORMAT

Compiled	Run-time
	X

FUNCTION:

Defines the format of the constants and the FLOAT and INTEGER type variables for the input/output operations to be executed with the ~WRITE, ~PRINT, ~REPORT, ~READ_SCR, ~READ_IEEE, SEND_IEEE, ~PAUSE instructions.

SYNTAX:

~FORMAT integers, decimals, format [AUTOCR]

integer — Whole numbers to consider in the INTEGER and FLOAT type variables.

decimals — Decimal number to consider (only in FLOAT type variables)

format — Defines the type of notation for the variables, which can be:

EXP: specifies the exponential notation

DEC: specifies the decimal notation

OCT: specifies the octal notation

HEX: specifies the hexadecimal notation

ASC: converts the number introduced in the corresponding ASCII (41=A) format.

AUTOCR — Eliminates the necessity of pressing the ENTER key to confirm the entry of the data in the subsequent READ_SCR instructions. This option remains active until the next ~FORMAT definition.

DEFAULT VALUES:

If the ~FORMAT instruction is not specified, by default the number of decimals is 3 while the whole part is considered in its entirety by it is rounded to the fifteenth figure. The notation is decimal.

SEE ALSO:

~READ_SCR

EXAMPLE:

```
DECLARE RUNTIME STRING SCREEN;
START PRG1 STATIC;
~FORMAT 10,10, EXP;
~READ_SCR SCREEN;
~FORMAT 1,0, AUTOCR;
~READ_SCR SCREEN;
~FORMAT 2,3,HEX;
ENDTEST;
```

~GNO Instruction ~GNO

Compiled	Run-time
	X

FUNCTION:

Connects or disconnects the ground of the specified channels to/from the output connector of the channel board. This instruction works considering the whole board, not the single channel. Hence the ground is connected or disconnected for all channels in the channel board (Example 1/32). It is not possible to have the ground connected/disconnected on a single channel (Example: Channel 3,12,15)

SYNTAX:

~GNO=[*,][*][channel]

channel

Indicates the channel or channels that corresponds to a channel board. (Example: ch. 1 corresponds to channel board that contains channels from 1 to 32). The channels ground of the indicated board will be connected/disconnected from the output connector according to the [*,][*] fields. The channel can be:

- a single channel
- a group of channels
- a FLOAT type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

If the channel is not specified in the instruction (Example: ~GNO;), the ground of all channels in the test system will be disconnected from the output connector.

[*,]

Indicates the INCREMENTAL mode to connect/disconnect the ground. That is, the preceding connections are maintained and the actions specified in the current ~GNO instruction are executed upon this situation.

[,]

If no character is specified in place of the "*" character, this indicates the NORMAL connect/disconnect mode. That is, the ground of the channel board is connected while the ground of the channel boards not specified are disconnected. If the current ~GNO instruction includes the connection of channel grounds which are already connected, the status of these channel board remains the same.

[*]

Indicates that the channel board indicated, must be disconnected from the ground on the output connector.

[ ]

If no character is specified in place of the "*" character, this indicates that the channel board must be connected on the output connector.

NOTE: If the value of the channel is passed on via a variable, any errors will come out during execution and not during compilation.

EXAMPLE: We assume to work with two F32 boards in the following example:

~LI2=12;	Connects channel 12 to line 2
~GNO=12;	Connects the ground of channels from 1 to 32 on the output connector (because channel 12 is on channel board from 1 to 32).
~GNO=*12;	Disconnects the ground of channels from 1 to 32 from the output connector
~GNO;	Disconnects the ground of all channels from the output connector
~CALC CH=34;	Connects the ground of channels from 33 to 64 on the output connector (because channel 34 is on channel board from 33 to 64).
~LI2=CH;	
~GNO=CH;	
~GNO;	
~LI2;	Sconnects all channels from line 2
ENDTEST;	

~GROUP Instruction ~GROUP

Compiled	Run-time
	X

FUNCTION: Defines different families of channels which will be programmed at the levels defined in the ~LEVEL instruction.

SYNTAX: **~GROUPn**= group of channels

n Letter, which identifies the family of channels. It can be any letter from A to Z.

channels group Group of channels to be associated to the specified family.

The specified family varies according to the channel boards in the system. If the channels are the P32 type, each family is made of a minimum of 4 or a multiple of 4, up to a maximum of 32. If the channel boards are the F32 type, each family is composed of 32 channels. If the boards are the F40 type, each family is composed of 8 channels minimum or multiples of 8.

It is NOT possible to define contiguous channel groups : a group composed of the channels 1,6,9,23 is not correct, you need to define: 1,2,3,4 or 9,10,11,12,13,14,15,16 and so on.

N.B. It is only necessary to write one channel belonging to a family for all of the other channels to be considered automatically. Therefore be careful not to declare channels belonging to the same family in different ~GROUP instructions. Refer to EXAMPLE 2.

The channels can be expressed as:

- physical channel numbers
- channel names (those declared in the DECLARE CHANNEL instruction)
- variables

The "/" character is used to indicate a contiguous group of channels and the "," character or the "-" character are used to indicate non-contiguous channels.

SEE ALSO: ~LEVEL

NOTE: For correct functioning, a ~LEVEL instruction has to be defined before a ~GROUP and not vice versa.
More ~LEVEL instructions can be defined and associated to the same family in different instants of the program.

EXAMPLE:

EXAMPLE 1

Assuming that we have two L32 channel cards. (Each channel card constitutes on e family of 32 channels)

```
~LEVELA,IL=0.1,IH=4.7,OL=1.5,OH=2.4;
~GROUPA=1;          !Channels 2 to 32 are automatically
                    associated to group A!

~LEVELB,IL=0.8,IH=4.5,OL=1.2,OH=2.4;
~GROUPB=33;         !Channels 34 to 64 are automatically
                    associated to group B!

.
.
```

EXAMPLE 2

Assuming we have one P32 channel card (There a 8 families of 4 channels on the channel card)

```
~LEVELA,IL=-0.1,IH=4.7,OL=1.5,OH=2.4;
~GROUPA=1,5;
~LEVELB,IL=1,IH=4.5,OL=1.5,OH=2.4;.
~GROUPB=9;
~LEVELC,IL=1,IH=5,OL=1,OH=2.4;.
~GROUPC=2,13;       ! Warning!!! channel 2 was
                    already defined in the preceding
                    ~GROUPA instruction. With this
                    new declaration channels 1,2,3,4
                    belong to group C!
```

EXAMPLE 3

```
DECLARE RUNTIME INTEGER IHCH
.
.
~WRITE "Enter the channel number: ";
~READ_SCR IHCH;
~LEVELD,IL=0,IH=5,OL=0.8,OH=2.4;
    ~GROUPD=IHCH;
```

~IF-~ELSE-~ENDIF **Instruction** **~IF-~ELSE-~ENDIF**

☛

Compiled	Run-time
	X

FUNCTION: Verifies the condition indicated; if the condition is true, the body of the ~IF branch is executed, otherwise the body of the ~ELSE is executed.

SYNTAX:

~IF [NOT] condition variable constant/variable2/expression
...
...
~ELSE
...
...
~ENDIF

[NOT] Enables the execution of a NOT of the specified condition.

variable Variable for which the specified condition is to be verified. The variable can be INTEGER, FLOAT, STRING type and the system variable NUM[1…15], STR[1…15].

condition Specifies the condition to apply to the variable. To express the condition the following characters can be used:

< or “lt” meaning “less than ...”
> or “gt” meaning “greater than...”
= or “eq” meaning “equal to....”
<= or “le” meaning “less than or equal to ...”
>= or “ge” meaning “greater than or equal to ...”
<> or “ne” meaning “different from ...”

constant/variable2/expression This can be:

- a constant
- a runtime variable of the type INTEGER, FLOAT, STRING

- an arithmetic/logic expression whose operands can be:

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Xor
&	And
|	Or
<n	Shift n bits to the left
>n	Shift n bits to the right

The arithmetic/logical expression must be enclosed in parentheses (). It is possible to have several levels of parentheses inside the global parentheses.

NOTE: The ~IF- ~ELSE- ~ENDF instruction accepts up to 10 nesting levels.

EXAMPLE:

```
DECLARE RUNTIME INTEGER A,B;

START PRG1 STATIC;
/;
~CALC A=1;
~CALC B=1;
/;
~IF A>10;
~WRITE “A>10”;
~ELSE;
~WRITE “A<10”;
~ENDIF;
/;
~IF A lt 10;
~WRITE “A>10”;
~ELSE;
~WRITE “A<10”;
~ENDIF;
/;
~CALC B=B+1;
/;
~IF A<(B*2);
~WRITE "A<B";
~ELSE;
~WRITE "A>B";
~ENDIF;
/;
~IF B>(A+C*(D+2));
~CALC B=B+1;
~ELSE;
~CALC E=E+1;
~ENDIF;
/;
ENDTEST;
```

~IH Instruction ~IH

Module	Compiled	Run-time
F32/P32		**X**

FUNCTION:

Executes the monitored forcing of the driver channel to high level. The channel driving voltage is the one indicated in the ~LEVEL instruction. As soon as the instruction is interpreted, the forcing of the channel is activated even without specifying the instruction for the closing of the pattern (/;).

SYNTAX:

~IH channel

channel

Indicates the channel or channels to drive to high level. This can be expressed as:

- a single channel
- a group of channels
- an integer type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUES:

If non-forcing instruction is specified, the driver channels are in masked sensor mode.

SEE ALSO:

~IL

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;
START PRG1 STATIC;
~IH CS;
/;
~IH 23;
/;
~IH 2,4,6,8;
/;
~IH 1/8;
/;
~WRITE "ENTER THE CHANNEL NUMBER: ";
~READ_SCR CAN;
/;
~IH CAN;
ENDTEST;
```

~IL Instruction ~IL

☛

Module	Compiled	Run-time
F32/P32		**X**

FUNCTION:

Executes the monitored forcing of the driver channel to low level. The channel driving voltage is the one indicated in the ~LEVEL instruction. As soon as the instruction is interpreted, the forcing of the channel is activated even without specifying the instruction for the closing of the pattern (/;).

SYNTAX:

~IL channel

channel

Indicates the channel to drive to low level. This can be expressed as:

- a single channel
- a group of channels
- an integer type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUES:

If non-forcing instruction is specified, the driver channels are in masked sensor mode.

SEE ALSO:

~IH

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;
START PRG1 STATIC;
~IL CS;
/;
~IL 23;
/;
~IL 2,4,6,8;
/;

~IL 1/8;
/;
~WRITE "ENTER THE CHANNEL NUMBER: ";
~READ_SCR CAN;
/;
~IL CAN;
ENDTEST;
```

~LEVEL Instruction ~LEVEL

Module	Compiled	Run-time
F32/P32/F40		**X**

FUNCTION: Determines the programming threshold of the driver and sensor channels for the families defined in the ~GROUP instruction.

SYNTAX: **~LEVELn**,**IL**=value,**IH**=value,**OL**=value, **OH**=value,**CUR**=value

n — Letter, which identifies the channel family. This can be a letter from A to Z. This is related to what is defined in the ~GROUP instruction.

IL=value — Specifies the value in volts of the low level of the driver channels. The value can also be defined with a FLOAT or INTEGER type runtime variable. See the EXAMPLE.

IH=value — Specifies the value in volts of the high level of the driver channels. The value can also be defined with a FLOAT or INTEGER type runtime variable.

OL=value — Specifies the value in volts of the low sensor level and the low driver level of the channels . The value can also be defined by a FLOAT or INTEGER type variable.

OH=value — Specifies the value in volts of the high sensor level and the high driver level of the channels . The value can also be defined by a FLOAT or INTEGER type variable.

CUR=value — Specifies the maximum current level of the channels expressed in mA. The current range is:

from 20mA to 500mA

The current value is automatically multiplied by 2, therefore if the following is defined: ~LEVELA IL=.....,CUR=10 it means the current will be 20mA.

The CUR parameter is not valid for F40 boards.

DEFAULT VALUES: If the ~LEVEL instruction is not specified, the default values used are in the File.srg instruction, that is: DVH=4.5V, DVL=0.8V, THH=2.4V, THL=0.8V, CUR=200mA.

SEE ALSO: ~GROUP

NOTE: For correct functioning the ~LEVEL instruction must be defined before the ~GROUP instruction.
For correct functioning the four levels (IH, IL, OH, OL) must always be present in the ~LEVEL instruction. Besides, the following conditions must be true: |IL| > |OL| ed |IH| > |OH|. ***Example:*** ~LEVELA IL=1, IH=5, OL=0.8, OH=2.4

The voltage levels defined with the ~LEVEL instruction, have priority on the ones defined in the SRG file with the DVH, DVL, THH, THL instructions.

EXAMPLE:

```
DECLARE RUNTIME INTEGER TRICH;
DECLARE RUNTIME FLOAT TRINEGV,TRIPOS,TEMP1;

START PRG1 STATIC;

~LEVELA,IL=1.5,IH=4.7,OL=0.8,OH=2.4;
~GROUPA=1/64;
.
.
~LEVELB,IL=TRINEGV,IH=TRIPOSV,OL=TEMP1,OH=TEMP1;
~GROUPB=TRICH;
.
.
~LEVELC IL=0.8,IH=4,OL=0.4,OH=3,CUR=10;
~GROUPC 1,2,33;
.
.
~LEVELD,IL=-1,IH=4.7,OL=1.5,OH=2.4;
~GROUPD=1/32;
.
.
ENDTEST;
```

~LIn — Instruction — ~LIn

Compiled	Run-time
	X

FUNCTION: Connects and disconnects the specified channels from the line indicated. The instruction works in the following way:

If we write	**The following linea will be opened/closed**
~LI1	LI1 and LI5
~LI2	LI2 and LI6
~LI3	LI3 and LI7
~LI4	LI4 and LI8
~LI5	The digital line: LI5

SYNTAX: **~LI line**=[*,][*][channel]

line — Indicates the line to connect/disconnect the channel to/from. Expressed as a number from 1 to 5. The lines from 1 to 4 are analog while the line 5 is digital.

channel — Indicates the channel or channels to connect/disconnect to/from the specified line. The channel can be:

- a single channel
- a group of channels
- a FLOAT type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

If the channel is not specified in the instruction (Example: ~LI1;), all channels connected to the indicated line, will be disconnected.

[*,] — Indicates the incremental connect/disconnect of the channels. That is, the preceding connections are maintained and the actions specified in the current ~LI instruction are executed upon this situation.

[,] — If no character is specified in place of the "*" character, this indicates the normal channel connect/disconnect mode. That is, the channels specified in the current ~LI instruction are connected while those not specified are disconnected. If the current ~LI instruction includes the connection of channels which are already connected, the status of these channels remains the same.

[*] — Indicates the channel, which follows, must be disconnected from the indicated line.

[ ] If no character is specified in place of the "*" character, this indicates that the channel which follows must be connected to the indicated line.

NOTE: The ~LIn instruction is equivalent to the execution of two ~FLIn istruction according to the table:

The instruction	Is equivalent to the execution of
~LI1	~FLI1 and ~FLI5
~LI2	~FLI2 and ~FLI6
~LI3	~FLI3 and ~FLI7
~LI4	~FLI4 and ~FLI8
~LI5	It doesn't have a corrispondent ~FL istruction because LI5 is the digital line

If the value of the channel is passed on via a variable, any errors will come out during execution and not during compilation.

EXAMPLE:

```
DECLARE RUNTIME INTEGER CH;
DECLARE CHANNEL A3=65;
.
START PRG1 STATIC;
/;
.
.
~LI2=99;          !Connects channel 99 to line 2 and
                  disconnects all of the other channels
                  (normal connection mode)

~LI3=1,4,6,8/32;  ! Connects channels 1,4,6, and from
                  channel 8 to channel 32 included to line
                  3, and disconnects all of the other
                  channels (normal mode)

~LI4=1-3-5-7;     !Connects channels 1,3,5,7 to line 4 and
                  disconnects all of the other channels
                  (normal mode)

~LI2;             !Disconnects all of the channels present
                  on line 2


~CALC CH=15;
~LI1=CH;          !Connects channel 15 to line 1 and
                  disconnects all of the other (normal
                  mode)

~LI5=*,3          !Connects channel 3 while all of the
                  other channels remain in the preceding
                  situation (incremental connection mode)
```

~LI4=*,*88	!Disconnects channel 88 while all of the other channels remain in the preceding situation (incremental connection mode)
~LI4=*,TRICH;	
~LI3=A3;	!Connects signal 65 defined with the DECLARE CHANNEL instruction to line 3 and disconnects all of the other channels (normal mode)

~LOAD — Instruction — ~LOAD

Compiled	Run-time
	X

FUNCTION: Loads the value contained in the specified variable into the AR register.

SYNTAX: **~LOAD** value

value — Value to load into the AR register. It can be expressed as:

- a constant
- a runtime variable type INTEGER, FLOAT

SEE ALSO: ~STORE

NOTE: The AR register is a FLOAT type variable and can be loaded with FLOAT and INTEGER type variables or constants. It cannot be loaded with STRING type constants or variables.

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR;
DECLARE RUNTIME FLOAT VAR1;

START PRG1 STATIC;
/;
~LOAD 2.3;
/;
~CALC VAR=2;
~LOAD VAR;
/;
~CALC VAR1=4.5693;
~LOAD;
/;
ENDTEST;
```

~LOG Instruction ~LOG

Compiled	Run-time
	X

FUNCTION: Calculates the natural logarithm of the AR register.

SYNTAX: **~LOG**

SEE ALSO: ~EXP

EXAMPLE:

```
DECLARE RUNTIME FLOAT VAR;

START PRG1 STATIC;
/;
~LOAD 7;
~LOG;
~STORE VAR;
~WRITE "LOGARITHM: ",VAR;
/;
~PAUSE 1;
/;

ENDTEST;
```

~MEAS COUNTER **Instrument** **~MEAS COUNTER**

☛

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

Executes a measurement with the COUNTER instrument which has the following characteristics:

Time interval: from 500µs to 10s with a 500ns step
Counting: up to 32767 impulses per step
Input voltage: ±5V.

The result of the measurement is stored in the AR register (accumulator) and it is possible to control the value using the ~COMP or ~ATEST instructions.

SYNTAX:

~MEAS COUNTER LI4 TIM=value NRC=value MRC=value

LI4

Connects the input of the measurer to line 4.

TIM=value

Decimal number that indicates the duration of the measurement. The range accepted is: 0.0005ms ≤ TIM ≤ 60000ms. The value must be indicated without the unit of measurement. The value can be expressed as a constant, as a RUNTIME variable or as an expression enclosed in parentheses.

NRC=value

Defines the number of recycles of the test. Example: NRC=10 means that the test will be repeated 10 times. The value can be expressed as a constant, as a RUNTIME variable or as an expression enclosed in parentheses. This command is combined to the MRC=value command.

MCR=value

Defines the form of measurement and can assume the following meanings:

M — at every recycle the result of the measurement is stored. The interaction stops when the expected number of recycles is reached, providing the average of the results.

W — the interaction stops when the expected number of recycles is reached, and only at this point the measurement is executed.

J — the interaction stops when the test gives a positive result.

H — the interaction stops when the expected number of recycles is reached and the highest value measured is stored.

L — the interaction stops when the expected number of recycles is reached and the lowest value measured is stored.

P the interaction stops when the expected number of recycles is reached and the difference between the highest and lowest value measured is stored.

MRC=1|2|3|4|5|6 are alias of MRC=M|W|J|H|L|P (that is MCR=1 means MRC=M, MRC=2 means MRC=W…)

DEFAULT VALUES: If the LI4 parameter is not specified, the instrument is disconnected.

If the TIM parameter is not specified, TIM=1ms by default.

EXAMPLE:

```
~MEAS COUNTER LI4,TIM=1;
~MEAS COUNTER LI4,TIM=TEMP1;
```

~MEAS PMM MAX= TIM= NRC= MRC=

Module	Compile	Run-time
HV		**X**

FUNCTION: Executes a voltage or current measure accordion to the indicated parameters. The result of the measure is stored in the AR register (accumulator) and it is possible to control the value using the ~COMP or ~ATEST instructions.

SYNTAX: **~MEAS PMM MAX=** scale, **TIM=** time, **NRC**=value, **MRC**=value

scale — Indicates the value of the end scale chosen for the voltage measures. It can be chosen from the following values:

10 / 100 / 1000

For current measures, the end scale is not programmable and it is a default value fixed at 10A.

time — Indicates the time that the RMS converter needs to settle within a zone of error ≤ 1%.
The value is expressed in ms and can vary from 10 to 200.

NRC=value — Defines the number of recycles of the test. Example: NRC=10 means that the test will be repeated 10 times. The value can be expressed as a constant, as a RUNTIME variable or as an expression enclosed in parentheses. This command is combined to the MRC=value command.

MCR=value — Defines the form of measurement and can assume the following meanings:

M: at every recycle the result of the measurement is stored. The interaction stops when the expected number of recycles is reached, providing the average of the results.

W the interaction stops when the expected number of recycles is reached, and only at this point the measurement is executed.

J the interaction stops when the test gives a positive result.

H the interaction stops when the expected number of recycles is reached and the highest value measured is stored.

L the interaction stops when the expected number of recycles is reached and the lowest value measured is stored.

.

P the interaction stops when the expected number of recycles is reached and the difference between the highest and lowest value measured is stored.

MRC=1|2|3|4|5|6 are alias of MRC=M|W|J|H|L|P (that is MCR=1 means MRC=M, MRC=2 means MRC=W…)

SEE ALSO: ~TEST

EXAMPLE:

```
START GH1 STATIC;
~CALC A=5;
```

```
DECLARE RUNTIME INTEGER A;

~MEAS PMM MAX= 1000, TIM=200;
~MEAS PMM MAX= 100, TIM=20, NRC=(A+3) MRC=M;
ENDTEST;
```

~MEAS USER_GEN **Instruction** **~MEAS USER_GEN**

☛

Module	Compiled	Run-time
USB/GEN		**X**

FUNCTION: Executes a measure RMS to DC with the GEN module. The result is a continuous voltage value, contained in the AR register.

SYNTAX: **~MEAS USER_GEN HZ=**value, **TIM=**value, **MAX=**value, **[DIR | LI3 | GAIN] ;**

HZ=value Indicates the expected frequency of the signal to measure expressed in Hz. It can assume values from 5Hz to 5MHz. When the frequency is specified, the time of measure integration is automatically calculated.

TIM=value Indicates the integration time of the measure, expressed in ms. If the parameter is not specified the length of the measure is programmed to 1ms. (TIM=1).

MAX=value Indicates the end scale value in Volts. It can assume the following values: 2-10-50V.

DIR Indicates that the measure will be executed between GNDREF and the direct output of the booster.

LI3 Indicates that the measure will be executed between GNDREF and line LI3 (for this measure LI3 must be disconnected from the guarding generator).

GAIN Indicates that the measure executed will be the ratio between line LI1 and LI3 (for this measure LI3 must be disconnected from the guarding generator).

NOTA: For the measure it is necessary to specify the expected frequency or the integration time of the measure. Specifying one of them, the other one will be automatically calculated.

If both the parameters HZ=value and TIM=value are not specified the value for the integration time is10ms by default.

EXAMPLE: ~MEAS USER_GEN HZ=1000

~MEAS PW1…8 — Instrument — ~MEAS PW1…8

☛

Module	Compiled	Run-time
SE2		**X**

FUNCTION:

Measures a tension or a current on the indicated feeder. The result of the measure is present in the AR register.

SYNTAX:

~MEAS PWn type of measure, mode of measure

n

Parameter to identify the feeder, it can assume values from 1 to 8.

Type of measure

It can assume the following values:

I: indicates a current measure.
V: indicates a voltage measure.
At least one parameter (V or I) has to be specified in the ~MEAS instruction.

Mode of measure

Indicates reading mode. It can assume the following values:

INT: internal reading, made through the programming signals of the feeder, connected at the SE2 module.

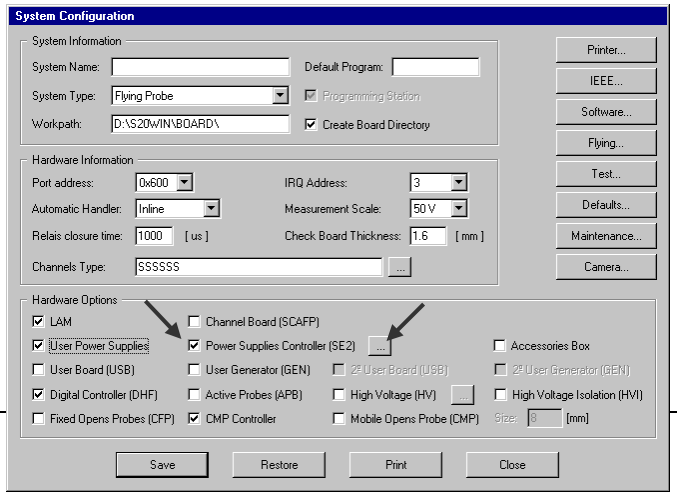
EXT external reading, made directly on the power supply sense signals.

DIG measures using the ADC converter (analog/digital converter). If not set to DIG, the reading is done on line 4 of the LAM board.

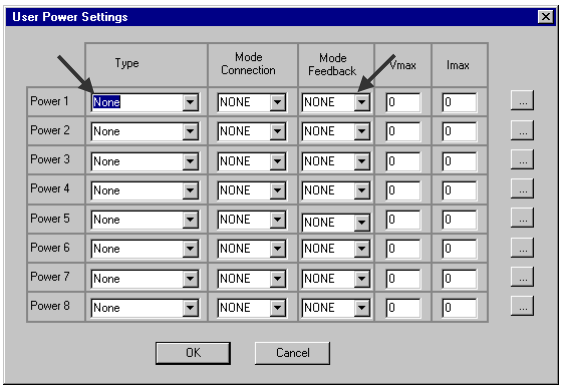
DEFAULT VALUES:

Mode of measure:

it is set according to the power supply mounted in the test system. It is possible to see the mode of measure in this way: in the S20WIN software, select the UTILITIES\Configuration menu, select the Power Supplies Controller (SE2) option and press the button beside as indicated in figure.



In the User Power Settings dialogue, consult the MODE FEEDBACK value.



If the value is:
INT: means INTERNAL mode of reading that is the reading is done through the programming signals of the SE2 module.

EXT: means EXTERNAL mode of reading that is the reading is done on the sense signals.

INT/EXT: means that the reading can be done both in INTERNAL and EXTERNAL mode.

EXAMPLE:

EXAMPLE 1 Tension and current measure on PW1 with internal measuring mode.

```
~SET PW1 V=0.5 I=6 SENSEON ON INT;
~MEAS PW1 V SERV;
~CURSOR 10,10;
~WRITE ”VOLT ON PW1=”,AR;
~MEAS PW1 I SERV;
~CURSOR 20,10;
~WRITE ”CURRENT ON PW1=”,AR;
```

EXAMPLE 2 Tension and current measure on PW2 with external measuring mode.

```
~SET PW2 V=1 I=2 SENSEON ON EXT;
~MEAS PW2 V EXT;
~CURSOR 10,10;
~WRITE ”VOLT ON PW2=”,AR;
~MEAS PW1 I EXT;
~CURSOR 20,10;
~WRITE ”CURRENT ON PW2=”,AR;
```

~MEAS VOLT **Instrument** **~MEAS VOLT**

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

Measures a voltage with the VOLT instrument, which has the following characteristics:

Voltage measures

from -2V to +2V step of 1mV
from -10V to +10V step of 5mV
from -50V to +50V step of 25mV
The result of the measure is stored in the AR register (accumulator) and it is possible to control the value using the ~COMP or ~ATEST instructions.

SYNTAX:

~MEAS VOLT [**LI1**] [**LI4**] **MAX=value TIM=value NRC=value MRC=value**

LI1

Connects the input of the measurer to line 1.

LI4

Connects the input of the measurer to line 4.

MAX=value

Indicates the end scale value of the instrument.
It can accept the following values:

2-10-50

The value can be expressed: as a constant, as a compilation variable, as a runtime variable, as a numerical expression indicated with parentheses.

TIM=value

Decimal value that indicates the length of the measure. The accepted range is:
0.0005ms ≤ TIM ≤ 60000ms. The value must be indicated without the unit of measurement.
The value can be expressed: as a constant, as a compilation variable, as a runtime variable, as a numerical expression indicated with parentheses.

NRC=value

Defines the number of recycles of the test. Example: NRC=10 means that the test will be repeated 10 times. The value can be expressed as a constant, as a RUNTIME variable or as an expression enclosed in parentheses. This command is combined with the MRC=value command.

MCR=value

Defines the measuring mode and can accept the following meanings:

M: at every recycle the result of the measurement is stored. The iteration stops when the expected number of recycles is reached, providing the average of the results.

W The iteration stops when the expected number of recycles is reached and, only at this point the measurement is executed.

J the interaction stops when the test gives a positive result.

H the interaction stops when the expected number of recycles is reached and the highest value measured is stored.

L the interaction stops when the expected number of recycles is reached and the lowest value measured is stored.

P the interaction stops when the expected number of recycles is reached and the difference between the highest and lowest value measured is stored.

MRC=1|2|3|4|5|6 are alias of MRC=M|W|J|H|L|P (that is MCR=1 means MRC=M, MRC=2 means MRC=W…).

DEFAULT VALUES: If the parameters LI1 and LI4 are not specified, the instrument is connected to line 4.
If the MAX=... parameter is not specified, MAX=50 by default.
If the TIM=... parameter is not specified, TIM=1 by default.

NOTE: If the specified value of the parameter MAX=... is greater than 2, the end scale is set to 10.

If the specified value of the parameter MAX=... is greater than 10, the end scale is set to 50.

EXAMPLE:

```
DECLARE RUNTIME INTEGER FS,TP;
START PRG1 STATIC;
/;
~MEAS VOLT LI1,MAX=2,TIM=1;
~MEAS VOLT LI4,MAX=10,TIM=0.5;
/;
~CALC FS=4;
~CALC TP=10;
~MEAS VOLT=LI1,MAX=FS,TIM=TP;
/;
ENDTEST;
```

~MH Instruction ~MH

Module	Compiled	Run-time
F32/P32		**X**

FUNCTION: Executes the unmonitored forcing of the driver channel to high level. The channel driving voltage is the one indicated in the ~LEVEL instruction. As soon as the instruction is interpreted, the forcing of the channel is activated even without specifying the instruction for the closing of the pattern (/;).

SYNTAX: **~MH** channel

channel: Indicates the channel to drive to high level. This can be expressed as:

- a single channel
- a group of channels
- an integer type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUES: If no forcing instruction is specified, the driver channels are in masked sensor mode.

NOTE: This instruction is used when there is a capacitive load (of some mF) connected to the network (i.e. a capacitor). In this situation, when the channel is set to the high level, the capacitor will charge and during its charging time, the channel is still low. The ~MH instruction set the channel high but it doesn't control it, in this way there isn't a false error.

SEE ALSO: ~ML

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;
START PRG1 STATIC;
~IH CS;
/;
~IH 23;
/;
~IH 2,4,6,8;
/;~MH 1/8;/;
~WRITE "ENTER THE CHANNEL NUMBER: ";
~READ_SCR CAN;
/;
~MH CAN;
/;
ENDTEST;
```

~ML Instruction ~ML

☛

Module	Compiled	Run-time
F32/P32		**X**

FUNCTION: Executes the unmonitored forcing to low level of the driver channel. The channel driving voltage is the one indicated in the ~LEVEL instruction. As soon as the instruction is interpreted, the forcing of the channel is activated even without specifying the instruction for the closing of the pattern (/;).

SYNTAX: **~ML** channel

channel: Indicates the channel to drive to low level. This can be expressed as:

- a single channel
- a group of channels
- an INTEGER type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUES: If no forcing instruction is specified, the driver channels are in masked sensor mode.

NOTE: This instruction is used when there is a capacitive load (of some mF) connected to the network (i.e. a capacitor). In this situation, when the channel is set to the low level, the capacitor will discharge and during the discharging time, the channel is still high. The ~ML instruction set the channel low but it doesn't control it, in this way there isn't a false error.

SEE ALSO: ~MH

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;
START PRG1 STATIC;
~IL CS;
/;
~IL 23;
/;
~ML 2,4,6,8;
/;
~ML 1/8;
/;
~WRITE "ENTER THE CHANNEL NUMBER: ";
~READ_SCR CAN;
/;~ML CAN;/;
ENDTEST;
```

~MSG — Instruction — ~MSG

Compiled	Run-time
	X

FUNCTION: Displays the indicated value inside the lower status bar of the Static terminal

SYNTAX: **~MSG** value

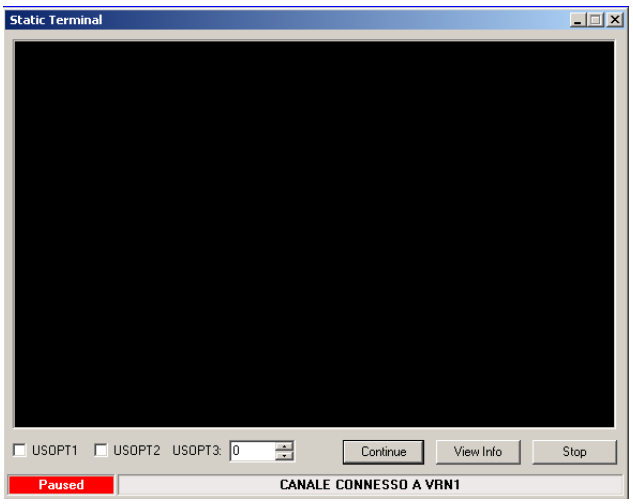
value: Value displayed in the Status bar of the Static Terminal window when the ~MSG instruction is executed. The value can be expressed as:

- a constant
- a runtime variable INTEGER, FLOAT, STRING type
- a string of characters enclosed by quotation marks with maximum length of 80 characters

SEE ALSO: ~PAUSE

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE RUNTIME FLOAT VAR1;
START PRG1 STATIC;
~CALC CAN=16;
~CURSOR 60,20;
~WRITE CAN;
~MSG "CANALE CONNESSO A VRN1";
~CALC CAN=27;
~CURSOR 60,21;
~WRITE CAN;
~MSG "CANALE CONNESSO A VRN1"; !channel connected to
        VRN1
ENDTEST;
```



~MULT — Instruction — ~MULT

Compiled	Run-time
	X

FUNCTION: Multiplies the specified value by the contents of the AR register.

SYNTAX: **~MULT** value

value — Value to multiply the AR register by. This can be expressed as:

- a constant
- a runtime variable INTEGER, FLOAT type

SEE ALSO: ~ADD ~SUB ~DIV

EXAMPLE:

```
DECLARE RUNTIME INTEGER V1;

START PRG1 STATIC;
/;
~CALC V1=6;
/;
~ MULT V1;
/;
~MULT 5;
/;
ENDTEST;
```

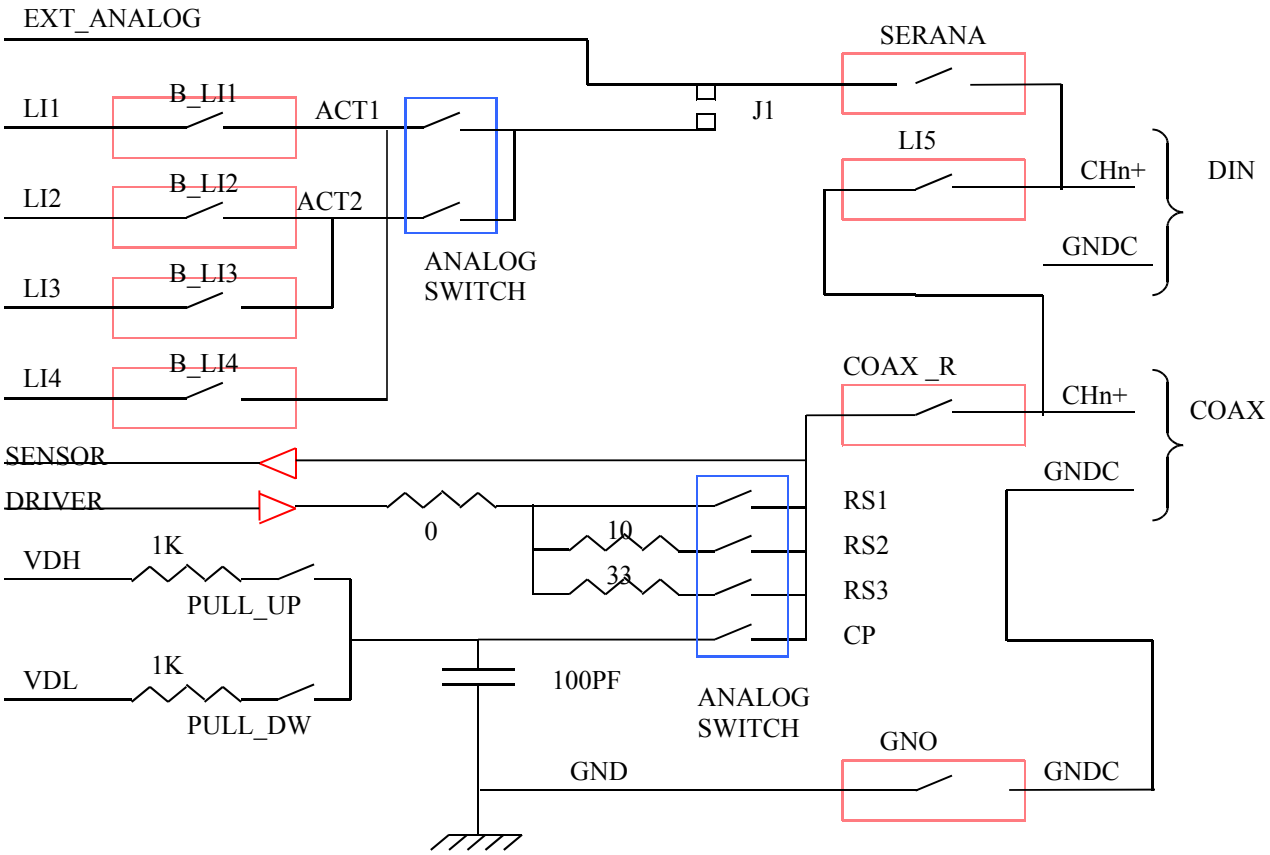
~MUXOUT Instruction ~MUXOUT

☛

Module	Compiled	Run-time
F40		**X**

FUNCTION:

Closes/opens the SERANA relay, LI5 and COAX_R of the output channel. The schematic of the F40 channel is given here below for better understanding. This schematic is referred to a channel with both the DIN and coax output.



SYNTAX:

~MUXOUT parameter [*,]channel

or

~MUXOUT parameter [*,]channel **parameter** channel

parameter

Parameter defining the relay of the channel. It can accept the following values:

DINANA
DIN
COAX
ANA

DINANA: connects the analog channel to the output connector closing the SERANA relay and the COAX_R relay. It is useful when you want to have on the output connector both the analog and coaxial channels. The analog channel can be of two types: internal to the F40 module (analog switch with 2 lines per channel) or external EXT_ANALOG from the S64 module that can be connected with 4 lines per channel. If the J1 jumper is not mounted, the EXT_ANALOG is present on the output. If the J1 jumper is mounted, the internal F40 analog channel is on the output.

DIN: disconnects the analog channel from the output connector opening the SERANA relay.

COAX: connects the coax channel on the output connector. It opens the SERANA and closes the COAX_R relays. Used to connect only the coax output on the output connector.

ANA: opens the COAX_R and closes the SERANA relays. Used to connect only the analog output on the output connector.

[*,] Indicates the incremental connect/disconnect of the channels. That is, the preceding connections are maintained and the actions specified in the current ~MUXOUT instruction are executed upon this situation.

[,] If no character is specified in place of the "*" character, the DINANA parameters is applied for all non-listed channels.

channel Indicates the channel or channels to connect/disconnect to/from the specified line. The channel can be:

- a single channel
- a group of channels
- a FLOAT type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUE: By default the SERANA relay is closed while the COAX_R relay is in the same state of the LI5 relay, previously set with the ~LI5 command.

NOTE: If there is a voltage level on my board that is out of the programmed range for the digital driver, the line 5 (LI5) relay must be opened otherwise there is the risk to damage the hardware.

~LI5 command closes/opens both the LI5 and COAX_R relay.
It is recommended to use the LI5 command first and then the MUXOUT.
If only the COAX_R relay must be closed, the LI5 relay must be opened first with the ~LI5 command and then the ~MUXOUT COAX command can be used.

The internal analog channels work in families of minimum 8 channels. If jumper J1 is mounted (internal analog channels selected), when a channel is connected to the output

connector and the SERANA relay is closed, we have to note that ALSO ALL OTHER 8 CHANNELS in the family, are connected.

It is not possible to have the digital resource only on the DIN connector. It is always present both on the DIN and coax connectors

EXAMPLE:

~MUXOUT DINANA *, 34;	Connects the analog channel 34 to the output connector
~MUXOUT COAX *, 31;	Connects channel 31 to the coax output
~LI5 13-31;	(to be read together with the following line)
~MUXOUT COAX 31, DIN 13	It connects channel 13 both to the digital and coax output.

~OH Instruction ~OH

Module	Compiled	Run-time
F32/P32		**X**

FUNCTION:

Executes the test at high level of the sensor channel. The channel test voltage is the one indicated in the ~LEVEL instruction. As soon as the instruction is interpreted, the forcing of the channel is activated even without specifying the instruction for the closing of the pattern (/;).

SYNTAX:

~OH channel

channel

Indicates the channel to test at high level. This can be expressed as:

- a single channel
- a group of channels
- an INTEGER type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUES:

If no sensing instruction is specified, the sensor channels are masked.

SEE ALSO:

~OL

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;

START PRG1 STATIC;

~IL CS;
/;
~IH 23;
/;
~OH 2,4,6,8;
/;
.
.
~OH 1/8;
/;
~WRITE "Enter channel number to test : ";
~READ_SCR CAN;
/;
~OH CAN;
/;
ENDTEST;
```

~OL Instruction ~OL

☛

Module	Compiled	Run-time
F32/P32		**X**

FUNCTION:

Executes the test at low level of the sensor channel. The channel test voltage is the one indicated in the ~LEVEL instruction. As soon as the instruction is interpreted, the channel on high level is controlled even without specifying the instruction for the closing of the pattern (/;).

SYNTAX:

~OL channel

channel

Indicates the channel to test at high level. This can be expressed as:

- a single channel
- a group of channels
- an INTEGER type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUES:

If no sensing instruction is specified, the sensor channels are masked.

SEE ALSO:

~OH

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;
START PRG1 STATIC;
~IL CS;
/;
~IH 23;
/;
~OL 2,4,6,8;
/;
~OL 1/8;
/;
~WRITE "Enter the channel number to test : ";
~READ_SCR CAN;
/;
~OL CAN;
/;
ENDTEST;
```

~OM Instruction ~OM

☛

Module	Compiled	Run-time
F32/P32		**X**

FUNCTION:

Masks the control on the sensor channel therefore it does not activate the global error flag of the program. As soon as the instruction is interpreted, As soon as the instruction is interpreted, the channel is masked without specifying instruction for the closing of the pattern (/;).

SYNTAX:

~OM channel

channel

Indicates the channel to mask. This can be:

- a single channel
- a group of channels
- an INTEGER type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUES:

The sensor channels are all masked.

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;
START PRG1 STATIC;
~IL CS;
/;
~IH 23;
/;
~OH 2,4,6,8;
/;
.
.
~OM 1/8;
/;
~WRITE "Enter the channel number to test : ";
~READ_SCR CAN;
/;
~OM CAN;
/;
ENDTEST;
```

~ONBREAK **Instruction** **~ONBREAK**

Module	Compiled	Run-time
F32/P32/F40		**X**

FUNCTION:

Defines the action to be carried out when a pause occurs in the program execution. The pause can be generated by the ~PAUSE, ~READ_SCR istructions or a fail condition in the clusater that opens the Debug dialog. (the Stop on functional errors button , must be selected).

SINTAX:

~ONBREAK option

option

the optino can be:

ABORT
CALL subroutine name

ABORT: when an error occurs, the execution of the current cluster will be interrupted (abort).

CALL subroutine name: when an error occurs, it calls the indicated subroutine. This option is very useful if, for example, the power supplies must be switched off after an error and the cluster aborted. Refer to Example 2 for details.

DEFAULT VALUES:

~ONBREAK CONTINUE . it doesn't stop the program execution.

EXAMPLE:

EXAMPLE 1

```
DECLARE CHANNEL CH1=45;
START PART2 STATIC;
~ONBREAK ABORT;        When a pause occurs, the cluster will be
                       aborted.
~LEVELA IL=0,IH=5,OL=0.2,OH=2.4;
~GROUPA CH1;
~GNO CH1;
~LI1=CH1;
~TEST VOLT LI1 MAX=20, TIM=10 LO=15 HI=18;
If the measured voltage is out of range, the debug dialog is opened
and the cluster is aborted
~CURSOR 1,10;
~WRITE "MEASURED VOLTAGE: "AR;
~DELAY 10S;
ENDTEST;
```

~ONERROR Instruction ~ONERROR

☛

Module	Compiled	Run-time
F32/P32/F40		**X**

FUNCTION: Defines the action to be carried out when an error occurs in the cluster, that is, when the global flag error is set.

SINTAX: **~ONERROR** option

option The optino can be:

ABORT
CALL subroutine name

ABORT: when an error occurs, the execution of the current cluster will be interrupted (abort).

CALL subroutine name: when an error occurs, it calls the indicated subroutine. This option is very useful if, for example, the power supplies must be switched off after an error and the cluster aborted. Refer to Example 2 for details.

DEFAULT VALUES: **~ONERROR** CONTINUE. When an error occurs, the execution is continued.

NOTE: If the global flag error is set by the ~CALC FAIL=1 istruction or ~CALC FAIL=FAIL+1, the ~ONERROR instruction has NO effect.

EXAMPLE: EXAMPLE 1

```
DECLARE CHANNEL CH1=45;
START PART2 STATIC;
~ONERROR ABORT;        When the global flag error is set, the
                       cluster will stop.
~LEVELA IL=0,IH=5,OL=0.2,OH=2.4;
~GROUPA CH1;
~GNO CH1;
~LI1=CH1;
~TEST VOLT LI1 MAX=20, TIM=10, LO=15 HI=18;
                       If the measured voltage is out of range,
                       the global flag error will be set and the
                       program will abort
~CURSOR 1,10;
~WRITE "MEASURED VALUE: "AR;
~DELAY 10M;
ENDTEST;
```

EXAMPLE 2

```
DECLARE CHANNEL CH1=45;
~SUBR ALIOFF;
~SET PW1 V=0 OFF DLY=500;
~SET PW2 V=0 OFF;
~RETURN;
~ENDSUBR;
DECLARE SUBROUTINE ALIOFF;
START PART3 STATIC;
~ONERROR CALL ALIOFF;
```

When the global flag error is set, the subroutine ALIOFF will be executed to switch off the power supply. After that the execution will continue.

```
~SET PW1 V=5 DLY=500M ON;
~LEVELA IL=0,IH=5,OL=0.2,OH=2.4;
~GROUPA CH1;
~GNO CH1;
~LI1=CH1;
~TEST VOLT LI1 MAX=20, TIM=10 LO=15, HI=18;
```

If the measured voltage is out of range, the global flag error will be set and the program will abort

```
~CURSOR 1,10;
~WRITE “MEASURED VALUE: “AR;
~DELAY 10M;
ENDTEST;
```

ESEMPIO 3

```
DECLARE CHANNEL CH1=45;
DECLARE CHANNEL CH2=56;
DECLARE CHANNEL CH3=89;
DECLARE RUNTIME ARGUMENT STRING RES;
~SUBR RES_ERR (RES);
~CURSOR 20,10;
~COLOR 0,12;
~WRITE "RESISTOR " RES " FAILS!";
~PAUSE 1;
~RETURN;
~ENDSUBR;
DECLARE SUBROUTINE RES_ERR;
START PART4 STATIC;
~ONERROR CALL RES_ERR (RES);!
```

When the global flag error is set, the subroutine RES_ERR will be executed to display on the screen the failing resistor. After that, the execution will continue.

```
~SET PW1 V=5 DLY=500M ON;
~LEVELA IL=0,IH=5,OL=0.2,OH=2.4;
~GROUPA CH1 CH2;
~GNO CH1 CH2;
~SET DR1 VL  I=100 V=3 LI1;
~SET GND LI2;
~LI1=CH1;
~LI2=CH2;
*****************RESISTOR R15 ************
~CALC RES="R15";
~TEST VOLT LI1 MAX=2, TIM=10 LO=0.3 HI=1;
```

If the measured voltage is out of range, the resistor is in error and the the subroutine RES_ERR will be executed
~LI1;
**********RESISTOR R27**********
~LI1=CH3;
~CALC RES="R27";
~TEST VOLT LI1 MAX=2, TIM=10 LO=0.3 HI=1;
If the measured voltage is out of range, the resistor is in error and the subroutine RES_ERR will be executed
~LI1;
~LI2;
~CLEAR GND;
~SET GNDREF OFF;
~CLEAR DR1;
~CURSOR 20,12;
~COLOR 0,15;
~WRITE "MEASURED VALUE: "AR;
~PAUSE 2;
ENDTEST;

~OR Instruction ~OR

Compiled	Run-time
	X

FUNCTION: Calculates the OR between the contents of the AR register and the specified argument.

SYNTAX: **~OR** value

value

It can be expressed as:

- a constant
- an INTEGER type runtime variable

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR1,VAR2;
DECLARE RUNTIME FLOAT VAR;

START PRG1 STATIC;
/;
~CALC VAR1=3;
~LOAD VAR1;
~OR B01101001;
~STORE VAR;
~WRITE "OR of AR : ",VAR;
/;
~PAUSE 1;
/;
~CALC VAR1=0X55;
~CALC VAR2=0X0F;
~LOAD VAR1;
~OR VAR2;
~STORE VAR;
~WRITE "OR of AR : " VAR;
/;
~PAUSE 2;
/;
ENDTEST;
```

~PAUSE **Instruction** **~PAUSE**

Compiled	Run-time
	X

FUNCTION: Executes a pause in the execution of the program, displaying the value, if specified, and waits for a keyboard entry by the operator.

SYNTAX: **~PAUSE** [value]

value Value displayed in the Pause Message box of the Static Terminal window when the ~PAUSE instruction is executed. The value can be expressed as:

- a constant
- a runtime variable INTEGER, FLOAT, STRING type
- a string of characters enclosed by quotation marks with maximum length of 80 characters

SEE ALSO: ~DELAY

EXAMPLE: EXAMPLE

```
DECLARE RUNTIME INTEGER CAN;
DECLARE RUNTIME FLOAT VAR1;
START PRG1 STATIC;
/;
~CALC CAN=12;
~CURSOR 60,20;
~WRITE CAN;
~PAUSE;
~CALC CAN=CAN+1;
/;
~CURSOR 60,21;
~WRITE CAN;
~PAUSE;
/;
~CALC CAN=CAN+1;
~CALC VAR1=3.56;
~PAUSE VAR1;            !Displays the contents of the
                        VAR1 variable in the Pause
                        Message box and executes
                        a pause!
/;
~PAUSE 11111;           !Displays the constant and
                        executes a pause!
/;
~PAUSE "PROBE SU U4-6";     !Displays the string and
                            executes a pause!
/;
```

```
ENDTEST;
```

~PCT Instruction ~PCT

☛

Module	Compiled	Run-time
HV		**X**

FUNCTION: Controls the closing/opening of a PCT contact of the High Voltage module.

SYNTAX: **~PCT** =[*,][*]contact

contact: Indicates the PCT contact to close/open. It is an integer variable from 1 to 144. ***Example:*** ~PCT=*17 is the first contact of the second board MSX (MXS2). The contact number can be expressed as:

[*,] Indicates the INCREMENTAL way of closing/opening contacts. That is, the previous connection is maintained and on this the actions specified in the current ~PCT instruction are executed. ***Example:*** ~PCT=*15 means that the contact 15 is closed while the opening/closing situation of the other contacts remains unchanged.

[,] If instead of the "*" character no character is specified, this indicates the NORMAL mode of closing/opening contacts. That is the specified contact or contacts in the current ~PCT instruction are closed, while the ones not specified are opened. ***Example:*** ~PCT=15 means the contact 15 is closed while the other ones are opened. If the current ~PCT instruction includes the closing of contacts, which are already closed, the status of these contacts remains the same.

[*] Indicates that the specified contact must be OPEN.

[ ] If no character is specified in place of the "*" character, this indicates that the specified contact must be CLOSED.

NOTE: If the value of the channel is specified on via a variable, any errors will come out during execution and not during compilation.

~PLn Instruction ~PLn

Module	Compiled	Run-time
HV		**X**

FUNCTION: Instruction for the High Voltage/High Voltage Lite module. Controls the closing or the opening of a channel of the HV module, on the indicated internal line. The internal lines are PL1, PL2, PL3, and PL4.

SYNTAX: **~PLn** =[*,][*]channel

n — Indicates the number of the internal line:

1 = PL1
2 = PL2
3 = PL3
4 = PL4

channel — Indicates the channel or channel to be connected/disconnected from the specified line. The channel can be:

- a single channel
- a group of channels
- a FLOAT type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

[*,] — Indicates the INCREMENTAL way of connecting/disconnecting channels. That is, the previous connection is maintained and on this the actions specified in the current ~PL instruction are executed. ***Example:*** ~PL1=*15 means that the channel 15 is connected on the internal line PL1 while the situation of connection/disconnection of the other channels remains unchanged.

[,] — If instead of the "*" character no character is specified, this indicates the NORMAL mode of connecting/disconnecting contacts. That is the specified channels in the current ~PCT instruction are connected, while the ones not specified are disconnected. ***Example:*** ~PL1=15 means the channel 15 is connected to the internal PL1 while the other channels are disconnected. If the current ~PL instruction includes the connecting of channels, which are already connected, the status of these channels remains the same.

[*] — Indicates that the following channel in the instruction, must be DISCONNECTED from the indicated lines.

[ ] — If no character is specified in place of the "*" character, this indicates that the following channel in the instruction must be CONNECTED on the indicated lines.

NOTE: If the value of the channel is specified on via a variable, any errors will come out during execution and not during compilation.

EXAMPLE:

```
DECLARE RUNTIME INTEGER CH;
DECLARE CHANNEL A3=65;
.
START PRG1 STATIC;
/;
.
.
~PL2=29;          Connects channel 29 to line 2 and
                  disconnects all the other channels
                  (normal connection mode)
~PL3=1,4,6,8/32;  Connects channels 1,4,6, and from 8 to 32 to
                  line 3 and disconnects the other channels
                  (normal connection mode)

~PL4=1-3-5-7;     Connects channels 1,3,5,7 to line 4 and
                  disconnects all the other channels

~PL2;             Disconnects all the channels from line 2

~CALC CH=15;
~PL1=CH;          Connects channel 15 to line 1 disconnects the
                  other channels

~PL4=*,3          Connects channel 3 to line 4 while
                  the connections of the other channels remain
                  unchanged (incremental connection mode)

~PL4=*,*88        Disconnects channel 88 from line 4
                  while the connections of the other channels
                  remain unchanged (incremental connection
                  mode)

~PL4=*,TRICH;

~PL3=A3;          Connects channel 65 defined with the instruction
                  DECLARE CHANNEL to line 3 and disconnects
                  the other channels (normal connection mode)
```

~PRINT **Instruction** **~PRINT**

Compiled	Run-time
	X

FUNCTION: Outputs the specified value or values to a printer.

SYNTAX: **~PRINT** value1,value2,...

value1,value2,... The value to be printed can be expressed as:

- a constant
- a runtime variable INTEGER, FLOAT, STRING type
- a string of characters

SEE ALSO: ~FORMAT ~CURSOR

EXAMPLE: EXAMPLE 1

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE RUNTIME FLOAT VAR1;
DECLARE RUNTIME INTEGER CAN;
DECLARE RUNTIME INTEGER R12,R13;
DECLARE RUNTIME STRING RISULT;

START PRG1 STATIC;
/;
~IH S1;
/;
~IL S1 S2;
/;
~CALC VAR1=33.56;
~CALC RISULT="PASS";
~CALC CAN=256;
/;
~PRINT 12;          !Prints a constant!

/;
~PRINT VAR1;        ! Prints a float runtime  variable!

/;
~PRINT "CHANNEL NR. ",CAN;      !Prints a string and an
                                 integer runtime variable!

/;
~PRINT RISULT;              !Prints a string type runtime
                             variable!
/;
```

```
~CALC R12=1;
~CALC R13=3;
~PRINT "REVISION SW = " R12 ".",R13;
/;
IH S2;
/;
/;
IH S1;
/;
ENDTEST;
```

EXAMPLE 2

The VAR1 variable will be printed ten times, incrementing its value starting from 10.

```
DECLARE RUNTIME INTEGER VAR1;

START PRG1 STATIC;
/;
~CALC VAR1=9;
~DO 10;
~CALC VAR1=VAR1+1;
~PRINT VAR1;
~DONE;
/;
ENDTEST;
```

~PROTECTION **Instruction** **~PROTECTION**

☛

Module	Compiled	Run-time
HV		**X**

FUNCTION: Executes the reading on the general protection (not the ones of the channel boards) of the High Voltage module. If the protection is active, the AR register is worth 1, otherwise it is worth 0.

SYNTAX: **~PROTECTION**

SEE ALSO: Instructions of the High Voltage module.

EXAMPLE:

```
START LETTURA STATIC;
~PROTECTION;
~IF AR=1;
~WRITE "PROTECTION ON THE HIGH VOLTAGE MODULE: ACTIVE ";
~ELSE;
~WRITE "\CR WARNING!!";
~WRITE "\CR PROTECTION ON THE HIGH VOLTAGE MODULE: NOT ACTIVE! ";
~ENDIF;
~PAUSE;
ENDTEST;
```

~PULL UP/DOWN — Instruction — ~PULL UP/DOWN

☛

Module	Compiled	Run-time
F40		**X**

FUNCTION: Connects/disconnects the 1KΩ pull-up or pull-down resistor to the output channels. The pull-up is to the high voltage threshold of the channel. The pull-down to the low one.

SYNTAX: **~PULL parameter** [*,]channel

or

~PULL parameter [*,]channel **parameter** [*,]channel

parameter — This parameter defines if the resistor is a pull-up or pull-down. It can accept the following values:

UP
DOWN

The UP DOWN parameters can also be specified in succession, in this case the channel in question will be connected both the pull-up and the pull-down resistor. Refer to the example.

[*,] — Indicates the INCREMENTAL way of connecting/disconnecting channels. That is, the previous connection is maintained and on this the actions specified in the current ~PULL instruction are executed.

[,] — If instead of the "*" character no character is specified, all the channels of the system are disconnected from the pull-up/down resistor while the ones specified are connected to the new value, if specified.

channel — Indicates the channel or the channels to be connected/disconnected to the specified resistor. The channel can be:

- a single channel
- a group of channels
- a FLOAT type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUES: All the channels of the system are disconnected from the pull-up and pull-down resistors by default.

EXAMPLE:

~PULL UP 13;	Connects channel 13 to the 1KΩ pull-up resistor. All the other channels are disconnected from pull-up/down resistors.
~PULL DOWN *,23;	Connects channel 23 to the pull-down resistor and leaves all the other channels unchanged.
~PULL DOWN *,1-4/32;	Connects channels 1, 4 and from channel 6 to channel 32 to the pull-down resistor, leaves all the other channels unchanged.
~PULL;	Disconnects all the channels connected to pull-up/down resistors.
~CALC CH=15; ~PULL UP CH;	Connects channel 15 to the pull-up resistor. All the other channels are unchanged.
~PULL UP 15 DOWN 6-8;	Connects channel 15 to the pull-up resistor while channels 6 and 8 to the pull-down resistor. Disconnects the other channels that might be connected.
~PULL UP 15 DOWN 6-8 UP DOWN 7;	Channel 7 in this case will be connected both to the pull-up and pull-down resistor. Disconnects the other channels that might be connected.

~READ BUSn **Instruction** **~READ BUSn**

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Reads data from the digital bus of the USB module. The reading address is present on lines USER A0- USER-A7 used in the previous instruction ~SET USER_BUSWn while the data to read, is present on lines USER-D0- USER-D7.

SYNTAX: **~READ BUS**n variable

n Whole number from 1 to 4 indicating the USB module from where the reading must be done.

variable INTEGER type runtime variable on which the value of the data can be read.

SEE ALSO: ~SET USER_BUSWn

NOTE: The ~READ_BUSn instruction reads the data present on lines USER-D0 - USER-D7 at the specified address, on lines USER-A0 - USER-A7, from the previous instruction ~SET USER_BUSWn.

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR1;
START PRG1 STATIC;
/;
~SET USER_BUSW1 A=0XAA D=0XAF;    !Writing on the address 0XAA of the 0XAF data on the USB module nr. 1

~READ BUS1 VAR1;    !Reading of the data 0XAF in the VAR1 variable
/;
~IF VAR1=0XAF;
~WRITE "TEST OK";
~ELSE;
~WRITE "TEST NOK";
~ENDIF;
/;
ENDTEST;
```

~READ_CLOCK **Instruction**
~READ_CLOCK

Compiled	Run-time
	X

FUNCTION: Reads the number from a timer that starts at the beginning of the program. Executing two subsequently READ_CLOCK instructions and making the subtraction between the values, it is possible to measure the time between one event and another one. The timer increases after every ms so it is necessary to do the right conversion to obtain the expected unit of measurement.

SYNTAX: **~READ_CLOCK** value

value Can be expressed as an INTEGER or FLOAT type runtime variable.

SEE ALSO: ~READ_TIME, READ_DATE

EXAMPLE:

```
DECLARE RUNTIME INTEGER CLO, CLO1 I;
DECLARE RUNTIME FLOAT  CLO2;
START LETTURA STATIC;
~READ_CLOCK CLO;
~WRITE "\CR CLOCK= " CLO;
~FOR I=1 TO 23;
~IL D0;/;~IH D0;/;
~ENDFOR;
~READ_CLOCK CLO1;
~WRITE "\CR CLOCK= " CLO1;
~CALC CLO1=CLO1-CLO;
~WRITE " \CR CLOCKD= " CLO1;
~CAlC CLO2=CLO1/1000;
~WRITE " \CR CLOCKD= " CLO2;
~PAUSE;
ENDTEST;
```

~READ_DATE — Instruction — ~READ_DATE

Compiled	Run-time
	X

FUNCTION: Reads the current date of the PC.

SYNTAX: **~READ_DATE** value

value — It must be expressed as a STRING type runtime variable.

SEE ALSO: ~READ_CLOCK, READ_TIME

EXAMPLE:

```
DECLARE RUNTIME STRING DATS;
START LETTURA STATIC;
~READ_DATE DATS;
~WRITE "Date is: " DATS;
~PAUSE;
ENDTEST;
```

~READ_IEEE **Instruction** **~READ_IEEE**

Compiled	Run-time
	X

FUNCTION: Reads a number in float format from the IEEE bus and stores it in the specified variable. The partial error flag is activated if the instrument does not answer.

SYNTAX: **~READ_IEEE** variable

value Runtime variable FLOAT and STRING type.

SEE ALSO: ~SEND_IEEE

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE RUNTIME FLOAT VAR1;

START PRG1 STATIC ;
~IH S1;
/;
~READ_IEEE VAR1;        !Reads a float number and stores
                         it in the VAR1 variable!

/;
~PRINT VAR1;
~IL S1 S2;
/;
~IH S1;
/;
ENDTEST;
```

~READ_OPTOn — Instruction — ~READ_OPTOn

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION:

Reads the status of the opto-coupled inputs present on the specified USB module.

SYNTAX:

~READ_OPTOn variable

n

Whole number (from 1 to 4) indicating the USB module to read.

variable

INTEGER type runtime variable on which it is possible to read the opto-coupled inputs.

The least significant bit of the variable corresponds to the opto-coupled input 1.

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR1;
DECLARE CHANNEL CH1=45;

START PRG1 STATIC;
/;
~READ OPTO1 VAR1;
~IF VAR1= 0X06;          !In this case the signals 1,4,5,6,7,8
                         are worth zero while 2 and 3 are
                         worth 1.

~IL CH1;
/;
~ENDIF;
/;
~IF VAR1= 8;             !In this case the signals 1,2,3,5,6,7,8 are
                         worth zero while 4 is worth 1.

/;
~IH CH1;
/;
~ENDIF;
/;

ENDTEST;
```

~READ_SCR **Instruction** **~READ_SCR**

Compiled	Run-time
	X

FUNCTION: Interrupts program execution and waits for the operator to enter a value. A box appears on the static terminal, in the cursor position, and the operator can enter its value there. The number is stored in the specified variable.

SYNTAX: **~READ_SCR** variable

value Value read from the keyboard, this can be expressed as:

- a runtime variable runtime INTEGER type.
- a runtime variable runtime FLOAT type.
- a runtime variable runtime STRING type.

SEE ALSO: ~READ_IEEE

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE RUNTIME FLOAT VAR1;
DECLARE RUNTIME INTEGER IN1;
DECLARE RUNTIME STRING ST1;
CLOCK=50N;
START TREAD STATIC ;
~IH S1;
/;
~WRITE “Enter a number: ”;        !Read of a float number
                                  and stores it in the VAR1
                                  variable!
~READ_SCR VAR1;
/;
~WRITE”Enter a whole number: ”;   !Read of a whole number and
                                  stores it in the IN1 variable !
                                  ~READ_SCR IN1;
/;
~WRITE”Enter a string: ”;         !Read of a string and stores it
                                  in the ST1 variable !
~READ_SCR ST1;
/;
~IL S1 S2;
/;
~IH S1;
/;
ENDTEST;
```

~READ STAUS PW1…8 **Instrument** **~READ STATUS PW1…8**

☛

Module	Compiled	Run-time
SE2		**X**

FUNCTION: Reads the status of the indicated power supply. The measure result is stored in the AR register.

SYNTAX: **~READ STATUS PWn parameter**

Parameter Indicates:

V: voltage programmed on the power supply.
I: current programmed on the power supply
NOTE: The voltage and current value read is the one PROGRAMMED on the power supply . To obtain the EFFECTIVE values, use the ~MEAS instruction.

VMAX: Power supply max. voltage
IMAX: Power supply max. current
STATUS power supply status (0=off, 1=on)

NOTE: It is possible to read the status even if the power supply is OFF.

EXAMPLE:

```
~READ STATUS PW1 STATUS;
~CURSOR 10,10;
~WRITE "POWER SUPPLY STATUS ",AR;

~SET PW1 V=0.1 I=5 SENSEOFF ON INT PREPARE;
~SET PW3 V=5 I=2 SENSEOFF ON INT PREPARE;
~READ STATUS PW1 STATUS;
~CURSOR 10,10;
~WRITE " PW1 STATUS: 0=OFF 1=ON ",AR;
~READ STATUS PW1 V;
~CURSOR 10,20;
~WRITE "PW1 VOLTAGE",AR;
~READ STATUS PW1 I;
~CURSOR 10,30;
~WRITE "PW1 CURRENT",AR;
~SET PW3 ON;
~READ STATUS PW3 STATUS;
~CURSOR 10,50;
~WRITE "PW3 STATUS: 0=OFF 1=ON ",AR;
~READ STATUS PW1 V;
~CURSOR 10,60;
~WRITE "PW3 VOLTAGE",AR;
~READ STATUS PW1 I;
~CURSOR 10,70;
~WRITE "PW3 CURRENT",AR;
```

~READ_TIME **Instruction**
~READ_TIME

Compiled	Run-time
	X

FUNCTION: Reads the Personal Computer Time.

SYNTAX: **~READ_TIME** value

value Must be expressed as a STRING type runtime variable.

SEE ALSO: ~READ_CLOCK, READ_DATE

EXAMPLE:

```
DECLARE RUNTIME STRING TIMS;
START LETTURA STATIC;
~READ_TIME TIMS;
~WRITE "Time is: " TIMS;
~PAUSE;
ENDTEST;
```

~REPORT **Instruction** **~REPORT**

Compiled	Run-time
	X

FUNCTION: Displays the specified value or values in the TEST REPORTS window of the test environment. It is also reported the name of the cluster in question.

SYNTAX: **~REPORT** value1,value2,...

value1,value2,... The values can be expressed as:

- a constant
- a runtime variable INTEGER, FLOAT, STRING type
- a string of characters

SEE ALSO: ~FORMAT

NOTE: The report of this instruction is written automatically in the statistics files if the statistics mode is enabled.

The ~CURSOR and ~COLOR. instructions are not managed for the ~REPORT instruction.

Several arguments can be specified (***Example***: arg1=constant, arg2= variable runtime float) in the same ~REPORT instruction.

EXAMPLE: EXAMPLE 1

```
DECLARE RUNTIME STRING ERR;
DECLARE RUNTIME INTEGER REP;

START PRG1 STATIC;
/;
~CALC REP=STATO_T1&0X02;
~COMP REP LO=0 HI=0 ONPASS REP11;
~CALC ERR="TEST1: 24V AC";
 ~REPORT ERR;
LABEL=REP11;
/;
ENDTEST;
```

EXAMPLE 2

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE RUNTIME FLOAT VAR1;
DECLARE RUNTIME INTEGER CAN;
DECLARE RUNTIME INTEGER R13,R13;
DECLARE RUNTIME STRING RISULT;


START PRG1 STATIC ;
~IH S1;
/;
~IL S1 S2;
/;
~CALC VAR1=33.56;
~CALC RISULT="PASS";
~CALC CAN=256;
/;
~REPORT 12;                    !Report of a constant!
/;
~REPORT VAR1;                  !Report of a float runtime
                               variable!
/;
~REPORT "CHANNEL NR. " CAN;  !Report of  a string
                                 and an integer
                                 runtime variable!
/;
~REPORT RISULT;                !Report of a string type runtime
                               variable!
/;
~REPORT "REVISIONE SW = " R12 "." R13;
/;
~IH S2;
/;
/;
~IH S1;
/;
ENDTEST;
```

~RG Instruction ~RG

Compiled	Run-time
	X

FUNCTION: Reads the value of the group of channels and assigns it to the specified variable. The value is assigned to the variable in binary form. The first channel declared in the group corresponds to the least significant bit of the variable.

SYNTAX: **~RG** value = channel group

value — Indicates the destination variable.
This can be:

- an INTEGER type runtime variable

channel group — Indicates the group of channels to read. The group of channels can be expressed as:

- physical channel numbers
- channel names defined with the DECLARE CHANNEL instruction

The "/" character is used to indicate a contiguous group of channels and the "," or "-" character is used to indicate non-contiguous channels.

SEE ALSO: ~WG

NOTE: The ~RG instruction simply executes a read of the status of the channels without testing them, therefore no error is generated.

This instruction does not modify the status of the channel.

Driver or masked channels can also be read; the value which the channels assume in that moment will be passed on to the variable.

Up to 32 channels can be read with ~RG instruction..

EXAMPLE:

```
DECLARE CHANNEL D0=23;
DECLARE CHANNEL D1=24
DECLARE CHANNEL D2=25;
DECLARE CHANNEL D3=26
DECLARE CHANNEL D4=27;
```

```
DECLARE CHANNEL D5=28
DECLARE CHANNEL D6=29;
DECLARE CHANNEL D7=30;
DECLARE CHANNEL DATB=139;

DECLARE RUNTIME INTEGER VAR1,VAR2,VAR3;

START PRG1 STATIC;
/;
~RG VAR1=23,24,25,26,27,28,29,30;        !The least significant
                                          bit of the VAR1
                                          variable will contain
                                          the status of channel
                                          23, while the most
                                          significant bit will
                                          contain that of
                                          channel 30!

/;
~RG VAR2=D0,D1,D2,D3;
/;
~RG VAR3=DATB;
/;
ENDTEST;
```

~ROUT RSn **Instruction** **~ROUT RSn**

☛

Module	Compiled	Run-time
F40		**X**

FUNCTION: Connects/disconnects the output resistor between the driver channel and the output of the connector to isolate the channel from the circuit under test or adapt the impedance of the transmission line of the channel.

SYNTAX: **~ROUT RSn** [*,]channel

or

~ROUT RSn [*,]channel **RSn** [*,]channel

RSn: Parameter that programs the resistor value between the driver channel and the output connector. It can assume the following values:

RS0 or OPEN: disconnects the driver channel from the output connector
RS1: insert a 0Ω resistor
RS2: insert a 8Ω resistor
RS3: insert a 10Ω resistor
RS4: insert a 33Ω resistor

[*,]: Indicates the INCREMENTAL way of connecting/disconnecting channels. In the incremental mode, the previous connection is maintained and new connections/disconnections indicated in the current ~ROUT instruction are performed on this situation.

[,]: If, instead of the "*" character no character is specified, all the channels of the system are connected to RS1 (0Ω) while the ones specified are disconnected/connected to the new value, if specified.

channel: Indicates the channel or the channels to connect/disconnect to the specified resistor. The channel can be:

- a single channel
- a group of channels
- a FLOAT type runtime variable
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," character or the "-" character are used to indicated non-contiguous channels.

DEFAULT VALUES: By default, all the channels of the system are connected to RS1.

EXAMPLE:

~ROUT RS0 13; Connects the driver channel 13 to the sensor with a 0Ω resistor while all the other channels are connected RS1.

~ROUT RS2 13; Connects the driver channel 13 to the sensor with a 8Ω Ω resistor while all the other channels are connected RS1.

~ROUT RS2 *,1-4/32;Connects channels 1, 4 and from channel 6 to channel 32 with a 8Ω resistor and leaves all the other channels unchanged.

~CALC CH=15;
~ROUT RS2 CH; Connects channel 15 with a 8Ω resistor and connects all the other channels to RS1.

~ROUT RS0 15 RS3 6-8; Connects channel 15 to RS0 and channels 6 up to 8 to RS3, all the other channels are connected to RS1.

~SEND_IEEE **Instruction** **~SEND_IEEE**

☛

Compiled	Run-time
	X

FUNCTION: Sends the specified value on the IEEE bus.

SYNTAX: **~SEND_IEEE** value

value The value can be expressed as:

- a constant
- a STRING type runtime variable

SEE ALSO: ~READ_IEEE

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE RUNTIME FLOAT VAR1;
DECLARE RUNTIME INTEGER CAN;
DECLARE RUNTIME INTEGER R12,R13;
DECLARE RUNTIME STRING RISULT;

START PRG1 STATIC ;
~IH S1;
/;
~CALC VAR1=12.56;
~CALC RISULT="PASS";
~CALC CAN=557;
/;
~SEND_IEEE 11.22;          !Sends a constant!
/;
~SEND_IEEE PAROLA;         !Sends a string type
                           runtime variable!
/;
~IL S1 S2;
/;
/;
~IH S2;
/;
/;
~IH S1;
/;
ENDTEST;
```

~SET ALM **Instruction** **~SET ALM**

☛

Module	Compiled	Run-time
LAM		**X**

FUNCTION: Controls the opening or the closing of the ALM relays of the LAM module.

SYNTAX: **~SET ALM V**=value

value: Value indicating the relays situation. The value can be expressed in binary, decimal, hexadecimal, octal format and can be expressed as a constant, as a compiled variable, as a runtime variable or as an arithmetic expression. The binary codification of the value has to interpreted the following way:
The least significant bit (LSB) corresponds to the ALM relays on LINE 1 while the most significant bit (MSB) corresponds to the ALM relays on LINE 8.
The bit must be set to 1 to open the relays. Must be 0 to close them.

NOTE: The execution of the ~CLEAR ALL instruction puts the ALM relays in default position, that is all closed.

EXAMPLE: Writing ~SET ALM V =15 the relays situation is the following:
Decimal value: 15 Binary value: 0 0 0 0 1 1 1 1

ALM (LINE 1): open
ALM (LINE 2): open
ALM (LINE 3): open
ALM (LINE 4): open
ALM (LINE 5): closed
ALM (LINE 6): closed
ALM (LINE 7): closed
ALM (LINE 8): closed

~SET BUS **Instruction** **~SET BUS**

☛

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

Sends the controls specified in the parameters of the instruction, on the IEEE bus. Used to talk with external instrumentation on the IEEE bus.

SYNTAX:

~SET BUS parameters

parameters

DCL Device clear
LLO Local Lock Out
SPD Serial Poll Disable
SPE Serial Poll Enable

GET Group Execute Trigger
GTL Go to Local
SDC Selected Device Clear

MLA=n My Listen Address (n=0 to 30 decimal)
MTA=n My Talk Address (n=0 to 30 decimal)
UNL Unlisten
UNT Untalk

MSA: My secondary Address (n=0 to 30 decimal)

EXAMPLE:

```
MODULE TL STATIC;

DECLARE RUNTIME STRING AA,REPORT;
DECLARE RUNTIME INTEGER I;
DECLARE RUNTIME FLOAT
MISURA_VOLT,MISURA_FREQ,MEDIA,CORRENTE,ASSORBI
MENTOMAX;

START IEEE STATIC;

*****************************************************
* freq: measurement and store of the value into MISURA_FREQ
*variable
*":CONF:FREQ 40000, set to freq. Measurement, expected
*VOLTAGE =40000 RIS.=0.001

*****************************************************
~CLEAR BUS;
~SET BUS,UNT,UNL,MLA,3,MTA,0;
* UNT=UNTOLKEN that is: do not speak
* UNL=UNLISTEN that is: do not listen
* MLA=my listen address is 3, that is: when I listen, I use address 3
* MTA=my talk address is 0, that is: when I talk, I use address 0

~DO 5;
* BEEP
~SEND_IEEE "BEL";
```

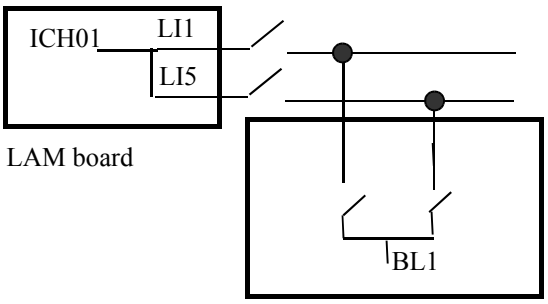
```
~DELAY 500M;
~DONE;
~SEND_IEEE "ID?";
*It asks to the instrument "Who are you?"
~SEND_IEEE ":INP1:FILT:LPAS:STAT 1";
* enable the low-pass filter (100KHz)
~SEND_IEEE ":INIT:CONT 1";
* enable the DC measurement
~PAUSE;

*****************************************************
SEND A MESURE REQUEST TO THE INSTRUMENT
*****************************************************

~SEND_IEEE ":MEAS:FREQ? 40000,1";
~SET BUS,UNT,UNL,MLA,0,MTA,3;
*The instrument is in listening mode, in fact MAL=0 and MTA=3
~READ_IEEE AA;
*The instrument answers" I'm a DMM, a VOLMETER,...."
~WRITE AA;
~PAUSE;
****************************************************
* Read of the measures value and store it in the "MISURA_FREQ"
variable
****************************************************
~READ_IEEE MISURA_FREQ;
* It performs the frequency measurement
~WRITE "READ VALUE (32758Hz-32778Hz) =
"MISURA_FREQ," Hz";
~COMP MISURA_FREQ LO=32758 HI=32778 ONPASS JUMP1;
~CALC FAIL=1;
~REPORT " ON THE QUARTZ FREQUENCY";
LABEL JUMP1;
~CURSOR CR;
ENDTEST;
ENMODULE;
```

~SET BUS_LINE SERV value **Instruction** **~SET BUS_LINE SERV value**

Module	Compiled	Run-time
SE2		**X**

FUNCTION:

Programs the connections of the LAM lines (from LI1 up to LI8) through the SE2 module. It is useful when the ALM relays of the LAM module are opened (therefore lines from LI1 to LI8 are no longer connected in the following way LI1-LI5, LI2-LI6, LI3-LI7, LI4-LI8). The ~SET BUS_LINE instruction is used to restore the connection (LI1-LI5, etc) through the SE2 module.



SYNTAX:

~SET BUS_LINE SERV value

value

The value can be:

OPEN: opens all lines from LI1 TO LI8 (0XFF)

CLOSE: closes all lines from LI1 TO LI8 (0X00)

V=value:
the situation of the lines can be specified through a value expressed in decimal notation where, if the value is converted in hexadecimal, the bit set to 0 means CLOSED line, the bit at 1 means OPEN line. The least significant bit (LSB) corresponds to line LI1. The most significant bit (MSB) corresponds to line LI8.
This value can also be expressed through an integer type RUNTIME variable (DECLARE RUNTIME INTEGER).

EXAMPLE:

EXAMPLE 1 Connection of LL1 to BL1
START PDD1 STATIC;
~SET BUS_LINE SERV V=254
ENDTEST;

The value 254 corresponds to 1 1 1 1 1 1 1 0 in hexadecimal notation where the LSB at 0 (CLOSED) is referred to line LI1 connected therefore with the BL1 signal of the SE2 module.

EXAMPLE 2 Connection of LL1 and LI5 to BL1 through the value in the H23 variable.

```
DECLARE RUNTIME INTEGER H23;
START PDD2 STATIC;
~CALC H23=238;
~SET BUS_LINE SERV V=H23;
ENDTEST;
```

The value 238 corresponds to 1 1 1 0 1 1 1 0 in hexadecimal notation where the bit at 0 (CLOSED) is referred to line LI1 and to line LI5 that are connected together with signal BL1 of the SE2 module.

~SET CKGEN **Instrument** **~SET CKGEN**

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

Programs the CKGEN instrument (clock generator) present on the LAM module. The instrument has the following characteristics:
Nr. Of impulses per step: from 1 to 32767
Period: from 500ns to 60s step of 500ns
Clamp current: 500mA
Output voltage: 5V
Rise time: 30ns
Fall time: 30ns

SYNTAX:

~SET CKGEN LI3 TIM= value NUM= value [CKN] [PREPARE] [START]

LI3

Connects the output of CKGEN to line 3 of the LAM module.

TIM=value

Indicates the time of the measure expressed in ms. The accepted range is :

0.0005ms ≤ TIM ≤ 60000ms

value can be expresses as a constant, a compiled variable, a run-time variable or an arithmetic expression.

NUM=value

Value in decimal format indicating the number of output pulses from the clock generator. The value can be expressed as a constant, as a compiled variable, as a runtime variable or as an arithmetic expression.

CKN

Indicates the polarity of the first edge of the clock signal. If this parameter is specified, the first will be a FALLING edge, if not, it will be a RISING one.

PREPARE

Indicates when the CKGEN will start to generate pulses. If the parameter is written, the CKGEN will be active on the next ~MEAS or ~TEST instruction. If PREPARE is not written, the CKGEN is active immediately.

START

Indicates that the CKGEN will be active immediately

DEFAULT VALUE:

If the LI3 parameter is not written, the instrument is disconnected.
If the TIM=… parameter is not written, by default TIM=0.
If the NUM=… parameter is not written, by default NUM=0.

EXAMPLE:

~SET CKGEN NUM=2,TIM=10,LI3;
It programs the CKGEN to generate 2 pulses with a period of 10ms each. It connects the instrument on line 3
~SET CKGEN NUM=100,TIM=1,LI3;
It programs the CKGEN to generate 100 pulses with a period of 1ms each. It connects the instrument on line 3

~SET DR1 **Instrument** **~SET DR1**

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

Programs the DR1 instrument (voltage/current generator) present on the LAM module. The instrument shows the following characteristics:

Voltage generator

Tension from -50V to +50V step of 50mV
from -10V to +10V step of 5mV

Current clamp 1mA, 10mA, 100mA, 200mA, 300mA, 400mA, 500mA

Current generator

Current from 0 to 100µA step of 25nA
from 100µA to 1µA step of 250nA
from 1mA to 10mA step of 2.5µA
from 10mA to 100mA step of 25µA

Voltage clamp from -5V to +5V step of 50mV
from -1V to +1V step of 0.5mV

SYNTAX:

~SET DR1 VL IL LI1 [PULSE] V=value I=value

VL Programs the instrument as current generator. The clamp voltage will be expressed by the parameter V=value.

IL Programs the instrument as voltage generator. The clamp current will be expressed by the parameter I=constant or runtime variable or arithmetic expression.

LI1 Connects the output of DR1 to line 1 of the LAM.

PULSE It defines when the DR1 will be active. If the parameter is written, the DR1 will be active on the next ~MEAS instruction and it will be active for the time indicated by TIM. If PULSE is not written, DR1 will be active on the next ~MEAS instruction and will remain active (until there is a CLEAR DR1 instruction).

V=value DR1 output voltage, expressed in Volt. The value can be expressed as a constant, a compiled variable, a run-time variable or an arithmetic expression

I=value DR1 output current, expressed in mA. The value can be expressed as a constant, a compiled variable, a run-time variable or an arithmetic expression

DEFAULT VALUE:

If the parameters VL or IL are not specified, DR1 is programmed as current generator (the VL parameter is active).

If the LI1 parameter is not specified, the instrument is not connected to the lines.

If the PULSE parameter is not specified, DR1 becomes active on the next ~MEAS instruction and remains active.

If the V=... parameter is not specified, V=0 by default.
If the I=... parameter is not specified, I=0 by default

NOTE: The DR1 instrument must be programmed in voltage clamp **or** current clamp, never in both.

EXAMPLE:

```
~SET DR1 V=TRIPOSV,I=TEMP0,VL,LI1;
~SET DR1 V=0,I=1,VL,LI1;
~SET DR1 V=1,IL,I=1,LI1,PULSE;
```

~SET DR2 **Instrument** **~SET DR2**

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

Programs the DR2 instrument (voltage generator) present on the LAM module. The instrument shows the following characteristics:

Voltage from -10.240V to +10.235V step of 5mV

Clamp current 1A max.

SYNTAX:

~SET DR2 LI2 LI3 LI4 [PULSE] RG V=value

LI2 Connects DR2 output to line 2 of the LAM.

LI3 Connects DR2 output to line 3.

LI4 Connects DR2 output to line 4.

PULSE It defines when the DR2 will be active. If PULSE is written, DR2 becomes active on the next ~MEAS instruction and remains active for all of the duration indicated by TIM If PULSE is not written, DR2 will be active immediately and remains active.

RG Connects a 100Ω resistor in series to the DR2 output.
NOTE: This parameters is only for LI4.

V=value DR2 output, expressed in Volt. Value can be expressed as a constant, a compiled variable, a run-time variable or an arithmetic expression.

DEFAULT VALUE:

If LI2, LI3, LI4 parameters are not specified, Dr2 is disconnected from the lines.

If PULSE is not spiced, DR2 becomes active immediately after it is programmed and remains active.

If the RG parameter is not specified, the 100Ω resistor is not inserted in series with the generator

If the parameter V=... is not specified, V=0 by default.

EXAMPLE:

```
~SET DR2 V=0,LI2;
~SET DR2 V=5,LI3,PULSE;
~SET DR2 V=VAL1,LI4,RG;
```

~SET EB0 **Instruction** **~SET EB0**

☛

Module	Compiled	Run-time
LAM		**X**

FUNCTION: Opens or closes the External bus (EB0) relays present on the LAM module. The External Bus has got 16 lines.

SYNTAX: **~SET EB0 V**=value

value: Value that indicates the relay status. The value can be expressed in binary, decimal, hexadecimal or octal format. The value can be a constant, a compiled variable, a run-time variable or an arithmetic expression.
The least significant bit (LSB) of the value, corresponds to the EB0 relay on LINE 1.
The most significant bit (MSB) of the value, corresponds to the EB0 relay on LINE 16
To OPEN the specified relay, the bit must be set to 1. To CLOSE it, to 0.

NOTE: A ~CLEAR ALL instruction sets all EB0 relays to the default condition that is: all relays closed.

EXAMPLE: If we write ~SET EB0 V =0x0ff0, the situation of the relay is:

Binary code: 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0

EB0 (LINE 1): closed
EB0 (LINE 2): closed
EB0 (LINE 3): closed
EB0 (LINE 4): closed
EB0 (LINE 5): open
EB0 (LINE 6): open
EB0 (LINE 7): open
EB0 (LINE 8): open
EB0 (LINE 9): open
EB0 (LINE 10): open
EB0 (LINE 11): open
EB0 (LINE 12): open
EB0 (LINE 13): closed
EB0 (LINE 14): closed
EB0 (LINE 15): closed
EB0 (LINE 16): closed

~SET EB1 **Instruction** **~SET EB1**

☛

Module	Compiled	Run-time
LAM		**X**

FUNCTION: Opens or closes the External bus (EB1) relays present on the LAM module. The External Bus has got 16 lines.

SYNTAX: **~SET EB1 V**=value

value Value that indicates the relay status. The value can be expressed in binary, decimal, hexadecimal or octal format. The value can be a constant, a compiled variable, a run-time variable or an arithmetic expression.
The least significant bit (LSB) of the value, corresponds to the EB1 relay on LINE 1.
The most significant bit (MSB) of the value, corresponds to the EB1 relay on LINE 16
To OPEN the specified relay, the bit must be set to 1. To CLOSE it, to 0.

NOTE: A ~CLEAR ALL instruction sets all EB1 relays to the default condition that is: all relays closed.

EXAMPLE: If we write ~SET EB1 V =0x0ff0, the situation of the relay is:

Binary code: 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0

EB1 (LINE 1): closed
EB1 (LINE 2): closed
EB1 (LINE 3): closed
EB1 (LINE 4): closed
EB1 (LINE 5): open
EB1 (LINE 6): open
EB1 (LINE 7): open
EB1 (LINE 8): open
EB1 (LINE 9): open
EB1 (LINE 10): open
EB1 (LINE 11): open
EB1 (LINE 12): open
EB1 (LINE 13): closed
EB1 (LINE 14): closed
EB1 (LINE 15): closed
EB1 (LINE 16): closed

~SET ELM **Instruction** **~SET ELM**

☛

Module	Compiled	Run-time
LAM		**X**

FUNCTION: Opens or closet the ELM relays present on the LAM module.

SYNTAX: **~SET ELM V**=value

value: Value that indicates the relay status. The value can be expressed in binary, decimal, hexadecimal or octal format. The value can be a constant, a compiled variable, a run-time variable or an arithmetic expression.
The least significant bit (LSB) of the value, corresponds to the ALM relay on LINE 1.
The most significant bit (MSB) of the value, corresponds to the ALM relay on LINE 8
To OPEN the specified relay, the bit must be set to 1. To CLOSE it, to 0.

NOTE: A ~CLEAR ALL instruction sets all ALM relays to the default condition that is: all relays open.

EXAMPLE: If we write ~SET ELM V=15, the situation of the relays is:
Decimal value : 15 — Binary value: 0 0 0 0 1 1 1 1

ELM (LINE 1): open
ELM (LINE 2): open
ELM (LINE 3): open
ELM (LINE 4): open
ELM (LINE 5): closed
ELM (LINE 6): closed
ELM (LINE 7): closed
ELM (LINE 8): closed

~SET EMX instrument=line **Instruction** **~SET EMX instrument=line**

☛

Module	Compiled	Run-time
SE2/EXL		**X**

FUNCTION:

It programs the connection between the external instruments and the board line of the SE2 and EXL modules. Instruments are physically on the SE2 and EXL connectors and they are connected to the board lines through the EMX matrix. Once the instruments are connected to the board lines, it is possible to connect them to the LAM lines (LI1-LI4) with the ~SET BUS_LINE SERV

External instruments are:

UNIVERSAL COUNTER1 (UCT1)
UNIVERSAL COUNTER2 (UCT2)
FUNCTION GENERATOR (FNT1)
MULTIMETER (DMM1, DMM2, DMMC)

SINTAX:

~SET EMX instrument=line

instrument= line

Indicates the instrument and line according to the following table:

INSTRUMENT	LINE	Module
UCT1+	FLI3 FLI4	SE2
UCT1-	FLI3 GND	SE2
UCT2+	FLI3 FLI4	SE2
UCT2-	FLI3 GND	SE2
FNT1+	FLI1 FLI2	SE2
FNT1-	FLI3 GND	SE2
DMM1+	FL1 FL3 FL4	SE2
DMM1-	FLI3 FLI4	SE2
DMM2+	FLI2 FLI3 FLI4	SE2
DMM2-	FLI3 FLI4	SE2
DMMC+	FLI3 FLI4	SE2
PORT1…64	From FLI1 to FLI8, REF1, REF2, LI1 (LI1 closes FLI1 and FLI5)	EXL

Where:

+ indicates the positive pin of the instrument
- indicates the negative pin of the instrument
DMMC indicates the current line of the DMM.

Instruments are connected through appropriate connectors on the SE2 ed EXL modules as illustrated below:

SE2 CONNECTORS PINOUT.

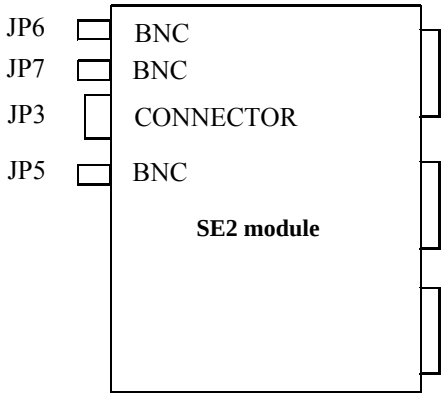
.

BNC CONNECTOR	INSTUM.
J5 CENTRAL	FNT+
J5 SHIELD	FNT-

BNC CONNECTOR	INSTUM.
J7 CENTRAL	UCT2+
J7 SHIELD	UCT2-

BNC CONNECTOR	INSTUM.
J6 CENTRAL	UCT1+
J6 SHIELD	UCT1-

CONNECTOR	INSTUM.
JP3-1	DMM2-
JP3-2	DMMCURR
JP3-3	DMM1-
JP3-4	DMM2+
JP3-5	DMM1+



EXAMPLE OF CONNECTION BETWEEN INSTRUMENTS AND EXL MODULE

port num.	board num	i/o/b	bnc	Instrument name	Signal name	Shield name
1	1	B	SI	PROBE_A	PROBE-SIGNAL	PROBE-GND
2	1	O	SI	FNT (FLUKE PM5138)	OUTPUTL	FNT-GND
3	1	O	SI	FNT (FLUKE PM5138)	TTL	FNT-GND
4	1	I	SI	FNT (FLUKE PM5138)	TRIGGER	FNT-GND
5	1	I	SI	SCOPE (TEKTRONIX TDS3032)	CH1	OSC-GND
6	1	I	SI	SCOPE (TEKTRONIX TDS3032)	CH2	OSC-GND
7	1	I	SI	SCOPE (TEKTRONIX TDS3032)	EXT TRIG	OSCGND
8	1	O	SI	HVDP1 (H.V. DIFF. PROBE)	OUTPUT	HVDP-GND
9	1	O	SI	HVDP2 (H.V. DIFF. PROBE)	OUTPUT	HVDP-GND
10	1	I	SI	HVDP2 (H.V. DIFF. PROBE)	IN +	N.U
11	1	I	NO	HVDP2 (H.V. DIFF. PROBE)	IN -	N.U
12	1	I	NO	DMM (AGILENT 34401A)	DMM1 +	N.U.
13	1	I	NO	DMM (AGILENT 34401A)	DMM1 -	N.U
14	1	I	NO	DMM (AGILENT 34401A)	DMM2 +	N.U.
15	1	I	NO	DMM (AGILENT 34401A)	DMM2 -	N.U.
16	1	I	NO	DMM (AGILENT 34401A)	DMMC +	……
17	2	I	SI	AMP (NF CORP. HSA4011)	INPUT A	A-GND
18	2	I	SI	AMP (NF CORP. HSA4011)	INPUT B	B-GND
19	2	O	.NO	USER_LOAD1 (PT-USERB000-00)	LOAD1 +	N.U.
20	2	O	NO	USER_LOAD1 (PT-USERB000-00)	LOAD1 -	N.U
21	2	O	NO	USER_LOAD2 (PT-USERB000-00)	LOAD2 +	N.U.
22	2	O	NO	USER_LOAD2 (PT-USERB000-00)	LOAD2 -	N.U
23	2	O	NO	USER_LOAD3 (PT-USERB000-00)	LOAD1 +	N.U.
24	2	O	NO	USER_LOAD3 (PT-USERB000-00)	LOAD1 -	N.U.
25	2	O	NO	USER_LOAD4 (PT-USERB000-00)	LOAD2 +	N.U.
26	2	O	NO	USER_LOAD4 (PT-USERB000-00)	LOAD2 -	N.U
27	2	O	NO	DC SIGNAL SOURCE 80V 0,6A	SENSE +	N.U.
28	2	O	NO	DC SIGNAL SOURCE 80V 0,6A	SENSE -	N.U.
29	2	O	NO	TERMOCOUPLE +		N.U
30	2	O	NO	TERMOCOUPLE -		
31	2	O	NO	FREE		
32	2	O	NO	FREE		

NOTE: Short circuits between instruments are not reported during program compilation, that is:

~SET EMX UCT1+=FLI3 UCT1-=FLI3 Doesn't give any report.

EXAMPLE:

Connection of the Universal Counter to Board line 3 and system GND.

~SET EMX UCT1+=FLI3 UCT1-=GND;

Connection of the Digital multimeter to Board line 1 and Board line 4

~SET EMX DMM1+=FLI1 DMM1-=FLI4;

~SET Esn PLn IP= Instruction ~SET Esn PLn IP=

☛

Module	Compiled	Run-time
HV		**X**

FUNCTION: Instruction for the High Voltage/High Voltage Lite module. It connects a resource to the internal line indicated.

SYNTAX: **~SET ES**n **PL**n [**IP=**current value]

Esn indicates the number of the resource that must be connected. It is a number that can vary from 1 to 12 for the High Voltage module and from 1 to 4 for High Voltage Lite module.

PLn indicates the number of internal line where the resource must be connected. It is a number that can vary from 1 to 4.

[IP= current value] Indicates the maximum current threshold over which, the hardware protection of the HV module will activate.
Accepted values: from 25 to 5000, expressed in mA.

EXAMPLE: ~SET ES1 PL2 IP=1000;
Connection of resource 1 to line 2. The current threshold is 1000 mA.

~SET GND — Instrument — ~SET GND

☛

Module	Compiled	Run-time
LAM		**X**

FUNCTION: Connects the instrument reference from line 2 of the LAM module.

SYNTAX: ~SET GND LI2;

LI2 Connects the reference on line 2.

DEFAULT VALUES: If the LI2 parameter is not written, the reference is disconnected from line 2.

EXAMPLE:

~SET GND LI2;

~SET GNDREF — Instruction — ~SET GNDREF

Module	Compiled	Run-time
SE2		**X**

FUNCTION: Opens/closes the GND_REF relay present on the SE2 module. This relay is used to refer both the power supplies and the UUT to a common ground.

SYNTAX: **~SET GNDREF Status**

Status — Indicates the relay status and can have the following values:

ON: the GNDREF relay is close when at least a power supply will be switched ON (with the ~SET PWn ON or ~SET PW ALL ON instruction).

OFF: Opens the GNDREF relay. It is therefore possible to open the GNDREF relay even if one or more power supplies are ON.

NOTE: The GND_REF relay opens automatically when all power supplies will be switched OFF.

DEFAULT VALUE: If the status field in the SET GNDREF instruction is not written, the GNDREF relay is closed by default when at least one power supply is switched ON.

If the GNDREF OFF instruction is NOT written, the relay is automatically open when all power supplies are switched OFF.

EXAMPLE: The GNDREF relay is closed when the power supply PW1 is switched ON and it is opened when the PW is switched OFF.

~SET GNDREF ON;
~SET PW1 V=5, I=1 SENSEOFF INT ON; — Here the GNDREF relay is closed.

~SET PW2 V=15 I=1 INT SENSEON ON; — PW2 is switched ON, the GNDREF relay is still closed

~SET GNDREF OFF;
~SET PW ALL OFF; — All Pws are switched off and the GNDREF relay is opened

NOTE: The GNDREF relays opened automatically when the PWs are switched Off even if the ~SET GNDREF OFF instruction is not written;

~SET GUA **Instrument** **~SET GUA**

☛

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

Programs the GUA instrument (guarding generator) present on the LAM module. The instrument shows the following characteristics:

Current	1A max
Voltage	from –10.240V to +10.235V

SYNTAX:

~SET LI1 LI3 EXT DR1

LI1 Connects the GUA output to line 1.

LI3 Connects the GUA output to line 3.

EXT Connects the GUA to the external line EXT. In this an external voltage drives the GUA and it puts a guard according to the external voltage supplied.

DR1 The GUA puts a guard according to the voltage on DR1.

DEFAULT VALUE:

If LI1 and LI3 parameters are not specified, the GUA is disconnected from the lines.

If EXT and DR1 parameters are not specified, by default, the GUA follows the voltage on DR1.

EXAMPLE:

~SET GUA DR1,LI3;
~SET GUA EXT,LI3;

~SET ISOLEV N= mV= ON ON **Instruction** **~SET ISOLEV N= mV=**

☛

Module	Compiled	Run-time
HV		**X**

FUNCTION: Instruction for the High Voltage/High Voltage Lite module. It programs a floating DC voltage reference in output to the module

SYNTAX: **~SET ISOLEV** [**N**= voltage ref. number] [**mV**= voltage value] **ON/OFF;**

[N= voltage ref. number] indicates the voltage reference numbers. It is a value that can vary from 1 to 20. The value can be expressed as a constant, a compiled variable, a run-time variable or an arithmetic expression.

[mV=voltage value] Indicates the voltage to program on the specified reference. Value can vary from 0 to 10000 and it is expressed in mV. The value can be expressed as a constant, a compiled variable, a run-time variable or an arithmetic expression.

ON activates the specified reference.

OFF disables the specified reference.

EXAMPLE: ~SET ISOLEV N=3 mV=5000 ON;

~SET ISORES N= OHM= Instruction ~SET ISORES N= OHM=

Module	Compiled	Run-time
HV		**X**

FUNCTION: Programs the two resistive loads on the High Voltage module.

SYNTAX: **~SET ISORES** [**N**= resistive loads] [**OHM**= value] **ON/OFF**

[N= resistive load] Integer number that can be 1 or 2 indicating the resistive load to program. The value can be expressed as a constant, a compiled variable, a run-time variable or an arithmetic expression.

[OHM=value] Resistive load value, expressed in OHM, it can vary from 1 to 65.000. The value can be expressed as a constant, a compiled variable, a run-time variable or an arithmetic expression.

ON it connects the resistive load to J16 connector of the High Voltage module.

OFF it disconnects the resistive load from J16 connector of the High Voltage module.

SEE ALSO: ~CLEAR ISORES

EXAMPLE: ~SET ISORES N=1 OHM=10500 ON;

~SET PMM PP= PN= MODE= AC/DC **Instruction** **~SET PMM PP= PN= MODE= AC/DC**

☛

Module	Compiled	Run-time
HV		**X**

FUNCTION: Connects the meter to the lines of the High voltage/High Voltage Lite module and programs it for a voltage/current measurements.

SYNTAX: **~SET PMM PP** = positive, **PN** = negative **MODE** = coupling

positive indicates where the positive pin of the instrument will be connected. The pin can be connected to the following points for a VOLTAGE measurement:

PL1N: line 1, neutral
PL1P: line 1, phase
PL2N: line 2, neutral
PL2P: line 2, phase
PL3N: line 3, neutral
PL3P: line 3, phase
PL4N: line 4, neutral
PL4P: line 4, phase
GND: Ground of the system
EXTP: external line, phase (*)
EXTN: external line, neutral (*)

(*) If the external line is a low voltage line it must be connected to the MHV connector, on 20 (BT_EXT_P) and 22 (BT_EXT_N) pins. If the external line is a high voltage line it must be connected to 1 AT_EXT_P and 11 AT_EXT_N

The positive pin of the meter can be connected to the following points for a CURRENT measurement:

I1: connection on PL1
I2: connection on PL2
I3: connection on PL3
I4: connection on PL4

negative indicates where the negative pin of the instrument will be connected. It follows the same rules explained before, for the positive pin

mode Indicates the measurement type. It can be:

RMS: RMS measurement (it measures the effective value, an RMS to DC converted is used).
DIR: direct voltage measure.

coupling	Indicates the coupling and can be: AC (it connects a capacitor on the meter to eliminate the possible waveform offset) DC
EXAMPLE: Voltage measurement	~SET PMM PP=PL1N, PN=PL1P MODE=DIR DC;
	~SET PMM PP=I2, MODE=RMS DC; Current measurement

~SET OPC **Instruction** **~SET OPC**

Module	Compiled	Run-time
LAM		**X**

FUNCTION: Driver the Open Collector signals of the LAM module

SYNTAX: **~SET OPC V**=value

value: Value that drives the Open Collector signals. The value can be in binary, decimal, hexadecimal or octal format. The value can be expressed as a constant, a compiled variable, a run-time variable or an arithmetic expression.
The least significant bit (LSB) of the value, corresponds to the OPC0 signal.
The most significant bit (MSB) of the value, corresponds to the OPC8 signal.
If the specified bit is set to 1, the Open Collector output will be 0.
If the specified bit is set to 0, the Open Collector output will be FLOATING.

NOTE: A ~CLEAR ALL instruction sets all Open Collector signals to a floating state.

EXAMPLE: If we write: ~SET OPC=B11110000, the Open collector signals will be:

OPC0: at zero
OPC1: at zero
OPC2: at zero
OPC3: at zero
OPC4: floating
OPC5: floating
OPC6: floating
OPC7: floating
OPC8: floating

~SET PUL **Instrument** **~SET PUL**

Module	Compiled	Run-time
LAM		**X**

INSTRUMENT:

Programs the resistive matrix present on the LAM module. The matrix shows the following characteristics:

Range

from 100Ω to 1.5KΩ step of 100Ω
from 2KΩ to 15KΩ step of 1KΩ
from 20KΩ to 150KΩ step of 10KΩ
from 200KΩ to 1.5MΩ step of 100KΩ

SYNTAX:

~SET PUL EXT UP DOWN OHM=value

EXT

The programmable resistor output is connected to the external line EXT so that it is possible to connect a pull-up/pull-down resistor externally to the LAM module.

UP

it programs a pull-up resistor connected to +5V.

DOWN

it programs a pull-down resistor connected to GND.

OHM=value

Value of the resistor expressed in K The value can be expressed as a constant, a compiled variable, a run-time variable or an arithmetic expression.

DEFAULT VALUES:

If EXT, UP and DOWN parameters are not written, the resistor is disconnected.

If the OHM=... parameter is not written, by default it is: OHM=0.

EXAMPLE:

```
~SET PUL UP,OHM=5;
~SET PUL DOWN,OHM=1;
~SET PUL EXT,OHM=2.2;
```

~SET PW1…8 **Instruction** **~SET PW1…8**

☛

Module	Compiled	Run-time
SER/SE2		**X**

FUNCTION: Programs the specified power supply

SYNTAX: **~SET PWn [ALL] V=voltage [I=current] [SENSE] [CONNECTION] [ON/OFF] [DLY=time] [PREPARE]**

PWn — Indicates the power supply to program where n can vary from 1 to 8 (from PW1 to PW8)

ALL — Switches all power supplies ON/OFF at the same time. The power supplies had to be prepared with the ~SET PWn and PREPARE option, previously.

V=voltage — indicates the voltage to program on the power supply. It is expressed in Volt. The value can be expressed as a constant or an INTEGER or FLOAT run-time variable. The voltage range varies according to the power supply mounted in the test system.

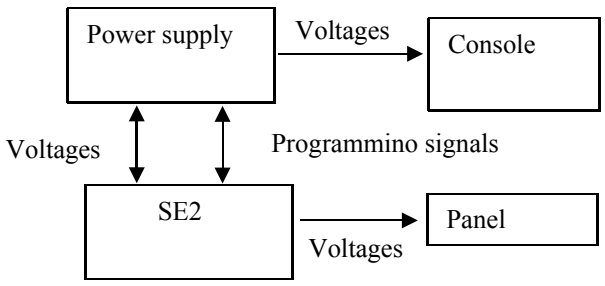
I=current — it programs the current limit on the power supply, expressed in Ampere. The value can be expressed as a constant or an INTEGER or FLOAT run-time variable. The current range varies according to the power supply mounted in the test system.

SENSE — Indicates the connection mode for the SENSE signals and can be:

SENSEON: connects the sense signals to the test fixture.

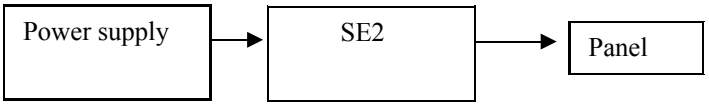
SENSEOFF: connects the sense signals internally to the SE2 module

CONNECTION — Indicates the connecting mode of the POWER SUPPY. Power supply are physically connected both to the system console (EXTERNAL mode), and internally to the SE2 module. From here, the power supply outputs are then connected to the system panel (INTERNAL mode). Connections are illustrated in the following figure.

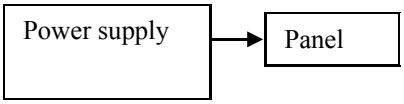


It is possible to choose the requested connection mode through a software command. (INT or EXT or INT/EXT).

INT o INT/EXT: Indicates an INTERNAL mode of connection:

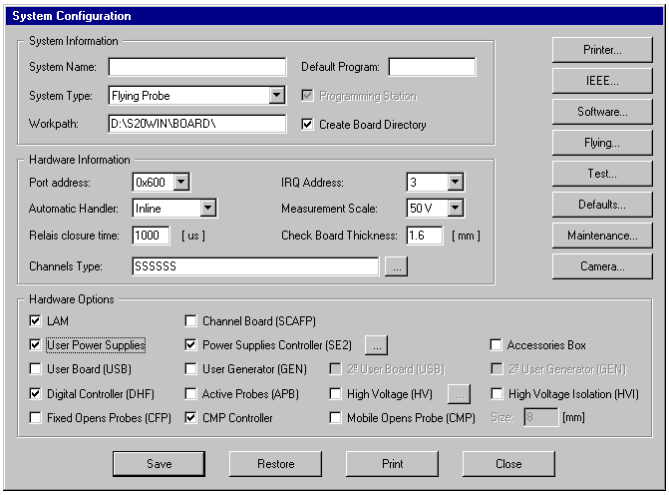


EXT o INT/EXT: Indicates an EXTERNAL mode of connection :

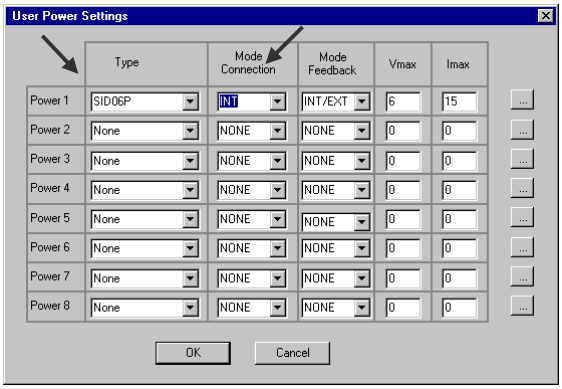


To choose the connection mode:

In the S20WIN environment, select the UTILITIES\Configuration menu. Here select the Power Supplies Controller (SE2) option and press the button beside as illustrated in figure:



In the User Power Settings dialog, select the power supply requested that is listed in the Type column. Select the requested connection that is listed in the Mode Connection column.



ON — Switches the power supply ON.

OFF — Switches the power supply OFF.

DLY=time

It programs a delay time that will be executed after the power supply is switched ON. The delay time is expressed in seconds

PREPARE

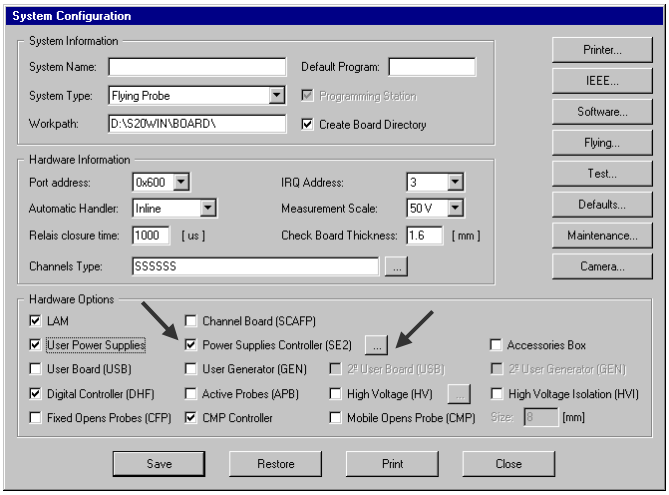
If this parameter is written, the specified power supply is only prepared but not switched ON/OFF. This operation can be executed later on in the test program and can be an individual operation (with the ~SET PWN ON/OFF instruction) or together with others, (with the ~SET PW ALL instruction)

ALL

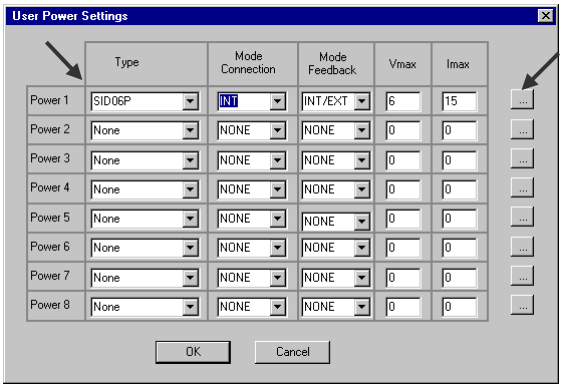
It switches more that one power supply ON/OFF. This parameter is usually used after the preparation of one or more power supply (with the PREPARE option). In this case, there is only one global delay time during switching ON/OFF and not subsequently delays.

DEFAULT VALUES:

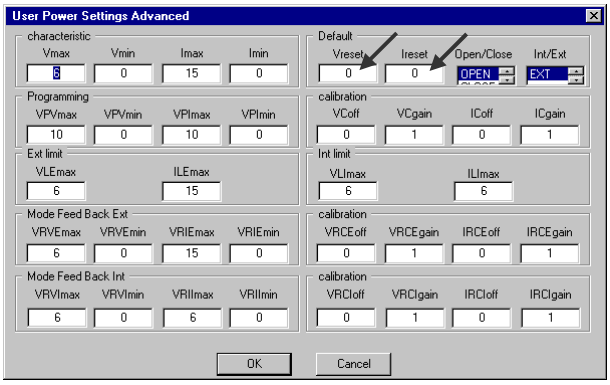
V=voltage and I=current:
they are set according to the type of power supply mounted in the test system. It is possible to check them in the following way: in the S20WIN environment, select the UTILITIES\Configuration menu. Here select the Power Supplies Controller (SE2) option and press the button beside as illustrated in figure:



In the User Power Settings dialog, select the power supply requested that is listed in the Type column and press the button beside.

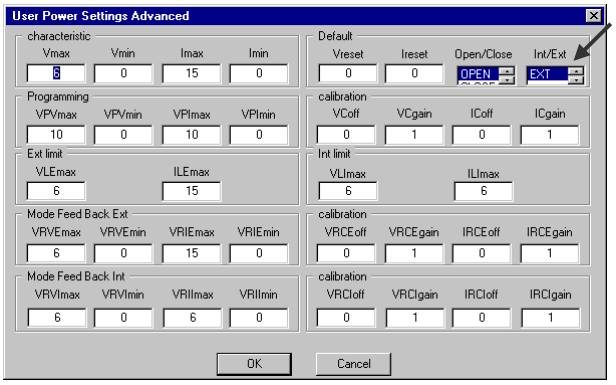


In the User Power Settings Advantage dialog, the Vreset and Ireset indicates the default values.



SENSEON: by default is active

INT: follow the same procedure explained before for the voltage and current default values.
In the User Power Settings Advantage dialog, the INT/EXT field indicates the default value.



OFF: by default is active
DLY: by default, 100ms for switching ON and 200ms for switching OFF.
PREPARE: by default is disabled.

NOTE: It is advisable to prepare the power supplies at the beginning of the program and to switch them ON/OFF together with the ~SET PW ALL ON/OFF instruction.

EXAMPLE:

START PDE1 STATIC;

~SET PW1 V=5 I=1 SENSEOFF INT ON;	PW1 ON to 5Vwith current limit of 1A
~SET PW1 V=5 I=1 OFF;	PW1 OFF
~SET PW3 V=5 I=1 PREPARE ON;	Preparation of the PW3 for switching ON. PW3 is still OFF.
~SET PW3 ON ;	PW3 ON

~SET PW1 V=5 I=1 PREPARE ON; ~SET PW4 V=10 I=1.2 PREPARE ON;	Preparation of PW1 and PW4.
~SET PW ALL ON DLY=1;	PW1 and PW4 are switched ON simultaneously. After switching ON, there is a wait time (delay) of 1s.
~SET PW ALL OFF;	PW1 and PW4 are switched OFF simultaneously.

~SET SERVICE_BUS **Instrument** **~SET SERVICE_BUS**

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

It programs the connection of the SERVICE BUS present on the LAM module. The service bus is composed of the following signals:

EXL3
EXL4
EXST
EXGUARD
EXPUL

SYNTAX:

~SET SERVICE_BUS LI3 LI4 EXS EXG PUL

LI3 Connects the EXL3 signal to line 3. Used to measure an external signal.

LI4 Connects the EXL4 signal to line 4. Used to measure an external signal.

EXS An external signal present on the EXST line is used as the strobe of the measure.

EXG Connects the guarding generator (GUA) on EXGUARD line of the service bus.

PUL Connects the resistive matrix (PUL) in external mode so that it is possible to have pull-up/pull-down resistors referred to the DR2 voltage (and not +5V or GND).

DEFAULT VALUES:

If LI3, LI4, EXG, PUL parameters are not written, the associated signals are disconnected from the lines.

If the EXS parameter is not written, the internal strobe is used by default

EXAMPLE:

~SET SERVICE_BUS LI3;
~SET SERVICE_BUS EXG;
~SET PUL UP OHM=1;
~SET DR2 V=7;
~SET SERVICE_BUS PUL; Here there is a 1KΩ pull-up resistor referred to 7V.

~SET USER_BUSWn **Instruction** **~SET USER_BUSWn**

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION:

Writes a data on the digital bus of the USB module. The address is on USER A0- USER-A7 lines while data is on USER-D0- USER-D7 lines. This busses are available in the USB breadboard area for customization.

SYNTAX:

~SET USER_BUSWn A=address D=data

n

Integer number from 1 to 4 that indicates the USB to write. (there can be up to 4 USB modules mounted in a test system)

A=address

Address from 0 to 255 that can be expressed as:

- integer number in decimal or hexadecimal format
- integer run-time variable

D=data

Data from 0 to 255 that can be expressed as:

- integer number in decimal or hexadecimal format
- integer run-time variable

SEE ALSO:

~READ BUSn
~CLEAR USER_BUSW

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR1;
START PRG1 STATIC;
/;
~SET USER_BUSW1 A=0X55 D=0X3A;      Writing of data 0X3A
at address 0X55 on USB number 1

~READ BUS1 VAR1;        !Reading of data 0X3A. Data is stored
                        into VAR1 variable.
/;
~IF VAR1=0X3A;
~WRITE "TEST PASS";
~ELSE;
~WRITE "TEST FAIL";
~ENDIF;
/;
ENDTEST;
```

~SET USER_DACn USER_DACn **Instruction** **~SET**

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Programs the 8 DAC present on the USB module.

SYNTAX: **~SET USER_DAC**n V=value

n Integer number between 1 and 32 indicating the DAC to program, on the USB module.

V=value DAC output voltage. Value between 0 and 255. The 0 value corresponds to a 0V voltage while 255 to 2.56V (end scale voltage). The value can be expressed in:

- decimal format 0-255
- hexadecimal format 0X00-0XFF

SEE ALSO: ~CLEAR USER_DAC

NOTE: On the USB module there is a DAC voltage reference of 2.56V. The 98 analog lines are then programmed from 0V to 2.56V.

EXAMPLE: ~SET USER_DAC1 V=0X64; It programs the DAC number 1 at 1V. The DAC is on USB number 1.

~SET USER_DAC27 V=150; It programs the DAC number 3 at 1.5V. The DAC is on USB number 4.

~SET USER_GEN **Instruction** **~SET USER_GEN**

☛

Module	Compiled	Run-time
GEN		**X**

FUNCTION: Programs the function generator present on the GEN module.

SYNTAX: **~SET USER_GEN HZ=**value, **VOUT=** value, **VOFF=** value, **ROUT=** value, **[SIN | TRI | RECT], [VPP] , [GUA]**

HZ=value Indicates frequency of output signal in Hz. It can have values ranging from 5Hz to 5MHz.

VOUT=value Indicates output signal voltage and is expressed in Volts. The value can be expressed in Vrms (Veff) or Vpp (peak to peak value) if the VPP parameter is specified.

VOFF=value Indicates output signal voltage offset value and is expressed in Volts. It can have values ranging from +/- 10Vmax. If the parameter is not specified its value is 0.

ROUT=value Indicates resistance value series-connected to the output of the waveform generator and is expressed in Ω. Its value is between 0 and 1M Ω. If this parameter is not specified the parameter value will be 0. (ROUT=0)

SIN The output signal has a sinusoidal waveform

TRI The output signal has a triangular waveform

RECT The output signal has a rectangular waveform

If the SIN, TRI and RECT parameters are not specified, By default, SIN is active.

VPP Indicates that the value expressed with the parameter **VOUT=***value* will be considered as a peak-to-peak voltage value. If the **VPP** parameter is not specified, it indicates that the **VOUT=***value* parameter is considered as a Vrms voltage value

GUA Indicates that the guarding generator will be connected to line 3. If the parameter is not specified, the guarding generator will be disconnected.

EXAMPLE: ~SET USER_GEN HZ=1000 Vout=3 SIN VPP It generates a sine waveform 1Kz frequency, and Vpp=3V

~SET USER_LINEn **Instruction** **~ SET USER_LINEn**

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Closes the 8 USB relays connected on the system lines.

SYNTAX: **~USER_LINE**n

n Integer number from 1 to 32 indicating the relay to close on the correspondent line.

SEE ALSO: ~CLEAR USER_LINE

NOTE: If there are 4 USB modules in the test system, it is possible to drive up to 32 relays keeping into account that relays number 1,9,17,25, will be connected to system line 1 (LI1) and relays 2,10,18,26 will be connected on line 2 (LI2), etc.

EXAMPLE:

~USER_LINE1 !Closes the relay 1 on system line 1. The relay is on USB number 1

~USER_LINE10 !Closes relay 2 on system line 2. The relay is on USB number 2.

~SET USER_LOADn — Instruction — ~SET USER_LOADn

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Programs the two resistive loads on the USB module.

SYNTAX: **~USER_LOAD**n **OHM**=value [**ONLINE**]

n — Integer number from 1 to 8 indicating the resistive load to program. Number 1 and 2 will indicate resistive loads on USB module 1. Number 3 and 4 will indicate resistive loads on USB 2 (if present) and so on.

OHM=value — Indicates the resistive value expressed in Ohm. It can be:

- an integer number from 1 to 1000000
- an INTEGER run-time variable.

ONLINE — If this parameter is specified, the resistive loads are connected also to the system lines in this way: resistive load 1 (LD1) is connected between line 1 (LI1) and line 2 (LI2). Resistive load 2 (LD2) is connected between line 2 (LI2) and line 4 (LI4).

DEFAULT VALUES: If the OHM parameter is not specified, by default is OHM=0.

If the ONLINE parameter is not specified, by default the resistive loads are disconnected from the system lines.

SEE ALSO: ~CLEAR USER_LOAD

EXAMPLE:

~SET USER_LOAD1 OHM=210; Programs load LD1 to 210Ω. The load is on USB number 1

~SET USER_LOAD8 OHM=1357; Programs load LD2 to 1357Ω. The load is on USB number 4.

~SET USER_LOAD2 OHM=1 ONLINE; Programs load LD2 to 1Ω and connects it between LI2 and LI4. The load is on USB number 1

~SET USER_WORDn **Instruction** **~SET USER_WORDn**

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Writes a data on the CUSTOM1-15 lines of the USB module. This bus is available in the USB breadboard area for customization on JP18 connector.

SYNTAX: **~SET USER_WORD**n V=value

n Integer number from 1 to 4 that indicates the USB to write.

V=value Data to write on CUSTOM1-15 lines. Data can be expressed in binary octal, decimal or hexadecimal format. Data can be a constant, a compiled variable, a run-time variable or an arithmetic expression. Line CUSTOM-1 corresponds to the least significant bit (LSB) of the data (binary code). Line CUSTOM-15 corresponds to the most significant bit (MSB).

SEE ALSO: ~CLEAR USER_WORD

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR1;
START PRG1 STATIC;
/;
~SET USER_WORD1 V=0X5AF2;   Write on CUSTOM bus, data:
                            0X5AF2. The user Board
                            module is USB nr. 1

/;
ENDTEST;
```

~SET ZTEST **Instruction** **~SET ZTEST**

☛

Module	Compiled	Run-time
SE2		**X**

FUNCTION: There are high precision components mounted on the SE2 module that can be connected to the system lines and measured with the LAM module for LAM calibration. This instruction programs the connection between the above-mentioned components and system lines.

SYNTAX: **~SET ZTEST Z=**value **P1=**line **P2=**line

Z=value

Indicates the component to connect and ca be:

Z1: 1MΩ resistor
Z2: 100KΩ resistor
Z3: 10KΩ resistor
Z4: 1KΩ resistor
Z5: 50Ω resistor
Z6: 1.2Ω resistor
Z7: 100nF capacitor
Z8: SPARE

It is also possible to use the following syntax:

Zn=1M, 100K, 10K, 1K, 50, 1.2, 100N, SPARE

P1=line and P2=line

Indicates where the components must be connected and can be:

LL1: Indicates the connection to LI1
LL2: Indicates the connection to LI2
LL3: Indicates the connection to LI3
LL4: Indicates the connection to LI4

EXAMPLE: Connection of 10KΩ precision resistor between line 4 and line 2 of LAM module. The resistor is then measured on the LAM module.

~SET ZTEST Z=Z3 P1=LL2 P2=LL4;

```
~SET DR1 VL I=30 V=1 PULSE;
~SET VOLT LI4 MAX=2 TIM=100;
~COMP AR LO = 0.28 HI=0.32 ONERROR BRANCH H1;
…
…
LABEL=H1;
~CURSOR 10,10;
~WRITE” LAM NOT CALIBRATED, VALUES OUT OF RANGE”;
ENDTEST;
```

~SHL — Instruction — ~SHL

Compiled	Run-time
	X

FUNCTION: Shifts the contents of the AR register to the left by the specified number of bits. It enters the “0” value during shift.

SYNTAX: **~SHL** value

value: Value, which specifies by how many bits the contents must be shifted to the left. This can be expressed as:

- a constant
- an INTEGER type runtime variable

SEE ALSO: ~SHR

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR1;
DECLARE RUNTIME FLOAT VAR;

START PRG1 STATIC;
/;
~CALC VAR1=5;
~LOAD 0XFE;
~SHL VAR1;
~STORE VAR;
~WRITE "SHIFT TO THE LEFT BY " VAR1 " BIT : ",VAR;
/;
~PAUSE 1;
/;
~SHL 4;
~STORE VAR;
~WRITE "SHIFT TO THE LEFT BY 4 BITS : ",VAR;
/;
~PAUSE 2;
/;
ENDTEST;
```

~SHR **Instruction** **~SHR**

Compiled	Run-time
	X

FUNCTION: Shifts the contents of the AR register to the right by the specified number of bits. It enters the “0” value during shift.

SYNTAX: **~SHR** value

value Value which specifies by how many bits the contents must be shifted to the right. This can be expressed as:

- a constant
- an INTEGER type runtime variable

SEE ALSO: ~SHL

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR1;
DECLARE RUNTIME FLOAT VAR;

START PRG1 STATIC;
/;
~CALC VAR1=2;
~LOAD 0X55;
~SHR VAR1;
~STORE VAR;
~WRITE "SHIFT TO THE RIGHT BY " VAR1 " BIT : ",VAR;
/;
~PAUSE 1;
/;
~SHR 2;
~STORE VAR;
~WRITE "SHIFT TO THE RIGHT BY 2 BITS : ",VAR;
/;
~PAUSE 2;
/;
ENDTEST;
```

~SIN **Instruction** **~SIN**

Compiled	Run-time
	X

FUNCTION: Calculates the sine of the contents of the AR register. The value of AR must be expressed in radians.

SYNTAX: **~SIN**

SEE ALSO: ~COS ~ATAN

EXAMPLE:

```
DECLARE RUNTIME INTEGER V1;
DECLARE RUNTIME INTEGER TR;

START PRG1 STATIC;
/;
~LOAD 2.35619449;
~ SIN;
/;
~STORE TR;
~WRITE TR
/;
ENDTEST;
```

~SQR **Instruction** **~SQR**

Compiled	Run-time
	X

FUNCTION: Calculates the square root of the contents of the AR register.

SYNTAX:
.

~SQR

SEE ALSO: ~SQUARE ~EXP

EXAMPLE:

```
DECLARE RUNTIME FLOAT VAR;

START PRG1 STATIC;
/;
~LOAD 454;
~SQR;
~STORE VAR;
~WRITE "SQUARE ROOT: ",VAR;
/;
~PAUSE 1;
/;

ENDTEST;
```

~SQUARE **Instruction** **~SQUARE**

Compiled	Run-time
	X

FUNCTION: Calculates the square of the contents of the AR register.

SYNTAX: **~SQUARE**

SEE ALSO: ~SQR ~EXP

EXAMPLE:

```
DECLARE RUNTIME FLOAT VAR;

START PRG1 STATIC;
/;
~LOAD 16;
~SQUARE;
~STORE VAR;
~WRITE "SQUARE: ",VAR;
/;
~PAUSE 1;
/;

ENDTEST;
```

~STORE **Instruction** **~STORE**

Compiled	Run-time
	X

FUNCTION: Unloads the contents of the AR register into the specified variable.

SYNTAX: **~STORE** variable

variable Runtime variable in which the contents of the AR register are stored. The variable must be FLOAT type.

SEE ALSO: ~LOAD

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR;
DECLARE RUNTIME FLOAT VAR1;

START PRG1 STATIC;
/;
~CALC VAR=25;
~LOAD VAR;
~SQR;
~STORE VAR1;
~WRITE "THE VALUE OF AR IS : ",VAR1;
/;
~PAUSE 1;
/;

ENDTEST;
```

~SUB **Instruction** **~SUB**

Compiled	Run-time
	X

FUNCTION: Subtracts the specified value from the contents of the AR register.

SYNTAX: **~SUB** value

value Value to subtract from the AR register. It can be expressed as:

- a constant
- an INTEGER, FLOAT type runtime variable

SEE ALSO: ~ADD ~MULT ~DIV

EXAMPLE:

```
DECLARE RUNTIME INTEGER V1;

START PRG1 STATIC;
/;
~CALC V1=6;
/;
~SUB V1;
/;
~SUB 5;
/;
ENDTEST;
```

~SUBR **Instruction** **~SUBR**

Compiled	Run-time
	X

FUNCTION:

Defines a sequence of instructions, which can be called in a functional program.
The subroutine allows you to declare RUNTIME type variables that will be local to the subroutine and will be used to acquire values from the keyboard or to manage the cursor, the colors and the messages in the Static terminal. The subroutines can be grouped in a library file, separated from the pattern file, and include it later in the main program.
All of the subroutines must be declared in the declarations section and must be written before the START instruction. To recall a subroutine within the program, use the ~CALL instruction. Refer to the examples.

SYNTAX:

~SUBR name subroutine (argument type argument 1, argument 2,...)
...
...
...
~RETURN
~ENDSUBR

name subroutine

Alphanumerical string maximum length 20 characters which identifies the subroutine. The name of the subroutine must not be the same as the name given to a label.

argument type

Specifies the type of parameter/parameters passed in the subroutine, which can be:
- INTEGER
- FLOAT
- STRING

The arguments can be defined as global or local. See the description in NOTE.
If the argument type is not declared, by default is INTEGER

argument1, argument2,...

Alphanumerical string maximum length 20 characters which identifies the argument of the subroutine. The argument can also assume the value %. Example: ~SUBR N1 (%) in this case, the argument has got the value calculated by the previous subroutines. Refer to the example 1.

SEE ALSO:

~CALL

NOTE: Up to 256 subroutines can be declared in a functional program.

There can be a maximum of 200 between arguments and variables in a functional program.

Arguments can be defined as global or local. The global arguments can be defined and used in all parts of the program and must be defined with the DECLARE RUNTIME ARGUMENT instruction. Refer to the example 1. The local parameters are “seen” only inside the subroutine in which they are used and must be defined in the ~SUBR instruction as follows: ~SUBR rout1 (INTEGER, par1, par2) both par1 and par2 are local arguments of the type INTEGER.
Different types of parameters can be defined within the parentheses ***Example:*** ~subr ruot2 (INTEGER par1, par2, FLOAT par3, par4, STRING par5). Refer to example 4.

It is possible to have more than one ~RETURN instruction in the subroutine. The ~RETURN instructions can be conditioned by the ONERROR or ONPASS options which control the partial error flag. The return of the subroutine, therefore, can be executed or not, depending on whether or not an error has occurred.
In any case the subroutine must be ended with the instruction ~ENDSUBR.

To recall a subroutine in the program, use the ~CALL name subroutine instruction.

EXAMPLE:

EXAMPLE 1
Subroutine declaration

```
DECLARE SUBROUTINE PROVA;

~SUBR PROVA;
~CURSOR 1,1;
~WRITE "TEST SUBROUTINE "
~PAUSE 1;
~RETURN;
~ENDSUBR;

START PROFUN STATIC ;
~CALL PROVA;
ENDTEST;
```

EXAMPLE 2
Subroutine with internal variable declaration. Note that the variable J defined in the subroutine PROVA is DIFFERENT from the variable J defined in PROVA2 even though the have the same name. Therefore the variables are local to their subroutine.

```
DECLARE SUBROUTINE PROVA PROVA2;

~SUBR PROVA;
DECLARE RUNTIME FLOAT J;
~CALC J=4.4;
~CURSOR 1,1;
~WRITE "VALUE 1" J;
~PAUSE J;
~RETURN;
~ENDSUBR;
```

```
~SUBR PROVA2;

DECLARE RUNTIME FLOAT J;
~CALC J=3.4;
~CURSOR 1,2;
~WRITE "VALUE 2";
~PAUSE J;
~RETURN;
~ENDSUBR;

START PROFUN STATIC ;
~CALL PROVA;
~CALL PROVA2;
ENDTEST;
```

EXAMPLE 3
Inclusion of a subroutine library file.

File MIESUB.lib written under the
directory S20WIN\BOARD\nomeboard

```
DECLARE SUBROUTINE PROVA PROVA2;

~SUBR PROVA;
DECLARE RUNTIME FLOAT J;
~CALC J=4.4;
~CURSOR 1,1;
~WRITE "VALUE 1" J;
~PAUSE J;
~RETURN;
~ENDSUBROUTINE;

~SUBR PROVA2;
DECLARE RUNTIME FLOAT J;
~CALC J=3.4;
~CURSOR 1,2;
~WRITE "VALUE 2";
~PAUSE J;
~RETURN;
~ENDSUBR;
```

File MIOPAT.pat written under the
directory S20WIN\BOARD\nomeboard

```
#INCLUDE MIESUB;
START PROFUN STATIC ;
~CALL PROVA;
~CALL PROVA2;
ENDTEST;
```

EXAMPLE 4
Definition of a subroutine with global arguments.

```
DECLARE CHANNEL CS=40;
DECLARE CHANNEL RD=17;
DECLARE CHANNEL CLK=19;
```

```
DECLARE CHANNEL DAT0=67;
DECLARE RUNTIME ARGUMENT INTEGER SUBERR;
DECALARE SUBROUTINE SUB1, SUB2;
~SUBR SUB1 (SUBERR);
~DELAY SUBERR;
~WRITE “DELAY TIME EQUAL TO : ”, SUBERR;
~RETURN;

~SUBR SUB2 (SUBERR);
~FOR I=1 TO SUBERR;
~IL CLK;
/;
~IH CLK;
/;
~ENDFOR;
~RETURN;



START PROG1 STATIC
IL CS;
/;
IL RD;
/;
~CALL SUB2 (100);
~CALL SUB1 (10);
~CALL SUB2 (%);
```

In this case the previous value of SUBERR is recalled, which is equal to 10.

```
OH DAT0;
/;
ENDTEST;
```

EXAMPLE 5
Subroutine definition with local arguments.

```
DECLARE CHANNEL CS=40;
DECLARE CHANNEL RD=17;
DECLARE CHANNEL CTR=143;
DECLARE CHANNEL LINE0=10;
DECLARE CHANNEL DAT0=67;
DECLARE SUBROUTINE SUB2, TEST_TRI;

~SUBR SUB2 (INTEGER P1, STRING P2);
~DELAY P1;
~WRITE P2;
~RETURN;

~SUBR TEST_TRI (INTEGER TRICH);
~CALC SUBERR=0;
~LI4=TRICH;
~SET PUL UP OHM=1K;
~TEST VOLT HI=POSVOLT,LO=4,MAX=6,LI4,TIM=0.1;
~RETURN

START PGR1 STATIC;
IL CS;
/;
IL RD;
```

```
/;
OH DAT1;
/;
IH CTR;
/;
~CALL SUB2 (100, CICLO1); /;/;~CALL TEST_TRI (10)..
ENDTEST;
```

EXAMPLE 6

WARNING!!: This example is supplied only to illustrate the compatibility with the subroutine declarations as it was executed in the software versions S20WIN previous to the 3.14. In all other cases it is advisable to use the method used in EXAMPLE 3.

Inclusion of the LIBR.LIB library (present under the directory S20WIN\BOARD\PROVA) in PROVA.PAT.

File LIBR.LIB:

```
DECLARE RUNTIME INTEGER INT1,INT2;
@MACRO SUBRO;          !The subroutine file must be enclosed in
                       a macro that in this case is the macro
                       SUBROUTINE!

~SUBR SUB1;
~CALC INT1=1;
~RETURN;
~SUBR SUB2;
~CALC INT2=INT1+1;
~RETURN;
ENDMACRO;
```

File PROVA.PAT:

```
DECLARE RUNTIME FLOAT REG1;
#INCLUDE LIBR; Library inclusion

START test STATIC;

~START_SUB;    Key word to define the beginning of the part of
               program reserved to the writing of the
               subroutines

@SUBRO;        From this point on, all library subroutines are
               "seen" in the file PROVA.PAT

~END_SUB;      Key word to define the end of the part of
               program reserved to the writing of the
               subroutines
/;
/;
~CALL SUB1;    Call to library's subroutine
/;
~CALL SUB2;    Call to library's subroutine
~WRITE INT2;
/;
~PAUSE 1
/;
ENDTEST;
```

~SYS **Instruction** **~SYS**

Compiled	Run-time
	X

FUNCTION: Enables the execution of the specified application. The return parameter of the application is stored in the API register.

SYNTAX: **~SYS** [display], [execution mode], “application name”,”parameters”

display

Specifies the display mode of the application to be executed. The mode can be:

HIDE	hidden mode
NORMAL	normal display mode
MINIMIZED	display reduced to an icon
MAXIMIZED	full screen display

execution mode

Specifies the execution mode of the called application, which can be:

SYNCHRON control of the functional program passes to the executed application and only after it has ended does it return to the calling program.

ASYNCHRON the application is executed simultaneously with the execution of the calling functional program.

application name

Indicates the name of the called application for which it is also possible to specify the associated path.

parameters

Parameters of the called application.

DEFAULT VALUES: ~SYS NORMAL, SYNCHRON,...

SEE ALSO: ~API

EXAMPLE: EXAMPLE 1

Opening of the NEWS.TXT document with the NOTEPAD editor.

```
START PRG1 STATIC;
/;
~SYS MAXIMIZED,“NOTEPAD”,“C:\S20WIN\NEWS.TXT”;
/;
ENDTEST;
```

EXAMPLE 2

Opening of the NEWS.TXT document with the WRITE editor

```
START PGR2 STATIC;
/;
~SYS MAXIMIZED,SYNCHRON,
"C:\WINDOWS\WRITE.EXE”,”C:\S20WIN\NEWS.TXT”;
/;
ENDTEST;
```

EXAMPLE 3

Print of the AUTOEXEC.BAT file during the execution of the test burst.

```
START PRG3 STATIC;
/;
~SYS HIDE,ASYNCHRON,
”C:\DOS\PRINT.EXE”,C:\AUTPEXEC.BAT”;
/;
ENDTEST;
```

~TEST COUNTER **Instrument** **~TEST COUNTER**

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

Executes a measurement with the COUNTER instrument which has the following characteristics:
Time interval: from 500µs to 10s step of 500ns
Counting: up to 32767 impulses per step
Input voltage: :±5V. The result of the measure is stored in the AR register (accumulator) and it is possible to control the value directly in this instruction with the parameters LO and HI and jump to a specific label with the ONERROR and ONPASS instructions depending on the result of the measurement. The global error flag is activated in case the measure is out of the specified range.

SYNTAX:

~TEST COUNTER LI4 TIM=value [**HI=**value] [**LO=**value] [**ONERROR** label] [**ONPASS** label]

LI4

Connects the input of the measurer to line 4.

TIM=value

Indicates the time of the measure expressed in ms. The accepted range is:

0.0005ms ≤ TIM ≤ 60000ms

HI=value

Defines the upper limit of the control range. The value can be expressed as a constant, as a compiled variable, as a runtime variable or as an arithmetic expression.

LO=value

Defines the lower limit of the control range. The value can be expressed as a constant, as a compiled variable, as a runtime variable or as an arithmetic expression.

NRC=value

Defines the number of recycles of the test. Example: NRC=10 means that the test will be repeated 10 times. The value can be expressed as a constant, as a RUNTIME variable or as an expression enclosed in parentheses. The NRC is combined with the MRC parameter.

MCR=value

Defines the measurement modality and can accept the following values:

M: at every recycle the result of the measurement is stored. The iteration stops when the expected number of recycles is reached, providing the average of the results.

W The iteration stops when the expected number of recycles is reached and, only at this point the measurement is executed.

J the interaction stops when the test gives a positive result.

H — the interaction stops when the expected number of recycles is reached and the highest value measured is stored.

L — the interaction stops when the expected number of recycles is reached and the lowest value measured is stored.

P — the interaction stops when the expected number of recycles is reached and the difference between the highest and lowest value measured is stored.

MRC=1|2|3|4|5|6 are alias of MRC=M|W|J|H|L|P (that is MCR=1 means MRC=M, MRC=2 means MRC=W…).

ONERROR Jumps to the specified label in case of an error.

ONPASS Jumps to the specified label in case of a correct measurement.

DEFAULT VALUES: If the LI4 parameter is not specified, the instrument is disconnected.

If the TIM parameter is not specified, by default TIM=1ms.

EXAMPLE:

```
~TEST COUNTER LI4,TIM=1 ONERROR GH1;
~TEST COUNTER LI4,TIM=TEMP1 ONPASS GH2;
```

~TEST PW1…8 **Instrument** **~TEST PW1…8**

Module	Compiled	Run-time
SE2		**X**

FUNCTION:

Executes a measurement on the specified power supply. The result of the measure is stored in the AR register (accumulator) and it is possible to control the value directly in this instruction with the parameters LO and HI and jump to a specific label with the ONERROR and ONPASS instructions depending on the result of the measurement. The global error flag is activated in case the measure is out of the specified range.

SYNTAX:

~TEST PWn I/V measure mode [**HI=**value] [**LO=**value] [**ONERROR** label] [**ONPASS** label]

PWn

Parameter to identify the power supply, it can assume the values from PW1 up to PW8.

I/V

Parameter indicating the type of measurement. I indicates a current measure. V indicates a voltage measure. At least one parameter (V or I) must be specified in the ~TEST instruction.

Measure mode

Indicates how to do the reading. It can accept the following values:

SERV: internal type reading done through the programming signals of the power supply the SE2 module.

EXT external type reading, taken directly on the sense signals of the feeder.

HI=value

Defines the upper limit of the control range.

LO=value

Defines the lower limit of the control range.

ONERROR

Jumps to the specified label in case of an error.

ONPASS

Jumps to the specified label in case of a correct measurement.

EXAMPLE:

```
~SET PW1 V=0.5 I=6 SENSEON ON INT;
~TEST PW1 V SERV ONERROR HG12;
…
…
LABEL=HG12;
```

~TEST PMM MAX= TIM= LO= HI= ONERROR= ONPASS

Compiled	Run-time
	X

FUNCTION:

Measures a voltage or a current. The measurement is executed according to the indicated parameters. The result of the measurement is stored in the AR register (accumulator) and it is possible to check the value using the ~COMP or ~ATEST instructions. It is also possible to check a range specified with the HI=...,LO=... parameters.
The global error flag is activated in case the measure is out of the specified range.

SYNTAX:

~TEST PMM MAX= scale **TIM**= time, **HI=**value , **LO=**value **ONERROR** label **ONPASS** label;

scale

Indicates the end scale value for voltage measurements. It can be chosen from one of the following values:

20 / 200 / 1000

The end scale value is not programmable for current measurements and its fixed value is 10A.

time

Indicates the time required from the RMS converter to settle within an error band ≤ 1%.
The value is expressed in ms and can vary from 10 to 200

HI=value

Defines the upper limit of the control range. The value can be expressed as a constant, as a compiled variable, as a runtime variable or as an arithmetic expression.

LO=value

Defines the lower limit of the control range. The value can be expressed as a constant, as a compiled variable, as a runtime variable or as an arithmetic expression.

ONERROR

Jumps to the specified label in case of an error.

ONPASS

Jumps to the specified label in case of a correct measurement.

SEE ALSO:

~MEAS

EXAMPLE:

```
~TEST PMM MAX=200, TIM=20
~TEST PMM MAX=1000, TIM=200, HI=1,LO= -1;
~TEST PMM MAX=1000, TIM=200, HI=300, LO=180 ONPASS PASS_1;
```

~TEST VOLT **Instruction** **~TEST VOLT**

Module	Compiled	Run-time
LAM		**X**

FUNCTION:

Measures a voltage with the VOLT instrument, which has the following characteristics:

Voltage measurements

from -2V to +2V step of 1mV
from -10V to +10V step of 5mV
from -50V to +50V step of 25mV

The result of the measurement is stored in the AR register (accumulator) and it is possible to check the value directly in this instruction with the LO and HI parameters and jump to a specific label with the instructions ONERROR and ONPASS according to the result of the measurement. The global error flag is activated in case the measurement is out of the specified range.

SYNTAX:

~TEST VOLT [**LI1**] [**LI4**] **MAX=value TIM=value** [**HI=**value] [**LO=**value] [**ONERROR** label] [**ONPASS** label]

LI1

If this parameter is specified, connects the input of the measurer to line 1.

LI4

If this parameter is specified, connects the input of the measurer to line 4.

MAX=value

Indicates the end scale value of the instrument. It can accept the following values:

2-10-50

TIM=value

Indicates the time of the measure expressed in ms. The accepted range is:

0.0005ms ≤ TIM ≤ 60000ms

HI=value

Defines the upper limit of the control range. The value can be expressed as a constant, as a compiled variable, as a runtime variable or as an arithmetic expression.

LO=value

Defines the lower limit of the control range. The value can be expressed as a constant, as a compiled variable, as a runtime variable or as an arithmetic expression.

NRC=value

Defines the number of recycles of the test. Example: NRC=10 means that the test will be repeated 10 times. The value can be expressed as a constant, as a RUNTIME variable or as an expression enclosed in parentheses. The NRC is combined with the MRC parameter.

MCR=value

Defines the measurement modality and can accept the following values:

M: at every recycle the result of the measurement is stored. The iteration stops when the expected number of recycles is reached, providing the average of the results.

W The iteration stops when the expected number of recycles is reached and, only at this point the measurement is executed.

J the interaction stops when the test gives a positive result.

H the interaction stops when the expected number of recycles is reached and the highest value measured is stored.

L the interaction stops when the expected number of recycles is reached and the lowest value measured is stored.

P the interaction stops when the expected number of recycles is reached and the difference between the highest and lowest value measured is stored.

MRC=1|2|3|4|5|6 are alias of MRC=M|W|J|H|L|P (that is MCR=1 means MRC=M, MRC=2 means MRC=W…).

ONERROR Jumps to the specified label in case of an error.

ONPASS Jumps to the specified label in case of a correct measurement.

SEE ALSO: ~MEAS VOLT

EXAMPLE:

~TEST VOLT MAX=123,LI4,TIM=0.1;

~TEST VOLT HI=TEMP3,LO=TEMP2,MAX=TEMP3,
LI4,TIM=20 ONPASS BENE0;

~TEST VOLT HI=1,LO=-1,MAX=2,LI4,TIM=0.1;

~TEST VOLT MAX=123,LI4;

~URP Instruction ~URP

Module	Compiled	Run-time
USB		**X**

FUNCTION: Controls the double contact power relay, of the USB module.

SYNTAX: **~URP=**[*], [*][relay]

relay

Integer number between 1 and 32, which indicates the relay or relays to close/open.

The relays between 1 and 8 are on USB nr 1
The relays between 9 and 16 are on USB nr 2
The relays between 16 and 24 are on USB nr 3
The relays between 24 and 32 are on USB nr 4

The relay can be expressed as:

- a whole number indicating a single relay.
- a group of relays
- a FLOAT type runtime variable
- an INTEGER type runtime variable

The "/" character is used to indicate a contiguous group of relays and the "," or "-" character is used to indicate non-contiguous relays.

If the relay is not specified in the instruction (Example: ~URP;), all relais previously closed will be opened.

[*,]

Indicates the incremental way of opening/closing relays. That is the previous connection is maintained and the actions specified in the ~URP current instruction are executed upon this situation.

[,]

If instead of the "*" character no character is specified, it indicates the normal mode to open/close relays. That is the relays specified in the current ~URP instruction are closed while the ones not specified are opened. If in the current ~URP requires to close relays that are already closed, these relays don't change their state.

[*]

Indicates that the following relay or relays must be opened.

[ ]

If instead of the "*" character no character is specified, this indicates that the following relay or relays must be closed.

NOTE: If the value of the channel is passed on via a variable, any errors will come out during execution and not during compilation.

EXAMPLE:

```
DECLARE RUNTIME INTEGER RL;
.
```

START PRG1 STATIC;
/;
.
~URP=5; !Closes relay 5 and opens all the other relays (normal connection mode)

~URP=1,4,6,8/12; !Closes relays 1,4,6, and from relay 8 to relay 12 included and opens all the other relays (normal connection mode)

~URP=1-3-5-7; !Closes relays 1,3,5,7 and opens all the other relays (normal connection mode)

~URP; !Opens all relays

~CALC RL=10;
~URP=RL; !Closes relay 10

~URP=*,3; !Closes relay 3 while all the other relays remain in the preceding situation (incremental connection mode)

~URP=*,*12; !Opens relay 12 while all the other relays remain in the preceding situation (incremental connection mode)

~URS Instruction ~URS

☛

Module	Compiled	Run-time
USB		**X**

FUNCTION: Controls reed type relays of the USB module.

SYNTAX: **~URS**=[*,][*][relay]

relay

Indicates the relay or relays to close/open. The relay can be expressed as:

- a whole number indicating a single relay
- a group of relays
- a FLOAT type runtime variable
- an INTEGER type runtime variable

The "/" character is used to indicate a contiguous group of relays and the "," or "-" character is used to indicate non-contiguous relays.

If the relay is not specified in the instruction (Example: ~URS;), all relais previously closed will be opened.

[*,]

Indicates the incremental way of opening/closing relays. That is the previous connection is maintained and the actions specified in the ~URS current instruction are executed upon this situation

[,]

If instead of the "*" character no character is specified, it indicates the normal mode to open/close relays. That is the relays specified in the current ~URS instruction are closed while the ones not specified are opened. If in the current ~URS requires to close relays that are already closed, these relays don't change their state.

[*]

Indicates that the following relay or relays must be opened.

[ ]

If instead of the "*" character no character is specified, this indicates that the following relay or relays must be closed.

NOTE: If the value of the channel is passed on via a variable, any errors will come out during execution and not during compilation.

EXAMPLE:

```
DECLARE RUNTIME INTEGER RL;
.
START PRG1 STATIC;
/;
.
.
~URS=5;   ! Closes relay 5 and open all the other relays (normal connection mode)

~URS=1,4,6,8/12;   !Closes relays 1,4,6, and from relay 8 to relay 12 included and opens all the other relays (normal connection mode)
```

~URS=1-3-5-7;! Closes relays 1,3,5,7 and opens all the other relays (normal connection mode)

~URS; !Opens all relays

~CALC RL=10;
~URS=RL; !Closes relay 10

~URS=*,3; ! Closes relay 3 while all the other relays remain unchanged (incremental connection mode)

~URS=*,*12;!Opens relay 12 while all the other relays remain unchanged (incremental connection mode)

~UOP — Instruction — ~UOP

Module	Compiled	Run-time
USB		**X**

FUNCTION: Controls the opto-coupled transistors of the USB module.

SYNTAX: **~UOP**=[*,][*][transistor]

[transistor] Indicates the il transistor or transistors to cut off or saturate. The transistor can be expressed as:

- a single transistor
- a group of transistors
- a FLOAT type runtime variable
- an INTEGER type runtime variable

The "/" character is used to indicate a contiguous group of transistors and the "," or "-" character is used to indicate non-contiguous transistors.

If the transistor is not specified in the instruction (Example: ~UOP;), all transistors previously saturated will be cut off.

[*,] Indicates the incremental way of cut off/saturate transistors. That is the previous situation is maintained and the actions specified in the current ~UOP instruction on are executed.

[,] If instead of the "*" character no character is specified, it indicates the normal mode to cut off/saturate transistors. That is the transistors specified in the current ~UOP instruction are saturated while the ones not specified are cut off. If in the current ~UOP requires to saturate transistors that are already saturated, these transistors don't change their status.

[*] Indicates that the following transistor or transistors must be opened.

[ ] If instead of the "*" character no character is specified, this indicates that the following transistor or transistors must be closed.

NOTE: If the value of the channel is passed on via a variable, any errors will come out during execution and not during compilation.

EXAMPLE:

```
DECLARE RUNTIME INTEGER TR;
.
START PRG1 STATIC;
/;
.
.
~UOP=5;    ! Transistor 5 is in saturation and all other transistors are
cut off (normal connection mode)
```

~UOP=1,4,6,8/12; ! Transistors 1,4,6, and from transistor 8 to transistor 12 are in saturation. All other ones are cut off (normal mode)

~UOP=1-3-5-7; ! Transistors 1,3,5,7 are in saturation and all others are cut off (normal mode)

~UOP; ! All transistors are cut off.

~CALC RL=10;
~UOP=RL; ! Transistor10 in saturation

~UOP=*,3; !Transistor 3 in saturation while all others remain in the previous situation (incremental connection mode)

~UOP=*,*12; !Transistor 12 cut off while all others remain in the previous situation (incremental connection mode)

~WG Instruction ~WG

Compiled	Run-time
	X

FUNCTION: Assigns a value (expressed by a constant or a variable) to a group of channels. The least significant bit of the constant or variable is assigned to the first declared channel in the group.

SYNTAX: **~WG** value = channel group

value — Indicates the value, which the group of channels will have to assume. The value can be expressed as:

- an INTEGER type runtime constant
- an INTEGER type runtime variable

channel group — Indicates the group of channels, which will assume the previously specified value. The channel group can be expressed as:

- physical channel numbers
- channel names defined with the DECLARE CHANNEL instruction

The "/" character is used to indicate a contiguous group of channels and the "," or "-" character is used to indicate non-contiguous channels..

- name of a group of channels defined with the DECLARE GROUP instruction.

SEE ALSO: ~RG

NOTE: The ~WG instruction modifies the status of the destination channel but does not alter either the direction or the mask.. Therefore it is necessary to ensure that the destination channels are already in driver before executing the write.

Up to 32 channels can be defined in a ~WG instruction.

EXAMPLE:

```
DECLARE CHANNEL D0=45;
DECLARE CHANNEL D1=46
DECLARE CHANNEL D2=47;
DECLARE CHANNEL D3=48;
DECLARE CHANNEL DT0=23;
DECLARE CHANNEL DT1=24;
DECLARE CHANNEL DT2=25;
DECLARE CHANNEL DT3=26;
```

```
DECLARE CHANNEL DT4=27;
DECLARE CHANNEL DT5=28;
DECLARE CHANNEL DT6=29;
DECLARE CHANNEL DT7=30;
DECLARE CHANNEL DATB=139;
DECLARE GROUP DATB=34,35,36,37,38,39,40,41;

DECLARE RUNTIME INTEGER VAL1,VAL2;

START PRG1 STATIC;
/;

~WG B1110=45,46,47,48;     !The low logic level (0)
                           will be associated to
                           channel 45 while the high
                           logic level (1) will be
                           associated to channels
                           46,47,48!

~WG 0X55=23,24,25,26,27,28,29,30;
/;
~CALC VAL1=0X59;
~CALC VAL2=0XA6;

/;
~WG VAL1=23,24,25,26,27,28,29,30;
/;
~WG VAL2=D0,D1,D2,D3;
/;
.
.
~WG VAL3=DATB;
/;
ENDTEST;
```

~WHILE-~ENDWHILE **Instruction** **~WHILE-~ENDWHILE**

☛

Compiled	Run-time
	X

FUNCTION: Executes the body of the ~WHILE loop until the condition is true.

SYNTAX:

~WHILE [NOT] variable1 condition constant/variable2/expression
...
...
...
~ENDWHILE

[NOT] Enables the execution of a NOT of the specified condition.

variable Variable for which the specified condition is to be verified. The variable can be INTEGER, FLOAT, STRING type and the system variables NUM[1…15], STR[1…15].

condition Specifies the condition to apply to the variable. The following characters can be used to express the condition:

< or "lt" meaning "less than ..."
> or "gt" meaning "greater than ..."
= or "eq" meaning "equal to ..."
<= or "le" meaning "less than or equal to ..."
>= or "ge" meaning "greater than or equal to ..."
<> or "ne" meaning "different from ..."

constant/variable2/expression Can be:

- a constant
- an INTEGER, FLOAT, STRING type runtime variable

- an arithmetic/logical expression whose operands can be:

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Xor
&	And
|	Or
<n	Shift n bits to the left
>n	Shift n bits to the right

The arithmetic/logical expression must be enclosed by parentheses (). There can be several levels in the arithmetic/logical expression inside the global parentheses.

NOTE: The ~WHILE- ~ENDWHILE instruction accepts up to 10 nesting levels.

If operations between variables and constants of different types or formats are performed, each operand is converted into the type and format of the destination variable unless it is modified using the ~FORMAT instruction.

For string type variables or constants, the only valid operand is addition (+), which is the same as chaining.

EXAMPLE:

```
DECLARE RUNTIME INTEGER A,B;

START PRG1 STATIC;
/;
~CALC A=100;
~CALC B=1;
/;
~WHILE A >= 95 ;
~CALC A=A-1;
~WRITE "A=  "A;
~ENDWHILE;
/;
~CALC B=B+1;
/;
~WHILE B<(A*2);
~WRITE "A<B";
~ENDWHILE;
```

~WRITE — Instruction — ~WRITE

Compiled	Run-time
	X

FUNCTION: Outputs the specified value or values to the screen.

SYNTAX: **~WRITE** attribute value1,value2,...

value1,value2,... The value can be expressed as:

- a constant
- an INTEGER, FLOAT, STRING type runtime variable
- a string of characters enclosed by quotation marks
- a FLOAT type register (REG1-REG100) enclosed by quotation marks and preceded by the "\" character
- an octal value enclosed by quotation marks and preceded by the % character.

attribute Attributes for the ~WRITE instruction which can be:

CR	Carriage return	new line.
EL	Erase line	deletes from cursor at the end of the line
IC	Insert character	inserts character to the right of cursor
IL	Insert line	inserts a line above the cursor
CS	Clear screen	deletes the contents of the Static Terminal window
RI	Right	moves cursor one position to the right
DO	Down	moves cursor a line under
HO	Home	moves cursor in line 1 column 1 position
EP	Erase page	deletes from cursor at the end of the page
DC	Delete character	deletes the character to the left of the cursor
DL	Delete line	deletes the line of the cursor
LE	Left	moves cursor a character to the left
UP	Up	moves cursor a line above

The attributes must be inserted in the instruction with this syntax: ~WRITE “\string attribute”; that is: ~WRITE “\CR Protection active”; A new line is inserted and the instruction writes “Protection active”.

SEE ALSO: ~FORMAT ~CURSOR ~COLOR

NOTE: It is possible to display an ASCII character by indicating the corresponding value in octal format. The octal value can be contained in a FLOAT type register or can be passed on by a constant. Refer to EXAMPLES 3 and 4.

EXAMPLE:

EXAMPLE 1

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE RUNTIME FLOAT VAR1;
DECLARE RUNTIME INTEGER CAN;
DECLARE RUNTIME INTEGER R13,R13;
DECLARE RUNTIME STRING RISULT;
START PRG1 STATIC ;
~IH S1;
/;
~IL S1 S2;
/;
~CALC VAR1=33.56;
~CALC RISULT="PASS";
~CALC CAN=256;
/;
~WRITE 12;                    !Displays a constant!

/;
~WRITE VAR1;                  !Displays a float runtime variable!
/;
~WRITE "\CR CHANNEL NR. " CAN;        !New line, and
                                      displays a string and
                                      an integer runtime
                                      variable!
/;
~WRITE RISULT;                !Displays a string type
                              runtime variable!
/;
~CURSOR 10,10;
~WRITE "\DO REVISIONE SW = " R12 "." R13;      !Moves to
                                               line and
                                               column 10,
                                               moves a line
                                               under
                                               (attribute
                                               \DO) and
                                               writes
/;
~IH S2;
/;
/;
~IH S1;
/;
ENDTEST;
```

EXAMPLE 2

The VAR1 variable will be displayed ten times, incrementing its value starting from 10.

```
.
.
.
~CALC VAR1=9;
~DO 10;
~CALC VAR1=VAR1+1;
~WRITE VAR1;
~DONE;
.
.
.
```

EXAMPLE 3

```
DECLARE RUNTIME FLOAT REG1;
START PRG1 STATIC;
~CALC REG1=46;          !Octal value corresponding to the
                        character " &"!
~WRITE "\R01";          !Displays the contents of the
                        REG1 register and that is the
                        character "&"!
/;
~PAUSE 1;
/;
ENDTEST;
```

EXAMPLE 4

```
START PRG2 STATIC;

~WRITE "%007";          !This instruction produces a sound, in
                        fact the value 007 corresponds to the
                        character BELL!
~PAUSE 1;
/;
~WRITE "%043"           !This instruction displays the
                        character #!
~PAUSE 2;
/;
ENDTEST;
```

~XOR — Instruction — ~XOR

Compiled	Run-time
	X

FUNCTION:

Calculates the XOR between the contents of the AR register and the specified argument.

SYNTAX:

~XOR value

value

This can be expressed as:

- constant
- an INTEGER type runtime variable

EXAMPLE:

```
DECLARE RUNTIME INTEGER VAR1,VAR2;
DECLARE RUNTIME FLOAT VAR;

START PRG1 STATIC;
/;
~CALC VAR1=3;
~LOAD VAR1;
~XOR B01101001;
~STORE VAR;
~WRITE "XOR di AR : ",VAR;
/;
~PAUSE 1;
/;
~CALC VAR1=0X55;
~CALC VAR2=0X0F;
~LOAD VAR1;
~XOR VAR2;
~STORE VAR;
~WRITE "XOR di AR : " VAR;
/;
~PAUSE 2;
/;
ENDTEST;
```

AG1…9 Instruction AG1…9

☛

Module	Compiled	Run-time
F40	**X**	

FUNCTION: Moves the group of channels at the instant specified by the ASSERT time (declared in the section of timing declaration).

SYNTAX: **AG1…9** group of channels=value

group of channels: Indicates the name of the group of channels to move. This name must be the same one defined in the DECLARE GROUP instruction.

value: Expresses the channels value where bit at 1 indicates a high level (IH) on the channel while bit a 0 indicates a low level (IL). The value can be expressed as:

- an INTEGER type constant
- an INTEGER type variable.

Integer constants can have the following prefixes:

B	indicates a constant in binary format
D or nothing	indicates a constant in decimal format
H or 0X	indicates a constant in hexadecimal format
O	indicates a constant in octal format

N.B. The first channel defined in the DECLARE GROUP instruction corresponds to the least significant bit (LSB) of the variable while the last signal defined in DECLARE GROUP instruction corresponds to the most significant bit (MSB) of the variable. Refer to the EXAMPLE.

SEE ALSO: DECLARE GROUP
OG1…9 group of channels, MG group of channels, ZG group of channels, GR group of channels.

NOTE: When the channels switch from SENSOR to DRIVER, the passage occurs at the associated assert time.

EXAMPLE:

```
DECLARE GROUP ADDLL = 21,22,23,24,25,26,27,28;
DECLARE GROUP ADDLH = 29,30,31,32,33,34,35,36;
DECLARE GROUP ADDHL = 37,38,39,40,41,42,43,44;
DECLARE GROUP ADDHH = 45,46,47,48,49,50,51,52;
DECLARE GROUP DATHH =185,186,187,188,
189,190,191,192;
DECLARE GROUP DATHL =177,178,179,180,
181,182,183,184;
DECLARE VARIABLE VAL,VAL1;

TIMING TIM1;
PERIOD 30C;
ASSERT1 2C;
STROBE1 28C;
ENDTIMING;
START PRG1;

START CLUS1;
.
.
.
IL ADDHH;
/;
AG1 ADDHH=0X55;
IG ADDHL=0X05;
IG ADDLH=0X55;
IG ADDLL=0XAA;
/;
@CALCOLATE VAL=0X4F;
IG DATHH=VAL;
/;
@CALCOLATE VAL1=B01110001;
IG DATHL=VAL1;
/;
.
.
ENDTEST;
```

AH1…9 Instruction AH1…9

☛

Module	Compile	Run-time
F40/DHF	**X**	

FUNCTION:

Moves the driver channel to high logic level at the instant defined by the ASSERT time (declared in the declaration section of the timing). For the F40 modules, the AH1..9 instruction can be used because the module can handle up to 9 assert times. For the DHF modules, only the instruction AH1 or AH are used.

SYNTAX:

AH1…9 channel

channel

Indicates the channel to switch at the ASSERT TIME. The channel can be:

- a single channel
- a group of channels
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," or "-" character is used to indicate non-contiguous channels.

SEE ALSO:

ASSERT1…9, AL1…9

NOTE:

When the channel goes from SENSOR to DRIVER, the passage occurs in the associated assert time.
The AH1 instruction is as alias of the AH instruction.

EXAMPLE:

```
DECLARE CHANNEL DAT0=34;
DECLARE CHANNEL DAT1=35;
DECLARE CHANNEL DAT2=36;
DECLARE CHANNEL DAT3=37;
DECLARE GROUP DAT=DAT0,DAT1,DAT2,DAT3;

CLOCK=150N;
TIMING TIM1;
PERIOD 30C;
ASSERT1 2C;
STROBE 28C;
ENDTIMING;

START PRG1;
IG DAT=B0101;
/;
IL DAT1,DAT3;
/;
AH1 DAT0,DAT1,DAT2,DAT3;
/;
AH DAT3;/;
ENDTEST;
```

AL1…9 Instruction AL1…9

☛

Module	Compiled	Run-time
F40/DHF	**X**	

FUNCTION: Moves the driver channel to low logic level at the instant defined by the ASSERT time (declared in the declaration section of the timing). For the F40 modules, the AL1..9 instruction can be used because the module can handle up to 9 assert times. For the DHF modules, only the instruction AL1 or AL are used..

SYNTAX: **AL1…9** channel

channel Indicates the channel to switch at the ASSERT TIME. The channel can be:

- a single channel
- a group of channels
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," or "-" character is used to indicate non-contiguous channels.

SEE ALSO: ASSERT1…9, AH1…9

NOTE: When the channel goes from SENSOR to DRIVER, the passage occurs at the associated assert time. The AL1 instruction has as alias the AL instruction.

EXAMPLE:

```
DECLARE CHANNEL DAT0=34;
DECLARE CHANNEL DAT1=35;
DECLARE CHANNEL DAT2=36;
DECLARE CHANNEL DAT3=37;
DECLARE GROUP DAT=DAT0,DAT1,DAT2,DAT3;
CLOCK=150N;

TIMING TIM1;
PERIOD 30C;
ASSERT1 2C;
STROBE 28C;
ENDTIMING;
START PRG1;
IG DAT=B0101;
/;
IH DAT0,DAT3;
/;
AL1 DAT0,DAT1,DAT2,DAT3;
/;
AL DAT3;/;
ENDTEST;
```

AS **Instruction** **AS**

Compiled	Run-time
X	

FUNCTION:

Switches the driver channel to the logic level opposite to the current one at the instant defined by the ASSERT time (declared in the declaration section of the timing).

SYNTAX:

AS channel

channel

Indicates the channel to switch at the ASSERT TIME. The channel can be:

- a single channel
- a group of channels
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," or "-" character is used to indicate non-contiguous channels.

SEE ALSO:

ASSERT, AL, AH

EXAMPLE:

```
DECLARE CHANNEL DAT0=34;
DECLARE CHANNEL DAT1=35;
DECLARE CHANNEL DAT2=36;
DECLARE CHANNEL DAT3=37;
DECLARE GROUP DAT=DAT0,DAT1,DAT2,DAT3;

CLOCK=150N;

TIMING TIM1;
PERIOD 30C;
ASSERT 2C;
STROBE 28C;
ENDTIMING;

START PRG1;
.
.
IG DAT=B0101;
/;
/;
AS DAT0,DAT1,DAT2,DAT3;
/;
ENDTEST;
```

ASSERT1…/9 **Instruction ASSERT1…/9**

☛

Module	Compiled	Run-time
F40/DHF	**X**	

FUNCTION: Indicates the switching time of the channels programmed with the AH, AL instructions inside the timing. All of the channels programmed with the instructions will switch at the same instant, which is the one defined by the ASSERT time. For the F40 modules, the ASSERT1..9 instruction can be used because the module can handle up to 9 assert times. For the DHF modules, only the instruction ASSERT1 or ASSERT are used.

SYNTAX: **ASSERT1…9 =**values

values Expresses the instant in which the channels are switched. The value must be included in the period indicated in the PERIOD instruction and can be expressed:

- as a multiple of the clock (3c, 7c, 8c,...)
- as time (1µs, 2µs,...)
- as frequency (2.5KHz, 1mhz)

To express it as time, the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

To express it as frequency, use the following letters:

KHZ	for	KHz	(kilo hertz)
MHZ	for	MHz	(mega hertz)

For the F40 modules the accepted range of values is : 20ns to 4980ns
For the DHF modules the minimum accepted value is 50ns.

SEE ALSO: PERIOD, AH, AL

NOTE: If the AH, AL, AS instructions are used in the program., make sure that the ASSERT time is defined in the timing, otherwise the channel behavior is unpredictable.

It is possible to define up to nine different assert times in a test program. For the F40 modules it is possible to use only four assert times for each module at the same time. But if inside the pattern a

phase on the channels of the board is already used, every phase precludes the use of an assert time.
The ASSERT1 instruction is alias of the ASSERT instruction.

EXAMPLE:

EXAMPLE 1 DEFINITION AND APPLICATION OF A TIMING WITH AN ASSERT TIME.

```
CLOCKF INT=20N;
TIMING TIM1;
STROBE1 900N;
ASSERT1 100N;
PERIOD 1000N;
ENDTIMING;
DECLARE CHANNEL D0=13;
DECLARE CHANNEL A0=16;

START PROG1 STATIC;
/ DINAMIC;
IL D0;
/;
AH1 D0;
/;
AL1 D0;
/;
ENDTEST;
```

EXAMPLE 2 DEFINITION OF THREE PHASES WITH UNCORRECT AND CORRECT APPLICATION.

```
CLOCKF=20N;
TIMING TIM1;
STROBE1 900N;
PHASE1 100N 300N;
ASSERT1 200N;
ASSERT2 400N;
ASSERT3 500N;
ASSERT4 120N;
ASSERT5 220N;
PERIOD 1000N;
ENDTIMING;
DECLARE CHANNEL D0=13;
DECLARE CHANNEL A0=16;
DECLARE CHANNEL CS1=20;
DECLARE CHANNEL CS2=22;

DECLARE CHANNEL CS3=25;

DECLARE CHANNEL ADS=40;

START PROG1 STATIC;
/ DINAMIC;
IL D0;
IL A0;
IL CS;
IL ADS;
/;
AH1 D0;
AH2 CS1;
```

```
AH3,CS2;
AH4 CS3;
AH5 ADS; ← CORRECT application, more than four phases at the
                                same time but not on the same
                                module ( the ADS signal is on a
                                different F40 module)
/;
P1 A0;
/;← UNCORRECT application, because 4 assert times are being used
                                and a phase on channels from the
                                same F40 module (the plans
                                defined in the previous pattern
                                remain active)
/;

ENDTEST;
```

BEGINLOOP-ENDLOOP Instruction BEGINLOOP-ENDLOOP

Module	Compiled	Run-time
DHF/F40	**X**	**X**

FUNCTION: Executes a loop, which is repeated as many times as specified by "num".

SYNTAX:

BEGINLOOP num
...
instructions
...
ENDLOOP Valid syntax for the DHF module

BEGINLOOP num
...
instructions
...
ENDLOOP [ONERROR] [ONPASS] Valid syntax for the F40 module

num: Whole number that indicates how many times the loop must be repeated. The range of num is as follows:

1 ≤ num ≤ 16384 For the DHF module
2 ≤ num ≤ 65000 For the F40 module

[ONERROR]: If specified, executes the loop until the error is present or until the set number of loops is reached.

[ONPASS]: If specified, executes the loop until the error is NOT present or until the set number of loops is reached.

NOTE: There can be up to 3 nesting levels with the BEGINLOOP/ENDLOOP instruction.

With repetitive patterns it is recommended to use the NOSTEP option so that the numeration of all repeated patterns is NOT listed.

The pattern with the ENDLOOP option with the ONERROR/ONPASS options, doesn't report any error. The error is generated by the next pattern.

It is not advisable to use the MASK option when the options ONERROR or ONPASS are present.

EXAMPLE:

```
BEGINLOOP 10;
COPY;

OM 1 2 3 4;
```

```
/;
COPY;
OL 1 2;
OH 3 4;
/ MASK NOPROBE;
/ ONERROR JUMP SOTTO MASK NOPROBE;

ENDLOOP;
```

Example for the F40 module

```
CLOCKF INT=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 100N, 300N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL D0=12;
DECLARE CHANNEL OUT1=12;

START PROG1;
/ TIMING TIM1;
IL D0;
/;
BEGINLOOP 10;
P1 D0;
OL OUT1;
/;
ENDLOOP ONERROR;
/;
BEGINLOOP 100;
IL D0;
/;
IH D0;
ENDLOOP;
/;
ENDTEST;
```

C1…9 Instruction C1…9

☛

Module	Compiled	Run-time
DHF/F40	**X**	

FUNCTION:

Executes the unmonitored forcing of the driver channels according to the movements indicated in the associated phase. . For the F40 modules, the C1..9 instruction can be used because the module can handle up to 9 phases. For the DHF module, which has three phases, only the instruction C1…3 instructions are used.

SYNTAX:

C1…9 channel

channel

Indicates the channel to drive according to what is specified in phase 1. The channel can be:

- a single channel
- a group of channels
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," or "-" character is used to indicate non-contiguous channels.

SEE ALSO:

PHASE1…9,P1…9

EXAMPLE:

```
EXAMPLE

DECLARE CHANNEL CLK=25;
DECLARE CHANNEL CLK1=74;
CLOCK= 100N;

TIMING TIM1;
PERIOD 10C;
PHASE1 3C,7C;
STROBE 9C;
ENDTIMING;

START PROG1;

IL CLK,CLK1;
/ TIMING TIM1;
C1 CLK;
/;
C1,CLK,CLK1;
/;

ENDTEST;
```

CALC — Instruction — CALC

Compiled	Run-time
X	

FUNCTION: In a COMPILED session of the program it executes logical arithmetic calculations and assignments between RUN-TIME constants and variables. This instruction is tipically used with WG and RG instructions.

SYNTAX: **CALC** variable=expression

variable — Destination variable. It can be a RUN-TIME INTEGER, FLOAT or STRING type variable.

expression — The expression can be a simple assignment (CALC variable=value) or an arithmetic/logic expression:

The values used in the expression can be:

- runtime variables of the type INTEGER, FLOAT, STRING
- constants of the type INTEGER, FLOAT, STRING

The operators, which can be used in the arithmetic/logic expression, are:

+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Xor
&	And
|	Or
<n	Shift n bits to the left
>n	Shift n bits to the right

The priority between the operands is from left to right if no parentheses are used. If parentheses are used, the entire expression must be enclosed between them.

NOTE: If operations are performed between variables and constants of different types and formats, each operand is converted into the type and format of the destination variable. See the chapter **Variables and constants** for the conversion of variables and constants of different types.

For string type variables and constants, the only valid operand is addition (+), which is the same as chaining.

EXAMPLE:

EXAMPLE 1

```
CLOCK 100N;
TIMING T1;
PERIOD 10C;
STROBE 9C;
ENDTIMING;

DECLARE CHANNEL D0=36;
DECLARE CHANNEL D1=88;
DECLARE CHANNEL D2=93;
DECLARE CHANNEL D3=123;

DECLARE CHANNEL A0=36;
DECLARE CHANNEL A1=88;
DECLARE CHANNEL A2=93;
DECLARE CHANNEL A3=123;
DECLARE CHANNEL A4=123;
DECLARE CHANNEL A5=123;
DECLARE CHANNEL A6=123;
DECLARE CHANNEL A7=123;

DECLARE RUNTIME INTEGER ADR PAGE ADL ADDRESS
DATA PATTERN;
DECLARE RUNTIME INTEGER ARRAY RIS[256];
DECLARE RUNTIME INDEX IDX;
DECLARE VARIABLE I;
DECLARE GROUP DAT=D0,D1,D2,D3;
DECLARE GROUP ADD=A0,A1,A2,A3,A4,A5,A6,A7;

START TEST4 STATIC;

~CALC PAGE=0;
~CALC PATTERN=H55;
~FOR PAGE=0 TO HFF STEP H10;
/ DMASTER TIMING T1;

CALC DATA=PATTERN;
CALC ADL=0;
CALC ADDRESS=PAGE+ADL;
WG ADD=ADDRESS;
WG DAT=DATA;

*WRITE CYCLE
/;
@FOR I=0 TO 255;
CALC ADDRESS=ADDRESS+1;
CALC DATA=DATA^HFF;
WG DAT=DATA;
/;
@ENDFOR;

*READ CYCLE
/;
CALC IDX=0;
```

```
CALC ADDRESS=PAGE+ADL;
WG ADD=ADDRESS;
RG RIS[IDX]=DAT;
/;
@FOR I=0 TO 255;
CALC ADDRESS=ADDRESS+1;
CALC IDX=IDX+1;
RG RIS[IDX]=DAT;
/;
@ENDFOR;

/ STATIC;
~ENDFOR;

ENDTEST;
```

EXAMPLE 2

```
DECLARE RUNTIME INTEGER RIS,K;
DECLARE RUNTIME INDEX J;
DECLARE RUNTIME INTEGER ARRAY VALORI[3];
DECLARE CHANNEL D0=36;
DECLARE CHANNEL D1=88;
DECLARE CHANNEL D2=93;
DECLARE CHANNEL D3=123;
DECLARE GROUP DAT=D0,D1,D2,D3;
START TEST4 STATIC;
~CALC VALORI[2]=1;
~CALC K=0;
/ DMASTER TIMING T1;
CALC J=K;
RG VALORI[J]=DAT;
/;
CALC J=J+1;
RG VALORI[J]=DAT;
/;
WG VALORI[2]=DAT;
ENDTEST;
```

When the compiled (dynamic) session of the program is executed (/ DMASTER), the compiler executes all CALC and WG insructions in sequence. In a second execution phase, the compiler executes the CALC and RG instructions in sequence. This mechanism allows the execution of a dynamic session changing its parameters each time. (i.e. changing the memory address each read/write operation in EXAMPLE 1).
Another advantage of writing/reading a memory using this method is that the entire ~FOR/~ENDFOR cycle its an interpreted instruction that takes few space in the object code. This feature is very important when dealing with memories or components that have a lot of data to write/read.

CLOCKF **Instruction** **CLOCKF**

☛

Module	Compiled	Run-time
F40	**X**	

FUNCTION: Sets the period of the clock referred to the F40 module. This signal is used as reference for the timing generation.
WARNING: This instruction is only listed for compatibility with the previous versions. From the rel. 3.14 the CLOCKF can be omitted because times are automatically set (to 20ns for clockf and to 100ns for clock).

SYNTAX: **CLOCKF [INT]=**value

value The value can only be:
20N that means 20ns

INT This parameter must be specified only if in the system there is not the DHF module and indicates that the clock used as reference is generated inside the F40 module.

NOTE: The CLOCK instruction set at 100N must be specified besides the CLOCKF if there is also the DHF module in the system.

EXAMPLE:

EXAMPLE 1: System equipped with only F40 modules.

```
CLOCKF INT=20N;
TIMING TIM1;
STROBE 900N;
PERIOD 1000N;
ENDTIMING;
```

EXAMPLE 2: System equipped with DHF and F40 modules.

```
CLOCKF=20N;
CLOCK=100N;
TIMING TIM1;
STROBE 900N;
PERIOD 1000N;
ENDTIMING;
```

CLOCK — Instruction — CLOCK

☛

Module	Compiled	Run-time
F40/DHF	**X**	**X**

FUNCTION:

Indicates the frequency (or the period) of the clock used to execute the functional program. the times expressed in the timing declaration (t. STROBE, t. ASSERT, t.DEAD etc.) will be referred to the clock value defined with this instruction.

SYNTAX:

CLOCK value

value

Indicates the clock value; it can be expressed either as a period or a frequency. The accepted range is:

50ns ≤ value ≤ 1µs or
1MHz ≤ value ≤ 20MHz

If the value is expressed as a period, the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

If the value is expressed as a frequency, the following letters are used:

KHZ	for	KHz	(kilo hertz)
MHZ	for	MHz	(mega hertz)

DEFAULT VALUES:

If the CLOCK instruction is not specified in the test program, the clock period is considered to be equal to 50ns.

NOTE: The resolution of the clock is 1 % of the defined value.

EXAMPLE:

EXAMPLE 1

Definition of the clock = 1 MHz with resolution of 10ns (1% of 1Mhz)

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE VARIABLE VAR1;
CLOCK 1MHZ;
TIMING TIM1;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
```

```
START PRG1;
IH S1 S2;
.
.
ENDTEST;
```

EXAMPLE 2

Definition of the clock = 0.550□s with resolution of 55ns (1% of 0.550µs)

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE VARIABLE VAR1;
CLOCK 0.550U;
TIMING TIM2;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG2;
IH S1 S2;
.
.
ENDTEST;
```

EXAMPLE 3

Definition of the clock =100ns with resolution of 10ps (1% of 100ns)

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE VARIABLE VAR1;
CLOCK 100N;
TIMING TIM3;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG3;
IH S1 S2;
.
.
ENDTEST;
```

EXAMPLE 4

Clock value omitted, in this case the value of 50ns is taken for the clock period by default.

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE VARIABLE VAR1;
TIMING TIM4;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG4;
IH S1 S2;
.
.
```

```
ENDTEST;
```

CLOCK2 — Instruction — CLOCK2

☛

Module	Compiled	Run-time
DHF	**X**	**X**

FUNCTION:

Indicates the oscillating frequency (or period) of the auxiliary clock present on the BNC of the digital controller during the execution of the functional program. The auxiliary clock is a square waveform with a width of 5V referenced to system ground. The resolution of the clock is !% of the defined frequency. It can be used to synchronize with external instruments.

SYNTAX:

CLOCK2 value

value

Indicates the value of the frequency of period of the clock and the accepted range is:

16.66ns ≤ value ≤ 6666.66ns or
150Khz ≤ value ≤ 60Mhz

To express the value as time, the following letters are used:

P for ps (pico seconds)
N for ns (nano seconds)
U for µs (micro seconds)
M for ms (milli seconds)

To express the value as frequency, use the following letters:

KHZ for KHz (kilo hertz)
MHZ for MHz (mega hertz)

P = ps pico seconds
N = ns nano seconds
U = µs micro seconds
M = ms milli seconds

kHz = KHz kilo hertz
MHZ = MHz mega hertz

DEFAULT VALUES:

By default the auxiliary clock frequency is 14.192Mhz, during the execution of the functional program.

EXAMPLE:

EXAMPLE 1

Clock signal = 1 MHz with resolution of 10ns (1% of 1MHz)

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE VARIABLE VAR1;
```

```
CLOCK2 1U;

TIMING CTS1;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG1;
IH S1 S2;
/;
ENDTEST;
```

EXAMPLE 2

Clock signal = 200 KHz with resolution of 2KHz (1% of 200 KHz)

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE VARIABLE VAR1;
CLOCK2 200KHZ;

TIMING CTS1;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG1;
IH S1 S2;
/;
ENDTEST;
```

CONFIGURATION — Compiler Directives — CONFIGURATION

Compiled	Run-time
X	**X**

FUNCTION:

Allows the development of a test program as if the system hardware configuration were different from the current one. This is useful when developing programs for another system, which will be where the program will actually be installed.

SYNTAX:

CONFIGURATION=channel board1,channel board2,...

channel board 1,channel board2,...

Indicates the type of channel board used in the configuration. The channel board is expressed as a letter of the alphabet as indicated below:

D	RTTL channel board
S	P32, H32 or L32 channel board
R	R40 channel board
F	F40 channel board
X	X64 channel board

DEFAULT VALUES:

If the CONFIGURATION instruction is not specified, the current system configuration is taken by default.

NOTE: The CONFIGURATION instruction must be written before the START - ENDTEST instruction

CAREFUL!!

It is not advisable to execute a test program on a system with a configuration different from the one specified in the CONFIGURATION instruction.

EXAMPLE:

Assuming that we are working on a hardware configuration, which has one RTTL board, one R40 board, three H32 boards and two P32 boards.

```
#INCLUDE FUNZMAC
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=4;
DECLARE VARIABLE VAR1;
CLOCK=50N;
TIMING CTS1;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
@COMPILER COMPLEX=5000;
@COMPILER MAX_ERROR=400;
CONFIGURATION= DRSSSSS;
START PRG1;
…..
.ENDTEST;
```

COPY — Instruction — COPY

Compiled	Run-time
X	**X**

FUNCTION: Specifying this instruction, the compiler reprograms the channel situation for each pattern.

SYNTAX: **COPY**

DEFAULT VALUES: If the COPY instruction is not specified, the INCREMENTAL compilation mode is taken by default.

SEE ALSO: INCREMENTAL

EXAMPLE:

```
DECLARE CHANNEL CH1=48;
START PRG1;
/;          !pattern1!
COPY;
IH CH1;
/;          !pattern2       CH1 is programmed high!
/;          !pattern3       CH1 is programmed high!
/;          !pattern4       CH1 is programmed high!
/;          !pattern5       CH1 is programmed high!
/;          !pattern6       CH1 is programmed high!
IH CH1;
/;          !pattern7       CH1 is programmed high!
ENDTEST;
```

We will now compare the EXAMPLE just described of the COPY instruction with that of the INCREMENTAL instruction.

```
DECLARE CHANNEL CH1=48;
START PRG1;
/;          !pattern1!
INCREMENTAL;
IH CH1;
/;          !pattern2       CH1 is programmed high!
/;          !pattern3       the channel is not programmed!
/;          !pattern4       the channel is not programmed!
/;          !pattern5       the channel is not programmed!!
/;          !pattern6       the channel is not programmed!
IH CH1;
/;          !pattern7       the channel is not programmed
                            because the channel is already
                            high!
ENDTEST;
```

CR **Instruction** **CR**

☛

Module	Compiled	Run-time
F40	**X**	

FUNCTION: Sets the signal as repetitive clock inside the period. From the moment the instruction is specified, the channel becomes repetitive inside the period. That is: if we have a period of 4µs and a phase1 of 100n, 200n, the channel will commute every 100ns within the period, until it reaches 4µs.

SYNTAX: **CR** channel

channel Indicates the channel that must become repetitive clock. It can be:

- a single channel
- a group of channels
- a channel name defined with the DECLARE CHANNEL instruction.

The "/" character is used to indicate a group of contiguous channels and the "," or "-" character is used to indicate non-contiguous channels.

NOTE: The duration of the period must be a time multiple of the clock period.

To "turn-on" a repetitive channel, it must be associated to a phase and must be "not controlled" using the instructions C1…9. See example 1.

To "turn-off" a repetitive channel, the NR instruction must be used and the channel returns to behave in the way specified in its associated phase.

When the repetitive clock is "turned off", the channel executes one more commutation. To obviate this disadvantage, use a timing longer than 20ns. Refer to the EXAMPLE 2.

SEE ALSO: NR

EXAMPLE: EXAMPLE 1

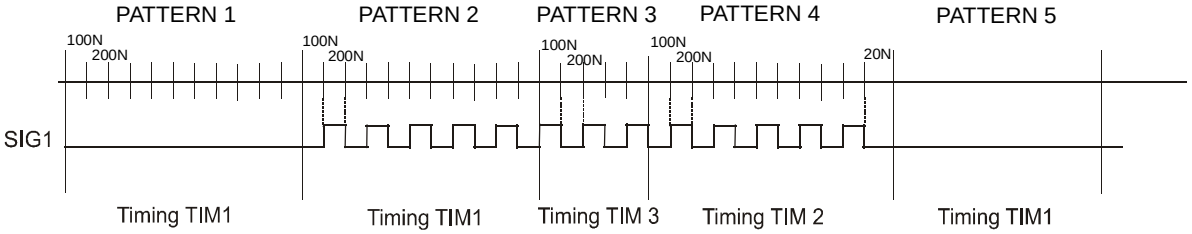
```
CLOCKF=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 100N, 200N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL SIG1=12;
```

```
START PROG1;
/ TIMING TIM1;
IL SIG1;
C1 SIG1;    Channels associated to phase 1 not controlled
/;
CR SIG1;
/;
NR SIG1;
/;
/;
ENDTEST;
```

EXAMPLE 2

```
CLOCKF INT =20N;
TIMING TIM1;
PERIOD 1000N;
PHASE1 100N, 200N;
STROBE 900N;
ENDTIMING;

TIMING TIM2;
PERIOD 1020N;
PHASE1 100N, 200N;
STROBE 900N;
ENDTIMING;

TIMING TIM3;
PERIOD 600N;
PHASE1 100N, 200N;
STROBE 500N;
ENDTIMING;

DECLARE CHANNEL SIG1=12;
START PROG1;
IL SIG1;            channel initialization
/ TIMING TIM1;
C1 SIG1;            Channel associated to phase 1, not controlled
CR SIG1;            Here the repetitive clock is “turned-on”
/;                  Pattern with repetitive clock
/ TIMING TIM3;      Pattern with a different timing
/ TIMING TIM2;      A timing longer than 20N is used, to ”turn-off”
                    the repetitive clock rightly
NR SIG1;            Here the repetitive clock is “turned-off”
/ TIMING TIM1;      Here, TIM1 timing is used again
ENDTEST;
```

Example 2 is illustrated here:



DEAD — Instruction — DEAD

Compiled	Run-time
X	

FUNCTION: Indicates the minimum time for programming the channels before the operative phase of the pattern.

SYNTAX: **DEAD** value

value

Expresses the minimum time dedicated to programming the channels before the operative phase of the pattern.

The value can be expressed as:

- as a multiple of the clock (3c, 7c, 8c,...)
- as time (1µs, 2µs,...)
- as frequency (2.5KHz, 1MHz)

To express it as time, the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

To express it as frequency, use the following letters:

KHZ	for	KHz	(kilo hertz)
MHZ	for	MHz	(mega hertz)

DEFAULT VALUES: If the dead time is not specified, the time available between the start of the operative phase of the pattern and the first event is used (which can be a t OVERLAY, t ASSERT, PHASE1/2/3).

NOTE: The dead time is inserted outside of the pattern duration defined by the PERIOD instruction.
If the DEAD time is insufficient for programming the channels of the pattern, an EXTRADEAD time is inserted automatically and a WARNING is generated during the compilation of the test program.
If the channel programming time is less than the duration of the DEAD time, this time is inserted anyway.

EXAMPLE:

```
CLOCK 150N;
TIMING TIM1;
PERIOD 12C;
DEAD 4C;
STROBE 11C;
ENDTIMING;
START PRG1;
.
.
ENDTEST;
```

DECLARE ARGUMENT ARGUMENT **Instruction** **DECLARE**

Compiled	Run-time
X	

FUNCTION: Allows the declaration of an argument, which will then be used as a parameter in the calls to the macros. The argument thus defined is global, that is, it can be used in any part of the functional program.

SYNTAX: **DECLARE ARGUMENT** = name argument1, name argument2,...

name argument1,
name argument2,... Alphanumerical string maximum length 20 characters which indicates the name of the argument in question.

NOTE: Up to 100 arguments/variables can be defined in the functional program.

EXAMPLE:

```
DECLARE CHANNEL FC0=14;
DECLARE CHANNEL FC1=15;
DECLARE CHANNEL FC2=16;
DECLARE CHANNEL SIZE0=45;
DECLARE CHANNEL ASN=76;

DECLARE ARGUMENT=VAHH,VAHL;

@MACRO RDROM (VAHH,VAHL);
IG ADDHH=VAHH;
IG ADDHL=VAHL;
IL FC0,FC2,SIZE0;
IH RW,FC1;
IL ASN;
@ENDMACRO;


START MAC;
.
.
@RDROM (0X00, 0X01);
/;
@RDROM (0X00, 0X02);
.
.
ENDTEST;
```

DECLARE CHANNEL **Instruction** **DECLARE CHANNEL**

☛

Compiled	Run-time
X	**X**

FUNCTION: Assigns a name to the physical system channel. The name is then used in the functional program instructions with the advantage that when the physical channel number is modified, the instructions are referred to the new one automatically.

SYNTAX: **DECLARE CHANNEL** channel name = channel number

channel name: Alphanumerical string, which indicates the name, assigned to the physical channel. The channel name length can be a maximum of 20 characters.

channel number: Indicates the number of the physical channel.

NOTE: Up to 1024 DECLARE CHANNEL/DECLARE PIN instructions can be declared in a functional program.

SEE ALSO: DECLARE GROUP

EXAMPLE:

```
EXAMPLE

DECLARE CHANNEL RESETN = 2;
DECLARE CHANNEL S2=12;
DECLARE CHANNEL S4=22;

START PRG1;

IL RESETN;                    !board reset!
/;
/;
IH RESETN;
/;
/;
IL S2;
/;
IH S4;
/;
.
.
ENDTEST;
```

DECLARE GROUP **Instruction** **DECLARE GROUP**

Compiled	Run-time
X	**X**

FUNCTION: Assigns a name to a group of channels.

SYNTAX: **DECLARE GROUP** group name =channel group

group name: Alphanumerical string that indicates the name assigned to the group of channels. The group name can be a maximum of 20 characters long.

channel group: Indicates the group of physical channels associated to the group. The channels are separated by a comma “,”.
It is possible to express the channels with a physical number or with the channel names defined previously in the DECLARE CHANNEL instruction. See the EXAMPLE.

A maximum of 64 channels can be defined for each DECLARE GROUP instruction, but when we assign a value to the group, the instructions must be executed on two groups of 32.

SEE ALSO: DECLARE CHANNEL, IG, OG, MG, ZG, GR

NOTE: A maximum of 100 DECLARE GROUP instructions can be defined per functional program.

Names of groups previously defined can also be used for new DECLARE GROUP. Refer to the EXAMPLE 2.

The movement or read of the channels defined in a group is carried out via the IG, OG, MG, ZG, GR instructions.

N.B. The first channel defined in the DECLARE GROUP instruction will correspond to the least significant bit (LSB) of the value expressed with the abovementioned instructions while the last signal defined in the group will correspond to the most significant bit (MSB).

Refer to the EXAMPLE.

EXAMPLE:

EXAMPLE 1

```
DECLARE CHANNEL RESETN=156;
DECLARE CHANNEL D0 = 1;
DECLARE CHANNEL D1 = 2;
DECLARE CHANNEL D2 = 3;
DECLARE CHANNEL D3 = 4;
DECLARE CHANNEL D4 =5;
DECLARE CHANNEL A0 = 20;
DECLARE CHANNEL A1 = 21;
DECLARE CHANNEL A2 = 22;
DECLARE CHANNEL A3 = 23;
```

```
DECLARE GROUP DATI =1,2,3,4;
DECLARE GROUP ADDR =A0,A1,A2,A3;

START PRG12;

IL RESETN;                    !board reset!
/;
/;
IH RESETN;
/;
/;
IL S2;
/;
IH S4;
/;
IG DATI=B0101;    !assigns the low value to channels 1 and 3 and
                  the high value to channels 2 and 4!

IG ADDR=0XC;      !assigns the high value to channels A0 and A1
                  and the low value to A2 and A3!
.
.
ENDTEST;
```

EXAMPLE 2

```
DECLARE CHANNEL D0 = 1;
DECLARE CHANNEL D1 = 2;
DECLARE CHANNEL D2 = 3;
DECLARE CHANNEL D3 = 4;
DECLARE CHANNEL D4 =5;
DECLARE CHANNEL D5 = 20;
DECLARE CHANNEL D6 = 21;
DECLARE CHANNEL D7 = 22;
DECLARE CHANNEL D8 = 23;

DECLARE GROUP DATL =DO,D1,D2,D3;
DECLARE GROUP DATH =D4,D5,D6,D7;

DECLARE GROUP DATABUS =DATL,DATH;

START PRG13;

IG DATL=B0101; !assigns values to the data D0..D3
/;
IG DATH=0XC; !assigns values to the data D4..D7
/
IG DATABUS=0XA4; assigns a value to the bus
/;
IL DATABUS;
/;
IH DATABUS;
/;
ENDTEST;
```

DECLARE PIN — Instruction — DECLARE PIN

Compiled	Run-time
X	**X**

FUNCTION: Allows the association of the channel name used in the library (P1, P2,...Pn) to the channel name defined in the DECLARE CHANNEL instruction.

SYNTAX: **DECLARE PIN** library name = channel name

library name — Name used in the library to describe the channels.

channel name — Channel name defined in the DECLARE CHANNEL instruction.

SEE ALSO: DECLARE CHANNEL

NOTE: It is possible to define up to 1024 DECLARE PIN/ DECLARE CHANNEL per functional program.

EXAMPLE:

```
DECLARE CHANNEL S1=34;
DECLARE CHANNEL S2=39;
DECLARE CHANNEL S3=40;

DECLARE PIN P1=S1;
DECLARE PIN P2=S2;
DECLARE PIN P3=S3;
DECLARE PIN P4=S4;

START PRG1;
/;
IL S1,S3;
/;
OH S2,S4;
/;
IH S1,S3;
/;
OL S2,S4;
/;
ENDTEST;

Library  file

*************************
*************************
*****FILE S20LOG.LIB *****
*************************
*************************
```

```
#7400(COMM,P1,P2,P3,P4,P5,P6,P7,P8,P9,P10,P11,P12,P13,P14)
!D,4,14;
*PI(IIOIIO.OIIOII.);
*TY(......G......V);
!3,2,1,0,0,P1,P2,P3;
!3,2,1,0,0,P4,P5,P6;
!3,2,1,0,0,P9,P10,P8;
!3,2,1,0,0,P12,P13,P11;
@C(4.7,0,2.4,.8,.1,1,P,C);
@L(HH0...........);
@L(Lh1...........);
@L(hL1...........);
@L(...HH0........);
@L(...Lh1........);
@L(...hL1........);
@L(.......0HH....);
@L(.......1Lh....);
@L(.......1hL....);
@L(..........0HH.);
@L(..........1Lh.);
@L(..........1hL.);
.
.
.
```

!These names
are associated to
the channel names!

DECLARE PINLIST **Instruction** **DECLARE PINLIST**

Compiled	Run-time
X	**X**

FUNCTION: Allows the declaration of a group of channels which will be moved with the @L instruction in which the status each channel must assume is specified (driver high, driver low,...)

SYNTAX: **DECLARE PINLIST=**channel name1, channel name2,...

channel name1, channel name2,... List of channels in the pinlist. The "/" is used to indicated a contiguous group of channels and the "," character is used to indicate non-contiguous channels.
It is possible to express the channels with the physical numbers or with the channel names defined previously in the DECLARE CHANNEL instruction.
Up to 256 channels can be defined for each pinlist.

SEE ALSO: @L, DECLARE CHANNEL

NOTE: If more DECLARE PINLIST instructions are defined in a functional program, only the last one is considered valid.
The DECLARE PINLIST allows the declaration of up to 256 channels.

The correspondence between @L instruction and the DECLARE instruction is as follows:

DECLARE PINLIST=CH1,CH2,...CHn;

@L(S1,S2,.....Sn);

The first status defined in the @L instruction (S1) is associated to the first channel defined in the DECLARE PINLIST instruction (CH1), status 2 (S2) to channel 2 (CH2),.... status n (Sn) is associated to channel n (Chn). See EXAMPLE 1.

EXAMPLE:

EXAMPLE 1

```
DECLARE PINLIST = 1,2,3,4,5,6,7,8,9,10,11,12,13,14;

START PRG1;

@L(HHL01.........);      !Channel 1 and 2 are forced high, channel 3
                         is forced low, channel 4 is measured low,
```

channel 5 is measured high, the other channels are masked!

```
@L(Lh1...........);
@L(hL1...........);
@L(...HH0........);
@L(...Lh1........);
@L(...hL1........);
@L(.......0HH....);
@L(.......1Lh....);
@L(.......1hL....);
@L(..........0HH.);
@L(..........1Lh.);
@L(..........1hL.);

ENDTEST;
```

EXAMPLE 2

```
DECLARE CHANNEL A1=22;
DECLARE CHANNEL A2=23;
DECLARE CHANNEL A3=24;
DECLARE CHANNEL A4=25;

DECLARE PINLIST= A1,A2,A3,A4;
.
.
.
```

DECLARE RUNTIME **Instruction** **DECLARE RUNTIME**

Compiled	Run-time
	X

FUNCTION: Declares a runtime variable used in the program.

SYNTAX: **DECLARE RUNTIME** variable type = variable name1, variable name2,...

variable

Indicates the type of variable declared, which can be:

- INTEGER
- FLOAT
- STRING

The INTEGER type variables express whole numbers, and the range of values is:

-0x7FFFFFFF ≤ variable ≤ +0x7FFFFFFF

The FLOAT type variables express floating-point numbers and the range of values is:

-1.7E+308 ≤ variable ≤ + 1.7E+308

The STRING type variables express strings of characters, and can accept:

up to 80 characters enclosed in quotation marks (“”)

variable name 1,
variable name2,...

Alphanumerical string maximum 20 characters which indicates the name of the variable.
The underscore “_” is accepted in the variable name while other special characters such as @ # $ % ~ etc... are not accepted.

The variable name can also be a register REG1-100 which will be used in the ~CURSOR instruction. When a register of this type is defined, it is only necessary to define the one with the highest index, the other ones are automatically defined. Refer to the EXAMPLE 2.

Up to 750 variables/arguments can be defined.

SEE ALSO: DECLARE VARIABLE

EXAMPLE: EXAMPLE 1

```
DECLARE RUNTIME INTEGER A
DECLARE RUNTIME INTEGER B,J,IN1;
DECLARE RUNTIME FLOAT TEMP,CUR;
```

```
DECLARE RUNTIME STRING NOME;
START PRG1 STATIC;

~CALC A=3;
~CALC B=5;
~CALC TEMP=14.7;
~CALC CUR=150;
~CALC NOME=“CHANNEL nr. 1”;
/;
ENDTEST;
```

EXAMPLE 2
Register declaration.

```
DECLARE VARIABLE FLOAT REG10;
DECLARE VARIABLE INTGER I;

START FGGF STATIC;
~CALC REG1=1;
~CALC REG10=1;
~FOR I=1 TO 10;
~CURSOR REG1,REG10;
~WRITE “x”
~CALC REG1=REG1+1;
~ENDFOR;
~PAUSE;
~ENDFOR;
ENDTEST;
```

DECLARE RUNTIME ARGUMENT ARGUMENT **Instruction** **DECLARE RUNTIME**

Compiled	Run-time
	X

FUNCTION:

Declares a runtime argument used afterwards in a subroutine. The argument thus defined is a global type that is it is "seen" by all subroutines of the program.

SYNTAX:

DECLARE RUNTIME ARGUMENT argument type name argument 1, argument 2,...

variable type

Indicates the type of variable declared, which can be:

- INTEGER
- FLOAT
- STRING

The INTEGER type variables express whole numbers, and the range of values is:

-0x7FFFFFFF ≤ variable ≤ +0x7FFFFFFF

The FLOAT type variables express floating-point numbers and the range of values is:

-1.7E+308 ≤ variable ≤ + 1.7E+308

The STRING type variables express strings of characters, and can accept:

up to 80 characters enclosed in quotation marks ("")

argument name 1,
argument name 2,...

Alphanumerical string maximum 20 characters which indicates the name of the argument.
The underscore "_" is accepted in the variable name while other special characters such as @ # $ % ~ etc... are not accepted.

SEE ALSO:
DECLARE VARIABLE

DECLARE RUNTIME VARIABLE

EXAMPLE:

```
DECLARE CHANNEL CLK=34;
DECLARE RUNTIME INTEGER I;
DECLARE RUNTIME ARGUMENT INTEGER NUMCLK

START PRG1 STATIC;
~START_SUB;
```

```
~SUBR CLOCK(NUMCLK);
~FOR I=1 TO NUMCLK;
~IL CLK;
/;
~IH CLK;
/;
~ENDFOR;
~RETURN;
~END_SUB;

~CALL CLOCK(23);
~CALL CLOCK(*);
```

Since the argument is global, if it is recalled in this way, the previous value of NUMCLK is taken, that is 23.

```
ENDTEST;
```

DECLARE RUNTIME ARRAY ARRAY **Instruction** **DECLARE RUNTIME**

Compiled	Run-time
	X

FUNCTION: Declares a runtime array that will be used later in the program. The static arrays must be considered as a group of runtime variables and for this reason, all properties of run-time variables can be here applied. Arrays are very useful to compare the contents of a memory.

Arrays can be of three types: INTEGER, FLOAT and STRING.

SYNTAX:

DECLARE RUNTIME INTEGER ARRAY array name[dimension]
DECLARE RUNTIME FLOAT ARRAY array name[dimension]
DECLARE RUNTIME STRING ARRAY array name[dimension]

array name

Alphanumerical string maximum 20 characters which indicates the name of the array. The underscore "_" is accepted in the variable name while other special characters such as @ # $ % ~ etc... are not accepted.

dimension

Array dimensions. Can be expressed as a whole number or as a INTEGER type variable. The maximum dimension is 65500 considered as the total sum of all INTEGER type arrays and of INTEGER runtime variables of the program.

The maximum dimension is 4100 considered as the total sum of all FLOAT type arrays and of FLOAT runtime variables of the program.

The maximum dimension is 2050 considered as the total sum of all STRING type arrays and of STRING runtime variables of the program.

The array index can be:

- a constant
- an integer type variable (defined with the declare variable instruction)
- an index type variable (defined with the DECLARE RUNTIME INTEGER INDEX instruction). It is mandatory to use this type when the index within the ~FOR, ~WHILE loops varies (increase or decrease). Up to 7 different indexes can be defined. When an index is used, it must be preceded by the character %. Refer to the example and to the description of the DECLARE RUNTIME INDEX control.

SEE ALSO: DECLARE RUNTIME INDEX instruction.

NOTE: Arrays must be defined before the START instruction of the program or in subroutines that declare local variables.

The first array cell starts from address 0. Hence if an array with dimension 3 is defined, it has got four cells.

EXAMPLE: Definition of a float type array of five cells, array initialization and contents display.

```
DECLARE RUNTIME FLOAT ARRAY VET[4];
DECLARE VARIABLE IND;
DECLARE RUNTIME INTEGER INDEX A;
START PRO STATIC;

~CALC VET[1]=3.3;          Initialization of cell 0. In this case the
                           index is a constant.

@CALCOLATE IND=1;
~CALC VET[%IND]=4.6;       Initialization of cell 2. In this case the
                           index is an integer variable .

~CALC A=2;
~CALC VET[A]=8.6;          Initialization of cell 3. In this case the
                           index is an INDEX type runtime
                           variable.

~CALC VET[3]=VET[%IND]+1; Cells addition.
                                 In VET[3] the value is 5.6 (4.6+1)

~FOR A=0 TO 3;     contents display of the vector, on video. In this
                   case, since the index changes within a ~FOR
                   loop, it must be defined as INDEX type.

~CURSOR 10,A;
~WRITE VET[A];
~ENDFOR;

ENDTEST;
```

DECLARE RUNTIME INDEX INDEX | **Instruction** | **DECLARE RUNTIME**

Compiled	Run-time
	X

FUNCTION: Declares an INDEX type variable to use as index for runtime type arrays. It is necessary to define this variable if the array index changes within the ~FOR or ~WHILE loops.

SYNTAX: **DECLARE RUNTIME INDEX** index name

index name: Alphanumerical string maximum 20 characters, which indicates the name of the index. The underscore "_" is accepted in the variable name while other special characters such as @ # $ % ~ etc... are not accepted.

SEE ALSO: DECLARE RUNTIME ARRAY

NOTE: Up to seven different indexes can be defined in the program.

EXAMPLE: Definition of a two-cells integer type array and initialization.

```
DECLARE RUNTIME INTEGER ARRAY VET[2];
DECLARE RUNTIME INTEGER INDEX A;
START PRO STATIC;

~CALC A=1;
~CALC VET[A]=8;          Initialization of cell 1.

~CALC A=A+1;
~CALC VET[A]=2;          Initialization of cell 2.


~FOR A=1 TO 2;     Display of the contents of the vector, on video.
~CURSOR 10,A;
~WRITE VET[A];
~ENDFOR;

ENDTEST;
```

DECLARE SIGNAL SIGNAL | **Instruction** | **DECLARE**

Compiled	Run-time
X	**X**

FUNCTION: Defines a signal. The defined signal can be handled like a channel name or as a true variable. It is therefore possible to execute both arithmetic/logic calculations and move it as if it were a channel.

SYNTAX: **DECLARE SIGNAL** signal name

signal name: Alphanumerical string, which indicates the name, assigned to the signal. The signal name can be a maximum of 20 characters long.

SEE ALSO: DECLARE CHANNEL

NOTE: It is possible to define up to 1024 signals within a functional program.

EXAMPLE: EXAMPLE

```
DECLARE SIGNAL CAN1;
START PRG1;
@CALCOLATE CAN1=1;
@FOR CAN1=1 TO 15;          !Executes the subsequent
IH CAN1;                    movement to the high
/;                          level of channels 1 to 14!
@ENDFOR;
.
.
ENDTEST;
```

DECLARE SUBROUTINE SUBROUTINE **Instruction** **DECLARE**

Compiled	Run-time
	X

FUNCTION: When subroutines are used in the program, they have to be declared at the beginning with the DECLARE SUBROUTINE instruction.

SYNTAX: **DECLARE SUBROUTINE** subroutine name 1, subroutine name 2,…

subroutine name 1,
subroutine name 2,... Alphanumerical string maximum 20 characters which indicates the name of the variable.

NOTE: Up to 256 arguments/variables can be defined in the functional program.

EXAMPLE:

```
DECLARE SUBROUTINE PROVA;

~SUBR PROVA;
~CURSOR 1,1;
~WRITE "TEST SUBROUTINE"
~PAUSE 1;
~RETURN;
~ENDSUBR;

START PROFUN STATIC ;
~CALL PROVA;
ENDTEST;
```

DECLARE VARIABLE **Instruction** **DECLARE VARIABLE**

Compiled	Run-time
X	

FUNCTION: Declares an INTEGER type variable used in the program.

SYNTAX: **DECLARE VARIABLE** variable name1, variable name2,...
or
DECLARE VARIABLE variable name=initialization value

variable name 1,
variable name 2,... Alphanumerical string maximum 20 characters which indicates the name of the variable. The underscore "_" is accepted in the variable name while other special characters such as @ # $ % ~ etc... are not accepted.

Initialization value Integer number to initialize the variable

SEE ALSO: DECLARE RUNTIME INTEGER
DECLARE RUNTIME FLOAT
DECLARE RUNTIME STRING

NOTE: Variables must be defined before the START instruction.
The INTEGER type variables express whole numbers, and the range of values is:

-0x7FFFFFFF ≤ variable ≤ +0x7FFFFFFF

Up to 128.000 variables/arguments can be defined in a functional program.

EXAMPLE:

```
DECLARE VARIABLE A,B,IN1; Variable declarations
DECLARE VARIABLE J=5; Declaration and initialization
START PRVAR;
@CALC A=2;
@CALC B=1;
@CALC IN1=15;
/;
@CALC A=A+B/J;
@CALC IN1=IN1-A;
ENDTEST;
```

DECLARE VARIABLE ARRAY ARRAY **Instruction** **DECLARE VARIABLE**

Compiled	Run-time
X	**X**

FUNCTION: Declares an array of integer numbers that will be used in the program.

SYNTAX: **DECLARE ARRAY** array name[dimension] or
DECLARE ARRAY array name[dimension]=initialization values

array name — Alphanumerical string maximum 20 characters which indicates the name of the array. The underscore "_" is accepted in the variable name while other special characters such as @ # $ % ~ etc... are not accepted.

dimension — Array dimension. It can be expressed as a whole number or as a INTEGER type variable. The maximum dimension is 65000 considered as the total sum of all arrays in the program.

Initialization values — Whole numbers separated by commas, which initialize the contents of the array. It is possible to initialize the array during its definition.

NOTE: The arrays must be defined before the START instruction of the program.

The first array cell starts from address 0. Hence if an array with dimension 3 is defined, it has got four cells.

Arrays are very useful to compare contents of a memory.

EXAMPLE:

```
DECLARE ARRAY A1[4];                Definition of array A1 of five elements
DECLARE ARRAY A2[4]=1,4,78,23,89;   Definition of array A2 of 5 elements and initialization
DECLARE CHANNEL CS=13;
DECLARE CHANNEL RD=77;
DECLARE CHANNEL D0=346;
DECLARE CHANNEL D1=745
DECLARE CHANNEL D2=76;
DECLARE CHANNEL D3=78;
DECLARE GROUP DATA=D0,D1,D2,D3;
DECLARE VARIABLE I;
START PRO;
@CALCOLATE A1[0]=23;                Array A1. Initialization of cell 0
@CALCOLATE A1[1]=A2[0];             Array A1. Initialization of cell 1
@CALCOLATE A1[2]=35;                Array A1. Initialization of cell 2
```

```
                                    @FOR I=0 TO 3;
IL CS;
/;
IL RD;
/;
OG DATA=A2[I];                          Data comparison
                                    /;
                                    IH RD;
                                    /;
                                    IH CS;
                                    /;
                                    @ENDFOR;
                                    ENDTEST;
```

This loop reads from a memory and compares the contents read with the contents of the array.

DELAY **Instruction** **DELAY**

☛

Module	Compiled	Run-time
F40	**X**	

FUNCTION:

Sets a delay time between the current pattern and the following one. During the delay time the sensor signals are not controlled, the channels associated to phases and repetitive clock are not generated.

SYNTAX:

DELAY time

time

Indicates the waiting time and can be expressed with the following letters:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)
S	for	s	(seconds)

The accepted value is from 40ns to 30s.

EXAMPLE:

```
CLOCKF INT=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 100N, 300N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL D0=12;
DECLARE CHANNEL OUT1=12;

START PROG1;
/ TIMING TIM1;
IL D0;
/;
OL OUT1;
DELAY 1S; ← In this pattern the DELAY time is executed first and
            then there is the control on the sensor channel
/;
ENDTEST;
```

EOF — Instruction — EOF

Compiled	Run-time
X	**X**

FUNCTION: Defines the end of file.pat. Useful to insert comments or notes after the end of the program.

SYNTAX: **EOF**

EXAMPLE:

```
DECLARE RUNTIME INTEGER A,B,IN1; Variable declarations
DECLARE RUNTIME INTEGER J;
START PRVAR STATIC;
~CALC A=2;
~CALC B=1;
~CALC IN1=15;
/;
~CALC A=A+B/J;
~CALC IN1=IN1-A;
ENDTEST;
EOF;
*comments*
```

DECLARE RUNTIME FLOAT DF, variable once used to store the tension value.

GR — Instruction — GR

Compiled	Run-time
X	**X**

FUNCTION:

Allows the modification of the logic level of a group of driver or sensor channels according to the specified value. The new value assumed by the channel following the GR instruction is illustrated in the following table:

PREVIOUS STATUS OF THE CHANNEL	VALUE SPECIFIED IN THE "GR" INSTRUCTION	NEW STATUS OF THE CHANNEL
IL	1	IH
IL	0	IL
IH	1	IH
IH	0	IL
OL	1	OH
OL	0	OL
OH	1	OH
OH	0	OL

SYNTAX:

GR group name = value

group name

Indicates the name of the group of channels to modify. This name must equal to the one defined in the DECLARE GROUP instruction.

value

Expresses the modification values of the channel group. Refer to the table in the FUNCTION section.
The value can be :

- an INTEGER type constant
- an INTEGER type variable

The integer constants can have the following prefixes:

B	indicates a constant in binary format
D or no character	indicates a constant in decimal format
H or 0X	indicates a constant in hexadecimal format
O	indicates a constant in octal format

N.B. The first channel defined in the DECLARE GROUP instruction will correspond to the least significant bit (LSB) of the variable, while the last signal defined in the DECLARE GROUP instruction will

correspond to the most significant bit (MSB) of the variable. See the EXAMPLE.

SEE ALSO:

DECLARE GROUP
OG channel group, IG channel group, MG channel group, ZR channel group.

EXAMPLE:

```
DECLARE GROUP ADDHH = 45,46,47,48,49,50,51,52;
DECLARE GROUP DATHH =185,186,187,188,
189,190,191,192;
DECLARE VARIABLE VAL,VAL1;
START CLUS1;
/;
IG ADDHH=0X00;
/;
GR ADDHH=0X0F;          !Channels 45,46,47,48 go from the IL
                        status to IH status while channels
                        49,50,51,52 remain in the IL status.

@CALCOLATE VAL=0X55;
OG DATHH=VAL;
/;
@CALCOLATE VAL1=0X0A;
GR DATHH=VAL1           !Channels 185,187,189,191 go from OH
                        status to the OL status, channels
                        186,188 go from the OL status to the
                        OH status, channels 190,192 remain in
                        the OL status.
/;
ENDTEST;.
```

HI Instruction HI

Compiled	Run-time
X	**X**

FUNCTION:

Executes the conversion of the channel to high logic level. That is, if the channel was low driver, it becomes high driver; if it was low sensor it becomes high sensor.

SYNTAX:

HI channel

channel

Indicates the channel to convert. This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

SEE ALSO:

LO, IN, OU

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;

START PRG1;

IL CS;
/;
HI CS;      !The channel is converted into IH CS;
/;
OL 1;
/;
HI 1;       !The channel is converted into OH 1;
/;

IH 1/8;
/;
ENDTEST;
```

IG — Instruction — IG

Compiled	Run-time
X	**X**

FUNCTION:

Executes a monitored forcing of a group of driver channels according to the specified value.

SYNTAX:

IG channel group=value

channel group

Indicates the name of the channel group to be forced. This name must be the same as the one defined in the DECLARE GROUP instruction.

value

Expresses the value to assign to the channels where the bits at 1 indicate a high level on the channel while the bits at 0 indicate a low level. The value can be expressed as:

- an INTEGER type constant
- an INTEGER type variable

The integer constants can have the following prefixes:

B	indicates a constant in binary format
D or no character	indicates a constant in decimal format
H or 0X	indicates a constant in hexadecimal format
O	indicates a constant in octal format

N.B. The first channel defined in the DECLARE GROUP instruction will correspond to the least significant bit (LSB) of the variable, while the last signal defined in the DECLARE GROUP instruction will correspond to the most significant bit (MSB) of the variable. See the EXAMPLE.

SEE ALSO:

DECLARE GROUP
OG channel group, MG channel group, ZG channel group, GR channel group.

EXAMPLE:

```
DECLARE GROUP ADDLL = 21,22,23,24,25,26,27,28;
DECLARE GROUP ADDLH = 29,30,31,32,33,34,35,36;
DECLARE GROUP ADDHL = 37,38,39,40,41,42,43,44;
DECLARE GROUP ADDHH = 45,46,47,48,49,50,51,52;
DECLARE GROUP DATHH =185,186,187,188,
```

```
189,190,191,192;
DECLARE GROUP DATHL =177,178,179,180,
181,182,183,184;
DECLARE VARIABLE VAL,VAL1;

START CLUS1;
.
.
.
IG ADDHH=0X00;
IG ADDHL=0X05;
IG ADDLH=0X55;
IG ADDLL=0XAA;
/;
@CALCOLATE VAL=0X4F;
IG DATHH=VAL;
/;
@CALCOLATE VAL1=B01110001;
IG DATHL=VAL1;
/;
.
.
ENDTEST;
```

IH — Instruction — IH

Compiled	Run-time
X	**X**

FUNCTION:

Executes the monitored forcing of the driver channel to high level. The channel is forced at the beginning of the pattern and controlled at strobe time when the DHF module is used. It is controlled at the end of the pattern when the F40 modules are used.

SYNTAX:

IH channel

channel

Indicates the channel to force to high level. This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

DEFAULT VALUES:

If no forcing instruction is specified, the driver channels are in masked sensor mode.

SEE ALSO:

IL

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;

START PRG1;

IH CS;
/;
IH 23;
/;
IH 2,4,6,8;
/;

IH 1/8;
/;
ENDTEST;
```

IL — Instruction — IL

Compiled	Run-time
X	**X**

FUNCTION:

Executes the monitored forcing of the driver channel to low level.
The channel is forced at the beginning of the pattern and controlled at strobe time when the DHF module is used. It is controlled at the end of the pattern when the F40 modules are used.

SYNTAX:

IL channel

channel

Indicates the channel to force to low level. This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

DEFAULT VALUES:

If no forcing instruction is specified, the driver channels are in masked sensor mode.

SEE ALSO:

IH

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;

START PRG1;

IL CS;
/;
IL 23;
/;
IL 2,4,6,8;
/;

IL 1/8;
/;
ENDTEST;
```

IN **Instruction** **IN**

Compiled	Run-time
X	**X**

FUNCTION: Executes the conversion of the channel from driver to sensor. That is, if the channel was low/high driver, it becomes low/high sensor.

SYNTAX: **IN** channel

channel Indicates the channel to convert. This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

SEE ALSO: OU, LO, HI

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;

START PRG1;

IH CS;
/;
IN CS;      !The channel is converted to OH CS!
/;
IL 1;
/;
IN 1;       ! The channel is converted to OL 1;
/;
IL 1/8;
/;
ENDTEST;
```

INCREMENTAL **Instruction INCREMENTAL**

Compiled	Run-time
X	**X**

FUNCTION: Specifying this instruction, the compiler reprograms the channel situation for those patterns where there are no variations in the stimuli.

SYNTAX: **INCREMENTAL**

DEFAULT VALUES: If the instruction is not specified, the INCREMENTAL compilation mode is taken by default.

SEE ALSO: COPY

NOTE: The INCREMENTAL option is very useful because it significantly decreases the time dedicated to programming the channels; however it can be disadvantageous in the case of a program with jumps. See EXAMPLE 3.

EXAMPLE:

```
DECLARE CHANNEL CH1=48;

START PRG1;
/;          !pattern1!
INCREMENTAL;
IH CH1;
/;          !pattern2      CH1 is programmed high!
/;          !pattern3      the channel is not programmed!

IH CH1
/;          !pattern4      CH1 is programmed low!
/;          !pattern5      the channel is not programmed!

/;          !pattern6      the channel is not programmed!

IL CH1;
/;          !pattern7      the channel is not programmed
                           because the channel is already
                           low!
ENDTEST;
```

We now compare the example just described of the INCREMENTAL instruction with that of the COPY instruction.

```
DECLARE CHANNEL CH1=48;
```

```
START PRG1;
/;          !pattern1!
COPY;
IH CH1;
/;          !pattern2       CH1 is programmed high!
/;          !pattern3       CH1 is programmed high!
/;
IL CH1;     !pattern4       CH1 is programmed low!

/;          !pattern5       CH1 is programmed low!

/;          !pattern6       CH1 is programmed low!

IL CH1;
/;          !pattern7       CH1 is programmed low!

ENDTEST;
```

```
EXAMPLE 3
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
START PRG1;
/;          !pattern 1!
INCREMENTAL;
IH S1;
/;          !pattern 2!
~ATEST VAR1 LO=2 HI=5 ONERROR LAB1;
/;          !pattern 3!
IL S1;
/;          !pattern 4!
/;          !pattern 5!
LABEL=LAB1;
/;          !pattern 6!
ENDTEST;
```

LABEL=label name — Instruction — LABEL= label name

Compiled	Run-time
X	**X**

FUNCTION: Define a label, which is taken as reference for the jump instructions.

SYNTAX: **LABEL**=label name

label name: Expresses the name of the LABEL in question. This can be an alphanumerical string maximum 20 characters. The underscore "_" (underscore) is accepted in the label name while the use of special characters such as @ # $ % ~ etc... are not accepted.

SEE ALSO: Pattern option: /label name
@LABEL

NOTE: Up to 1000 labels can be defined in one functional program.

EXAMPLE:

```
START PRG1 STATIC;
~TEST VOLT HI=TOLP,LO=TOLM,MAX=15,
LI4,TIM=0.5 ;
~CURSOR 10,7;
~WRITE " LIVELLO ALTO = ";
~WRITE AR;
~BRANCH ONPASS SOTTO5;
~CALC SUBERR=SUBERR+1;
LABEL=SOTTO5;
~OM IHCH;
.
.
LABEL=FULL;
~CALC STATO_T1=0;
~CALC STATO_T2=0;
.
.
~COMP SCREEN HI="C" LO="C" ONPASS FULL;
~COMP SCREEN HI="F" LO="F" ONPASS FINSTOP;
.
.
LABEL=FINSTOP;
~CALL FINESTOP;
/;
ENDTEST;
```

LO **Instruction** **LO**

Compiled	Run-time
X	**X**

FUNCTION:

Executes the conversion of the channel to low logic level. That is, if the channel was high driver, it becomes low driver; if it was high sensor it becomes low sensor.

SYNTAX:

LO channel

channel

Indicates the channel to convert to low level. This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

SEE ALSO:

HI, IN, OU

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;

START PRG1;

IH CS;
/;
LO CS;      !The channel is converted to IL CS!
/;
OH 1;
/;
LO 1;       !The  channel is converted to OL 1;
/;
IL 1/8;
/;
ENDTEST;
```

LOAD — Instruction — LOAD

Compiled	Run-time
X	

FUNCTION:

Allows the programming of the channel data to be moved within the executive phase of the pattern. This enables faster pattern execution. The LOAD time can be used only if the timing is specified with the OPTIMIZE option (see the TIMING-ENDTIMING instruction).

SYNTAX:

LOAD value

value

Expresses the time, within the executive phase of the pattern, in which the channel data are programmed.

The value can be expressed:

- as a multiple of the clock (3c, 7c, 8c,...)
- as time (1µs, 2µs,...)
- as frequency (2.5KHz, 1mhz)

To express it as time, the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

To express it as frequency, use the following letters:

KHZ	for	KHz	(kilo hertz)
MHZ	for	MHz	(mega hertz)

SEE ALSO:

OPTIMIZE option of the TIMING- ENDTIMING instruction.

NOTE: The LOAD time can be used only if the timing is specified with the OPTIMIZE option.

EXAMPLE:

```
CLOCK 150N;
TIMING TIM1;
PERIOD 12C;
DEAD 4C;
LOAD 1C;
STROBE 11C;
ENDTIMING;

START PRG1;
.
.
.
ENDTEST;
```

MG — Instruction — MG

Compiled	Run-time
X	**X**

FUNCTION:

Executes the unmonitored forcing of a group of driver channels according to the specified value.

SYNTAX:

MG group name =value

group name

Indicates the name of the group of channels to force. This name must equal to the one defined in the DECLARE GROUP instruction.

value

Expresses the modification values of the channel group. Refer to the table in the FUNCTION section.

The value can be :

- an INTEGER type constant
- an INTEGER type variable

The integer constants can have the following prefixes:

B	indicates a constant in binary format
D or no character	indicates a constant in decimal format
H or 0X	indicates a constant in hexadecimal format
O	indicates a constant in octal format

N.B. The first channel defined in the DECLARE GROUP instruction will correspond to the least significant bit (LSB) of the variable, while the last signal defined in the DECLARE GROUP instruction will correspond to the most significant bit (MSB) of the variable. See the EXAMPLE.

SEE ALSO:

DECLARE GROUP
OG channel group, IG channel group, ZG channel group, GR channel group.

EXAMPLE:

```
DECLARE GROUP ADDLL = 21,22,23,24,25,26,27,28;
DECLARE GROUP ADDLH = 29,30,31,32,33,34,35,36;
DECLARE GROUP ADDHL = 37,38,39,40,41,42,43,44;
DECLARE GROUP ADDHH = 45,46,47,48,49,50,51,52;
```

```
DECLARE GROUP DATHH =185,186,187,188,
189,190,191,192;
DECLARE GROUP DATHL =177,178,179,180,
181,182,183,184;
DECLARE VARIABLE VAL, VAL1;

START CLUS1;
/;
IG ADDHH=0X00;
IG ADDHL=0X05;
MG ADDLH=0X55;
MG ADDLL=0XAA;
/;
@CALCOLATE VAL=0X4F;
IG DATHH=VAL;
/;
@CALCOLATE VAL1=B01110001;
MG DATHL=VAL1;
/;
ENDTEST;.
```

MH **Instruction** **MH**

Compiled	Run-time
X	**X**

FUNCTION: Executes the unmonitored forcing of a group of driver channels to high level.

SYNTAX: **MH** channel

channel

Indicates the channel to force to high level.
This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

DEFAULT VALUES: If no forcing instruction is specified, the driver channels are in masked sensor mode.

SEE ALSO: ML

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;

START PRG1;

MH CS;
/;
IH 23;
/;
MH 2,4,6,8;
/;

IH 1/8;
/;
ENDTEST;
```

ML	Instruction	ML

Compiled	Run-time
X	**X**

FUNCTION: Executes the unmonitored forcing of a group of driver channels to low level.

SYNTAX: **ML** channel

channel

Indicates the channel to force to low level.
This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

DEFAULT VALUES: If no forcing instruction is specified, the driver channels are in masked sensor mode.

SEE ALSO: MH

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;

START PRG1;

IL CS;
/;
ML 23;
/;
IL 2,4,6,8;
/;

ML 1/8;
/;
ENDTEST;
```

MODULE-ENDMODULE Instruction MODULE-ENDMODULE

Compiled	Run-time
X	**X**

FUNCTION: Allows a part of a program to be grouped in a module inside the functional program. This module can be used as a "stand-alone" section during debug.

SYNTAX:

MODULE module name
...
...
...
ENDMODULE

module name: Alphanumerical string, which represents the name of the module. The can be a maximum of 9 characters long.

NOTE: The part of the functional program included between the words MODULE-ENDMODULE is automatically extracted from a File.pat upon entry into the debug environment, where the module can be modified and then recompiled.

The module can always be modified and then recompiled. In addition, the programmer can decide whether or not to accept the modifications and to save them in the source program.

At each module declaration all of the variable and macro declarations are reset while the clock definition remains the same for all of the modules.

It is possible to declare up to 256 modules within one functional program.

EXAMPLE:

```
CLOCK 200NS;
TIMING DEFAULT;
STROBE 8C;
PERIOD 10C;
PHASE1 4C,6C;
ENDTIMING;

MODULE U1;

DECLARE PINLIST =
191,31,32,33,34,35,36,123,136,75,100,0,188,187,186,185
184,183,2,117;
DECLARE GROUP DBUS=31,32,33,34,35,36;
DECLARE GROUP OBUS=183,184,185,186,187,188;
DECLARE CHANNEL WRCK = 191;
DECLARE CHANNEL RD4 = 100;
```

```
DECLARE VARIABLE = I,J;


@MACRO WRITE;
IG DBUS=I;
IH WRCK RD4;
/;
IL WRCK;
/;
IH WRCK;
/;
/;
@ENDMACRO;

@MACRO READ;
/;
IH RD4;
/;
IL RD4;
OG OBUS=J;
/;
OM OBUS;
IH RD4;
/;
@ENDMACRO;


START U1;

!!! DIGITAL GUARDING

MH 157 103 162 89 78 171 79 100 172 192 173 99;
/ timing default;
/;
!!! DIGITAL GUARDING
/;
@CALCOLATE I=0X3F;
@WRITE;
/;
@CALCOLATE J=0X3F;
@READ;
/;
ENDTEST;

ENDMODULE;

MODULE U10;

DECLARE PINLIST =
103,5,23,24,6,7,31,32,8,75,121,9,33,34,10,11,35,36,12

START U10;

!!! DIGITAL GUARDING
MH 157 103 162 89 78 171 79 100 172 192 173 99;
/ timing default;
/;
!!! DIGITAL GUARDING

@L(L.........L.........);
@L(L1H.......T.........);
```

```
@L(L0L.......T.........);
@L(L..H1.....T.........);
@L(L..L0.....T.........);
@L(L....1H...T.........);
@L(L....0L...T.........);
@L(L......H1.T.........);
@L(L......L0.T.........);
@L(L.........T1H.......);
@L(L.........T0L.......);
@L(L.........T..H1.....);
@L(L.........T..L0.....);
@L(L.........T....1H...);
@L(L.........T....0L...);
@L(L.........T......H1.);
@L(L.........T......L0.);

ENDTEST;

ENDMODULE;
```

NR	Instruction	NR

☛

Module	Compiled	Run-time
F40	**X**	

FUNCTION: "Turns-off" the repetitive clock. From this moment on, the channel behaves according to the associated phase.

SYNTAX: **NR** channel

channel Indicates the channel to turn-off. It can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

SEE ALSO: CR

EXAMPLE:

```
CLOCKF=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 100N, 200N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL SIG1=12;
START PROG1;
/ TIMING TIM1;
IL SIG1;
/;
CR SIG1;
/;
NR SIG1;
/;
/;
ENDTEST;
```

OG0…9 Instruction OG0…9

☛

Module	Compiled	Run-time
DHF/F40	**X**	**X**

FUNCTION:

Executes a measurement of a group of sensor channels according to the associated strobe time. The binary coding of the value specifies the state of the channel to be checked, where the bit at 1 indicate that the corresponding channels have to be checked high (OH) while the bit a 0 indicate that the corresponding channels have to be checked low (OL). If the OG0 instruction is specified, the channels are checked at the end of the pattern.
For the F40 modules it is used the instruction in the form OG0..9 while for the DHF modules only the OG1 or OG instruction is allowed.

SYNTAX:

OG0…9 group name =value

group name

Indicates the name of the channel group to be measured. This name must be the same as the one defined in the DECLARE GROUP instruction.

value

Expresses the value to be measured on the channels. The value can be expressed as:

- an INTEGER type constant
- an INTEGER type variable

The integer constants can have the following prefixes:

B	indicates a constant in binary format
D or no character	indicates a constant in decimal format
H or 0X	indicates a constant in hexadecimal format
O	indicates a constant in octal format

N.B. The first channel defined in the DECLARE GROUP instruction will correspond to the least significant bit (LSB) of the variable, while the last signal defined in the DECLARE GROUP instruction will correspond to the most significant bit (MSB) of the variable. See the EXAMPLE.

SEE ALSO:

DECLARE GROUP
IG channel group, MG channel group, ZG channel group, GR channel group.

NOTA: For the F40 module: if the OG0 instruction is specified there are three different strobe times for every module because the two of the phases and the end pattern can be used.
The OG instruction is an alias of OG.

EXAMPLE:

```
DECLARE GROUP ADDLL = 21,22,23,24,25,26,27,28;
DECLARE GROUP ADDLH = 29,30,31,32,33,34,35,36;
DECLARE GROUP ADDHL = 37,38,39,40,41,42,43,44;
DECLARE GROUP ADDHH = 45,46,47,48,49,50,51,52;
DECLARE GROUP DATHH =185,186,187,188,
189,190,191,192;
DECLARE GROUP DATHL =177,178,179,180,
181,182,183,184;
DECLARE VARIABLE VAL,VAL1;
TIMING TIM1;
PERIOD 10C;
PHASE1 3C,7C;
STROBE1 9C;
ENDTIMING;
START PRG1;

START CLUS1;
/;
IG ADDHH=0X00;
IG ADDHL=0X05;
IG ADDLH=0X55;
IG ADDLL=0XAA;
/;
@CALCOLATE VAL=0X4F;
OG DATHH=VAL;          !The 189,190 and
                       192 channels are checked
                       low (OL) while the
                       185,186,187,188,191
                       channels are checked high
                       (OH)!
/;
@CALCOLATE VAL1=B01110001;
OG1 DATHL=VAL1;        !The 178,179,180 and
                       184 channels are checked
                       low (OL) while the
                       177,181,182,183 channels
                       are checked high (OH)!
/;
.
.
ENDTEST;
```

OH0…9 Instruction OH0…9

Module	Compiled	Run-time
DHF/F40	**X**	**X**

FUNCTION:

Executes the measurement of the high level of the sensor channel.
The F40 modules use the instruction in the form OH0..9 while for the DHF modules, only the OH1 or OH instruction is allowed.
The OH0 instruction checks the high level at the end of the pattern.
With the instructions, OH1…9, the channel is checked high at the associated strobe time.

SYNTAX:

OH1…9 channel

channel

Indicates the channel to measure at high level.
This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

DEFAULT VALUES:

If no forcing instruction is specified, the driver channels are in masked sensor mode.

SEE ALSO:

OL0…9

NOTE: For the F40 module: if the OH0 instruction is specified there are three different strobe times for every module because the two of the phases and the end pattern can be used.

The OH instruction is alias of OH1.

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;
TIMING TIM1;
PERIOD 10C;
PHASE1 3C,7C;
STROBE1 9C;
ENDTIMING;
START PRG1;
IL CS;
/;
IH 23;
/;
OH1 2,4;
/;
OH 6,8;
/;
OL 1/8;
/;
ENDTEST;
```

OL0…9 Instruction OL0…9

Module	Compiled	Run-time
DHF/F40	**X**	**X**

FUNCTION:

Executes the measurement of the low level of the sensor channel. For The F40 modules use the instruction in the form OL0..9 while for the DHF modules, only the OL1 or OL instruction is allowed.
The OL0 instruction checks the high level at the end of the pattern. With the instructions, OL1…9, the channel is checked high at the associated strobe time.

SYNTAX:

OL0…9 channel

channel

Indicates the channel to measure at low level.
This can be:
- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

DEFAULT VALUES:

If no forcing instruction is specified, the driver channels are in masked sensor mode.

SEE ALSO:

OH0…9

NOTE: For the F40 module: if the OL0 instruction is specified there are three different strobe times for every module because the two of the phases and the end pattern can be used.
The OL instruction is alias of OL1.

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;
TIMING TIM1;
PERIOD 10C;
PHASE1 3C,7C;
STROBE1 9C;
ENDTIMING;
START PRG1;
IL CS;
/;
IH 23;
/;
OL1 2,4;
/;
OL 6,8;
/;
OL 1/8;
/;
ENDTEST;
```

OM	Instruction	OM

Compiled	Run-time
X	**X**

FUNCTION: Masks the measurement of the sensor channel.

SYNTAX: **OM** channel

channel

Indicates the channel to mask.

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

DEFAULT VALUES: All of the channels are in masked sensor mode.

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;

START PRG1;

IL CS;
/;
IH 23;
/;
OH 2,4,6,8;
/;
.
.
OM 1/8;
/;
ENDTEST;
```

OU — Instruction — OU

Compiled	Run-time
X	**X**

FUNCTION:

Converts the channel from sensor to driver that is if the channel was low/high sensor it becomes low/high driver.

SYNTAX: **OU** channel

channel

Indicates the channel to be converted.
This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

DEFAULT VALUES: If no forcing instruction is specified, the driver channels are in masked sensor mode.

SEE ALSO: IN, LO, HI

EXAMPLE:

```
DECLARE RUNTIME INTEGER CAN;
DECLARE CHANNEL CS=125;
DECLARE CHANNEL WR=23;
DECLARE GROUP DATA=1,2,3,4,5,6,7,8;
START PRG1;
IL CS;
/;
IH CS;
/;
OL 1;
/;
OU 1;        !The channel is converted to IL 1;
/;
OH 1;
/;
OU 1;        !The channel is converted to IH 1;!
/;

IH 1/8;
/;
ENDTEST;
```

OVERLAY **Instruction** **OVERLAY**

☛

Module	Compiled	Run-time
DHF	**X**	

FUNCTION:

Indicates the time dedicated to programming the channels within the executive phase of the pattern.

SYNTAX:

OVERLAY value

value

Expresses the time dedicated to programming the channels within the executive phase of the pattern.

The value can be expressed:

- as a multiple of the clock (3c, 7c, 8c,...)
- as time (1µs, 2µs,...)
- as frequency (2.5KHz, 1mhz)

To express it as time, the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

To express it as frequency, use the following letters:

KHZ	for	KHz	(kilo hertz)
MHZ	for	MHz	(mega hertz)

DEFAULT VALUES:

If the OVERLAY time is not specified, the system automatically uses the part available from the start of the timing up to the first declaration.

(t ASSERT, PHASE1/2/3,...).

EXAMPLE:

```
CLOCK 150N;
TIMING TIM1;
PERIOD 12C;
DEAD 4C;
OVERLAY 1C;
PHASE1 3C 7C;
STROBE 11C;
ENDTIMING;

START PRG1;
.
.
.
ENDTEST;
```

The figure represents the timing TIM1.

P1…9 Instruction P1…9

Module	Compiled	Run-time
DHF/F40	**X**	**X**

FUNCTION:

Executes the forcing of the driver channels according to the movement indicated in the associated phase. The driver channels are monitored on the strobe time. For the F40 modules the instruction is used in the form P1…9 because nine different phases are available. For the DHF module the instruction in the form P1…3 is used.

SYNTAX:

P1…9 channel

channel

Indicates the channel to force according to the indications in the phase. This can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

SEE ALSO:

PHASE1…9, C1…9

NOTE: **For the DHF module:** 1 to 250 commutation instants can be defined for each phase.

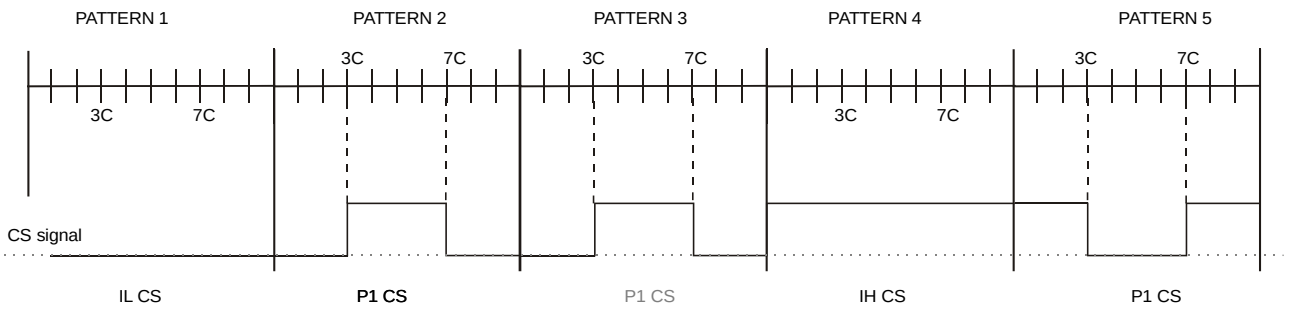
For the F40 module: only 2 commutation instants can be defined for each phase.

EXAMPLE:

```
EXAMPLE

DECLARE CHANNEL CS=25;
CLOCK= 100N;

TIMING TIM1;
PERIOD 10C;
PHASE1 3C,7C;
STROBE 9C;
ENDTIMING;

START PROG1;

IL CS;
/ TIMING TIM1;          !pattern 1, timing TIM1 is
                        associated to the patterns and the
                        channel is forced low. In this
```

	pattern the channel does not follow the timing because it is moved by the IL instruction!
P1 CS;	
/;	!**pattern 2**, the CS channel is moved according to the timing TIM1!
/;	!**pattern 3**, no instruction is specified so it is implied that the channel continues to move according to the timing TIM1!
IH CS;	
/;	!**pattern 4,** the channel is forced high and it does not follow the timing TIM1 in this pattern because it is moved by the IH instruction!
P1 CS;	
/;	!**pattern 5** , here the CS channel is moved according to the timing TIM1!
ENDTEST;	

The following figure illustrates the example described above.



PERIOD — Instruction — PERIOD

☛

Module	Compiled	Run-time
DHF/F40	**X**	

FUNCTION: Indicates the duration of the executive phase of the pattern.

SYNTAX: **PERIOD** value

value Represents the duration of the executive phase of the pattern.

The range of accepted values is:

100ns ≤ value ≤ 3.3ms or
10Mhz ≤ value ≤ 0.30Khz

The value can be expressed:

- as a multiple of the clock (3c, 7c, 8c,...)
- as time (1µs, 2µs,...)
- as frequency (2.5KHz, 1mhz)

To express it as time, the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

To express it as frequency, use the following letters:

KHZ	for	KHz	(kilo hertz)
MHZ	for	MHz	(mega hertz)

NOTE: The PERIOD time must always be specified in the timing declaration.

EXAMPLE:

```
TIMING TIM1;
PERIOD 14C;
STROBE 12C;
ENDTIMING;

START PRG1;
.
.
.
ENDTEST;
```

PHASE1…/9 **Instruction** **PHASE1…/9**

☛

Module	Compiled	Run-time
DHF/F40	**X**	

FUNCTION:

Defines the instants of commutation of the driver channels associated to the phase within the period. The channels associated to the phase will be moved by the P1… or C1...9 instruction.
For the F40 module the instruction in the form PHASE1…9 can be used while for the DHF module the instruction in the form PHASE1…3 is used.

For the F40 module the R1, R0 and RZ instructions can be used.

SYNTAX:

PHASE1…9 =values

values

Defines the instant or instants of commutation within the period and can be defined:

- as multiples of the clock (3c, 7c, 8c,...)
- as time (1µs, 2µs,...)
- as frequency (2.5KHz, 1mhz)

To express it as time, the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

To express it as frequency, use the following letters:

KHZ	for	KHz	(kilo hertz)
MHZ	for	MHz	(mega hertz)

For the F40 modules the accepted range of values is: 20ns to 5000ns.
For the DHF modules the minimum accepted value is 50ns.

N.B. It is advisable to express the commutation times of the phase in multiples of the clock (3c, 5c,...) because when the clock value is modified these times are automatically referenced to the new value.

DEFAULT VALUES:

If no PHASE instruction is defined and the ASSERT time is not defined, the channels move at the beginning of the pattern.

If no PHASE instruction is defined and the ASSERT time is defined, the channels move according to this value.

SEE ALSO: P1…9, C1…9, S1..3,R1,R0,RZ

NOTE: **General notes:** The instant or instants of commutation must be included in the duration of the period.

For the DHF module: From 1 to 250 commutations can be defined within the period.

If in the timing PHASE2 and PHASE3 are defined or PHASE1 and PHASE3 with the same commutation values, a WARNING is inserted in the File.lis and the instants are automatically moved , one from the other, by one clock pulse . Refer to EXAMPLE 3.

For the F40 module: only two commutation times can be specified in the phase.

Up to nine different phases can be defined in a test program but only two phases can be used at the same time for each module.

EXAMPLE:

EXAMPLE 1 DEFINITION AND APPLICATION OF A TIMING WITH PHASE .

```
CLOCKF INT=20N;
TIMING TIM1;
STROBE1 900N;
PHASE1 100N 300N;
PERIOD 1000N;
ENDTIMING;
DECLARE CHANNEL D0=13;
DECLARE CHANNEL A0=16;

START PROG1 STATIC;
/ DINAMIC;
IL D0;
/;
P1 D0;
/;
ENDTEST;
```

EXAMPLE 2 DEFINITION OF THREE PHASES WITH CORRECT AND WRONG APPLICATION.

```
CLOCKF=20N;
TIMING TIM1;
STROBE1 900N;
PHASE1 100N 300N;
PHASE2 200N 600N;
PHASE3 20N 100N;
PHASE4 20N 200N;
PERIOD 1000N;
ENDTIMING;
DECLARE CHANNEL D0=13;
DECLARE CHANNEL A0=16;
DECLARE CHANNEL CS=23;
```

```
DECLARE CHANNEL ADS=40;

START PROG1 STATIC;
/ DINAMIC;
IL D0;
IL A0;
IL CS;
IL ADS;
/;
P1 D0;
P2 A0; ← WRONG application, more that two simultaneous phases
                                    on channels of the same module!!
P3 CS;
/;
IH CS;
P1 D0;
P2 A0; ← P1,P2,P4= CORRECT application, the signals are not on
                                    the same F40 module
P4 ADS;
/;
ENDTEST;
```

EXAMPLE 3

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE VARIABLE VAR1;
CLOCK=50N;

TIMING TIM1;
PERIOD 10C;
PHASE1 2C 4C;
PHASE3 2C 4C;          ! PHASE3 is moved to 3C
STROBE 9C;              and a WARNING is inserted in the file
                       Prg1.lis!
ENDTIMING;

START PRG1;
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
@CALCOLATE VAR1=3;
/;
IH S2;
/;
ENDTEST;
```

R0 Instruction R0

☛

Module	Compiled	Run-time
F40	**X**	

FUNCTION:

Return to 0. If the signal is in low driver (IL), it is controlled with a high logic level impulse, then it returns in low logic level. On the contrary if the signal is in high driver (IH), it remains low for all the pattern. The impulse times are regulated by the phase associated to the channel.
The R0 instruction remains active in the following patterns until a NR instruction is not specified. Therefore the channel remains in low logic level.

SYNTAX:

R0 channel

channel

Indicates the channel to be put in Return to 0 mode. It can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

SEE ALSO:

R1

EXAMPLE:

```
CLOCKF=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 100N, 300N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL D0=12;


START PROG1;
/ TIMING TIM1;
IH D0;
/;
```

R0 D0; In this pattern and in the following a low level impulse with duration of 200ns is generated (descent 100ns, and rise 300ns).

```
/;
/;
ENDTEST;
```

R1 Instruction R1

Module	Compiled	Run-time
F40	**X**	

FUNCTION:

Return to 1. If the signal is in high driver (IH), it is controlled with a low logic level impulse, then it returns in high logic level. On the contrary if the signal is in low driver (IL), it remains high for all the pattern. The impulse times are regulated by the phase associated to the channel.
The R1 instruction remains active in the following patterns until a NR instruction is not specified. Therefore the channel remains in high logic level.

SYNTAX:

R1 channel

channel

Indicates the channel to be put in Return to 1 mode. It can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

SEE ALSO:

R0

EXAMPLE:

```
CLOCKF=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 100N, 300N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL D0=12;


START PROG1;
/ TIMING TIM1;
IL D0;
/;
R1 D0; In this pattern and in the following a low level impulse with
duration of 200ns is generated (descent 100ns,  and rise 300ns).
/;
/;
ENDTEST;
```

RG Instruction RG

Module	Compiled	Run-time
F40	**X**	

FUNCTION: Reads the value of the group of channels and assigns it to the specified variable. The value is assigned to the variable in binary format. The first declared channel in the group corresponds to the least significant bit of the variable.

SYNTAX: **RG** value = group of channels

value: Indicates the destination variable. It can be:

- an INTEGER type RUNTIME variable defined with the DECLARE RUNTIME INTEGER instruction or an integer type runtime array, defined with the DECLARE RUNTME INTEGER ARRAY instruction.

group of channels: Indicates the group of channels to read. The group of channels can be expressed as:

- physical numbers of channel
- channel names defined with the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

SEE ALSO: WG

NOTE: The RG instruction executes a reading in real time of the channels condition and the result contained in the variable will be available in a following STATIC session.

The RG instruction reads the channels condition without checking them, therefore no error is generated.

This instruction does not modify the condition of the channel.

Also driver channels or masked channels can be read, the value stored in a variable will be the one of the channel in that particular instant.

Up to 32 channels can be read with the RG instruction.

EXAMPLE:

```
CLOCKF INT=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 100N, 300N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL D0=18;
DECLARE CHANNEL D1=167;
DECLARE CHANNEL D2=124;
DECLARE CHANNEL D3=52;
DECLARE GROUP DBUS=D0,D1,D2,D3;
DECLARE CHANNEL CS=12;
DECLARE CHANNEL RD=14;
DECLARE CHANNEL A0=144;
DECLARE CHANNEL A1=143;
DECLARE RUNTIME INTEGER REA;
DECLARE RUNTIME INTEGER ARRAY DAT[10];
DECLARE RUNTIME INTEGER INDEX J;

START PROG1;
/ TIMING TIM1;
IL A0;
IL A1;
/;
IL CS;
/;
IL RD;
/;
RG REA=DBUS;
/;
RG DAT[1]=DBUS;
/;
RG DAT[2]=DBUS;
/;

IH CS,RD;
/ STATIC;
!The result of the variables/array used in the dynamic session is now
available (when switching back to the STATIC session)
~ATEST REA HI=0X55 ONERROR MES1;
STATICA!
~CURSOR 10,10;
~WRITE “DATA OK”;
~FOR J=1 TO 2;
~CURSOR CR
~WRITE “DATA =” DAT[J];
~ENDFOR;
/ MES1;
~CURSOR 20,10;
~WRITE “DATA NOT OK”;
/;
ENDTEST;
```

RZ1…9 **Instruction** **RZ1…9**

☛

Module	Compiled	Run-time
F40	**X**	

FUNCTION: The channels associated to this instruction are controlled only between the first time of the phase and the second and then they are put in high impedance.
The logic value assumed by the channels between the two times depends from the value giving in the IG/IL/IH instructions. It is used mostly on a digital signal bus.

SYNTAX: **RZ1…9** channel

channel Indicates the channel to associate to the RZ instruction. It can be:

- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

NOTE: To remove the RZ mode, use the NR instruction. Therefore the channel remains in high impedance until the next instruction, which will define its future condition (IG/IH/IL).

SEE ALSO: NR

EXAMPLE:

```
CLOCKF=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 200N, 800N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL D0=12;
DECLARE CHANNEL D1=13;
DECLARE CHANNEL D2=14;
DECLARE CHANNEL D3=15;
DECLARE GROUP DBUS =D0,D1,D2,D3;

START PROG1;
/ TIMING TIM1;
IG DBUS=0X5;
RZ1 DBUS;        The data bus is controlled with the value of 0X5
                 between 200ns to 800ns then it put in high
                 impedance.
/;               This pattern behaves as a the previous one
NR DBUS;
```

```
/;          The bus is turned-off from the RZ mode, therefore the
            channels go in high impedance.

IH DBUS;    The channels are all controlled  in high logic level
/;
ENDTEST;
```

S1/2/3 **Instruction S1/2/3**

Module	Compiled	Run-time
DHF	**X**	

FUNCTION: Executes the measurement of the sensor channels at the strobe time according to what is declared in the respective phase.

SYNTAX:

S1 channel
S2 channel
S3 channel

channel

Indicates the channel to measure according to what is specified in the respective phase. This can be:

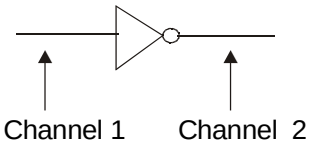
- a single channel
- a group of channels
- a channel name defined in the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non-contiguous channels.

SEE ALSO: PHASE1/2/3

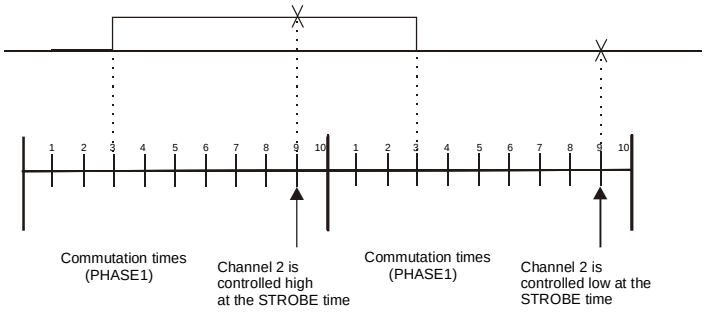
EXAMPLE:

Let's assume that we have a logical NOT gate with channel 1 on the input and channel 2 on the output. Channel 1 is associated to phase 1.




```
TIMING TIM1;
PERIOD 10C;
STROBE 9C;
PHASE1 4C;
ENDTIMING;

START PRG1;
/;
IL 1;
/;
P1 1;
S1 2;          !Channel 1 will be measured high at the strobe time (9C)!
/;
```

```
P1 1;
S1 2;          !Channel 1 will be measured low at the strobe time (9C)!
/;
ENDTEST;
```



PATTERN 3 PATTERN 4

START-ENDEST **Instruction** **START-ENDTEST**

Compiled	Run-time
X	**X**

For the sake of clarity, the START-ENDTEST instruction is indicated with a brief description of its options, which will explained in detail on the following pages. Therefore for greater detail, refer to each single option.

FUNCTION: Indicates the start and end of a functional test program.

SYNTAX: **START** program name [option 1, option 2,...]
...
...
...
ENDTEST

program name — Alphanumerical string maximum 9 characters which specifies the name associated to the functional program in question.

option1, option2,... — START-ENDTEST options, which can be:

DUTY_CYCLE=value — Indicates the rest time between the execution of one functional program and another. (Used, for example, if it is necessary to let the components under test cool off between one test and another).

STATUS_LEARN_FROM — Creates a File.pac from the information contained in the status file File.sts.

PROBE_OFF — Disables the probe for the program being examined.

PROBE_ON — Enables the probe for the program being examined.

SYNC_LOAD_ON — Enables the sync signal during the channel programming phase.

SYNC_LOAD_OFF — Disables the sync signal during the channel programming phase.

STATIC — Specifies that the program is a runtime functional program.

STATUS_GENERATE — Creates a status file File.sts from the information contained in the file File.pat.

STATUS_UPDATE — Updates the file File.sts from the information contained in the file File.pat. If there are signals in the status file that not appear in the, these remain unchanged.

STATUS_LEN=num — Determines the dimensions of the status file File.sts.

THRESHOLD=value — Indicates the value of the probe threshold expressed in V/10

$-127 \leq$ value $\leq +127$.

TIMEOUT=value — Indicates the timeout expressed in ms.

NOTE: More than one option can be specified in the same START-ENDSTART instruction, separated by a comma. If the FULL_FUN or STATIC options are not specified the program is executed in dynamic mode.

EXAMPLE:

```
CLOCK 100NS;
TIMING TIM1;
PERIOD 10C;
STROBE 8C;
ENDTIMING;
START PRG1;
/ TIMING TIM1;
COPY;
OM 1 2 3 4;
/ LABEL SOPRA MASK NOPROBE;
COPY;
OL 1 2;
OH 3 4;
/ MASK NOPROBE;
/ ONERROR JUMP SOPRA MASK NOPROBE;
COPY;
OM 1 2 3 4;
/;ENDTEST;
```

START DUTY_CYCLE=value — Option — START DUTY_CYCLE=value

Compiled	Run-time
X	

FUNCTION:

Indicates the rest time between the execution of one functional program and another. (Used, for example, if it is necessary to let the components under test cool off between one test and another).

SYNTAX:

START program name **DUTY_CYCLE=**value
...
...
...
ENDTEST

program name — Alphanumerical string maximum 9 characters which specifies the name associated to the functional program in question.

value — Value of the duty cycle expressed in milli seconds.

DEFAULT VALUES:

If not specified, the duty cycle time is automatically calculated and it is equal to 50% of the timeout.

SEE ALSO:

TIMEOUT Option of the START-ENDTEST instruction.

NOTE:

CAREFUL!!

The DUTY_CYCLE time can be specified both in the File.pat and in the File.srg. The declaration of the DUTY_CYCLE time in the File.srg has priority over the one executed in the File.pat.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG1 DUTY_CYCLE =100
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
.
ENDTEST;
```

START PROBE_OFF **Option** **START PROBE_OFF**

Compiled	Run-time
X	

FUNCTION: Disables the use of the probe for the current program. The probe can, in any case, be enabled at pattern level with the PROBE pattern option, where necessary.

SYNTAX: **START** program name **PROBE_OFF**...
...
...
...
ENDTEST

program name: Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

DEFAULT VALUES: **START** program name **PROBE_ON**

SEE ALSO: Pattern options PROBE and NOPROBE.

NOTE: The pattern option PROBE enables the probe only in the pattern in which it is written, even having specified the program as PROBE_OFF

EXAMPLE: EXAMPLE 1

The program has the probe disabled.

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG1 PROBE_OFF
IH S1 S2;
/ TIMING TIM1;      !*pattern 1
IL S1 S2;
/;                  !*pattern 2
IH S1 S2;
/ PROBE;            !*pattern 3
IL S1;
/;                  !*pattern 4
IL S2;
```

```
/ PROBE;          !*pattern 5
IH S2;
/;                !*pattern 6
/;                !*pattern 7
ENDTEST
```

EXAMPLE 2

The program has the probe disabled but it is enabled in patterns 3 and 5.

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG1 PROBE_OFF
IH S1 S2;
/ TIMING TIM1;    !*pattern 1
IL S1 S2;
/;                !*pattern 2
IH S1 S2;
/ PROBE;          !*pattern 3
IL S1;
/;                !*pattern 4
IL S2;
/ PROBE;          !*pattern 5
IH S2;
/;                !*pattern 6
/;                !*pattern 7
ENDTEST
```

START PROBE_ON — Option — START PROBE_ON

Compiled	Run-time
X	

FUNCTION: Enables the use of the probe for the current program. The probe can, in any case, be disabled at pattern level with the NOPROBE pattern option, where necessary.

SYNTAX:

START program name **PROBE_ON**
...
...
...
ENDTEST

program name: Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

DEFAULT VALUES: **START** program name **PROBE_ON**

SEE ALSO: Pattern options PROBE and NOPROBE.

NOTE: The NOPROBE disables the clock only in the pattern in which it is written, even having specified the program as PROBE_ON

EXAMPLE:

EXAMPLE 1

The program in question has the probe enabled

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG1 PROBE_ON
IH S1 S2;
/ TIMING TIM1;      !*pattern 1
IL S1 S2;
/;                  !*pattern 2
IH S1 S2;
/ PROBE;            !*pattern 3
```

```
IL S1;
/;                    !*pattern 4
IL S2;
/ PROBE;              !*pattern 5
IH S2;
/;                    !*pattern 6
/;                    !*pattern 7
ENDTEST
```

EXAMPLE 2

The program in question has the probe enabled, but it is disabled in patterns 3 and 5.

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG1 PROBE_OFF
IH S1 S2;
/ TIMING TIM1;        !*pattern 1
IL S1 S2;
/;                    !*pattern 2
IH S1 S2;
/ NOPROBE;            !*pattern 3
IL S1;
/;                    !*pattern 4
IL S2;
/ NOPROBE;            !*pattern 5
IH S2;
/;                    !*pattern 6
/;                    !*pattern 7
ENDTEST
```

START STATIC **Option** **START STATIC**

Compiled	Run-time
	X

FUNCTION: Specifies that the current program is a runtime functional program.

SYNTAX:

START program name **STATIC**
...
...
...
ENDTEST

program name: Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

SEE ALSO: FULL_FUN option of the START - ENDTEST instruction

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG1 STATIC;
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
.
.
ENDTEST;
```

START STATUS_GENERATE STATUS_GENERATE **Option** **START**

Compiled	Run-time
X	

FUNCTION: Specifies that during the current program compilation a status file File.sts is created from the information contained in File.pat

SYNTAX:

START program name **STATUS_GENERATE**
...
...
...
ENDTEST

program name: Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

SEE ALSO: STAUS_LEARN_FROM option and STATUS_UPDATE option of the START – ENDTEST instruction.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG1 STATUS_GENERATE;
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
.
.
ENDTEST;
```

START STATUS_LEARN_FROM — Option — START STATUS_LEARN_FROM

Compiled	Run-time
X	

FUNCTION:

Specifies that during the current functional program compilation a listing file (File.pac) is created from the information contained in the status file.

SYNTAX:

START program name **STATUS_LEARN_FROM**
...
...
...
ENDTEST

program name

Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

SEE ALSO:

STATUS_GENERATE option of the START – ENDTEST instruction.
STATUS_UPDATE option of the START – ENDTEST instruction.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
CLOCK=50N;
TIMING TIM1;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG1 STATUS_LEARN_FROM
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
.
.
ENDTEST;
```

START STATUS_LEN=num — Option — START STATUS_LEN=num

Compiled	Run-time
X	

FUNCTION: Defines the length of the status file.

SYNTAX:

START program name **STATUS_LEN=**num
...
...
...
ENDTEST

program name — Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

num — Length of the file expressed in number of patterns.

NOTE: The number of patterns specified in this option must be equal or higher that the number of patterns of the File.pat.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG1 STATUS_LEN=1302;
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
.
.
ENDTEST;
```

START STATUS_UPDATE **Option** **START STATUS_UPDATE**

☛

Compiled	Run-time
X	

FUNCTION:

Specifies that during the compilation current functional program the status file File.sts is up dated from the information in container in the File.pat. If there are signals in the status file that do not appear in the File.pat, these remain unchanged.

SYNTAX:

START program name **STATUS_UPDATE**
...
...
ENDTEST

program name

Alphanumerical string maximum 9 characters, which specifies the name, associated to the program in question.

SEE ALSO:

STATUS_LEARN_FROM option and STATUS_GENERATE option of the START – ENDTEST instruction

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
CLOCK=50N;
TIMING TIM1;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG1 STATUS_UPDATE;
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
.
.
ENDTEST;
```

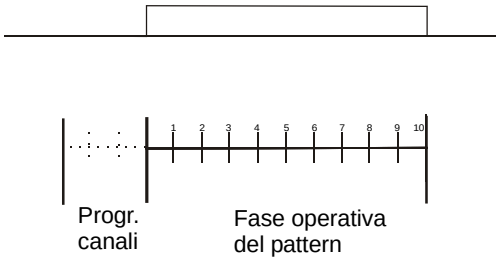
START SYNC_LOAD_OFF **Option** **START SYNC_LOAD_OFF**

☛

Module	Compiled	Run-time
DHF	**X**	

FUNCTION:

Defines the output mode of the synchronism signal. SYNC_LOAD_OFF disables the sync signal during the channel programming phase of each pattern, as illustrated in the following figure:



After having defined the output mode of the synchronism signal, it is necessary to use the pattern option SYNC to output the signal in the desired patterns.

SYNTAX:

START program name **SYNC_LOAD_OFF**...
...
...
...
ENDTEST

program name

Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

DEFAULT VALUES:

If the SYNC_LOAD_OFF or SYNC_LOAD_ON are not specified, the synchronism signal is considered to be SYNC_LOAD_OFF by default.

SEE ALSO:

SYNC pattern option

EXAMPLE:

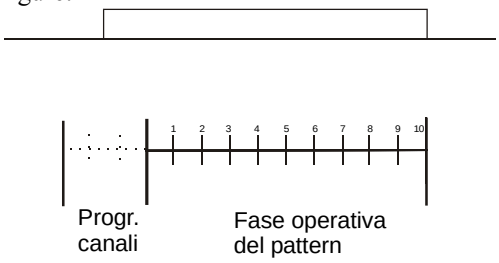
```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG1 SYNC_LOAD_OFF
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
.
.
ENDTEST;
```

START SYNC_LOAD_ON — Option — START SYNC_LOAD_ON

Module	Compiled	Run-time
DHF	**X**	

FUNCTION:

Defines the output mode of the synchronism signal. SYNC_LOAD_ON enables the sync signal during the channel programming phase of each pattern, as illustrated in the following figure:



After having defined the output mode of the synchronism signal, it is necessary to use the pattern option SYNC to output the signal in the desired patterns.

SYNTAX:

START program name **SYNC_LOAD_ON**
...
...
ENDTEST

program name — Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

DEFAULT VALUES:

If the SYNC_LOAD_OFF or SYNC_LOAD_ON are not specified, the synchronism signal is considered to be SYNC_LOAD_OFF by default.

SEE ALSO:

SYNC pattern option

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG1 SYNC_LOAD_ON
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
.
.
ENDTEST;
```

START THRESHOLD=value — Option — START THRESHOLD=value

☛

Module	Compiled	Run-time
DHF	**X**	**X**

FUNCTION:

Specifies the probe threshold for the functional program in question. The threshold is expressed in V/10.

SYNTAX:

START program name **THRESHOLD=**value
...
...
...
ENDTEST

program name

Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

value

Voltage value of the probe threshold expressed in V/10

$$-127 \leq \text{value} \leq +127$$

Thus, if the value to specify is 11,5V, write 115.

DEFAULT VALUES:

The default value of the probe threshold is 2V.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
CLOCK=50N;
TIMING TIM1 ;
PERIOD 10C;
STROBE 3C;
ENDTIMING;

START PRG1 THRESHOLD=22;
IH S1 S2;
/ TIMING TEMPO1;
IL S1 S2;
/;
.
.
ENDTEST;
```

START TIMEOUT=value — Option — START TIMEOUT=value

☛

Module	Compiled	Run-time
F40/DHF	**X**	

FUNCTION:

Indicates the maximum duration of the functional program during its execution. Once this time has expired, the execution is terminated to avoid damaging the components under test if it is faulty.

SYNTAX:

START program name **TIMEOUT**=value
...
...
ENDTEST

program name — Alphanumerical string maximum 9 characters which specifies the name associated to the program in question.

value — Value of the timeout expressed in milli seconds.

DEFAULT VALUES:

If it is not specified, the timeout is calculated automatically from the clock time. The loops expressed in the BEGINLOOP - ENDLOOP instruction are considered, but JUMP instructions, jumps to SUBROUTINES and exits from the program for errors are not.

SEE ALSO:

DUTY_CYCLE option of the START – ENDTEST instruction

NOTE:

CAREFUL!!

The TIMEOUT can be specified both in the File.pat and
The File.srg. (with the TOU command).
The declaration of the TIMEOUT in the File.srg has priority
over the one in the File.pat.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE GROUP BUS= 1,2;
CLOCK=50N;
TIMING TIM1;
PERIOD 10C;
STROBE 3C;
ENDTIMING;
START PRG1 TIMEOUT=1
IH S1 S2;
/ TIMING TIM1;
IL S1 S2;
/;
ENDTEST;
```

STROBE1…/9

Instruction STROBE1…/9

☛

Module	Compiled	Run-time
DHF/F40	**X**	

FUNCTION:

Defines the instant of measurement of the sensor and driver channels within the pattern.
The F40 modules use the instruction in the form STROBE1..9 while for the DHF modules, only the STROBE or STROBE1 instruction is allowed.

SYNTAX:

STROBE1…9 value/values

value/values

Defines the instant of measurement or the strobe window of the channel within the period. The value/values can be expressed:

- as a multiple of the clock (3c, 7c, 8c,...)
- as time (1µs, 2µs,...)
- as frequency (2.5KHz, 1mhz)

To express it as time, the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

To express it as frequency, use the following notation:

KHZ	for	KHz	(kilo hertz)
MHZ	for	MHz	(mega hertz)

For the DHF module: If two or more values are indicated for the strobe time, the associated signal is sampled twice, only in the indicated values.

For the F40 module: If two values are indicated for the strobe time, the signal is continuously strobed between the two values (strobe window).

NOTE:

The STROBE1 instruction is an alias of the STROBE instruction.
The STROBE time must always be declared in the timing.
The STROBE0 is implicit, therefore it does not have to be declared.

For the F40 module:
Up to nine different strobe times can be defined in a test program but only two can be used at the same time on channels of the same module.

Only one continuous strobe (with two values) can be defined for each F40 module.

If the channel is measured with the OH0 or OL0 instruction, there can be three different strobe times for each module because the two of the phase and the end of pattern can be used.

EXAMPLE:

EXAMPLE 1 DEFINITION OF A TIMING WITH A STROBE TIME

```
CLOCKF=20N;
TIMING TIM1;
STROBE1 900N;
PHASE1 100N 300N;
PERIOD 1000N;
ENDTIMING;
DECLARE CHANNEL D0=13;
DECLARE CHANNEL A0=16;

START PROG1 STATIC;
/ DINAMIC;
IL A0;
/;
OH1 D0;
/;
ENDTEST;
```

EXAMPLE 2 DEFINITION AND APLLICATION OF A TIMING WITH CONTINUOUS STROBE

```
CLOCKF=20N;
TIMING TIM1;
STROBE1 900N;
STROBE2 300N 900N;
PHASE1 100N 300N;
PHASE2 200N 600N;
PERIOD 1000N;
ENDTIMING;
DECLARE CHANNEL D0=13;
DECLARE CHANNEL DB0=23;
DECLARE CHANNEL A0=16;

START PROG1 STATIC;
/ DINAMIC;
IL D0,DB0;
IL A0;
/;
OH1 D0;
OH2 DB0;
/;
ENDTEST;
```

SYNC_ON/OFF **Instruction** **SYNC_ON/OFF**

☛

Module	Compiled	Run-time
F40	**X**	

FUNCTION: Generates a synchronism signal that can be used for external instrumentation. The synchro signal is generated from the pattern in which the instruction is written, on.

SYNTAX: SYNC_ON and SYNC_OFF

SEE ALSO: SYNC pattern option

EXAMPLE:

```
DECLARE CHANNEL C1;
DECLARE CHANNEL C2;

START PRG1;
/;
IL C1;
/;
IH C1;
SYNC_ON;
IL C2;
/               !Synchro signal here!
IH C2;
/;              ! Synchro signal here!
/;              ! Synchro signal here!
SYNC_OFF;
/;
ENDTEST;
```

TIMING - ENDTIMING Instruction TIMING - ENDTIMING

Compiled	Run-time
X	

FUNCTION: Defines the declaration section of the timing. The pattern duration, the strobe, the assert time, the various phases. etc. are defined in this section. The timing will then be used at pattern level during the program.

SYNTAX:

TIMING timing name option
...
...
ENDTIMING

timing name: Alphanumerical string maximum 20 characters which defines the name of the timing.

option: Indicates the option which can be associated to the timing; this can be:

optimize. The programming of the channel data is inserted in the first clock of the executive phase of the pattern. This enables faster program execution, up to pattern rates of 10MHz.

share. Indicates that the current timing can also be used by another functional program File.pat without having to be declared again.

noshare. Indicates that the timing can only be used in the current functional program and not in other File.pat functional programs.

SEE ALSO: TIMING pattern option.

NOTE: The TIMING - ENDTIMING section must always be declared.

Up to 220 timings can be declared in the functional program. However, if there are timings with more than 128 clock pulses, these occupy the space of two or more timings, in multiples of 128.

Up to 250 commutations can be declared for each timing.

To recall the timing during the program, it is sufficient to use the pattern option:

TIMING timing name;

The timing is associated to the pattern in which it is written and is maintained also for the patterns which follow, until a new timing is associated.

EXAMPLE:

```
TIMING TIM1;
STROBE 8C;
PERIOD 10C;
PHASE1 4C,6C;
ENDTIMING;

TIMING TIM2;
STROBE 4C;
PERIOD 5C;
DEAD 2C;
ENDTIMING;

TIMING TIM3 NOSHARE;
STROBE 9C;
PERIOD 10C;
OVERLAY 2C;
PHASE1 3C,5C;
ENDTIMING;

START PRG1;

/ TIMING TIM1;
/;
/;
.
.
/ TIMING TIM2;
/;
.
.
/ TIMING TIM3;
/;
/;
.
.
ENDTEST;
```

WG Instruction WG

Module	Compiled	Run-time
F40	**X**	

FUNCTION: INSTRUCTION NOT YET IMPLEMENTED. Assigns the value (expressed through a variable) at a group of channels. The least significant bit of the constant or of the variable is assigned to the first channel declared in the group.

SYNTAX: **WG** value = group of channels

value: Indicates the destination variable. It can be:

- a runtime variable defined with the DECLARE RUNTIME INTEGER instruction.

group of channels: Indicates the group of channels to read. The group of channels can be expressed as:

- physical channel numbers
- channel names defined with the DECLARE CHANNEL instruction

The "/" character is used to indicate a group of contiguous channels and the ","or the "-" character is used to indicated non contiguous channels.

SEE ALSO: RG

NOTE: The WG instruction modifies the status of the destination channel but it does not change the direction nor the mask. Therefore it is necessary to guarantee that the destination channels are already in driver before executing the writing.

Up to 32 channels can be defined in a ~WG instruction.

EXAMPLE: To be defined

ZG **Instruction** **ZG**

Compiled	Run-time
X	**X**

FUNCTION:

Allows a group of sensor channels to be masked according to the specified value. The bits at "1" of the specified value indicate the channels to be masked, while the bits at "0" indicate the channels which are not masked.

SYNTAX:

ZG group name =value

group name

Indicates the name of the group of channels to mask. This name must be the same as the one defined in the DECLARE GROUP instruction.

value

Expresses the variation value of the group of channels. Refer to the table described in the FUNCTION section.
The value can be:

- an INTEGER type constant
- an INTEGER type variable

The integer constants can have the following prefixes:

B	indicates a constant in binary format
D or no character	indicates a constant in decimal format
H or 0X	indicates a constant in hexadecimal format
O	indicates a constant in octal format

N.B. The first channel defined in the DECLARE GROUP instruction will correspond to the least significant bit (LSB) of the variable, while the last signal defined in the DECLARE GROUP instruction will correspond to the most significant bit (MSB) of the variable. See the EXAMPLE.

SEE ALSO:

DECLARE GROUP
OG channel group, IG channel group, MG channel group, GR channel group.

EXAMPLE:

```
DECLARE GROUP ADDLL = 21,22,23,24,25,26,27,28;
DECLARE GROUP ADDLH = 29,30,31,32,33,34,35,36;
DECLARE GROUP ADDHL = 37,38,39,40,41,42,43,44;
DECLARE GROUP ADDHH = 45,46,47,48,49,50,51,52;
DECLARE GROUP DATHH =185,186,187,188,
```

```
189,190,191,192;
DECLARE GROUP DATHL =177,178,179,180,
181,182,183,184;
DECLARE VARIABLE VAL;

START CLUS1;
/;
IG ADDHH=0X00;
IG ADDHL=0X05;
IG ADDLH=0X55;
IG ADDLL=0XAA;
/;
@CALCOLATE VAL=0X0F;
ZG DATHH=VAL;
/;
ZG DATHL=0XF0;
/;
ENDTEST;
```

/ DEAD — Pattern option — / DEAD

Module	Compiled	Run-time
F40/DHF	**X**	**X**

FUNCTION: Indicates, at pattern level, the minimum time provided for programming the channels before the executive phase of the pattern.

SYNTAX: **DEAD** value

value

Expresses the minimum time provided for programming the channels before the executive phase of the pattern.
The value can be expressed:

- as a multiple of the clock (3c, 7c, 8c,...)
- as time (1µs, 2µs,...)
- as frequency (2.5KHz, 1mhz)

To express it as time, the following letters are used:

P	for	ps	(pico seconds)
N	for	ns	(nano seconds)
U	for	µs	(micro seconds)
M	for	ms	(milli seconds)

To express it as frequency, use the following notation:

KHZ	for	KHz	(kilo hertz)
MHZ	for	MHz	(mega hertz)

NOTE: The dead time is inserted outside of the duration of the pattern defined in the PERIOD instruction. If the DEAD time is not sufficient for programming the channels of the pattern, an EXTRADEAD time is automatically inserted and a WARNING is generated during the compilation of the functional program. If the programming time of the channels is less than the duration of the DEAD time, this time is inserted anyway.

For the F40 module in a STATIC session of the program, the dead time establishes the time between the application of stimuli and the reading of the answers (strobe of channels) and can be programmed in the interval from 40ns and 30s. Default=5Us.

EXAMPLE:

```
DECLARE CHANNEL S1;
DECLARE CHANNEL S2;
CLOCK=150N;
TIMING TIM1;
PERIOD 12C;
STROBE 11C;
ENDTIMING;
START PRG1;
/ TIMING TIM1;
IH S1,S2;
/;
/ DEAD 2C;
```

```
IL S1;
/;
IL S2;.
ENDTEST;
```

/ DMASTER — Pattern option — / DMASTER

Compiled	Run-time
X	

FUNCTION: Enables the system to go from the runtime work environment to the compiled environment.

SYNTAX: **DMASTER**

SEE ALSO: STATIC pattern option

EXAMPLE:

```
DECLARE CHANNEL CH1=1;
DECLARE CHANNEL CH2=2;

START PRG1 STATIC;
/;
~IH CH1;
/;
~IL CH2;
/DMASTER;
IH CH1;
/;
IL CH2;
/;
ENDTEST;
```

/ JUMP — Pattern option — / JUMP

☛

Module	Compiled	Run-time
DHF/F40	**X**	

FUNCTION:

Executes the jump to the specified label. The jump can also be executed only in the case of error.

SYNTAX:

Syntax for the **DHF** module

[**ONERROR**] **JUMP** syntax label for the DHF module

Syntax for the **F40** module

/ JUMP label **[ONERROR] [ONPASS]**

label — Specifies the label to jump to. The label name can be a maximum of 9 characters.

[ONERROR] — If specified, the jump is executed if there is an error (partial error flag activated).

[ONPASS] — If specified, the jump is executed if there is not an error (partial error flag deactivated).

EXAMPLE:

EXAMPLE1
Jump example with DHF module

```
DECLARE VARIABLE VAR1;
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE CHANNEL S3=3;
DECLARE CHANNEL S4=4;

START PGR1;
/;
@CALCOLATE VAR1=6;
/ JUMP LAB1;
@CALCOLATE VAR1=VAR1+4;
/;
LABEL=LAB1;
/;
OM S1,S2,S3,S4;
/;
LABEL=ERRORE;
OL S1,S2;
OH S3,S4;
/;
/ ONERROR JUMP ERRORE;
OM S1,S2,S3,S4;
/;
ENDTEST;
```

EXAMPLE 2
Jump example with F40 module

```
CLOCKF INT=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 100N, 300N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL D0=12;
DECLARE CHANNEL OUT1=12;

START PROG1;
/ TIMING TIM1;
IL D0;
/;
OL OUT1;
/ JUMP ONERROR MIS1;
P1 D0;
/;
OH OUT1;
/ MIS1;
IH D0;
/;
ENDTEST;
```

/ MASK — Pattern option — / MASK

Compiled	Run-time
X	**X**

FUNCTION: Masks the measurement on all of the sensor channels of the pattern. In case or error neither the partial error flag or the global error flag are activated.

SYNTAX: **MASK**

DEFAULT VALUES: The measurement on the sensor channels is not masked.

SEE ALSO: MASKC

EXAMPLE:

```
IL S1,S2;
OH S3;
/;
IH S1;
OL S3;
/ MASK;     !The low measurement on channel 3 is masked!
IL S1;
/;
OH S3;
/;
```

/ MASKC — Pattern option — / MASKC

Compiled	Run-time
	X

FUNCTION: Masks the measurement on all of the sensor channels of the pattern. In case of error the global error flag is not activated, but only the partial error flag is activated; therefore it is possible to execute a measurement with the ONERROR and ONPASS instructions.

SYNTAX: **MASKC**

DEFAULT VALUES: The measurement on the sensor channels of the pattern are not masked.

SEE ALSO: MASK

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE CHANNEL S3=3;
DECLARE RUNTIME INTEGER OHCH,SUBERR;
START PRG1 STATIC;
~CALC SUBERR=0;
/;
~IL S1,S2;
~OH S3;
/;
~IH S1;
~OL S3;
/ MASKC;          !The low measurement on channel 3 is
                   masked, the partial error flag is activated,
                   while the global flag is not!

~BRANCH ONERROR NOK;      !If the partial error flag is
                           active, jump to label
                           NOK!
/;
~LI1 *,OHCH;
~OH OHCH;
/ MASKC ONERROR NOK;
/;
LABEL=NOK;
~CURSOR 65,9;
~COLOR 1,5;
~WRITE "FAIL";
~COLOR 1,15;
~REPORT SUBERR;
~CALC SUBERR=SUBERR+1;
/;
ENDTEST;
```

/ label name — Pattern option — / label name

Compiled	Run-time
X	**X**

FUNCTION: Defines a label which is taken as reference by the jump instructions.

SYNTAX: **/label name**

label name — Expresses the name of the LABEL. It can be an alphanumerical string of up to 20 characters.

NOTE: There must not be a space between / and label (**Example:** /messag1).
Up to 1000 labels can be defined in a functional program.

EXAMPLE:

```
DECLARE CHANNEL S1=1;
DECLARE CHANNEL S2=2;
DECLARE VARIABLE A;
START PRG1;
/;
@CALC A=1;
/LAB1;
/;
IL S1;
IH S2;
/;
IH S1;
IL S2;
/;
@CALC A=A+1;
/;
@IF A <14
/JUMP LAB1;
@ENDIF;
/
ENDTEST;
```

/ NOPROBE — Pattern option — / NOPROBE

Compiled	Run-time
X	

FUNCTION: Disables the use of the probe for the current pattern.

SYNTAX: NOPROBE

DEFAULT VALUES: If the NOPROBE option is not specified, the status of the probe is the one defined by the PROBE_ON/OFF option of the START/ENDTEST instruction.

SEE ALSO: START program name PROBE_ON/OFF,
PROBE pattern option

EXAMPLE:

```
DECLARE CHANNEL RESET =39;
DECLARE CHANNEL S1=67;
DECLARE CHANNEL S3=86;

START PRG1 PROBE_ON;

IL RESET;
/;
IH RESET;
/;
IL S1;
IH S2;
/;
IL S2;
/NOPROBE;        !The probe is enabled for all of the
                 program, but is disabled for this pattern!
IL S2;
/;
ENDTEST;
```

/ NOSTEP — Pattern option — / NOSTEP

Compiled	Run-time
X	**X**

FUNCTION: Disables the counter for the current pattern. The pattern number will not appear in the list file File.pac and it will not be considered in the status file of the simulator.

SYNTAX: NOSTEP

NOTE: This instruction is used when it is necessary to execute initialization or synchronization patterns on the board under test before running the simulated program. These patterns, in fact, must not be considered in the status file, which must start only from the first simulated pattern.

EXAMPLE:

```
START PRG1;
/;              !pattern 1!
IL 2;
IH 3;
/;              !pattern2!
IH 3;
IL 2;
/ NOSTEP;       !pattern3 this pattern is not stored
                in the File.pac or in the
                status file!
IL 34;
/;              !pattern4!
IH 35;
/ NOSTEP;       !pattern5 this pattern is not stored
                in the File.pac or in the
                status file!
/;              !pattern6!
ENDTEST;
```

EXAMPLE 2

At the beginning of the program a synchronization on signal the FF2 low is carried out, then the simulated program is executed. Patterns 1 to 205 therefore are not stored in the status file because they have the NOSTEP pattern option.

```
DECLARE CHANNEL RES=59;
DECLARE CHANNEL CLK=2;
DECLARE CHANNEL FF2=63;
DECLARE CHANNEL S1=5;
DECLARE CHANNEL S2=89;

START PRG2;
/ NOSTEP;       !pattern1!
```

```
IL RES;
@FOR I=1 TO 100
IL CLK;
/ NOSTEP;
IH CLK;
/ NOSTEP;          !pattern 203!
@ENDFOR;
/LAB1 NOSTEP;      !pattern 204!
IL CLK;
OL FF2;
/ NOSTEP;          !pattern 205!
IH CLK;
JUMP LAB1;
/;                              !Start simulated program!
IH S1;
/;
IL S2;
/;
.
.
.
ENDTEST;
```

/ ONERROR — Pattern option — / ONERROR

Compiled	Run-time
X	**X**

FUNCTION: Checks the partial error flag; if it is active, it jumps to the specified label.

SYNTAX: **ONERROR** label

label — Alphanumerical string maximum 9 characters which identifies the jump label.

SEE ALSO: ONPASS

EXAMPLE:

```
DECLARE CHANNEL OHCH=45;

START PRG1 STATIC;
.
.
.
~LI1 *,OHCH;
~OH OHCH;
/ ONERROR NOK;
~OM OHCH;
.
.
LABEL = NOK;
.
.
ENDTEST;
```

/ ONPASS — Pattern option — / ONPASS

Compiled	Run-time
X	**X**

FUNCTION: Checks the partial error flag; if it is not active, it jumps to the specified label.

SYNTAX: **ONPASS** label

label: Alphanumerical string maximum 9 characters which identifies the jump label.

SEE ALSO: ONERROR

EXAMPLE:

```
DECLARE CHANNEL OHCH=45;

START PRG1 STATIC;
.
.
.
~LI1 *,OHCH;
~OH OHCH;
/ ONPASS OK;
~OM OHCH;
.
.
LABEL = OK;
.
.
ENDTEST;
```

/ PROBE — Pattern option — / PROBE

Compiled	Run-time
X	

FUNCTION: Enables the use of the probe for the current pattern.

SYNTAX: PROBE

DEFAULT VALUES: If the PROBE pattern option is not specified, the status of the probe is the one defined by the PROBE_ON/OFF option of the START/ENDTEST instruction.

SEE ALSO: START program name PROBE_ON/OFF
NOPROBE pattern option

EXAMPLE:

```
DECLARE CHANNEL RESET =39;
DECLASE CHANNEL S1=67;
DECLARE CHANNEL S3=86;

START PRG1 PROBE_OFF;

IL RESET;
/;
IH RESET;
/;
IL S1;
IH S2;
/;
IL S2;
/PROBE;    !The probe is disabled for all of the program but
           is enabled in this pattern!
/;
ENDTEST;
```

/ REPEAT — Pattern option — / REPEAT

Module	Compiled	Run-time
F40	**X**	

FUNCTION: Repeats the pattern in question for the indicated number of times.

SYNTAX: **/ REPEAT** number **[ONERROR] [ONPASS]**

number — Indicates how many times the pattern must be repeated. It is a number between 2 and 65000.

[ONERROR] — If specified, the pattern repetition is executed until the error condition is present (that is until the partial error flag is active).

[ONPASS] — If specified, the pattern repetition is executed until no errors are present (that is until the partial error flag is not active).

NOTE: In the presence of repetitive patterns it is advisable to use the NOSTEP option so that the pattern numeration is not listed.

The pattern with the REPEAT option with the ONERROR/ONPASS options, does not give errors. The error is generated on the following pattern.

EXAMPLE:

```
CLOCKF INT=20N;
TIMING TIM1;
PERIOD 4U;
PHASE1 100N, 300N;
STROBE 3.5U;
ENDTIMING;
DECLARE CHANNEL D0=12;
DECLARE CHANNEL OUT1=12;

START PROG1;
/ TIMING TIM1;
IL D0;
/;
P1 D0;
/ REPEAT 1000 NOSTEP;
/;
ENDTEST;
```

/ STATIC — Pattern option — / STATIC

Compiled	Run-time
	X

FUNCTION: Enables the system to go from the Compiled work environment to the Runtime environment.

SYNTAX: **STATIC**

SEE ALSO: DMASTER pattern option

NOTE: To use the STATIC option it is necessary to begin the program in STATIC, then go to the dynamic work environment with the DMASTER option, and return to static with the STATIC option.

EXAMPLE:

```
DECLARE CHANNEL CH1=1;
DECLARE CHANNEL CH2=2;

START PRG1 STATIC;
/DMASTER;
IH CH1;
/;
IL CH2;
/STATIC;
~IH CH1;
/;
~IL CH2;
/;
ENDTEST;
```

/ SYNC — Pattern option — / SYNC

Compiled	Run-time
X	

FUNCTION: Generates a synchronism signal, as defined by the SYNC_LOAD_OFF and SYNC_LOAD_ON options of the START_ENDTEST instruction, in the pattern in which it is written. The synchronism signal can be used for external instruments.

SYNTAX: SYNC

SEE ALSO: SYNC_LOAD_ON and SYNC_LOAD_OFF options of the START-ENDTEST instruction.

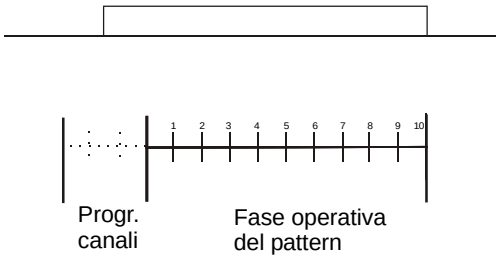
NOTE: Several program patterns can generate the synchronism signal.

EXAMPLE:

EXAMPLE 1

```
DECLARE CHANNEL C1;
DECLARE CHANNEL C2;

START PRG1 SYNC_LOAD_ON;
/;
IL C1;
/;
IH C1;
/ SYNC;     !The synchronism signal is generated!
IL C2;
/ SYNC;     !The synchronism signal is generated!
IH C2;
/;
ENDTEST;
```

The timing of the synchronism signal is as follows:

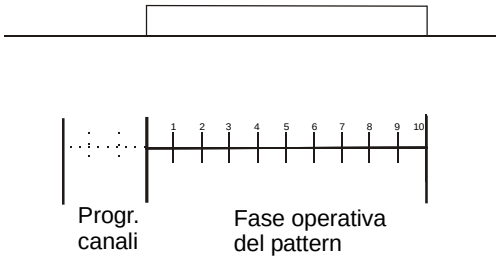


As you can see, the signal is generated even during the channel programming phase because it was enabled by the SYNC_LOAD_ON option of the START-ENDTEST instruction.

EXAMPLE 2

```
DECLARE CHANNEL C1;
DECLARE CHANNEL C2;

START PRG1 SYNC_LOAD_OFF;
/;
IL C1;
/;
IH C1;
/ SYNC;     !The synchronism signal is generated!
IL C2;
/ SYNC;     !The synchronism signal is generated!
IH C2;
/;
ENDTEST;
```

The timing of the synchronism signal is as follows:



As you can see, the signal is not generated during the channel programming phase because it was disabled by the SYNC_LOAD_ON option of the START-ENDTEST instruction.

/ TIMING — Pattern option — / TIMING

Compiled	Run-time
X	

FUNCTION: Associates the specified timing to the current pattern.

SYNTAX: TIMING timing name

timing name: Alphanumerical string maximum 20 characters which defines the name of the timing.

SEE ALSO: Declaration instructions of the TIMING - ENDTIMING timing.

NOTE: The specified timing remains active also for the following patterns until another timing association is declared.

EXAMPLE:

```
TIMING TIM1
PERIOD 10C;
PHASE1 3C 7C;
STROBE 9C;
ENDTIMING;
TIMING TIM2;
PERIOD 7C;
STROBE 5C;
DEAD 2C;
PHASE1 2C 3C;
ENDTIMING;
START PRG1;

/;
IL S2;
/ TIMING TIM1;        !Associates TIM1 to this pattern!
IH S2;
IL S1;
/;                    !TIM1 is implicitly associated to
                      this pattern!
IL S2;
/;                    !TIM1 is implicitly associated to
                      this pattern!
IH S2;
/ TIMING TIM2;        !Associates TIM2 to this pattern!
IL S2;
IH S1;
/;                    !TIM2 is implicitly associated
                      from here, on!
/;
/;
/;
ENDTEST
```

@CLUSTER **Instruction** **@CLUSTER**

File.srg	File.lib
X	

FUNCTION: It executes a Functional cluster

SYNTAX: **@CLUSTER** (“cluster name”,cluster type/diagnostic type,[channels])

cluster name

Alphanumeric string long up to 9 characters indicating the functional cluster.

cluster type/diagnostic type

This field can be one or two letters with the following meaning:

The first letter can be:

A fault area**:** if there is an error on the output pins, the diagnostic message will list all signals involved in the faulty chain. In this way, we are sure that the given fault area contains the faulty component or components.

F indicates a run-time cluster prepared to perform functional measurements. The cluster with F doesn’t program the LAM module when executed. Hence if two clusters are executed in sequence, the first cluster programs the LAM into the default condition while the second cluster DOESN’T program it. The LAM is therefore left in the the previous condition. The F letter must be present when referring to a Run-time cluster.

I in-circuit channels: the in-circuit channels will be automatically used to read the signals in the faulty chain. (the in-circuit channels can be in a bed of nail).

N no diagnostic: no diagnostic process will be executed for the present cluster. If there is an error, the test program ends indicating the failing channel and the failing step. The present cluster is a compiled cluster.

P probe: the present cluster will use the probe to read the signal during the diagnostic process. The present cluster is a compiled cluster.

S run-time cluster: the present cluster is a run-time cluster prepared for in-circuit measurements.

The second letter can be:

B bus guide: the bus guide routines will be used during the diagnostic process, to optimize it. In fact, in the diagnostic chain, the bus guide routine checks the chip select first so if the component involved is not enabled in that step, all its pins aren’t checked. In this way the diagnostic process is faster.

S the diagnostic message will list only the name of the single test that constitutes the test program but not the information about the different signals.

Q all test points in the network will be checked if this options selected (and not only one per network). In this way, if there is an interrupted track in the network, it will be detected.

[channels] lists the functional channels used during the functional test. The "/" character specifies a contiguous group of channels and the "-" character indicates a not-contiguous group.
The listed channel will be connected to LINE 5 during the test program.
It is possible also possible to list the channel used, inside the run-time functional cluster (file.pat) with the ~LI5 instruction, in this way the channels will be connected when the ~LI5 instruction is executed and not at the beginning of the cluster.

NOTE: The first letter in the "cluster type/diagnostic type" field must always be present, the second one is optional.

The diagnostic type can be specified in the @CLUSTER instruction or can be selected in the "Others Test Settings" dialog. (Select the Run\Test menu and press the button. Press the "Others" button). If the diagnostic type is selected in this dialog, it will be valid for all functional clusters listed in the file.srg

EXAMPLE:

@CLUSTER ("CTE1",P,1/192);
*Execution of the CTE1 functional cluster that use channels from 1 to 192. The diagnostic process will use the probe to read the signals"

@CLUSTER ("CL1",IB,1/224);
Execution of CL1 cluster that uses channels form 1 to 224. The diagnostic process will use the in-circuit channels and the bus guide routines

@CLUSTER ("CL2",N,1-5-7/32);
Execution of the CL2 functional cluster. No diagnostic process will be executed and only the failing channel and step are listed

@CLUSTER ("CTE1",P);
channels will be defined later on inside the file.pat

FUNCTIONAL INSTRUCTIONS of the File.SRG

The File.srg lists the in-circuit test and the functional clusters. There are some in-circuit commands that are used as parameters to the functional clusters (clock frequency setting, channel voltage thresholds, timout etc.)

CLK — Instruction — CLK

☛

Module	Compiled	Run-time
DHF	**X**	

FUNCTION: Indicates the clock period used for the functional cluster. The clock resolution is 1% of the specified value. The times that will be defined in the timing declaration inside the file.pat will be referred to the clock value defined in the CLK instruction.

SYNTAX: **CLK=**value

value: Value in ns (nano seconds) that indicates the clock period.

$50 \leq value \leq 1000$

DEFAULT VALUES : 50ns

SEE ALSO: CLOCK=val

NOTE: This instruction cancels the one declared in the File.pat with the instruction clock=val.

EXAMPLE: EXAMPLE 1

```
$include S20ANA.LIB;
#RES("R4",470,10,33,18);
.
.
.
@CLUSTER("CLUST1",S,1/32),CLK=100;
.
.
.
```

EXAMPLE 2

In this example, the clock period is defined both in the File.srg and the functional test (File.pat). The clock declaration in the fiel.srg, cancels the one declared in the File.pat therefore the final value will be clock=250ns.

File.srg

```
$include S20ANA.LIB;

#CON("C102",100,10,50,67);
.
.
.
@CLUSTER("CA3",S,1/64),CLK=250;
.
.
.
```

File.pat

```
DECLARE CHANNEL CS=2;
CLOCK=100N;
TIMING FT3;
PERIOD=10C;
STROBE=8C;
ENDTIMING;

START CA3 STATIC;
.
.
ENDTEST;
```

CUR **Instruction** **CUR**

☛

Module	Compiled	Run-time
F32/P32	**X**	**X**

FUNCTION: Indicates the value of the current clamp of the digital channels. The clamp value can be specified for a group of channel, but not for the individual channel.

SYNTAX: **CUR=**value:channels[**:ALL**]

value: Current value expressed in mA (milli Amps)

30 ≤ value ≤ 500

channels: Indicates the group of channels to which the specified current clamp is applied.
The "/" character is used to indicate a contiguous group of channels.

ALL: Specifying this option, the current clamp is applied to all of the digital channels present in the system.

DEFAULT VALUES: 200mA

EXAMPLE:

```
$include S20ANA.LIB;

#RES("R6",1000,10,45,68);
.
.
@CLUSTER("CP1",S,1/32),CUR=100:1/32;
.
.
LAB="LB1",CUR=450:ALL;
.
.
.
```

DVH — Instruction — DVH

Module	Compiled	Run-time
F40/P32/F32	**X**	**X**

FUNCTION: Defines the voltage value of the channel when it is programmed as high driver (IH).

SYNTAX: **DVH**=value:channels [**:ALL**]

value: Voltage value at which the driver channel will be programmed, expressed in Volts.

For P32/F32 channel boards: -5 ≤value≤ +12
For F40 channel boards: -3 ≤value≤ +12

channels: Indicates the group of channels which will assume the voltage value indicated when programmed as high driver.
In the case of P32 channels, groups of four channels can be formed for a total of eight different groups for each 32-channel card, The groups must begin from channels which are a multiple of 4 (***Example:*** 1-4,5-8 and not 2-5,6-9).

With one F40 board, it is possible to have up to 4 different channel groups (8 channels each).
For the F32 boards it is not possible to have different group (but only one composed by 32 channels).
The "/" is used to indicate a contiguous group of channels.

ALL: Specifying this option, the programming of the high threshold is applied to all of the digital channels present in the system.

DEFAULT VALUES: If the DVH instruction is not specified, the high threshold of the driver channels is 4.5V.

SEE ALSO: DVL

NOTE: More than one DVH instruction can be specified on the same line, using the "|" character. See the examples.

If the DVH, DVL instructions are specified for a Static cluster and the programming of the thresholds are also included in the File.pat using the ~LEVEL instruction, the latter instruction takes priority.

EXAMPLE:

```
DVH=5:ALL;
DVH=-4:1/64;
DVH=2:1/4|DVH=4.5:5/8;
DVH=10:1/32;
```

DVL **Instruction** **DVL**

Module	Compiled	Run-time
F40/P32/F32	**X**	**X**

FUNCTION: Defines the voltage value of the channel when it is programmed as low driver (IL).

SYNTAX: **DVL**=value:channels [**:ALL**]

value: Voltage value at which the driver channel will be programmed, expressed in Volts.

For the P32/F32 board: +5 ≤value≤ -12
For the F40 board: -3 ≤value≤ +12

channels: Indicates the group of channels which will assume the voltage value indicated when programmed as low driver.
In the case of P32 channels, groups of four channels can be formed for a total of eight different groups for each 32-channel card, The groups must begin from channels which are a multiple of 4 (***Example:*** 1-4,5-8 and not 2-5,6-9).
With one F40 board, it is possible to have up to 4 different channel groups (8 channels each).
With F32 boards it is not possible to have different groups.
The "/" is used to indicate a contiguous group of channels.

ALL: Specifying this option, the programming of the low threshold is applied to all of the digital channels present in the system.

DEFAULT VALUES: If the DVL instruction is not specified, the low threshold of the driver channels is 0V.

VEDERE ANCHE: DVH

NOTE: More than one DVL instruction can be specified on the same line, using the "|" character. See the examples.

If the DVH, DVL instructions are specified for a Static cluster and the programming of the thresholds are also included in the File.pat using the ~LEVEL instruction, the latter instruction takes priority.

If a Static cluster is executed in which the ~LEVEL instruction is specified, and then a dynamic cluster is executed without the specification of the DVH,DVL instructions, the threshold values present in the ~LEVEL instruction are maintained for the dynamic cluster.

EXAMPLE:

```
DVL=5:ALL;
DVL=-4:1/64;
DVL=2:1/4|DVL=4.5:5/8;
DVL=10:1/32;
```

GNO — Instruction — GNO

☛

Module	Compiled	Run-time
F40/P32/F32	**X**	**X**

FUNCTION: Connects or disconnects the ground of the channels indicated on the output connector of the channel module (and therefore on the board under test).

SYNTAX: **GNO=**ON/OFF:channels [**:ALL**]

ON — Connects the ground of the specified group of channels to the output connector..

OFF — Disconnects the ground of the specified group of channels from the output connector.

channels — Indicates the group of channels to connect/disconnect the ground to/from.
The "/" is used to indicate a contiguous group of channels.

ALL — Specifying this option, this instruction is applied to all of the digital channels present in the system.

DEFAULT VALUES: GNO=OFF for all of the digital channels.

EXAMPLE:

```
$include S20ANA.LIB;

#RES("R6",1000,10,45,68);
.
.
@CLUSTER("CP1",S,1/96),GNO=ON:1/96;
.
.
LAB="LB1",GNO=OFF:33/96;
.
.
LAB="LB2",GNO=ON:ALL;
```

LI5 Instruction LI5

Compiled	Run-time
X	**X**

FUNCTION: Connects one or more digital channels to the output connector of the relative channel module (and therefore to the board under test), closing line 5 (line dedicated to the functional test).

SYNTAX: **LI5=**channels [**:ALL**]

channels: Indicates the channel or group of channels connected via line 5. The "/" character is used to indicated a contiguous group of channels and the "-" character is used to indicated non contiguous channels

ALL: Enables the connection of all the digital channels of the system to line 5.

NOTE: It is possible to indicate more than one LI5 instruction on the same line of the program, using the "|" character. It is advisable to connect the digital channels inside the file.pat with the ~LI5 instruction because in this way they are connected only when needed.

EXAMPLE:

LI5=100; Channel 100 is connected to line 5.

LI5=5/10; Channels from 5 to 10 are connected to line 5.

LI5=1-4/32; Channel 1 and channel from 4 to 32 are connected to line 5.

LI5=ALL; All channels in the system are connected to line 5.

LI5=1-6-8|LI5=32/64; Two connection instructions are listed in the same line

NOS	Instruction	NOS

Compiled	Run-time
X	**X**

FUNCTION: Indicates whether or not the test program must continue if an error occurs on the measured channels

SYNTAX: **NOS**=Y/N

Y Program execution continues even if an error occurs.

N Program execution is interrupted when an error occurs

DEFAULT VALUES: NOS=N

NOTE: This instruction can be combined with the "Stop on functional error" button on the test environment palette. When active, this button causes the channel and the program pattern on which the error was found to be displayed in the S20 Windows Application dialog, program execution is stops, the power on the board is maintained and the system waits for a response from the operator to continue.

We can have four different situations:

1- NOS=N and the "Stop on functional error" button disabled: the test program ends on the first error found and the error is printed.

2 - NOS=N and the "Stop on functional error" button active: the test program stops on the first error found and indicates the channel in error. In addition, the power to the board is maintained, allowing the operator to diagnosis the nodes involved. Program execution ends after the first error is displayed.

3 - NOS=Y and the "Stop on functional error" button disabled: the test pogrom is executed to the end, even if errors are found; the last error found is printed.

4 - The File.srg includes this indication: NOS=Y and the "Stop on functional error" button is active: The test program stops on each error found, indicating the channel in error. In addition, power on the board is maintained, allowing the operator to diagnosis the node involved. The last error found is printed.

EXAMPLE:

```
$include S20FUN.LIB;
.
.
```

@CLUSTER(“FS5”,S,1/196), GNO=ON:ALL, CUR=400:1/32,NOS=Y;
.
.

PRB **Instruction** **PRB**

Compiled	Run-time
X	

FUNCTION: Programs the probe for 1 or 10

SYNTAX: **PRB=**1/10

1 Specifying PRB=1 the probe is programmed for 1.

10 Specifying PRB=10 the probe is programmed for 10.

DEFAULT VALUES: PRB=10

NOTE: It is possible to use the probe only if the real time controller (DMAST) is present.

EXAMPLE:

```
@CLUSTER("CL1",P,1/32),PRB=10;
.
.
.
@CLUSTER("CL2",P,33/64),PRB=1;
```

THH — Instruction — THH

Compiled	Run-time
X	**X**

FUNCTION: Defines the high threshold value of the channel when it is programmed as sensor (OH).

SYNTAX: **THH**=value:channels [**:ALL**]

value — Voltage value at which the sensor channel will be programmed, expressed in Volts.

For P32/F32 boards: -12.8 ≤value≤ +12.7
For F40 boards: -3 ≤value≤ +12

channels — Indicates the group of channels which will assume the voltage value indicated when programmed as high sensor.
In the case of P32 channels, groups of four channels can be formed for a total of eight different groups for each 32-channel card, The groups must begin from channels which are a multiple of 4 (***Example:*** 1-4,5-8 and not 2-5,6-9).
With the F40 board it is possible to have up to 4 groups of 8 channels each.
For the F32 board it is not possible to have different groups.
The "/" is used to indicate a contiguous group of channels.

ALL — Specifying this option, the programming of the high threshold is applied to all of the digital channels present in the system.

DEFAULT VALUES: If the THH instruction is not specified, the high threshold of the sensor channels is 2.4V.

SEE ALSO: THL

NOTE: More than one THH instruction can be specified on the same line, using the "|" character. See the examples.

If the THH,THL instructions are specified for a Static cluster and the programming of the thresholds are also included in the File.pat using the ~LEVEL instruction, the latter instruction takes priority.

If a Static cluster is executed in which the ~LEVEL instruction is specified, and then a dynamic cluster is executed without the specification of the THH,THL instructions, the threshold values present in the ~LEVEL instruction are maintained for the dynamic cluster.

If a Static cluster is executed in which the ~LEVEL instruction is specified, and then a dynamic cluster is executed which includes the

specification of the THH,THL instructions, the latter instructions have priority over the former one.

EXAMPLE:

```
THH=5:ALL;
THH=-4:1/64;
THH=2:1/4|THH=4.5:5/8;
THH=10:1/32;
```

THL **Instruction** **THL**

Compiled	Run-time
X	**X**

FUNCTION: Defines the low threshold value of the channel when it is programmed as sensor (OH).

SYNTAX: **THL**=value:channels [**:ALL**]

value: Voltage value at which the sensor channel will be programmed, expressed in Volts.

For P32/F32 boards: -12.8 ≤value≤ +12.7
For F40 boards: -3 ≤value≤ +12

channels: Indicates the group of channels which will assume the voltage value indicated when programmed as low sensor.
In the case of P32 channels, groups of four channels can be formed for a total of eight different groups for each 32-channel card, The groups must begin from channels which are a multiple of 4 (***Example:*** 1-4,5-8 and not 2-5,6-9).
With the F40 board it is possible to have up to 4 groups of 8 channels each.
For the F32 board it is not possible to have different groups.
The "/" is used to indicate a contiguous group of channels.

ALL: Specifying this option, the programming of the low threshold is applied to all of the digital channels present in the system.

DEFAULT VALUES: If the THL instruction is not specified, the low threshold of the sensor channels is 0.8V.

SEE ALSO: THL

NOTE: More than one THL instruction can be specified on the same line, using the "|" character. See the examples.

If the THH,THL instructions are specified for a Static cluster and the programming of the thresholds are also included in the File.pat using the ~LEVEL instruction, the latter instruction takes priority.

If a Static cluster is executed in which the ~LEVEL instruction is specified, and then a dynamic cluster is executed without the specification of the THH,THL instructions, the threshold values present in the ~LEVEL instruction are maintained for the dynamic cluster.

If a Static cluster is executed in which the ~LEVEL instruction is

specified, and then a dynamic cluster is executed which includes the specification of the THH,THL instructions, the latter instructions have priority over the former one.

EXAMPLE:

```
THL=5:ALL;
THL=-4:1/64;
THL=2:1/4|THL=4.5:5/8;
THL=10:1/32;
```

TH2 Instruction TH2

Module	Compiled	Run-time
F32/P32	**X**	**X**

FUNCTION: Indicates whether two thresholds (specified with the DVH,DVL,THH,THL instructions) are used for the current test or only one.

SYNTAX: **TH2**=Y/N

Y — If TH2=Y is specified, two thresholds are used for the current program.

N — If TH2=N is specified, only the higher threshold is used for the current program.

DEFAULT VALUES: TH2=N

NOTE: For the F40 board, the TH2=N command is ignored because two thresholds are always used.

SEE ALSO: DVH,DVL,THH,THL

EXAMPLE: TH2=Y;

TH2=N;

TOU **Instruction** **TOU**

Compiled	Run-time
X	**X**

FUNCTION:

Indicates the maximum duration of the functional test program execution phase. Once this time has expired, the execution is ended to avoid damaging the components under test in the case of a malfunction. This instruction is also useful in the case of synchronization on an external signal. If, for some reason, the synchronization does not occur, once the timeout time has expired, the program ends.

SYNTAX:

TOU=value

value

Value expressed in ms which indicates the timeout time.

$$0 \leq \text{value} \leq 65535$$

DEFAULT VALUES:

If the timeout is not specified, it is calculated automatically on the basis of the clock time. The loops expressed with the BEGINLOOP-ENDLOOP instruction are considered, but the Jump instructions, jumps to SUBROUTINES and exits on error are not considered in this calculation.

SEE ALSO:

TIMEOUT option of the START-ENDTEST instruction.

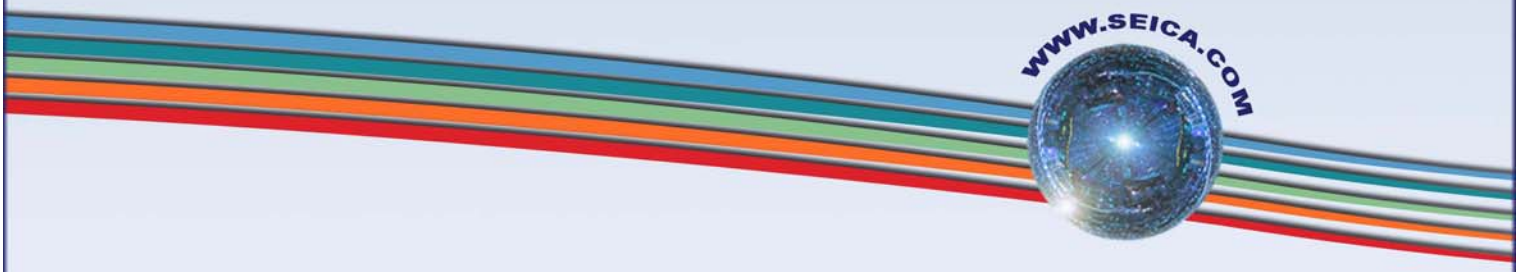
NOTE:

CAREFUL!!

The TIMEOUT can be specified both in the File.pat and the File.srg (TOU command). The TIMEOUT declaration in the File.srg has priority over the one executed in the File.pat.

EXAMPLE:

TOU=1000;



Seica
Test Solutions

SEICA SpA
via Kennedy 24
10019 Strambino - TO
ITALIA
Tel: +39 0125 6368.11
Fax: +39 0125 6368.99
Email: seica@seica.com